Gwent

2.1.3

Generated by Doxygen 1.15.0

1 Dandelion's Calculator 1
1.1 Overview . . . 1
1.2 Printing analog cards . . . 1
1.3 Documentation . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 9
4.1 File List . . . 9

5 Class Documentation 11
5.1 Ability Enum Reference . . . 11
5.1.1 Detailed Description . . . 11
5.1.2 Member Data Documentation . . . 11
5.1.2.1 BINDING . . . 11
5.1.2.2 HORN . . . 12
5.1.2.3 MORAL_BOOST . . . 12
5.1.2.4 NONE . . . 12
5.1.2.5 REVENGE . . . 12
5.2 AddCardDialog Class Reference . . . 12
5.2.1 Detailed Description . . . 14
5.2.2 Constructor & Destructor Documentation . . . 14
5.2.2.1 AddCardDialog() [1/2] . . . 14
5.2.2.2 AddCardDialog() [2/2] . . . 14
5.2.3 Member Function Documentation . . . 14
5.2.3.1 onCreate() . . . 14
5.2.4 Member Data Documentation . . . 15
5.2.4.1 disposables . . . 15
5.2.4.2 pickerAdapter . . . 15
5.2.4.3 row . . . 15
5.2.4.4 soundManager . . . 15
5.3 AppDatabase Class Reference . . . 16
5.3.1 Detailed Description . . . 16
5.3.2 Member Function Documentation . . . 17
5.3.2.1 rows() . . . 17
5.3.2.2 units() . . . 17
5.4 BondDamageCalculatorDecorator Class Reference . . . 17
5.4.1 Detailed Description . . . 19
5.4.2 Constructor & Destructor Documentation . . . 19
5.4.2.1 BondDamageCalculatorDecorator() . . . 19

5.4.3 Member Function Documentation . . . 19
5.4.3.1 calculateDamage() . . . 19
5.4.3.2 isBuffed() . . . 20
5.4.4 Member Data Documentation . . . 20
5.4.4.1 idToSquadSize . . . 20
5.5 BurnDialogUseCase Class Reference . . . 20
5.5.1 Detailed Description . . . 21
5.5.2 Member Function Documentation . . . 21
5.5.2.1 burn() [1/2] . . . 21
5.5.2.2 burn() [2/2] . . . 22
5.5.2.3 getBurnUnits() . . . 22
5.6 ResetAlertDialogBuilderAdapter.Callback Interface Reference . . . 23
5.6.1 Detailed Description . . . 23
5.6.2 Member Function Documentation . . . 23
5.6.2.1 reset() [1/2] . . . 23
5.6.2.2 reset() [2/2] . . . 23
5.7 ChangeFactionDialog.Callback Interface Reference . . . 24
5.7.1 Detailed Description . . . 24
5.7.2 Member Function Documentation . . . 24
5.7.2.1 onThemeSelect() . . . 24
5.8 CardListAdapter Class Reference . . . 25
5.8.1 Detailed Description . . . 26
5.8.2 Constructor & Destructor Documentation . . . 26
5.8.2.1 CardListAdapter() . . . 26
5.8.3 Member Function Documentation . . . 26
5.8.3.1 getItemId() . . . 26
5.8.3.2 onBindViewHolder() . . . 27
5.8.3.3 onCreateViewHolder() . . . 27
5.8.4 Member Data Documentation . . . 27
5.8.4.1 onCopy . . . 27
5.8.4.2 onRemove . . . 28
5.9 CardNumberPickerAdapter Class Reference . . . 28
5.9.1 Detailed Description . . . 29
5.9.2 Constructor & Destructor Documentation . . . 29
5.9.2.1 CardNumberPickerAdapter() . . . 29
5.9.3 Member Function Documentation . . . 30
5.9.3.1 addSelectedUnits() . . . 30
5.9.3.2 getDelayedOnValueChangeListener() . . . 30
5.9.4 Member Data Documentation . . . 30
5.9.4.1 abilityPicker . . . 30
5.9.4.2 damagePicker . . . 31
5.9.4.3 EPIC_UNIT_ABILITIES . . . 31

5.9.4.4 epicPicker . . . 31
5.9.4.5 numberPicker . . . 31
5.9.4.6 squadManager . . . 31
5.9.4.7 squadPicker . . . 31
5.10 CardUiState Class Reference . . . 32
5.10.1 Detailed Description . . . 33
5.10.2 Constructor & Destructor Documentation . . . 33
5.10.2.1 CardUiState() . . . 33
5.10.3 Member Function Documentation . . . 34
5.10.3.1 equals() . . . 34
5.10.3.2 getAbilityImageId() . . . 34
5.10.3.3 getDamageBackgroundImageId() . . . 34
5.10.3.4 getDamageString() . . . 34
5.10.3.5 getDamageTextColor() . . . 35
5.10.3.6 getSquadString() . . . 35
5.10.3.7 getUnitId() . . . 35
5.10.3.8 showAbility() . . . 36
5.10.3.9 showSquad() . . . 36
5.10.4 Member Data Documentation . . . 36
5.10.4.1 abilityImageId . . . 36
5.10.4.2 damageBackgroundImageId . . . 36
5.10.4.3 damageString . . . 36
5.10.4.4 damageTextColor . . . 37
5.10.4.5 DIFF_CALLBACK . . . 37
5.10.4.6 squadString . . . 37
5.10.4.7 unitId . . . 37
5.10.4.8 UNUSED . . . 37
5.11 CardUiStateFactory Class Reference . . . 38
5.11.1 Detailed Description . . . 38
5.11.2 Constructor & Destructor Documentation . . . 39
5.11.2.1 CardUiStateFactory() . . . 39
5.11.3 Member Function Documentation . . . 39
5.11.3.1 createCardUiState() [1/2] . . . 39
5.11.3.2 createCardUiState() [2/2] . . . 39
5.11.4 Member Data Documentation . . . 40
5.11.4.1 damageTextColors . . . 40
5.11.4.2 horn . . . 40
5.11.4.3 weather . . . 40
5.12 CardListAdapter.CardViewHolder Class Reference . . . 40
5.12.1 Detailed Description . . . 42
5.12.2 Constructor & Destructor Documentation . . . 42
5.12.2.1 CardViewHolder() . . . 42

5.12.3 Member Function Documentation . . . 42
5.12.3.1 getAbilityView() . . . 42
5.12.3.2 getBindingView() . . . 42
5.12.3.3 getDamageView() . . . 43
5.12.3.4 setItemId() . . . 43
5.12.4 Member Data Documentation . . . 43
5.12.4.1 abilityView . . . 43
5.12.4.2 bindingView . . . 44
5.12.4.3 damageView . . . 44
5.12.4.4 itemId . . . 44
5.13 ChangeFactionDialog Class Reference . . . 44
5.13.1 Detailed Description . . . 46
5.13.2 Constructor & Destructor Documentation . . . 46
5.13.2.1 ChangeFactionDialog() . . . 46
5.13.3 Member Function Documentation . . . 46
5.13.3.1 getOnThemeClickListener() . . . 46
5.13.3.2 onCreate() . . . 47
5.13.4 Member Data Documentation . . . 47
5.13.4.1 callback . . . 47
5.14 CoinFlipDialog Class Reference . . . 47
5.14.1 Detailed Description . . . 49
5.14.2 Constructor & Destructor Documentation . . . 49
5.14.2.1 CoinFlipDialog() . . . 49
5.14.3 Member Function Documentation . . . 49
5.14.3.1 onCreate() . . . 49
5.14.4 Member Data Documentation . . . 50
5.14.4.1 RANDOM . . . 50
5.15 DamageCalculator.Color Enum Reference . . . 50
5.15.1 Detailed Description . . . 50
5.15.2 Member Data Documentation . . . 50
5.15.2.1 BUFFED . . . 50
5.15.2.2 DEBUFFED . . . 50
5.15.2.3 DEFAULT . . . 51
5.16 DamageCalculator Interface Reference . . . 51
5.16.1 Detailed Description . . . 51
5.16.2 Member Function Documentation . . . 52
5.16.2.1 calculateDamage() . . . 52
5.16.2.2 isBuffed() . . . 52
5.17 DamageCalculatorBuildDirector Class Reference . . . 53
5.17.1 Detailed Description . . . 53
5.17.2 Member Function Documentation . . . 53
5.17.2.1 getCalculator() . . . 53

5.17.2.2 setHorns() . . . 54
5.17.2.3 setMoralBoosts() . . . 54
5.17.2.4 setSquads() . . . 55
5.18 DamageCalculatorBuilder Class Reference . . . 55
5.18.1 Detailed Description . . . 56
5.18.2 Member Function Documentation . . . 56
5.18.2.1 getResult() . . . 56
5.18.2.2 setBond() . . . 56
5.18.2.3 setHorn() . . . 57
5.18.2.4 setMoral() . . . 57
5.18.2.5 setWeather() . . . 57
5.18.3 Member Data Documentation . . . 57
5.18.3.1 calculator . . . 57
5.19 DamageCalculatorDecorator Class Reference . . . 58
5.19.1 Detailed Description . . . 59
5.19.2 Constructor & Destructor Documentation . . . 59
5.19.2.1 DamageCalculatorDecorator() . . . 59
5.19.3 Member Data Documentation . . . 59
5.19.3.1 component . . . 59
5.20 DamageCalculatorUseCase Class Reference . . . 59
5.20.1 Detailed Description . . . 60
5.20.2 Member Function Documentation . . . 60
5.20.2.1 getDamageCalculator() [1/2] . . . 60
5.20.2.2 getDamageCalculator() [2/2] . . . 60
5.21 DamageValuePicker Class Reference . . . 61
5.21.1 Detailed Description . . . 63
5.21.2 Constructor & Destructor Documentation . . . 63
5.21.2.1 DamageValuePicker() . . . 63
5.21.3 Member Function Documentation . . . 64
5.21.3.1 getDisplayString() . . . 64
5.21.3.2 getValue() . . . 64
5.21.3.3 setEpicValues() . . . 64
5.21.3.4 setSelectableValues() . . . 65
5.21.3.5 setValue() . . . 65
5.21.4 Member Data Documentation . . . 65
5.21.4.1 epicValues . . . 65
5.22 ResetDialogUseCase.DialogType Enum Reference . . . 66
5.22.1 Detailed Description . . . 66
5.22.2 Member Data Documentation . . . 66
5.22.2.1 DEFAULT . . . 66
5.22.2.2 MONSTER . . . 66
5.22.2.3 NONE . . . 66

5.23 FactionSwitchListener Class Reference . . . 67
5.23.1 Detailed Description . . . 68
5.23.2 Constructor & Destructor Documentation . . . 68
5.23.2.1 FactionSwitchListener() . . . 68
5.23.3 Member Function Documentation . . . 69
5.23.3.1 getListener() . . . 69
5.23.3.2 onSharedPreferenceChanged() . . . 69
5.23.3.3 setTheme() . . . 70
5.23.4 Member Data Documentation . . . 70
5.23.4.1 ballViews . . . 70
5.23.4.2 cardViews . . . 70
5.23.4.3 factionButton . . . 70
5.23.4.4 THEME_MONSTER . . . 71
5.23.4.5 THEME_NILFGAARD . . . 71
5.23.4.6 THEME_NORTHERN_KINGDOMS . . . 71
5.23.4.7 THEME_PREFERENCE_KEY . . . 71
5.23.4.8 THEME_SCOIATAEL . . . 71
5.23.4.9 unitViews . . . 71
5.24 GameBoardViewModel Class Reference . . . 71
5.24.1 Detailed Description . . . 73
5.24.2 Constructor & Destructor Documentation . . . 73
5.24.2.1 GameBoardViewModel() . . . 73
5.24.3 Member Function Documentation . . . 74
5.24.3.1 getMenuUiState() . . . 74
5.24.3.2 getModel() . . . 74
5.24.3.3 getRepository() . . . 75
5.24.3.4 getRowUiState() . . . 75
5.24.3.5 getSoundManager() . . . 75
5.24.3.6 onBurnButtonPressed() . . . 75
5.24.3.7 onFactionSwitchReset() . . . 76
5.24.3.8 onHornViewPressed() . . . 76
5.24.3.9 onResetButtonPressed() . . . 77
5.24.3.10 onWeatherButtonPressed() . . . 77
5.24.3.11 onWeatherViewPressed() . . . 77
5.24.3.12 reset() . . . 78
5.24.4 Member Data Documentation . . . 78
5.24.4.1 INITIALIZER . . . 78
5.24.4.2 menuUiState . . . 78
5.24.4.3 rowUiStates . . . 79
5.24.4.4 soundManager . . . 79
5.25 GwentApplication Class Reference . . . 79
5.25.1 Detailed Description . . . 80

5.25.2 Member Function Documentation . . . 81
5.25.2.1 getRepository() [1/2] . . . 81
5.25.2.2 getRepository() [2/2] . . . 81
5.25.2.3 onCreate() . . . 81
5.25.3 Member Data Documentation . . . 81
5.25.3.1 database . . . 81
5.25.3.2 repository . . . 82
5.26 HornDamageCalculatorDecorator Class Reference . . . 82
5.26.1 Detailed Description . . . 84
5.26.2 Constructor & Destructor Documentation . . . 84
5.26.2.1 HornDamageCalculatorDecorator() . . . 84
5.26.3 Member Function Documentation . . . 84
5.26.3.1 calculateDamage() . . . 84
5.26.3.2 doubleDamage() . . . 85
5.26.3.3 isBuffed() . . . 85
5.26.4 Member Data Documentation . . . 86
5.26.4.1 unitIds . . . 86
5.27 ImageViewSwitchAnimator Class Reference . . . 86
5.27.1 Detailed Description . . . 86
5.27.2 Member Function Documentation . . . 86
5.27.2.1 animatedSwitch() . . . 86
5.27.2.2 getOverlayView() . . . 87
5.28 IndicatorManager Class Reference . . . 87
5.28.1 Detailed Description . . . 88
5.28.2 Constructor & Destructor Documentation . . . 88
5.28.2.1 IndicatorManager() . . . 88
5.28.3 Member Function Documentation . . . 88
5.28.3.1 updateIndicators() . . . 88
5.28.4 Member Data Documentation . . . 89
5.28.4.1 currentView . . . 89
5.28.4.2 indicators . . . 89
5.29 IntroductionActivity Class Reference . . . 89
5.29.1 Detailed Description . . . 90
5.29.2 Member Function Documentation . . . 91
5.29.2.1 onCreate() . . . 91
5.29.2.2 onResume() . . . 91
5.29.2.3 updateBottomBar() . . . 91
5.29.3 Member Data Documentation . . . 91
5.29.3.1 callback . . . 91
5.29.3.2 finishButton . . . 91
5.29.3.3 indicatorManager . . . 92
5.29.3.4 nextButton . . . 92

5.29.3.5 skipButton . . . 92
5.29.3.6 viewPager . . . 92
5.30 IntroductionFragment Class Reference . . . 92
5.30.1 Detailed Description . . . 93
5.30.2 Member Function Documentation . . . 93
5.30.2.1 newInstance() . . . 93
5.30.2.2 onCreateView() . . . 94
5.30.3 Member Data Documentation . . . 94
5.30.3.1 ARG_SECTION_NUMBER . . . 94
5.30.3.2 imageIds . . . 94
5.30.3.3 PAGES_COUNT . . . 95
5.30.3.4 stringIds . . . 95
5.31 MainActivity Class Reference . . . 95
5.31.1 Detailed Description . . . 97
5.31.2 Member Function Documentation . . . 97
5.31.2.1 inflateCoinFlipPopup() . . . 97
5.31.2.2 inflateFactionPopup() . . . 97
5.31.2.3 initializeViewModel() . . . 97
5.31.2.4 onCreate() . . . 97
5.31.2.5 onDestroy() . . . 98
5.31.2.6 onResume() . . . 98
5.31.2.7 onWindowFocusChanged() . . . 98
5.31.3 Member Data Documentation . . . 98
5.31.3.1 disposables . . . 98
5.31.3.2 factionSwitchListener . . . 98
5.31.3.3 gameBoardViewModel . . . 99
5.32 MenuUiState Class Reference . . . 99
5.32.1 Detailed Description . . . 100
5.32.2 Constructor & Destructor Documentation . . . 100
5.32.2.1 MenuUiState() . . . 100
5.32.3 Member Function Documentation . . . 100
5.32.3.1 equals() . . . 100
5.32.3.2 getDamage() . . . 100
5.32.3.3 isBurn() . . . 101
5.32.3.4 isReset() . . . 101
5.32.3.5 isWeather() . . . 101
5.32.4 Member Data Documentation . . . 101
5.32.4.1 burn . . . 101
5.32.4.2 damage . . . 101
5.32.4.3 reset . . . 101
5.32.4.4 weather . . . 102
5.33 MenuUiStateObserver Class Reference . . . 102

5.33.1 Detailed Description . . . 103
5.33.2 Constructor & Destructor Documentation . . . 103
5.33.2.1 MenuUiStateObserver() . . . 103
5.33.3 Member Function Documentation . . . 103
5.33.3.1 accept() . . . 103
5.33.4 Member Data Documentation . . . 104
5.33.4.1 burnButton . . . 104
5.33.4.2 damageView . . . 104
5.33.4.3 resetButton . . . 104
5.33.4.4 weatherButton . . . 104
5.34 MoralDamageCalculatorDecorator Class Reference . . . 104
5.34.1 Detailed Description . . . 106
5.34.2 Constructor & Destructor Documentation . . . 106
5.34.2.1 MoralDamageCalculatorDecorator() . . . 106
5.34.3 Member Function Documentation . . . 107
5.34.3.1 calculateDamage() . . . 107
5.34.3.2 isBuffed() . . . 107
5.34.4 Member Data Documentation . . . 107
5.34.4.1 unitIds . . . 107
5.35 ValuePicker< T extends Comparable >.OnValueChangeListener< T extends Comparable > Interface Template Reference . . . 108
5.35.1 Detailed Description . . . 108
5.35.2 Member Function Documentation . . . 108
5.35.2.1 onValueChange() . . . 108
5.36 OverlayDialog Class Reference . . . 109
5.36.1 Detailed Description . . . 110
5.36.2 Constructor & Destructor Documentation . . . 110
5.36.2.1 OverlayDialog() [1/2] . . . 110
5.36.2.2 OverlayDialog() [2/2] . . . 110
5.36.3 Member Function Documentation . . . 111
5.36.3.1 onCreate() . . . 111
5.36.4 Member Data Documentation . . . 111
5.36.4.1 BACKGROUND . . . 111
5.36.4.2 dismissViewId . . . 111
5.36.4.3 layout . . . 111
5.36.4.4 NO_DISMISS_VIEW . . . 111
5.37 RemoveUnitsUseCase Class Reference . . . 112
5.37.1 Detailed Description . . . 112
5.37.2 Member Function Documentation . . . 112
5.37.2.1 getRevengeDialog() . . . 112
5.37.2.2 remove() [1/2] . . . 113
5.37.2.3 remove() [2/2] . . . 114

5.38 ResetAlertDialogBuilderAdapter Class Reference . . . 114
5.38.1 Detailed Description . . . 115
5.38.2 Constructor & Destructor Documentation . . . 115
5.38.2.1 ResetAlertDialogBuilderAdapter() . . . 115
5.38.3 Member Function Documentation . . . 116
5.38.3.1 create() . . . 116
5.38.3.2 setMonsterDialog() . . . 116
5.38.3.3 setTrigger() . . . 116
5.38.4 Member Data Documentation . . . 117
5.38.4.1 adapteeBuilder . . . 117
5.38.4.2 dialogCallback . . . 117
5.39 ResetDialogUseCase Class Reference . . . 117
5.39.1 Detailed Description . . . 118
5.39.2 Member Function Documentation . . . 118
5.39.2.1 getDialogType() . . . 118
5.39.2.2 reset() [1/2] . . . 118
5.39.2.3 reset() [2/2] . . . 119
5.40 ResetRepositoryUseCase Class Reference . . . 120
5.40.1 Detailed Description . . . 120
5.40.2 Member Function Documentation . . . 120
5.40.2.1 getRandomUnit() . . . 120
5.40.2.2 getRevengeDialog() . . . 121
5.40.2.3 reset() [1/2] . . . 121
5.40.2.4 reset() [2/2] . . . 122
5.41 RevengeAlertDialogBuilderAdapter Class Reference . . . 122
5.41.1 Detailed Description . . . 124
5.41.2 Constructor & Destructor Documentation . . . 124
5.41.2.1 RevengeAlertDialogBuilderAdapter() . . . 124
5.41.3 Member Function Documentation . . . 124
5.41.3.1 create() . . . 124
5.41.3.2 insertAvengers() . . . 124
5.41.3.3 setNegativeCallback() . . . 125
5.41.3.4 setPositiveCallback() . . . 125
5.41.4 Member Data Documentation . . . 125
5.41.4.1 adapteeBuilder . . . 125
5.41.4.2 AVENGER_ABILITY . . . 126
5.41.4.3 AVENGER_DAMAGE . . . 126
5.41.4.4 AVENGER_EPIC . . . 126
5.41.4.5 AVENGER_ROW . . . 126
5.41.4.6 AVENGER_SQUAD . . . 126
5.42 RowDao Interface Reference . . . 127
5.42.1 Detailed Description . . . 127

5.42.2 Member Function Documentation . . . 127
5.42.2.1 clearRows() . . . 127
5.42.2.2 clearWeather() . . . 128
5.42.2.3 insertRow() . . . 128
5.42.2.4 isHorn() . . . 128
5.42.2.5 isHornFlowable() . . . 128
5.42.2.6 isWeather() . . . 129
5.42.2.7 isWeatherFlowable() . . . 129
5.42.2.8 updateHorn() . . . 130
5.42.2.9 updateWeather() . . . 130
5.43 RowEntity Class Reference . . . 130
5.43.1 Detailed Description . . . 131
5.43.2 Constructor & Destructor Documentation . . . 131
5.43.2.1 RowEntity() . . . 131
5.43.3 Member Function Documentation . . . 131
5.43.3.1 getId() . . . 131
5.43.3.2 isHorn() . . . 132
5.43.3.3 isWeather() . . . 132
5.43.3.4 setHorn() . . . 132
5.43.3.5 setWeather() . . . 132
5.43.4 Member Data Documentation . . . 133
5.43.4.1 horn . . . 133
5.43.4.2 id . . . 133
5.43.4.3 weather . . . 133
5.44 RowType Enum Reference . . . 133
5.44.1 Detailed Description . . . 133
5.44.2 Member Data Documentation . . . 134
5.44.2.1 MELEE . . . 134
5.44.2.2 RANGE . . . 134
5.44.2.3 SIEGE . . . 134
5.45 RowUiState Class Reference . . . 134
5.45.1 Detailed Description . . . 135
5.45.2 Constructor & Destructor Documentation . . . 135
5.45.2.1 RowUiState() . . . 135
5.45.3 Member Function Documentation . . . 135
5.45.3.1 equals() . . . 135
5.45.3.2 getDamage() . . . 136
5.45.3.3 getUnits() . . . 136
5.45.3.4 isHorn() . . . 136
5.45.3.5 isWeather() . . . 136
5.45.4 Member Data Documentation . . . 136
5.45.4.1 damage . . . 136

5.45.4.2 horn . . . 137
5.45.4.3 units . . . 137
5.45.4.4 weather . . . 137
5.46 RowUiStateObserver Class Reference . . . 137
5.46.1 Detailed Description . . . 138
5.46.2 Constructor & Destructor Documentation . . . 139
5.46.2.1 RowUiStateObserver() . . . 139
5.46.3 Member Function Documentation . . . 139
5.46.3.1 accept() . . . 139
5.46.3.2 getObserver() . . . 139
5.46.4 Member Data Documentation . . . 140
5.46.4.1 damageView . . . 140
5.46.4.2 hornView . . . 140
5.46.4.3 unitView . . . 140
5.46.4.4 weatherResource . . . 140
5.46.4.5 weatherView . . . 141
5.47 RuleActivity Class Reference . . . 141
5.47.1 Detailed Description . . . 142
5.47.2 Member Function Documentation . . . 142
5.47.2.1 onCreate() . . . 142
5.47.2.2 onOptionsItemSelected() . . . 142
5.47.3 Member Data Documentation . . . 143
5.47.3.1 INTENT_EXTRA_TAG . . . 143
5.48 RuleSection Enum Reference . . . 143
5.48.1 Detailed Description . . . 143
5.48.2 Member Data Documentation . . . 143
5.48.2.1 CARD_ABILITIES . . . 143
5.48.2.2 CARDS . . . 144
5.48.2.3 COMMANDER . . . 144
5.48.2.4 COURSE . . . 144
5.48.2.5 FACTIONS . . . 144
5.48.2.6 GENERAL . . . 144
5.48.2.7 SPECIAL_CARDS . . . 144
5.49 ShowUnitsDialog.ScrollToEndObserver Class Reference . . . 144
5.49.1 Detailed Description . . . 145
5.49.2 Member Function Documentation . . . 145
5.49.2.1 onItemRangeInserted() . . . 145
5.50 SectionsPagerAdapter Class Reference . . . 145
5.50.1 Detailed Description . . . 146
5.50.2 Constructor & Destructor Documentation . . . 146
5.50.2.1 SectionsPagerAdapter() . . . 146
5.50.3 Member Function Documentation . . . 147

5.50.3.1 createFragment() . . . 147
5.50.3.2 getItemCount() . . . 147
5.51 SettingsActivity Class Reference . . . 147
5.51.1 Detailed Description . . . 149
5.51.2 Member Function Documentation . . . 149
5.51.2.1 onCreate() . . . 149
5.51.2.2 onOptionsItemSelected() . . . 149
5.51.2.3 onPreferenceStartFragment() . . . 149
5.51.2.4 onSaveInstanceState() . . . 150
5.51.3 Member Data Documentation . . . 150
5.51.3.1 CURRENT_FRAGMENT_KEY . . . 150
5.52 SettingsHeaderFragment Class Reference . . . 150
5.52.1 Detailed Description . . . 151
5.52.2 Member Function Documentation . . . 151
5.52.2.1 onCreatePreferences() . . . 151
5.53 SettingsRuleFragment Class Reference . . . 151
5.53.1 Detailed Description . . . 152
5.53.2 Member Function Documentation . . . 152
5.53.2.1 getSectionClickListener() . . . 152
5.53.2.2 onCreatePreferences() . . . 153
5.54 SettingsSoundFragment Class Reference . . . 153
5.54.1 Detailed Description . . . 154
5.54.2 Member Function Documentation . . . 154
5.54.2.1 onCreatePreferences() . . . 154
5.55 ShowUnitsDialog Class Reference . . . 155
5.55.1 Detailed Description . . . 156
5.55.2 Constructor & Destructor Documentation . . . 157
5.55.2.1 ShowUnitsDialog() . . . 157
5.55.3 Member Function Documentation . . . 157
5.55.3.1 getDialog() . . . 157
5.55.3.2 getRow() . . . 157
5.55.3.3 onCreate() . . . 158
5.55.4 Member Data Documentation . . . 158
5.55.4.1 cardListAdapter . . . 158
5.55.4.2 disposables . . . 158
5.55.4.3 row . . . 158
5.55.4.4 soundManager . . . 159
5.55.4.5 unitRecyclerView . . . 159
5.56 Sound Class Reference . . . 159
5.56.1 Detailed Description . . . 160
5.56.2 Constructor & Destructor Documentation . . . 160
5.56.2.1 Sound() . . . 160

5.56.3 Member Function Documentation . . . 160
5.56.3.1 createSound() . . . 160
5.56.3.2 getPreferenceKey() . . . 161
5.56.3.3 getSoundId() . . . 161
5.56.3.4 isActivated() . . . 161
5.56.3.5 setActivated() [1/2] . . . 161
5.56.3.6 setActivated() [2/2] . . . 162
5.56.4 Member Data Documentation . . . 162
5.56.4.1 activated . . . 162
5.56.4.2 preferenceKey . . . 162
5.56.4.3 soundId . . . 162
5.57 SoundManager Class Reference . . . 162
5.57.1 Detailed Description . . . 164
5.57.2 Constructor & Destructor Documentation . . . 164
5.57.2.1 SoundManager() . . . 164
5.57.3 Member Function Documentation . . . 165
5.57.3.1 playBurnSound() . . . 165
5.57.3.2 playCardAddSound() . . . 165
5.57.3.3 playCardRemovedSound() . . . 165
5.57.3.4 playClearWeatherSound() . . . 165
5.57.3.5 playCoinSound() . . . 166
5.57.3.6 playHornSound() . . . 166
5.57.3.7 playResetSound() . . . 166
5.57.3.8 playSound() . . . 166
5.57.3.9 playWeatherSound() . . . 167
5.57.4 Member Data Documentation . . . 167
5.57.4.1 changeListener . . . 167
5.57.4.2 SOUND_BURN . . . 167
5.57.4.3 SOUND_CARDS_EPIC . . . 167
5.57.4.4 SOUND_CARDS_MELEE . . . 168
5.57.4.5 SOUND_CARDS_RANGE . . . 168
5.57.4.6 SOUND_CARDS_SIEGE . . . 168
5.57.4.7 SOUND_COIN . . . 168
5.57.4.8 SOUND_HORN . . . 168
5.57.4.9 SOUND_RESET . . . 169
5.57.4.10 SOUND_WEATHER_FOG . . . 169
5.57.4.11 SOUND_WEATHER_FROST . . . 169
5.57.4.12 SOUND_WEATHER_GOOD . . . 169
5.57.4.13 SOUND_WEATHER_RAIN . . . 169
5.57.4.14 soundPool . . . 170
5.57.4.15 sounds . . . 170
5.58 SquadManager Class Reference . . . 170

5.58.1 Detailed Description . . . 171
5.58.2 Constructor & Destructor Documentation . . . 171
5.58.2.1 SquadManager() . . . 171
5.58.3 Member Function Documentation . . . 171
5.58.3.1 getFirstSquadWithMembers() . . . 171
5.58.3.2 onSquadChanged() . . . 171
5.58.4 Member Data Documentation . . . 172
5.58.4.1 MAX_NR_SQUADS . . . 172
5.58.4.2 states . . . 172
5.59 SquadState Class Reference . . . 172
5.59.1 Detailed Description . . . 173
5.59.2 Constructor & Destructor Documentation . . . 173
5.59.2.1 SquadState() . . . 173
5.59.3 Member Function Documentation . . . 174
5.59.3.1 getMemberBaseDamage() . . . 174
5.59.3.2 getSquadMembers() . . . 174
5.59.3.3 getSquadNumber() . . . 174
5.59.3.4 getState() . . . 175
5.59.3.5 hasMembers() . . . 175
5.59.4 Member Data Documentation . . . 175
5.59.4.1 memberBaseDamage . . . 175
5.59.4.2 squadMembers . . . 175
5.59.4.3 squadNumber . . . 176
5.60 StringValuePicker< T extends Comparable > Class Template Reference . . . 176
5.60.1 Detailed Description . . . 178
5.60.2 Constructor & Destructor Documentation . . . 178
5.60.2.1 StringValuePicker() [1/2] . . . 178
5.60.2.2 StringValuePicker() [2/2] . . . 179
5.60.3 Member Function Documentation . . . 179
5.60.3.1 getDisplayString() . . . 179
5.61 ResetDialogUseCase.Trigger Enum Reference . . . 180
5.61.1 Detailed Description . . . 180
5.61.2 Member Data Documentation . . . 180
5.61.2.1 BUTTON_CLICK . . . 180
5.61.2.2 FACTION_SWITCH . . . 180
5.62 UnitDao Interface Reference . . . 180
5.62.1 Detailed Description . . . 181
5.62.2 Member Function Documentation . . . 181
5.62.2.1 countUnits() [1/2] . . . 181
5.62.2.2 countUnits() [2/2] . . . 182
5.62.2.3 deleteUnit() . . . 182
5.62.2.4 deleteUnits() . . . 182

5.62.2.5 getUnit() . . . 182
5.62.2.6 getUnits() [1/2] . . . 183
5.62.2.7 getUnits() [2/2] . . . 183
5.62.2.8 getUnitsFlowable() . . . 183
5.62.2.9 hasNonEpicUnitsFlowable() . . . 184
5.62.2.10 insertUnit() [1/2] . . . 184
5.62.2.11 insertUnit() [2/2] . . . 184
5.63 UnitEntity Class Reference . . . 185
5.63.1 Detailed Description . . . 186
5.63.2 Constructor & Destructor Documentation . . . 186
5.63.2.1 UnitEntity() . . . 186
5.63.3 Member Function Documentation . . . 187
5.63.3.1 calculateDamage() . . . 187
5.63.3.2 collectionToString() . . . 187
5.63.3.3 getAbility() . . . 188
5.63.3.4 getDamage() . . . 188
5.63.3.5 getId() . . . 189
5.63.3.6 getRow() . . . 189
5.63.3.7 getSquad() . . . 189
5.63.3.8 isBuffed() . . . 189
5.63.3.9 isEpic() . . . 190
5.63.3.10 setAbility() . . . 190
5.63.3.11 setDamage() . . . 190
5.63.3.12 setEpic() . . . 190
5.63.3.13 setId() . . . 191
5.63.3.14 setRow() . . . 191
5.63.3.15 setSquad() . . . 191
5.63.3.16 toString() . . . 191
5.63.4 Member Data Documentation . . . 192
5.63.4.1 ability . . . 192
5.63.4.2 damage . . . 192
5.63.4.3 epic . . . 192
5.63.4.4 EPIC_DAMAGE_VALUES . . . 192
5.63.4.5 id . . . 192
5.63.4.6 NON_EPIC_DAMAGE_VALUES_UPPER_BOUND . . . 192
5.63.4.7 row . . . 193
5.63.4.8 squad . . . 193
5.64 UnitRepository Class Reference . . . 193
5.64.1 Detailed Description . . . 195
5.64.2 Constructor & Destructor Documentation . . . 195
5.64.2.1 UnitRepository() . . . 195
5.64.3 Member Function Documentation . . . 195

5.64.3.1 clearWeather() . . . 195
5.64.3.2 copy() . . . 195
5.64.3.3 countUnits() [1/2] . . . 196
5.64.3.4 countUnits() [2/2] . . . 196
5.64.3.5 delete() [1/2] . . . 196
5.64.3.6 delete() [2/2] . . . 197
5.64.3.7 getRepository() . . . 197
5.64.3.8 getUnit() . . . 197
5.64.3.9 getUnits() [1/2] . . . 198
5.64.3.10 getUnits() [2/2] . . . 198
5.64.3.11 getUnitsFlowable() . . . 198
5.64.3.12 hasNonEpicUnitsFlowable() . . . 199
5.64.3.13 initializeRows() . . . 199
5.64.3.14 insertUnit() [1/2] . . . 199
5.64.3.15 insertUnit() [2/2] . . . 200
5.64.3.16 isHorn() . . . 200
5.64.3.17 isHornFlowable() . . . 201
5.64.3.18 isWeather() . . . 201
5.64.3.19 isWeatherFlowable() . . . 201
5.64.3.20 reset() [1/2] . . . 202
5.64.3.21 reset() [2/2] . . . 202
5.64.3.22 switchHorn() . . . 202
5.64.3.23 switchWeather() . . . 203
5.64.4 Member Data Documentation . . . 203
5.64.4.1 database . . . 203
5.65 ValuePicker< T extends Comparable > Class Template Reference . . . 203
5.65.1 Detailed Description . . . 205
5.65.2 Constructor & Destructor Documentation . . . 205
5.65.2.1 ValuePicker() [1/2] . . . 205
5.65.2.2 ValuePicker() [2/2] . . . 205
5.65.3 Member Function Documentation . . . 206
5.65.3.1 getContext() . . . 206
5.65.3.2 getDisplayIntegers() . . . 206
5.65.3.3 getDisplayString() . . . 206
5.65.3.4 getPicker() . . . 207
5.65.3.5 getSelectableValues() . . . 207
5.65.3.6 getValue() . . . 207
5.65.3.7 setOnValueChangedListener() . . . 207
5.65.3.8 setSelectableValues() [1/2] . . . 208
5.65.3.9 setSelectableValues() [2/2] . . . 208
5.65.3.10 setValue() . . . 209
5.65.4 Member Data Documentation . . . 209

5.65.4.1 displayIntegers . . . 209
5.65.4.2 picker . . . 209
5.65.4.3 selectableValues . . . 210
5.66 WeatherDamageCalculator Class Reference . . . 210
5.66.1 Detailed Description . . . 211
5.66.2 Constructor & Destructor Documentation . . . 211
5.66.2.1 WeatherDamageCalculator() . . . 211
5.66.3 Member Function Documentation . . . 212
5.66.3.1 calculateDamage() . . . 212
5.66.3.2 isBuffed() . . . 212
5.66.4 Member Data Documentation . . . 213
5.66.4.1 weather . . . 213

6 File Documentation **215**

6.1 Ability.java . . . 215
6.2 AppDatabase.java . . . 215
6.3 RowDao.java . . . 215
6.4 RowEntity.java . . . 216
6.5 RowType.java . . . 216
6.6 UnitDao.java . . . 217
6.7 UnitEntity.java . . . 217
6.8 UnitRepository.java . . . 220
6.9 BurnDialogUseCase.java . . . 223
6.10 DamageCalculatorUseCase.java . . . 224
6.11 RemoveUnitsUseCase.java . . . 224
6.12 ResetAlertDialogBuilderAdapter.java . . . 225
6.13 ResetDialogUseCase.java . . . 226
6.14 ResetRepositoryUseCase.java . . . 227
6.15 RevengeAlertDialogBuilderAdapter.java . . . 228
6.16 BondDamageCalculatorDecorator.java . . . 229
6.17 DamageCalculator.java . . . 230
6.18 DamageCalculatorBuildDirector.java . . . 230
6.19 DamageCalculatorBuilder.java . . . 231
6.20 DamageCalculatorDecorator.java . . . 231
6.21 HornDamageCalculatorDecorator.java . . . 232
6.22 MoralDamageCalculatorDecorator.java . . . 232
6.23 WeatherDamageCalculator.java . . . 233
6.24 GwentApplication.java . . . 233
6.25 AddCardDialog.java . . . 233
6.26 CardNumberPickerAdapter.java . . . 234
6.27 DamageValuePicker.java . . . 236
6.28 SquadManager.java . . . 237

6.29 SquadState.java . . . 238
6.30 StringValuePicker.java . . . 239
6.31 ValuePicker.java . . . 239
6.32 CardListAdapter.java . . . 241
6.33 CardUiState.java . . . 242
6.34 CardUiStateFactory.java . . . 244
6.35 ShowUnitsDialog.java . . . 245
6.36 ChangeFactionDialog.java . . . 247
6.37 CoinFlipDialog.java . . . 248
6.38 OverlayDialog.java . . . 248
6.39 IndicatorManager.java . . . 249
6.40 IntroductionActivity.java . . . 249
6.41 IntroductionFragment.java . . . 250
6.42 SectionsPagerAdapter.java . . . 251
6.43 FactionSwitchListener.java . . . 251
6.44 GameBoardViewModel.java . . . 253
6.45 ImageViewSwitchAnimator.java . . . 256
6.46 MainActivity.java . . . 256
6.47 MenuUiState.java . . . 259
6.48 MenuUiStateObserver.java . . . 260
6.49 RowUiState.java . . . 260
6.50 RowUiStateObserver.java . . . 261
6.51 RuleActivity.java . . . 262
6.52 RuleSection.java . . . 263
6.53 SettingsActivity.java . . . 263
6.54 SettingsHeaderFragment.java . . . 264
6.55 SettingsRuleFragment.java . . . 265
6.56 SettingsSoundFragment.java . . . 266
6.57 Sound.java . . . 266
6.58 SoundManager.java . . . 267

Index **271**

Chapter 1

Dandelion's Calculator

1. Overview
2. Printing analog cards
3. Documentation

1.1 Overview

Dandelion's Calculator is an Android app (available in Play Store) that enables the user to play the Gwent card game from The Witcher 3: Wild Hunt with analog cards. This is accomplished by providing an easy-to-use interface which allows adding cards with certain attributes that correspond to the cards played analogously. The app then calculates the power score of the player including weather effects and card abilities. To see what this looks like, please visit the Play Store entry!

1.2 Printing analog cards

To actually play Gwent using *Dandelion's Calculator*, you need to print out analog Gwent cards. The card images can be downloaded from the following links:

Language	zip	tar
English	gwent_cards_en.zip	gwent_cards_en.tar.gz
German	gwent_cards_de.zip	gwent_cards_de.tar.gz

Please be aware that the Skellige faction is only available in the german packages! This PDF summarizes in which quantity you need to print which card.
When you installed the app, printed out your deck and found a friend, you are ready to go and can start playing the original Gwent game from The Witcher 3: Wild Hunt.

1.3 Documentation

If you are a developer and interested in improving the app by adding new features or fixing bugs, you are always welcome to open pull requests. An extensive reference specification of the whole app can be found on this website or in this PDF-file.

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

- Ability . . . 11
- AdapterDataObserver
  - ShowUnitsDialog.ScrollToEndObserver . . . 144
- AndroidViewModel
  - GameBoardViewModel . . . 71
- AppCompatActivity
  - IntroductionActivity . . . 89
  - MainActivity . . . 95
  - RuleActivity . . . 141
  - SettingsActivity . . . 147
- Application
  - GwentApplication . . . 79
- BurnDialogUseCase . . . 20
- ResetAlertDialogBuilderAdapter.Callback . . . 23
- ChangeFactionDialog.Callback . . . 24
- CardNumberPickerAdapter . . . 28
- CardUiState . . . 32
- CardUiStateFactory . . . 38
- DamageCalculator.Color . . . 50
- Consumer
  - MenuUiStateObserver . . . 102
  - RowUiStateObserver . . . 137
- DamageCalculator . . . 51
  - DamageCalculatorDecorator . . . 58
    - BondDamageCalculatorDecorator . . . 17
    - HornDamageCalculatorDecorator . . . 82
    - MoralDamageCalculatorDecorator . . . 104
  - WeatherDamageCalculator . . . 210
- DamageCalculatorBuildDirector . . . 53
- DamageCalculatorBuilder . . . 55
- DamageCalculatorUseCase . . . 59
- Dialog
  - OverlayDialog . . . 109
    - ChangeFactionDialog . . . 44
    - CoinFlipDialog . . . 47
    - AddCardDialog . . . 12
    - ShowUnitsDialog . . . 155
- ResetDialogUseCase.DialogType . . . 66
- Fragment
  - IntroductionFragment . . . 92

FragmentStateAdapter
SectionsPagerAdapter . . . 145
ImageViewSwitchAnimator . . . 86
IndicatorManager . . . 87
ListAdapter
CardListAdapter . . . 25
MenuUiState . . . 99
OnPreferenceStartFragmentCallback
SettingsActivity . . . 147
OnSharedPreferenceChangeListener
FactionSwitchListener . . . 67
ValuePicker< T extends Comparable >.OnValueChangeListener< T extends Comparable > . . . 108
PreferenceFragmentCompat
SettingsHeaderFragment . . . 150
SettingsRuleFragment . . . 151
SettingsSoundFragment . . . 153
RemoveUnitsUseCase . . . 112
ResetAlertDialogBuilderAdapter . . . 114
ResetDialogUseCase . . . 117
ResetRepositoryUseCase . . . 120
RevengeAlertDialogBuilderAdapter . . . 122
RoomDatabase
AppDatabase . . . 16
RowDao . . . 127
RowEntity . . . 130
RowType . . . 133
RowUiState . . . 134
RuleSection . . . 143
Sound . . . 159
SoundManager . . . 162
SquadManager . . . 170
SquadState . . . 172
ResetDialogUseCase.Trigger . . . 180
UnitDao . . . 180
UnitEntity . . . 185
UnitRepository . . . 193
ValuePicker< T extends Comparable > . . . 203
ValuePicker< Integer > . . . 203
DamageValuePicker . . . 61
ValuePicker< T > . . . 203
StringValuePicker< T extends Comparable > . . . 176
ViewHolder
CardListAdapter.CardViewHolder . . . 40

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Ability
: An enum listing the different card abilities possible . . . 11

AddCardDialog
: An OverlayDialog used to add new UnitEntity objects to a certain .row . . . 12

AppDatabase
: An abstract class extending RoomDatabase and representing the SQLite database of the application . . . 16

BondDamageCalculatorDecorator
: A DamageCalculator class responsible for calculating the tight bond buff if necessary, i.e., if there are units with the com.peternaggschga.gwent.data.Ability.BINDING ability . . . 17

BurnDialogUseCase
: A use case class responsible for dispatching a remove call to RemoveUnitsUseCase . . . 20

ResetAlertDialogBuilderAdapter.Callback
: An interface defining functions to propagate the user's decision back to the creator . . . 23

ChangeFactionDialog.Callback
: An interface used for propagating a selected theme back to the creator of a ChangeFactionDialog 24

CardListAdapter
: A ListAdapter used to provide CardListAdapter.CardViewHolder objects created from CardUiState objects to a RecyclerView . . . 25

CardNumberPickerAdapter
: A helper class responsible for initializing the NumberPicker views of an AddCardDialog (in .CardNumperPickerAdapter()) and adding UnitEntity objects with the selected attributes when .addSelectedUnits() is called . . . 28

CardUiState
: A data class encapsulating the visible state of a card in the card list shown by the ShowUnitsDialog . . . 32

CardUiStateFactory
: A factory class responsible for creating CardUiState objects from UnitEntity objects . . . 38

CardListAdapter.CardViewHolder
: A RecyclerView.ViewHolder class managing a card view that shows a representation of the UnitEntity referenced by .itemId . . . 40

ChangeFactionDialog
: An OverlayDialog class used to change the faction design . . . 44

CoinFlipDialog
: An OverlayDialog class showing the result of a coin-flip . . . 47

DamageCalculator.Color
: An enum containing values representing whether a unit is buffed, de-buffed or not affected by status effects . . . 50

DamageCalculator
An interface for a class capable of calculating the (de-)buffed damage of a unit for a given UnitEntity.id and UnitEntity.damage . . . 51
DamageCalculatorBuildDirector
A build director responsible for the creation of DamageCalculator instances from sets of units using DamageCalculatorBuilder . . . 53
DamageCalculatorBuilder
A builder class responsible for creating a decorator structure of DamageCalculator classes with the given parameters . . . 55
DamageCalculatorDecorator
An abstract DamageCalculator that is used as a superclass for the decorator classes implementing the damage calculation . . . 58
DamageCalculatorUseCase
A use case class responsible for creating DamageCalculator objects for rows using DamageCalculatorBuildDirector . . . 59
DamageValuePicker
A ValuePicker used for selection of damage values for UnitEntity.damage . . . 61
ResetDialogUseCase.DialogType
An Enum defining which form of Dialog should be shown . . . 66
FactionSwitchListener
A class implementing SharedPreference.OnSharedPreferenceChangeListener for the .THEME_PREFERENCE_KEY SharedPreference . . . 67
GameBoardViewModel
An AndroidViewModel class responsible for encapsulating and offering state of views in activity↩ _main.xml, i.e., that show the overall game board . . . 71
GwentApplication
An Application encapsulating the Gwent app . . . 79
HornDamageCalculatorDecorator
A DamageCalculator class responsible for calculating the horn buff if necessary, i.e., if there are units with the com.peternaggschga.gwent.data.Ability.HORN ability that are not the unit the damage is calculated for or if there is a commander's horn in this row . . . 82
ImageViewSwitchAnimator
A class providing functionality for changing the resource shown by an ImageView using a fading animation . . . 86
IndicatorManager
A class managing the indicator views shown in the bottom bar of the IntroductionActivity . . . 87
IntroductionActivity
An AppCompatActivity that gives the user an introduction into the usage of the application . . . 89
IntroductionFragment
A Fragment containing an ImageView and a TextView conveying introductory information . . . 92
MainActivity
An AppCompatActivity that is called on startup and that encapsulates the main view onto the game board . . . 95
MenuUiState
A data class encapsulating the visible state of the menu in the main view of the application . . . 99
MenuUiStateObserver
An observer class responsible for updating the menu views when notified, i.e., when a new MenuUiState is produced by the subscribed Flowable . . . 102
MoralDamageCalculatorDecorator
A DamageCalculator class responsible for calculating the moral buff if necessary, i.e., if there are units with the com.peternaggschga.gwent.data.Ability.MORAL_BOOST ability that are not the unit the damage is calculated for . . . 104
ValuePicker< T extends Comparable >.OnValueChangeListener< T extends Comparable >
An interface defining a callback for changes of the current value . . . 108
OverlayDialog
A Dialog class which is used for popups that are shown on top of the calling Activity . . . 109
RemoveUnitsUseCase
A use case class responsible for removing units from a UnitRepository . . . 112

ResetAlertDialogBuilderAdapter
An adapter class adapting AlertDialog.Builder to provide an interface for creating an AlertDialog asking the user whether he really wants to reset . . . 114
ResetDialogUseCase
A use case class responsible for dispatching a reset call to ResetRepositoryUseCase, possibly after a confirmation by the user obtained from a Dialog . . . 117
ResetRepositoryUseCase
A use case class responsible for resetting the UnitRepository . . . 120
RevengeAlertDialogBuilderAdapter
An adapter class adapting AlertDialog.Builder to provide an interface for creating an AlertDialog asking the user whether they want to invoke the Ability.REVENGE ability . . . 122
RowDao
An interface defining data access operations related to `rows` table . . . 127
RowEntity
A class representing the state of an attack row, i.e., weather and commanders horn . . . 130
RowType
An enum listing the different types of attack rows . . . 133
RowUiState
A data class encapsulating the visible state of a row in the main view of the application . . . 134
RowUiStateObserver
An observer class responsible for updating the views of the row defined in .row when notified, i.e., when a new RowUiState is produced the observed Flowable . . . 137
RuleActivity
An AppCompatActivity used to present the rules of the game . . . 141
RuleSection
An Enum used to discern the different possible sections shown by RuleActivity . . . 143
ShowUnitsDialog.ScrollToEndObserver
Smoothly scrolls to the last item in the RecyclerView to show the user that an item has been inserted . . . 144
SectionsPagerAdapter
A FragmentStateAdapter that returns an IntroductionFragment corresponding to one of the sections/tabs/pages . . . 145
SettingsActivity
An AppCompatActivity implementing PreferenceFragmentCompat.OnPreferenceStart↩ FragmentCallback that is used by the user to manage the SharedPreferences of the application 147
SettingsHeaderFragment
A PreferenceFragmentCompat class encapsulating the main preference screen, i.e., the Preferences defined in R.xmlheader_preferences . . . 150
SettingsRuleFragment
A PreferenceFragmentCompat class encapsulating the rule preference screen, i.e., the rule sections defined in R.xmlrule_preferences . . . 151
SettingsSoundFragment
A PreferenceFragmentCompat class encapsulating the sound preference screen, i.e., the Preferences defined in R.xmlsound_preferences . . . 153
ShowUnitsDialog
An OverlayDialog used to list the units of a certain row and enabling the user to copy, add, or delete new units . . . 155
Sound
A data class encapsulating information on a certain sound effect, e.g., its id in a SoundPool and whether it is activated in settings . . . 159
SoundManager
A class responsible for initializing and playing sounds when they are enabled . . . 162
SquadManager
A class used by the CardNumberPickerAdapter to determine the first squad with members or set the default damage according to the squads base damage . . . 170
SquadState
A data class encapsulating information about the squad defined by .squadNumber, i.e., how many .squadMembers are there and what is the .memberBaseDamage of units in this squad . 172

StringValuePicker< T extends Comparable >
A ValuePicker displaying a resource String for each element in selectableValues 176
ResetDialogUseCase.Trigger
An Enum listing the possible triggers of a reset 180
UnitDao
An interface defining data access operations related to `units` table 180
UnitEntity
A class representing a card on the game board 185
UnitRepository
A facade class managing public access to the data layer 193
ValuePicker< T extends Comparable >
A generic abstract wrapper class around NumberPicker used to select values of arbitrary types extending Comparable 203
WeatherDamageCalculator
A DamageCalculator class responsible for calculating the weather debuff if necessary, i.e., if .weather is true 210

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

gwent/GwentApplication.java . . . 233
gwent/data/Ability.java . . . 215
gwent/data/AppDatabase.java . . . 215
gwent/data/RowDao.java . . . 215
gwent/data/RowEntity.java . . . 216
gwent/data/RowType.java . . . 216
gwent/data/UnitDao.java . . . 217
gwent/data/UnitEntity.java . . . 217
gwent/data/UnitRepository.java . . . 220
gwent/domain/cases/BurnDialogUseCase.java . . . 223
gwent/domain/cases/DamageCalculatorUseCase.java . . . 224
gwent/domain/cases/RemoveUnitsUseCase.java . . . 224
gwent/domain/cases/ResetAlertDialogBuilderAdapter.java . . . 225
gwent/domain/cases/ResetDialogUseCase.java . . . 226
gwent/domain/cases/ResetRepositoryUseCase.java . . . 227
gwent/domain/cases/RevengeAlertDialogBuilderAdapter.java . . . 228
gwent/domain/damage/BondDamageCalculatorDecorator.java . . . 229
gwent/domain/damage/DamageCalculator.java . . . 230
gwent/domain/damage/DamageCalculatorBuildDirector.java . . . 230
gwent/domain/damage/DamageCalculatorBuilder.java . . . 231
gwent/domain/damage/DamageCalculatorDecorator.java . . . 231
gwent/domain/damage/HornDamageCalculatorDecorator.java . . . 232
gwent/domain/damage/MoralDamageCalculatorDecorator.java . . . 232
gwent/domain/damage/WeatherDamageCalculator.java . . . 233
gwent/ui/dialogs/ChangeFactionDialog.java . . . 247
gwent/ui/dialogs/CoinFlipDialog.java . . . 248
gwent/ui/dialogs/OverlayDialog.java . . . 248
gwent/ui/dialogs/addcard/AddCardDialog.java . . . 233
gwent/ui/dialogs/addcard/CardNumberPickerAdapter.java . . . 234
gwent/ui/dialogs/addcard/DamageValuePicker.java . . . 236
gwent/ui/dialogs/addcard/SquadManager.java . . . 237
gwent/ui/dialogs/addcard/SquadState.java . . . 238
gwent/ui/dialogs/addcard/StringValuePicker.java . . . 239
gwent/ui/dialogs/addcard/ValuePicker.java . . . 239
gwent/ui/dialogs/cards/CardListAdapter.java . . . 241
gwent/ui/dialogs/cards/CardUiState.java . . . 242
gwent/ui/dialogs/cards/CardUiStateFactory.java . . . 244
gwent/ui/dialogs/cards/ShowUnitsDialog.java . . . 245
gwent/ui/introduction/IndicatorManager.java . . . 249
gwent/ui/introduction/IntroductionActivity.java . . . 249

gwent/ui/introduction/IntroductionFragment.java . . . 250
gwent/ui/introduction/SectionsPagerAdapter.java . . . 251
gwent/ui/main/FactionSwitchListener.java . . . 251
gwent/ui/main/GameBoardViewModel.java . . . 253
gwent/ui/main/ImageViewSwitchAnimator.java . . . 256
gwent/ui/main/MainActivity.java . . . 256
gwent/ui/main/MenuUiState.java . . . 259
gwent/ui/main/MenuUiStateObserver.java . . . 260
gwent/ui/main/RowUiState.java . . . 260
gwent/ui/main/RowUiStateObserver.java . . . 261
gwent/ui/settings/RuleActivity.java . . . 262
gwent/ui/settings/RuleSection.java . . . 263
gwent/ui/settings/SettingsActivity.java . . . 263
gwent/ui/settings/SettingsHeaderFragment.java . . . 264
gwent/ui/settings/SettingsRuleFragment.java . . . 265
gwent/ui/settings/SettingsSoundFragment.java . . . 266
gwent/ui/sounds/Sound.java . . . 266
gwent/ui/sounds/SoundManager.java . . . 267

Chapter 5

Class Documentation

5.1 Ability Enum Reference

An enum listing the different card abilities possible.
Collaboration diagram for Ability:



Public Attributes

- NONE

  Represents a card that has no ability.
- HORN

  Represents a card that has the Commanders Horn ability, e.g.
- REVENGE

  Represents a card that has the Summon Avenger ability.
- BINDING

  Represents a card that has the Tight Bond ability.
- MORAL_BOOST

  Represents a card that has the Morale Boost ability.

5.1.1 Detailed Description

An enum listing the different card abilities possible.
Definition at line 6 of file Ability.java.

5.1.2 Member Data Documentation

5.1.2.1 BINDING

`BINDING`
Represents a card that has the Tight Bond ability.
Definition at line 25 of file Ability.java.
Referenced by CardNumberPickerAdapter.CardNumberPickerAdapter(), SquadManager.SquadManager(), UnitEntity.UnitEntity(), CardNumberPickerAdapter.addSelectedUnits(), DamageCalculatorBuildDirector.getCalculator(), UnitRepository.insertUnit(), UnitEntity.setSquad(), and DamageCalculatorBuildDirector.setSquads().

5.1.2.2 HORN

HORN
Represents a card that has the Commanders Horn ability, e.g.
Dandelion.
Definition at line 15 of file Ability.java.
Referenced by CardNumberPickerAdapter.CardNumberPickerAdapter(), DamageCalculatorBuildDirector.getCalculator(), and DamageCalculatorBuildDirector.setHorns().

5.1.2.3 MORAL_BOOST

MORAL_BOOST
Represents a card that has the Morale Boost ability.
Definition at line 30 of file Ability.java.
Referenced by CardNumberPickerAdapter.CardNumberPickerAdapter(), DamageCalculatorBuildDirector.getCalculator(), and DamageCalculatorBuildDirector.setMoralBoosts().

5.1.2.4 NONE

NONE
Represents a card that has no ability.
Definition at line 10 of file Ability.java.
Referenced by CardNumberPickerAdapter.CardNumberPickerAdapter().

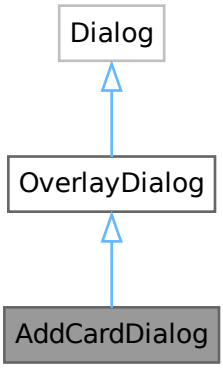
5.1.2.5 REVENGE

REVENGE
Represents a card that has the Summon Avenger ability.
Definition at line 20 of file Ability.java.
Referenced by CardNumberPickerAdapter.CardNumberPickerAdapter(), RemoveUnitsUseCase.remove(), and ResetRepositoryUseCase.reset().
The documentation for this enum was generated from the following file:

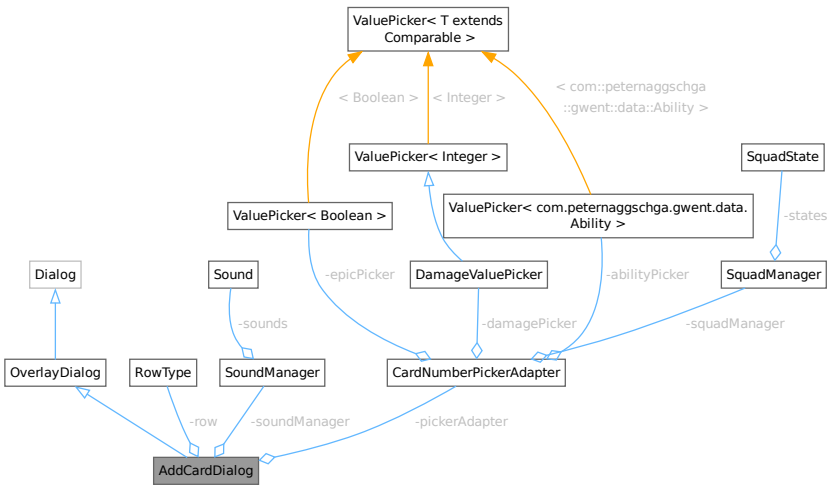
- gwent/data/Ability.java

5.2 AddCardDialog Class Reference

An OverlayDialog used to add new UnitEntity objects to a certain .row.
Inheritance diagram for AddCardDialog:



Collaboration diagram for AddCardDialog:



Public Member Functions

- AddCardDialog (@NonNull ShowUnitsDialog caller, @NonNull SoundManager soundManager)

  Constructor of an AddCardDialog called by the given ShowUnitsDialog using the given SoundManager.
- AddCardDialog (@NonNull Context context, @NonNull RowType row, @NonNull SoundManager soundManager)

  Constructor of an AddCardDialog in the given Context and for the given row.

Protected Member Functions

- void onCreate (Bundle savedInstanceState)

  Initializes the .pickerAdapter if not yet done so and sets View.OnClickListener for each button.

Protected Member Functions inherited from OverlayDialog

- OverlayDialog (@NonNull Context context, @LayoutRes int layout, @IdRes int dismissViewId)

  Constructor of an OverlayDialog in the given Context, with the given layout and dismissViewId.
- OverlayDialog (@NonNull Context context, @LayoutRes int layout)

  Constructor of an OverlayDialog in the given Context and with the given layout.

Private Attributes

- final RowType row

  RowType defining the row the new UnitEntity objects are added to.
- final SoundManager soundManager

  SoundManager used to play a Sound whenever a card is added.
- final CompositeDisposable disposables = new CompositeDisposable()

  CompositeDisposable keeping track of all subscriptions to observables made by this class.
- CardNumberPickerAdapter pickerAdapter = null

  CardNumberPickerAdapter managing the connection to the NumberPicker views of this Dialog.

Additional Inherited Members

Static Package Attributes inherited from OverlayDialog

- static final int NO_DISMISS_VIEW = -1

  Integer used as .dismissViewId, when the Dialog shouldn't be cancelable by clicking on a View.

5.2.1 Detailed Description

An OverlayDialog used to add new UnitEntity objects to a certain .row.
Definition at line 22 of file AddCardDialog.java.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 AddCardDialog() [1/2]

```
AddCardDialog (
            @NonNull ShowUnitsDialog caller,
            @NonNull SoundManager soundManager)
```

Constructor of an AddCardDialog called by the given ShowUnitsDialog using the given SoundManager.
Sets a Dialog.OnDismissListener disposing .disposables and showing the caller again. Wrapper of .AddCard↩Dialog(Context, RowType, SoundManager).

See also

.AddCardDialog(Context, RowType, SoundManager)

Parameters

caller	ShowUnitsDialog that called this Dialog.
soundManager	SoundManager used to play a Sound when cards are added.

Definition at line 67 of file AddCardDialog.java.
References disposables, and soundManager.

5.2.2.2 AddCardDialog() [2/2]

```
AddCardDialog (
            @NonNull Context context,
            @NonNull RowType row,
            @NonNull SoundManager soundManager)
```

Constructor of an AddCardDialog in the given Context and for the given row.
Sets a Dialog.OnDismissListener disposing .disposables and showing the caller again.

Parameters

context	Context this Dialog is shown in.
row	RowType defining which row the new UnitEntity objects are added to.
soundManager	SoundManager used to play a Sound when cards are added.

Definition at line 84 of file AddCardDialog.java.
References disposables, row, and soundManager.

5.2.3 Member Function Documentation

5.2.3.1 onCreate()

```
void onCreate (
```

```
Bundle savedInstanceState)  [protected]
```

Initializes the .pickerAdapter if not yet done so and sets View.OnClickListener for each button.

Parameters

savedInstanceState	If this dialog is being reinitialized after the hosting activity was previously shut down, holds the result from the most recent call to onSaveInstanceState, or null if this is the first time.

Reimplemented from OverlayDialog.
Definition at line 102 of file AddCardDialog.java.
References disposables, GwentApplication.getRepository(), UnitRepository.getUnits(), pickerAdapter, and row.

5.2.4 Member Data Documentation

5.2.4.1 disposables

```
final CompositeDisposable disposables = new CompositeDisposable()  [private]
```

CompositeDisposable keeping track of all subscriptions to observables made by this class.
Is being disposed in an android.content.DialogInterface.OnDismissListener that is set in .AddCardDialog(ShowUnitsDialog) and .AddCardDialog(Context, RowType).

See also

android.content.DialogInterface.OnDismissListener

.AddCardDialog(ShowUnitsDialog, SoundManager)

.AddCardDialog(Context, RowType, SoundManager)

Definition at line 47 of file AddCardDialog.java.
Referenced by AddCardDialog(), AddCardDialog(), and onCreate().

5.2.4.2 pickerAdapter

```
CardNumberPickerAdapter pickerAdapter = null  [private]
```

CardNumberPickerAdapter managing the connection to the NumberPicker views of this Dialog.
Is lazily initialized in .onCreate().

See also

.onCreate(Bundle)

R.id.card_layout

Definition at line 57 of file AddCardDialog.java.
Referenced by onCreate().

5.2.4.3 row

```
final RowType row  [private]
```

RowType defining the row the new UnitEntity objects are added to.
Definition at line 27 of file AddCardDialog.java.
Referenced by AddCardDialog(), and onCreate().

5.2.4.4 soundManager

```
final SoundManager soundManager  [private]
```

SoundManager used to play a Sound whenever a card is added.

See also

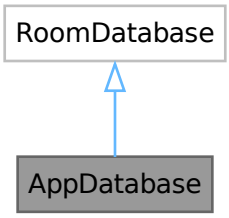
SoundManager.playCardAddSound(RowType, boolean)

Definition at line 35 of file AddCardDialog.java.
Referenced by AddCardDialog(), and AddCardDialog().
The documentation for this class was generated from the following file:

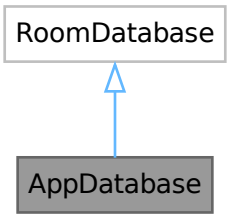
- gwent/ui/dialogs/addcard/AddCardDialog.java

5.3 AppDatabase Class Reference

An abstract class extending RoomDatabase and representing the SQLite database of the application.
Inheritance diagram for AppDatabase:



Collaboration diagram for AppDatabase:



Package Functions

- abstract UnitDao units ()

  Returns an object implementing the operations defined in UnitDao.
- abstract RowDao rows ()

  Returns an object implementing the operations defined in RowDao.

5.3.1 Detailed Description

An abstract class extending RoomDatabase and representing the SQLite database of the application.
The implementation is generated by the Room framework.
Definition at line 11 of file AppDatabase.java.

5.3.2 Member Function Documentation

5.3.2.1 rows()

`abstract RowDao rows () [abstract], [package]`
Returns an object implementing the operations defined in RowDao.

Returns

RowDao implementing DAO for `rows` table.

5.3.2.2 units()

`abstract UnitDao units () [abstract], [package]`
Returns an object implementing the operations defined in UnitDao.

Returns

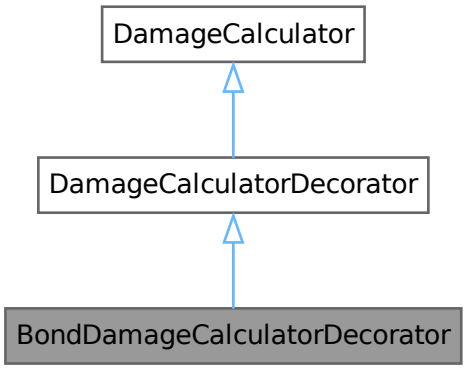
UnitDao implementing DAO for `units` table.

The documentation for this class was generated from the following file:

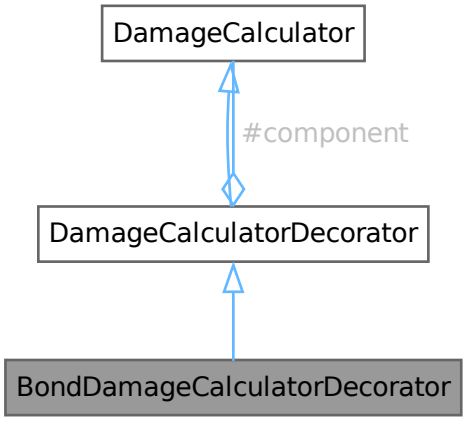
- gwent/data/AppDatabase.java

5.4 BondDamageCalculatorDecorator Class Reference

A DamageCalculator class responsible for calculating the tight bond buff if necessary, i.e., if there are units with the com.peternaggschga.gwent.data.Ability.BINDING ability.
Inheritance diagram for BondDamageCalculatorDecorator:



Collaboration diagram for BondDamageCalculatorDecorator:



Public Member Functions

- int calculateDamage (int id, @IntRange(from=0) int damage)

  Calculates the (de-)buffed damage of unit with the given id and the given base-damage.
- Color isBuffed (int id)

  Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT.

Package Functions

- BondDamageCalculatorDecorator (@NonNull DamageCalculator component, @NonNull Map< Integer, Integer > idToSquadSize)

  Constructor of a BondDamageCalculatorDecorator.

Package Functions inherited from DamageCalculatorDecorator

- DamageCalculatorDecorator (@NonNull DamageCalculator component)

  Constructor of a DamageCalculatorDecorator.

Private Attributes

- final Map< Integer, Integer > idToSquadSize

  A Map mapping the ids of all units with the com.peternaggschga.gwent.data.Ability.BINDING ability to the respective squad size.

Additional Inherited Members

Protected Attributes inherited from DamageCalculatorDecorator

- final DamageCalculator component

  A DamageCalculator which is decorated by this decorator.

5.4.1 Detailed Description

A DamageCalculator class responsible for calculating the tight bond buff if necessary, i.e., if there are units with the com.peternaggschga.gwent.data.Ability.BINDING ability.
Acts as a ConcreteDecorator in the implemented decorator pattern and should decorate a WeatherDamageCalculator for correct calculation.
Definition at line 17 of file BondDamageCalculatorDecorator.java.

5.4.2 Constructor & Destructor Documentation

5.4.2.1 BondDamageCalculatorDecorator()

```
BondDamageCalculatorDecorator (
            @NonNull DamageCalculator component,
            @NonNull Map< Integer, Integer > idToSquadSize)  [package]
```

Constructor of a BondDamageCalculatorDecorator.
Should only be called by DamageCalculatorBuilder. The given damage calculator should be of type WeatherDamageCalculator for correct damage calculation. The values of the given Map must contain non-`null` or positive Integers.

Parameters

component	DamageCalculator that is being decorated by this decorator.
idToSquadSize	Map mapping the ids of all units with the com.peternaggschga.gwent.data.Ability.BINDING ability to the respective squad size.

Exceptions

IllegalArgumentException	When idToSquadSize contains non-positive or null values.

See also

DamageCalculatorBuilder

Definition at line 35 of file BondDamageCalculatorDecorator.java.
References DamageCalculatorDecorator.component, and idToSquadSize.

5.4.3 Member Function Documentation

5.4.3.1 calculateDamage()

```
int calculateDamage (
            int id,
            @IntRange(from=0) int damage)
```

Calculates the (de-)buffed damage of unit with the given id and the given base-damage.
Returns the given damage times the number of units with the same squad if the unit has the com.peternaggschga.gwent.data.Ability.BI
ability.

Parameters

id	Integer representing the UnitEntity.id of the unit whose (de-)buff damage is calculated.
damage	Integer representing the base-damage of the unit whose (de-)buff damage is calculated.

Returns

Integer representing the (de-)buffed damage of the unit.

Exceptions

IllegalArgumentException	When damage is negative.

Implements DamageCalculator.
Definition at line 54 of file BondDamageCalculatorDecorator.java.
References DamageCalculatorDecorator.component, and idToSquadSize.

5.4.3.2 isBuffed()

```
Color isBuffed (
            int id)
```

Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT. Units are shown as Color.BUFFED when they are in a squad of two or more units, otherwise their Color is defined by .component.

Parameters

id	Integer representing the UnitEntity.id of the unit buff status is calculated.

Returns

Color representing whether the unit is buffed, de-buffed or not affected.

See also

Color

Implements DamageCalculator.
Definition at line 72 of file BondDamageCalculatorDecorator.java.
References DamageCalculatorDecorator.component, and idToSquadSize.

5.4.4 Member Data Documentation

5.4.4.1 idToSquadSize

```
final Map<Integer, Integer> idToSquadSize  [private]
```

A Map mapping the ids of all units with the com.peternaggschga.gwent.data.Ability.BINDING ability to the respective squad size.
Definition at line 22 of file BondDamageCalculatorDecorator.java.
Referenced by BondDamageCalculatorDecorator(), calculateDamage(), and isBuffed().
The documentation for this class was generated from the following file:

- gwent/domain/damage/BondDamageCalculatorDecorator.java

5.5 BurnDialogUseCase Class Reference

A use case class responsible for dispatching a remove call to RemoveUnitsUseCase.

Collaboration diagram for BurnDialogUseCase:



Static Public Member Functions

- static Single< Boolean > burn (@NonNull Context context, @NonNull SoundManager soundManager)

  Burns the strongest UnitEntity objects in UnitRepository.

Static Protected Member Functions

- static Single< Boolean > burn (@NonNull Context context, @NonNull UnitRepository repository, @NonNull SoundManager soundManager)

  Burns the strongest UnitEntity objects in UnitRepository.

Static Private Member Functions

- static Single< List< UnitEntity > > getBurnUnits (@NonNull UnitRepository repository)

  Returns the list of units that would be affected by a burn operation.

5.5.1 Detailed Description

A use case class responsible for dispatching a remove call to RemoveUnitsUseCase.
Definition at line 28 of file BurnDialogUseCase.java.

5.5.2 Member Function Documentation

5.5.2.1 burn() [1/2]

```
Single< Boolean > burn (
            @NonNull Context context,
            @NonNull SoundManager soundManager)  [static]
```

Burns the strongest UnitEntity objects in UnitRepository.
Invokes a Dialog asking whether the user really wants to remove those units. ResetRepositoryUseCase is used for resetting. Wrapper for .burn(Context, UnitRepository).

Parameters

context	Context where a Dialog can be inflated.
soundManager	SoundManager used, if an UnitEntity has the Ability.REVENGE ability.

Returns

A Single emitting a Boolean defining whether the units really were burned.

See also

.burn(Context, UnitRepository, SoundManager)

RemoveUnitsUseCase.remove(Context, UnitRepository, Collection, SoundManager)

Definition at line 85 of file BurnDialogUseCase.java.
References burn(), and GwentApplication.getRepository().
Referenced by burn(), and GameBoardViewModel.onBurnButtonPressed().

5.5.2.2 burn() [2/2]

```
Single< Boolean > burn (
            @NonNull Context context,
            @NonNull UnitRepository repository,
            @NonNull SoundManager soundManager)  [static], [protected]
```

Burns the strongest UnitEntity objects in UnitRepository.
Invokes a Dialog asking whether the user really wants to remove those units. ResetRepositoryUseCase is used for resetting.

Parameters

context	Context where a Dialog can be inflated.
repository	UnitRepository where units are burned.
soundManager	SoundManager used, if an UnitEntity has the Ability.REVENGE ability.

Returns

A Single emitting a Boolean defining whether the units really were burned.

See also

RemoveUnitsUseCase.remove(Context, UnitRepository, Collection, SoundManager)

Definition at line 101 of file BurnDialogUseCase.java.
References UnitEntity.collectionToString(), and getBurnUnits().

5.5.2.3 getBurnUnits()

```
Single< List< UnitEntity > > getBurnUnits (
            @NonNull UnitRepository repository)  [static], [private]
```

Returns the list of units that would be affected by a burn operation.
The returned list may be empty.

Parameters

repository	UnitRepository where units are fetched.

Returns

A Single emitting the List of UnitEntity objects that would be affected by the operation.

Definition at line 37 of file BurnDialogUseCase.java.
References DamageCalculatorUseCase.getDamageCalculator().
Referenced by burn().
The documentation for this class was generated from the following file:

- gwent/domain/cases/BurnDialogUseCase.java

5.6 ResetAlertDialogBuilderAdapter.Callback Interface Reference

An interface defining functions to propagate the user's decision back to the creator.
Collaboration diagram for ResetAlertDialogBuilderAdapter.Callback:



Public Member Functions

- void reset (boolean resetDecision, boolean keepUnit)

  Called when the user makes a decision.
- default void reset (boolean resetDecision)

  Wrapper for .reset(boolean, boolean).

5.6.1 Detailed Description

An interface defining functions to propagate the user's decision back to the creator.
Definition at line 99 of file ResetAlertDialogBuilderAdapter.java.

5.6.2 Member Function Documentation

5.6.2.1 reset() [1/2]

```
default void reset (
            boolean resetDecision)
```

Wrapper for .reset(boolean, boolean).
Can be used when the AlertDialog is not a monster dialog since it calls .reset(boolean, boolean) without keeping a unit.

Parameters

resetDecision	Boolean defining whether the user has confirmed the reset.

See also

.reset(boolean, boolean)

Definition at line 118 of file ResetAlertDialogBuilderAdapter.java.
References reset().

5.6.2.2 reset() [2/2]

```
void reset (
            boolean resetDecision,
            boolean keepUnit)
```

Called when the user makes a decision.
When the AlertDialog is a monster dialog, .reset(boolean) can be used.

Parameters

resetDecision	Boolean defining whether the user has confirmed the reset.
keepUnit	Boolean defining whether a random unit should be kept when resetting.

See also

.reset(boolean)

Referenced by reset().
The documentation for this interface was generated from the following file:

- gwent/domain/cases/ResetAlertDialogBuilderAdapter.java

5.7 ChangeFactionDialog.Callback Interface Reference

An interface used for propagating a selected theme back to the creator of a ChangeFactionDialog.
Collaboration diagram for ChangeFactionDialog.Callback:



Public Member Functions

- void onThemeSelect (@IntRange(from=THEME_MONSTER, to=THEME_SCOIATAEL) int theme)

  Callback being called when a theme is selected in the respective ChangeFactionDialog.

5.7.1 Detailed Description

An interface used for propagating a selected theme back to the creator of a ChangeFactionDialog.
Definition at line 84 of file ChangeFactionDialog.java.

5.7.2 Member Function Documentation

5.7.2.1 onThemeSelect()

```
void onThemeSelect (
            @IntRange(from=THEME_MONSTER, to=THEME_SCOIATAEL) int theme)
```

Callback being called when a theme is selected in the respective ChangeFactionDialog.

Parameters

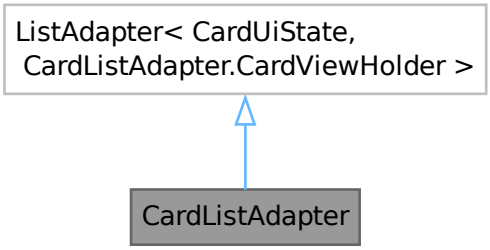
theme	Integer representing the selected theme.

The documentation for this interface was generated from the following file:

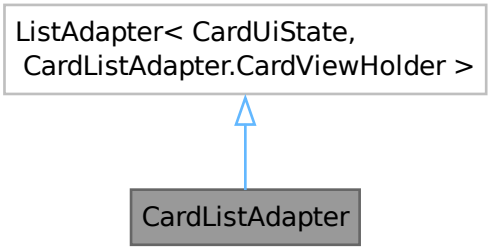
- gwent/ui/dialogs/ChangeFactionDialog.java

5.8 CardListAdapter Class Reference

A ListAdapter used to provide CardListAdapter.CardViewHolder objects created from CardUiState objects to a RecyclerView.
Inheritance diagram for CardListAdapter:



Collaboration diagram for CardListAdapter:



Classes

- class CardViewHolder

  A RecyclerView.ViewHolder class managing a card view that shows a representation of the UnitEntity referenced by .itemId.

Public Member Functions

- CardViewHolder onCreateViewHolder (@NonNull ViewGroup parent, int viewType)

  Creates a new CardViewHolder managing a new card view that is inflated from the given parent.
- void onBindViewHolder (@NonNull CardViewHolder holder, @IntRange(from=0) int position)

  Changes the views of CardViewHolder to represent the item at the given position.
- long getItemId (@IntRange(from=0) int position)

  Returns the UnitEntity.id of the UnitEntity represented at the given position.

Package Functions

- CardListAdapter (@NonNull Consumer< Integer > onCopy, @NonNull Consumer< Integer > onRemove)

Constructor of a CardListAdapter with the given .onCopy and .onRemove callbacks.

Private Attributes

- final Consumer< Integer > onCopy

  Consumer that is used in .onCreateViewHolder(ViewGroup, int) as CardListAdapter.CardViewHolder.onCopy.
- final Consumer< Integer > onRemove

  Consumer that is used in .onCreateViewHolder(ViewGroup, int) as CardListAdapter.CardViewHolder.onRemove.

5.8.1 Detailed Description

A ListAdapter used to provide CardListAdapter.CardViewHolder objects created from CardUiState objects to a RecyclerView.

See also

CardUiState

CardListAdapter.CardViewHolder

ListAdapter

RecyclerView

Definition at line 28 of file CardListAdapter.java.

5.8.2 Constructor & Destructor Documentation

5.8.2.1 CardListAdapter()

```
CardListAdapter (
            @NonNull Consumer< Integer > onCopy,
            @NonNull Consumer< Integer > onRemove)  [package]
```

Constructor of a CardListAdapter with the given .onCopy and .onRemove callbacks.
Calls super-constructor ListAdapter.ListAdapter(DiffUtil.ItemCallback) with CardUiState.DIFF_CALLBACK. Also calls .setHasStableIds() since .getItemId() returns the stable UnitEntity.id.

See also

ListAdapter.ListAdapter(DiffUtil.ItemCallback)

CardUiState.DIFF_CALLBACK

.setHasStableIds(boolean)

Parameters

onCopy	Consumer that is called with the UnitEntity.id, when the copy-button of the representing card is clicked.
onRemove	Consumer that is called with the UnitEntity.id, when the delete-button of the representing card is clicked.

Definition at line 65 of file CardListAdapter.java.
References CardUiState.DIFF_CALLBACK, onCopy, and onRemove.

5.8.3 Member Function Documentation

5.8.3.1 getItemId()

```
long getItemId (
            @IntRange(from=0) int position)
```

Returns the UnitEntity.id of the UnitEntity represented at the given position.

Parameters

position	Adapter position to query.

Returns

A Long referencing the UnitEntity.id of the UnitEntity represented at the given position.

Definition at line 122 of file CardListAdapter.java.

5.8.3.2 onBindViewHolder()

```
void onBindViewHolder (
            @NonNull CardViewHolder holder,
            @IntRange(from=0) int position)
```

Changes the views of CardViewHolder to represent the item at the given position.

Parameters

holder	The CardViewHolder which should be updated to represent the contents of the item at the given position in the data set.
position	The position of the item within the adapter's data set.

Definition at line 93 of file CardListAdapter.java.
References CardUiState.getAbilityImageId(), CardUiState.getDamageBackgroundImageId(), CardUiState.getDamageString(), CardUiState.getDamageTextColor(), CardUiState.getSquadString(), CardUiState.getUnitId(), CardUiState.showAbility(), and CardUiState.showSquad().

5.8.3.3 onCreateViewHolder()

```
CardViewHolder onCreateViewHolder (
            @NonNull ViewGroup parent,
            int viewType)
```

Creates a new CardViewHolder managing a new card view that is inflated from the given parent.

Parameters

parent	The ViewGroup into which the new View will be added after it is bound to an adapter position.
viewType	The view type of the new View.

Returns

A CardViewHolder object for a new card layout and with .onCopy and .onRemove callbacks.

Definition at line 81 of file CardListAdapter.java.
References onCopy, and onRemove.

5.8.4 Member Data Documentation

5.8.4.1 onCopy

```
final Consumer<Integer> onCopy  [private]
```

Consumer that is used in .onCreateViewHolder(ViewGroup, int) as CardListAdapter.CardViewHolder.onCopy.
Is called when the user clicks on the copy-button of an element with the UnitEntity.id of the represented UnitEntity.

See also

.onCreateViewHolder(ViewGroup, int)

CardListAdapter.CardViewHolder.onCopy

Definition at line 39 of file CardListAdapter.java.
Referenced by CardListAdapter(), CardListAdapter.CardViewHolder.CardViewHolder(), and onCreateViewHolder().

5.8.4.2 onRemove

`final Consumer<Integer> onRemove [private]`
Consumer that is used in .onCreateViewHolder(ViewGroup, int) as CardListAdapter.CardViewHolder.onRemove. Is called when the user clicks on the delete-button of an element with the UnitEntity.id of the represented UnitEntity.

See also

.onCreateViewHolder(ViewGroup, int)

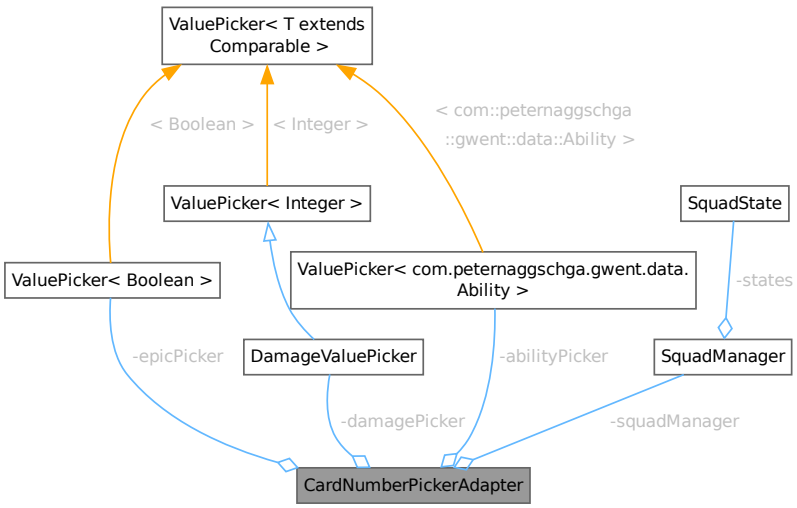
CardListAdapter.CardViewHolder.onRemove

Definition at line 50 of file CardListAdapter.java.
Referenced by CardListAdapter(), CardListAdapter.CardViewHolder.CardViewHolder(), and onCreateViewHolder().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/cards/CardListAdapter.java

5.9 CardNumberPickerAdapter Class Reference

A helper class responsible for initializing the NumberPicker views of an AddCardDialog (in .CardNumperPicker↩ Adapter()) and adding UnitEntity objects with the selected attributes when .addSelectedUnits() is called.
Collaboration diagram for CardNumberPickerAdapter:



Package Functions

- CardNumberPickerAdapter (@NonNull ViewGroup pickerGroup, @NonNull SquadManager squadManager)

  Constructor of a CardNumberPickerAdapter managing the NumberPicker views in the given ViewGroup using the given SquadManager.
- Single< Boolean > addSelectedUnits (@NonNull RowType row)

  Adds new UnitEntity objects the attributes selected by the managed pickers.

Static Package Functions

- static NumberPicker.OnValueChangeListener getDelayedOnValueChangeListener (@NonNull NumberPicker.OnValueChangeListener originalListener)

  Creates a NumberPicker.OnValueChangeListener that only executes the given NumberPicker.OnValueChangeListener after 500 ms if the value has not changed.

Private Attributes

- final ValuePicker< Boolean > epicPicker

  ValuePicker used to decide the value of UnitEntity.epic.
- final DamageValuePicker damagePicker

  DamageValuePicker used to decide the value of UnitEntity.damage.
- final ValuePicker< Ability > abilityPicker

  ValuePicker used to decide the value of UnitEntity.ability.
- final NumberPicker squadPicker

  NumberPicker used to decide the value of UnitEntity.squad.
- final SquadManager squadManager

  SquadManager containing current SquadState used to get squad information.
- final NumberPicker numberPicker

  NumberPicker used to decide the number of UnitEntity objects that are inserted.

Static Private Attributes

- static final List< Ability > EPIC_UNIT_ABILITIES = Arrays.asList(Ability.NONE, Ability.HORN, Ability.MORAL_BOOST)

  List of Ability values that epic units can have.

5.9.1 Detailed Description

A helper class responsible for initializing the NumberPicker views of an AddCardDialog (in .CardNumperPickerAdapter()) and adding UnitEntity objects with the selected attributes when .addSelectedUnits() is called.

See also

AddCardDialog

Definition at line 33 of file CardNumberPickerAdapter.java.

5.9.2 Constructor & Destructor Documentation

5.9.2.1 CardNumberPickerAdapter()

```
CardNumberPickerAdapter (
            @NonNull ViewGroup pickerGroup,
            @NonNull SquadManager squadManager)  [package]
```

Constructor of a CardNumberPickerAdapter managing the NumberPicker views in the given ViewGroup using the given SquadManager.
Sets value bounds and NumberPicker.OnValueChangedListener for the pickers in the ViewGroup. The ViewGroup must be the ConstraintLayout with the id R.id.card_layout from popup_add_card.xml.

See also

R.id.card_layout

Parameters

pickerGroup	ViewGroup containing the managed NumberPicker views.

squadManager	SquadManager containing up-to-date SquadState.

Definition at line 89 of file CardNumberPickerAdapter.java.
References abilityPicker, Ability.BINDING, damagePicker, EPIC_UNIT_ABILITIES, epicPicker, getDelayedOnValueChangeListener(), Ability.HORN, SquadManager.MAX_NR_SQUADS, Ability.MORAL_BOOST, Ability.NONE, numberPicker, Ability.REVENGE, squadManager, and squadPicker.

5.9.3 Member Function Documentation

5.9.3.1 addSelectedUnits()

```
Single< Boolean > addSelectedUnits (
            @NonNull RowType row)  [package]
```

Adds new UnitEntity objects the attributes selected by the managed pickers.

Parameters

row	RowType defining to which row the units are added.

Returns

A Single tracking operation status and containing whether the added unit is epic or not.

Definition at line 166 of file CardNumberPickerAdapter.java.
References abilityPicker, Ability.BINDING, damagePicker, UnitEntity.EPIC_DAMAGE_VALUES, epicPicker, GwentApplication.getRepository(), numberPicker, squadManager, and squadPicker.

5.9.3.2 getDelayedOnValueChangeListener()

```
NumberPicker.OnValueChangeListener getDelayedOnValueChangeListener (
            @NonNull NumberPicker.OnValueChangeListener originalListener)  [static], [package]
```

Creates a NumberPicker.OnValueChangeListener that only executes the given NumberPicker.OnValueChange↩ Listener after 500 ms if the value has not changed.

See also

NumberPicker.OnValueChangeListener

Parameters

originalListener	NumberPicker.OnValueChangeListener that is called when the value does not change.

Returns

A NumberPicker.OnValueChangeListener with delayed execution.

Definition at line 152 of file CardNumberPickerAdapter.java.
References getDelayedOnValueChangeListener().
Referenced by CardNumberPickerAdapter(), getDelayedOnValueChangeListener(), and ValuePicker< T extends Comparable >.setO

5.9.4 Member Data Documentation

5.9.4.1 abilityPicker

```
final ValuePicker<Ability> abilityPicker  [private]
```

ValuePicker used to decide the value of UnitEntity.ability.

See also

.squadPicker

Definition at line 58 of file CardNumberPickerAdapter.java.
Referenced by CardNumberPickerAdapter(), and addSelectedUnits().

5.9.4.2 damagePicker

```
final DamageValuePicker damagePicker  [private]
```

DamageValuePicker used to decide the value of UnitEntity.damage.
Definition at line 50 of file CardNumberPickerAdapter.java.
Referenced by CardNumberPickerAdapter(), and addSelectedUnits().

5.9.4.3 EPIC_UNIT_ABILITIES

```
final List<Ability> EPIC_UNIT_ABILITIES = Arrays.asList(Ability.NONE, Ability.HORN, Ability.MORAL_BOOST)
[static], [private]
```

List of Ability values that epic units can have.
Definition at line 38 of file CardNumberPickerAdapter.java.
Referenced by CardNumberPickerAdapter().

5.9.4.4 epicPicker

```
final ValuePicker<Boolean> epicPicker  [private]
```

ValuePicker used to decide the value of UnitEntity.epic.
Definition at line 44 of file CardNumberPickerAdapter.java.
Referenced by CardNumberPickerAdapter(), and addSelectedUnits().

5.9.4.5 numberPicker

```
final NumberPicker numberPicker  [private]
```

NumberPicker used to decide the number of UnitEntity objects that are inserted.
Definition at line 78 of file CardNumberPickerAdapter.java.
Referenced by CardNumberPickerAdapter(), and addSelectedUnits().

5.9.4.6 squadManager

```
final SquadManager squadManager  [private]
```

SquadManager containing current SquadState used to get squad information.
Definition at line 72 of file CardNumberPickerAdapter.java.
Referenced by CardNumberPickerAdapter(), and addSelectedUnits().

5.9.4.7 squadPicker

```
final NumberPicker squadPicker  [private]
```

NumberPicker used to decide the value of UnitEntity.squad.
Only visible if the value of .abilityPicker is set to Ability.BINDING.

See also

.abilityPicker

Definition at line 66 of file CardNumberPickerAdapter.java.
Referenced by CardNumberPickerAdapter(), and addSelectedUnits().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/addcard/CardNumberPickerAdapter.java

5.10 CardUiState Class Reference

A data class encapsulating the visible state of a card in the card list shown by the ShowUnitsDialog.
Collaboration diagram for CardUiState:



Public Member Functions

- CardUiState (int unitId, @DrawableRes int damageBackgroundImageId, @IntRange(from=UNUSED) int damage, @ColorInt int damageTextColor, @DrawableRes int abilityImageId, @Nullable @IntRange(from=1) Integer squad)

  Constructor of a CardUiState encapsulating the given data.
- boolean showAbility ()

  Returns whether the ability view is shown, i.e., if the unit has an ability other than Ability.NONE.
- boolean showSquad ()

  Returns whether the squad view is shown, i.e., if the unit has the Ability.BINDING ability.
- int getUnitId ()

  Returns the .unitId of the represented UnitEntity.
- int getDamageBackgroundImageId ()

  Returns the drawable resource shown as the background of the damage view.
- String getDamageString ()

  Returns the String shown in the damage view.
- int getDamageTextColor ()

  Returns the text color of the damage view.
- int getAbilityImageId ()

  Returns the drawable resource shown in the ability view.
- String getSquadString ()

  Returns the String shown in the squad view.
- boolean equals (Object o)

  Checks whether the given Object looks the same as this CardUiState.

Static Public Attributes

- static final DiffUtil.ItemCallback< CardUiState > DIFF_CALLBACK

  DiffUtil.ItemCallback used to compare different CardUiState objects in a ListAdapter, e.g., CardListAdapter.
- static final int UNUSED = -1

  Integer that is used instead of a drawable resource to indicate the absence of a meaningful .abilityImage.

Private Attributes

- final int unitId

  Integer referencing the UnitEntity.id of the represented UnitEntity.
- final int damageBackgroundImageId

  Integer referencing the drawable resource shown as the background of the damage view.
- final String damageString

String containing the number shown in the damage view or nothing if the represented UnitEntity is epic.

- final int damageTextColor

  Integer representing the text color of the damage view.

- final int abilityImageId

  Integer referencing the drawable resource shown in the ability view.

- final String squadString

  String containing the number shown in the squad view.

5.10.1 Detailed Description

A data class encapsulating the visible state of a card in the card list shown by the ShowUnitsDialog.
Can be created from the represented UnitEntity using CardUiStateFactory.

See also

CardUiStateFactory

ShowUnitsDialog

Definition at line 24 of file CardUiState.java.

5.10.2 Constructor & Destructor Documentation

5.10.2.1 CardUiState()

```
CardUiState (
            int unitId,
            @DrawableRes int damageBackgroundImageId,
            @IntRange(from=UNUSED) int damage,
            @ColorInt int damageTextColor,
            @DrawableRes int abilityImageId,
            @Nullable @IntRange(from=1) Integer squad)
```

Constructor of a CardUiState encapsulating the given data.

Parameters

unitId	Integer representing the UnitEntity.id of the represented UnitEntity.
damageBackground↩ ImageId	Integer referencing the drawable resource shown by the damage view.
damage	Integer representing the damage of the represented UnitEntity.
damageTextColor	Integer representing the text color of the damage view.
abilityImageId	Integer referencing the drawable resource shown by the ability image view or .UNUSED.
squad	Integer representing the UnitEntity.squad of the represented UnitEntity.

Exceptions

IllegalArgumentException	When damage is less than .UNUSED or squad is neither null nor greater than zero.

See also

CardUiStateFactory.createCardUiState(UnitEntity, DamageCalculator)

Definition at line 108 of file CardUiState.java.
References abilityImageId, damageBackgroundImageId, damageTextColor, unitId, and UNUSED.
Referenced by equals().

5.10.3 Member Function Documentation

5.10.3.1 equals()

```
boolean equals (
            Object o)
```

Checks whether the given Object looks the same as this CardUiState.
Does not compare .unitId since that field does not influence the visual representation of CardUiState objects.

Parameters

o	Object that is being compared with this CardUiState.

Returns

A Boolean defining whether the objects look the same.

Definition at line 212 of file CardUiState.java.
References CardUiState(), abilityImageId, damageBackgroundImageId, damageString, damageTextColor, and squadString.

5.10.3.2 getAbilityImageId()

```
int getAbilityImageId ()
```

Returns the drawable resource shown in the ability view.

Returns

An Integer referencing a drawable resource.

See also

.abilityImageId

.showAbility()

Definition at line 189 of file CardUiState.java.
References abilityImageId.
Referenced by CardListAdapter.onBindViewHolder().

5.10.3.3 getDamageBackgroundImageId()

```
int getDamageBackgroundImageId ()
```

Returns the drawable resource shown as the background of the damage view.

Returns

An Integer referencing a drawable resource.

See also

.damageBackgroundImageId

Definition at line 158 of file CardUiState.java.
References damageBackgroundImageId.
Referenced by CardListAdapter.onBindViewHolder().

5.10.3.4 getDamageString()

```
String getDamageString ()
```

Returns the String shown in the damage view.

Returns

A String containing the damage of the represented UnitEntity or nothing if the unit is empty.

See also

.damageString

Definition at line 168 of file CardUiState.java.
References damageString.
Referenced by CardListAdapter.onBindViewHolder().

5.10.3.5 getDamageTextColor()

`int getDamageTextColor ()`
Returns the text color of the damage view.

Returns

An Integer representing a color.

See also

.damageTextColor

Definition at line 178 of file CardUiState.java.
References damageTextColor.
Referenced by CardListAdapter.onBindViewHolder().

5.10.3.6 getSquadString()

`String getSquadString ()`
Returns the String shown in the squad view.

Returns

A String containing the squad of the represented UnitEntity or nothing.

See also

.squadString

.showSquad()

Definition at line 200 of file CardUiState.java.
References squadString.
Referenced by CardListAdapter.onBindViewHolder().

5.10.3.7 getUnitId()

`int getUnitId ()`
Returns the .unitId of the represented UnitEntity.

See also

.unitId

Returns

An Integer referencing the UnitEntity.id of the represented UnitEntity.

Definition at line 148 of file CardUiState.java.
References unitId.
Referenced by CardListAdapter.onBindViewHolder().

5.10.3.8 showAbility()

```
boolean showAbility ()
```
Returns whether the ability view is shown, i.e., if the unit has an ability other than Ability.NONE.

Returns

A Boolean defining whether the ability view is shown.

Definition at line 130 of file CardUiState.java.
References abilityImageId, and UNUSED.
Referenced by CardListAdapter.onBindViewHolder(), and showSquad().

5.10.3.9 showSquad()

```
boolean showSquad ()
```
Returns whether the squad view is shown, i.e., if the unit has the Ability.BINDING ability.

Returns

A Boolean defining whether the squad view is shown.

Definition at line 139 of file CardUiState.java.
References showAbility(), and squadString.
Referenced by CardListAdapter.onBindViewHolder().

5.10.4 Member Data Documentation

5.10.4.1 abilityImageId

```
final int abilityImageId  [private]
```
Integer referencing the drawable resource shown in the ability view.
May be .UNUSED if the view is not visible.

See also

.UNUSED

.getAbilityImageId()

Definition at line 88 of file CardUiState.java.
Referenced by CardUiState(), equals(), getAbilityImageId(), and showAbility().

5.10.4.2 damageBackgroundImageId

```
final int damageBackgroundImageId  [private]
```
Integer referencing the drawable resource shown as the background of the damage view.

See also

.getDamageBackgroundImageId()

Definition at line 64 of file CardUiState.java.
Referenced by CardUiState(), equals(), and getDamageBackgroundImageId().

5.10.4.3 damageString

```
final String damageString  [private]
```
String containing the number shown in the damage view or nothing if the represented UnitEntity is epic.

See also

.getDamageString()

Definition at line 72 of file CardUiState.java.
Referenced by equals(), and getDamageString().

5.10.4.4 damageTextColor

```
final int damageTextColor  [private]
```
Integer representing the text color of the damage view.

See also

.getDamageTextColor()

Definition at line 79 of file CardUiState.java.
Referenced by CardUiState(), equals(), and getDamageTextColor().

5.10.4.5 DIFF_CALLBACK

```
final DiffUtil.ItemCallback<CardUiState> DIFF_CALLBACK  [static]
```
Initial value:
```
= new DiffUtil.ItemCallback<CardUiState>() {
        @Override
        public boolean areItemsTheSame(@NonNull CardUiState oldItem, @NonNull CardUiState newItem) {
            return oldItem.unitId == newItem.unitId;
        }

        @Override
        public boolean areContentsTheSame(@NonNull CardUiState oldItem, @NonNull CardUiState newItem) {
            return Objects.equals(oldItem, newItem);
        }
    }
```
DiffUtil.ItemCallback used to compare different CardUiState objects in a ListAdapter, e.g., CardListAdapter.

See also

.unitId

.equals(Object)

Definition at line 33 of file CardUiState.java.
Referenced by CardListAdapter.CardListAdapter().

5.10.4.6 squadString

```
final String squadString  [private]
```
String containing the number shown in the squad view.

See also

.getSquadString()

Definition at line 95 of file CardUiState.java.
Referenced by equals(), getSquadString(), and showSquad().

5.10.4.7 unitId

```
final int unitId  [private]
```
Integer referencing the UnitEntity.id of the represented UnitEntity.
The only member that is not shown in the UI.

See also

.getUnitId()

Definition at line 57 of file CardUiState.java.
Referenced by CardUiState(), and getUnitId().

5.10.4.8 UNUSED

```
final int UNUSED = -1  [static]
```
Integer that is used instead of a drawable resource to indicate the absence of a meaningful .abilityImage.

See also

.abilityImageId

Definition at line 50 of file CardUiState.java.
Referenced by CardUiState(), CardUiStateFactory.createCardUiState(), and showAbility().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/cards/CardUiState.java

5.11 CardUiStateFactory Class Reference

A factory class responsible for creating CardUiState objects from UnitEntity objects.
Collaboration diagram for CardUiStateFactory:



Public Member Functions

- CardUiStateFactory (@NonNull Context context, boolean weather, boolean horn)

  Constructor of a CardUiStateFactory for a row with the given weather and horn (de-)buff values.
- List< CardUiState > createCardUiState (@NonNull Collection< UnitEntity > units)

  Creates a List of CardUiState objects from the given List of UnitEntity objects.
- CardUiState createCardUiState (@NonNull UnitEntity unit, @NonNull DamageCalculator calculator)

  Creates a CardUiState from the given UnitEntity.

Private Attributes

- final boolean weather

  Boolean defining the status of the weather debuff in the row for which CardUiState objects are created.
- final boolean horn

  Boolean defining the status of the horn buff in the row for which CardUiState objects are created.
- final Map< Color, Integer > damageTextColors = new HashMap<>(Color.values().length)

  Map containing a color integer for each Color value possible, i.e., Color.DEFAULT, Color.BUFFED, and Color.DEBUFFED.

5.11.1 Detailed Description

A factory class responsible for creating CardUiState objects from UnitEntity objects.

See also

CardUiState

Definition at line 28 of file CardUiStateFactory.java.

5.11.2 Constructor & Destructor Documentation

5.11.2.1 CardUiStateFactory()

```
CardUiStateFactory (
            @NonNull Context context,
            boolean weather,
            boolean horn)
```

Constructor of a CardUiStateFactory for a row with the given weather and horn (de-)buff values.
The Context parameter is used to retrieve the color values saved in .damageTextColors.

Parameters

context	Context object used to get colors.
weather	Boolean defining whether the weather debuff is active.
horn	Boolean defining whether the horn buff is active.

See also

Context.getColor(int)

Definition at line 58 of file CardUiStateFactory.java.
References damageTextColors, horn, and weather.

5.11.3 Member Function Documentation

5.11.3.1 createCardUiState() [1/2]

```
List< CardUiState > createCardUiState (
            @NonNull Collection< UnitEntity > units)
```

Creates a List of CardUiState objects from the given List of UnitEntity objects.
Basically calls .createCardUiState(UnitEntity, DamageCalculator) for each given UnitEntity.

See also

.createCardUiState(UnitEntity, DamageCalculator)

Parameters

units	List of UnitEntity objects that are converted to CardUiState objects.

Returns

A List of CardUiState objects from the given UnitEntity objects.

Definition at line 74 of file CardUiStateFactory.java.
References createCardUiState(), DamageCalculatorUseCase.getDamageCalculator(), horn, and weather.
Referenced by createCardUiState().

5.11.3.2 createCardUiState() [2/2]

```
CardUiState createCardUiState (
            @NonNull UnitEntity unit,
            @NonNull DamageCalculator calculator)
```

Creates a CardUiState from the given UnitEntity.

See also

.createCardUiState(Collection)

Parameters

unit	UnitEntity that is converted to a CardUiState.
calculator	DamageCalculator used to calculate damage and Color of the given UnitEntity.

Returns

A CardUiState obtained from the given UnitEntity.

Definition at line 89 of file CardUiStateFactory.java.
References damageTextColors, and CardUiState.UNUSED.

5.11.4 Member Data Documentation

5.11.4.1 damageTextColors

```
final Map<Color, Integer> damageTextColors = new HashMap<>(Color.values().length)  [private]
```

Map containing a color integer for each Color value possible, i.e., Color.DEFAULT, Color.BUFFED, and Color.DEBUFFED.
Definition at line 46 of file CardUiStateFactory.java.
Referenced by CardUiStateFactory(), and createCardUiState().

5.11.4.2 horn

```
final boolean horn  [private]
```

Boolean defining the status of the horn buff in the row for which CardUiState objects are created.
Definition at line 39 of file CardUiStateFactory.java.
Referenced by CardUiStateFactory(), and createCardUiState().

5.11.4.3 weather

```
final boolean weather  [private]
```

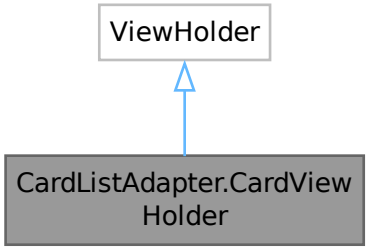
Boolean defining the status of the weather debuff in the row for which CardUiState objects are created.
Definition at line 33 of file CardUiStateFactory.java.
Referenced by CardUiStateFactory(), and createCardUiState().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/cards/CardUiStateFactory.java

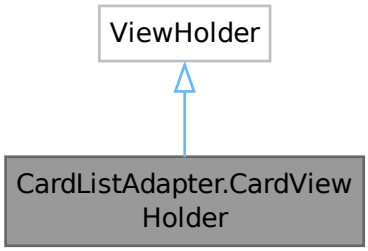
5.12 CardListAdapter.CardViewHolder Class Reference

A RecyclerView.ViewHolder class managing a card view that shows a representation of the UnitEntity referenced by .itemId.

Inheritance diagram for CardListAdapter.CardViewHolder:



Collaboration diagram for CardListAdapter.CardViewHolder:



Package Functions

- CardViewHolder (@NonNull View itemView, @NonNull Consumer< Integer > onCopy, @NonNull Consumer< Integer > onRemove)

  Constructor of a CardViewHolder for the given View initializing .damageView, .abilityView, and .bindingView.
- TextView getDamageView ()

  Returns the .damageView of the managed View.
- ImageView getAbilityView ()

  Returns the .abilityView of the managed View.
- TextView getBindingView ()

  Returns the .bindingView of the managed View.
- void setItemId (int itemId)

  Sets the .unitId of this ViewHolder.

Private Attributes

- final TextView damageView

  TextView showing the (de-)buffed damage of the represented UnitEntity.
- final ImageView abilityView

  ImageView showing an image of the UnitEntity.ability of the represented UnitEntity.

- final TextView bindingView

  TextView showing the UnitEntity.squad of the represented UnitEntity if UnitEntity.squad is Ability.BINDING.
- int itemId = (int) NO_ID

  Integer referencing the UnitEntity.id of the represented UnitEntity.

5.12.1 Detailed Description

A RecyclerView.ViewHolder class managing a card view that shows a representation of the UnitEntity referenced by .itemId.

See also

RecyclerView.ViewHolder

Definition at line 131 of file CardListAdapter.java.

5.12.2 Constructor & Destructor Documentation

5.12.2.1 CardViewHolder()

```
CardViewHolder (
            @NonNull View itemView,
            @NonNull Consumer< Integer > onCopy,
            @NonNull Consumer< Integer > onRemove)  [package]
```

Constructor of a CardViewHolder for the given View initializing .damageView, .abilityView, and .bindingView.
Sets View.OnClickListener on copy-button and delete-button calling the given onCopy and onRemove callbacks with .unitId.

Parameters

itemView	View with the card layout.
onCopy	Consumer that is called with .unitId, when the copy-button of the given View is clicked.
onRemove	Consumer that is called with .unitId, when the delete-button of the given View is clicked.

Definition at line 173 of file CardListAdapter.java.
References abilityView, bindingView, damageView, itemId, CardListAdapter.onCopy, and CardListAdapter.onRemove.

5.12.3 Member Function Documentation

5.12.3.1 getAbilityView()

```
ImageView getAbilityView ()  [package]
```

Returns the .abilityView of the managed View.

Returns

An ImageView showing the UnitEntity.ability of the represented UnitEntity.

See also

.abilityView

Definition at line 211 of file CardListAdapter.java.
References abilityView.

5.12.3.2 getBindingView()

```
TextView getBindingView ()  [package]
```

Returns the .bindingView of the managed View.

Returns

A TextView showing the UnitEntity.squad of the represented UnitEntity.

See also

.bindingView

Definition at line 221 of file CardListAdapter.java.
References bindingView.

5.12.3.3 getDamageView()

```
TextView getDamageView ()  [package]
```

Returns the .damageView of the managed View.

Returns

A TextView showing the damage of the represented UnitEntity.

See also

.damageView

Definition at line 201 of file CardListAdapter.java.
References damageView.

5.12.3.4 setItemId()

```
void setItemId (
            int itemId)  [package]
```

Sets the .unitId of this ViewHolder.

Parameters

itemId	Integer referencing the UnitEntity.id of the represented UnitEntity.

Exceptions

IllegalArgumentException	When itemId is negative.

See also

.itemId

Definition at line 232 of file CardListAdapter.java.
References itemId.

5.12.4 Member Data Documentation

5.12.4.1 abilityView

```
final ImageView abilityView  [private]
```

ImageView showing an image of the UnitEntity.ability of the represented UnitEntity.
If UnitEntity.ability is Ability.NONE, this view's visibility should be View.GONE.

See also

.getAbilityView()

Definition at line 144 of file CardListAdapter.java.
Referenced by CardViewHolder(), and getAbilityView().

5.12.4.2 bindingView

```
final TextView bindingView  [private]
```

TextView showing the UnitEntity.squad of the represented UnitEntity if UnitEntity.squad is Ability.BINDING.
If UnitEntity.ability is not Ability.BINDING, this view's visibility should be View.GONE.

See also

.getBindingView()

Definition at line 152 of file CardListAdapter.java.
Referenced by CardViewHolder(), and getBindingView().

5.12.4.3 damageView

```
final TextView damageView  [private]
```

TextView showing the (de-)buffed damage of the represented UnitEntity.

See also

.getDamageView()

Definition at line 137 of file CardListAdapter.java.
Referenced by CardViewHolder(), and getDamageView().

5.12.4.4 itemId

```
int itemId = (int) NO_ID  [private]
```

Integer referencing the UnitEntity.id of the represented UnitEntity.
Is initialized with .NO_ID and therefore must be set using .setItemId() in .onBindViewHolder(CardViewHolder, int).

See also

.setItemId(int)

.onBindViewHolder(CardViewHolder, int)

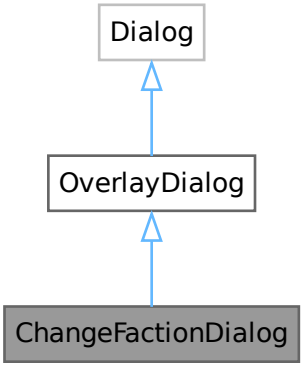
Definition at line 161 of file CardListAdapter.java.
Referenced by CardViewHolder(), and setItemId().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/cards/CardListAdapter.java

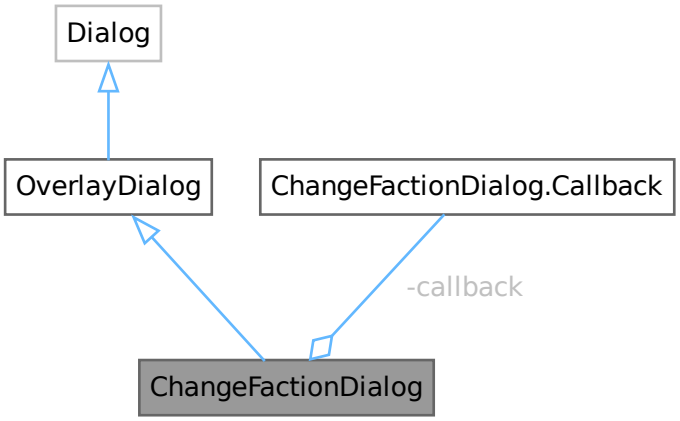
5.13 ChangeFactionDialog Class Reference

An OverlayDialog class used to change the faction design.

Inheritance diagram for ChangeFactionDialog:



Collaboration diagram for ChangeFactionDialog:



Classes

- interface Callback

  An interface used for propagating a selected theme back to the creator of a ChangeFactionDialog.

Public Member Functions

- ChangeFactionDialog (@NonNull Context context, @NonNull Callback callback)

  Constructor of a ChangeFactionDialog that calls the given Callback when one theme is selected.

Protected Member Functions

- void onCreate (Bundle savedInstanceState)

Initializes layout and sets listeners for each view.

Protected Member Functions inherited from OverlayDialog

- OverlayDialog (@NonNull Context context, @LayoutRes int layout, @IdRes int dismissViewId)

  Constructor of an OverlayDialog in the given Context, with the given layout and dismissViewId.
- OverlayDialog (@NonNull Context context, @LayoutRes int layout)

  Constructor of an OverlayDialog in the given Context and with the given layout.

Private Member Functions

- View.OnClickListener getOnThemeClickListener (@IntRange(from=THEME_MONSTER, to=THEME_↩ SCOIATAEL) int theme)

  Returns a View.OnclickListener instance that calls .cancel() and uses .callback to propagate the selected theme.

Private Attributes

- final Callback callback

  Callback that is called when a theme is selected.

Additional Inherited Members

Static Package Attributes inherited from OverlayDialog

- static final int NO_DISMISS_VIEW = -1

  Integer used as .dismissViewId, when the Dialog shouldn't be cancelable by clicking on a View.

5.13.1 Detailed Description

An OverlayDialog class used to change the faction design.
Definition at line 20 of file ChangeFactionDialog.java.

5.13.2 Constructor & Destructor Documentation

5.13.2.1 ChangeFactionDialog()

```
ChangeFactionDialog (
            @NonNull Context context,
            @NonNull Callback callback)
```

Constructor of a ChangeFactionDialog that calls the given Callback when one theme is selected.

Parameters

context	Context in which this Dialog is run.
callback	Callback that is called when a theme is selected.

Definition at line 33 of file ChangeFactionDialog.java.
References callback.

5.13.3 Member Function Documentation

5.13.3.1 getOnThemeClickListener()

```
View.OnClickListener getOnThemeClickListener (
            @IntRange(from=THEME_MONSTER, to=THEME_SCOIATAEL) int theme)  [private]
```

Returns a View.OnclickListener instance that calls .cancel() and uses .callback to propagate the selected theme.

Parameters

theme	Integer representing the selected theme.

Returns

A View.OnClickListener handling theme input.

See also

.cancel()

Callback.onThemeSelect(int)

Definition at line 73 of file ChangeFactionDialog.java.
References callback.
Referenced by onCreate().

5.13.3.2 onCreate()

```
void onCreate (
            Bundle savedInstanceState)  [protected]
```

Initializes layout and sets listeners for each view.

Parameters

savedInstanceState	If this dialog is being reinitialized after the hosting activity was previously shut down, holds the result from the most recent call to onSaveInstanceState, or null if this is the first time.

Reimplemented from OverlayDialog.
Definition at line 47 of file ChangeFactionDialog.java.
References getOnThemeClickListener().

5.13.4 Member Data Documentation

5.13.4.1 callback

```
final Callback callback  [private]
```

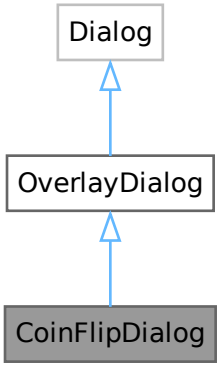
Callback that is called when a theme is selected.
Definition at line 24 of file ChangeFactionDialog.java.
Referenced by ChangeFactionDialog(), and getOnThemeClickListener().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/ChangeFactionDialog.java

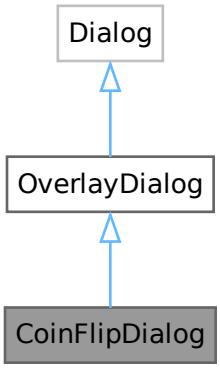
5.14 CoinFlipDialog Class Reference

An OverlayDialog class showing the result of a coin-flip.

Inheritance diagram for CoinFlipDialog:



Collaboration diagram for CoinFlipDialog:



Public Member Functions

- CoinFlipDialog (@NonNull Context context)

  Constructor of a CoinFlipDialog in the given Context.

Protected Member Functions

- void onCreate (Bundle savedInstanceState)

  Initializes layout and sets listeners for each view.

Protected Member Functions inherited from OverlayDialog

- OverlayDialog (@NonNull Context context, @LayoutRes int layout, @IdRes int dismissViewId)

  Constructor of an OverlayDialog in the given Context, with the given layout and dismissViewId.

- OverlayDialog (@NonNull Context context, @LayoutRes int layout)

  Constructor of an OverlayDialog in the given Context and with the given layout.

Static Private Attributes

- static final Random RANDOM = new Random()

  Random used to decide the result of the coin-flip.

Additional Inherited Members

Static Package Attributes inherited from OverlayDialog

- static final int NO_DISMISS_VIEW = -1

  Integer used as .dismissViewId, when the Dialog shouldn't be cancelable by clicking on a View.

5.14.1 Detailed Description

An OverlayDialog class showing the result of a coin-flip.
Shows heads or tails with 48.5% probability respectively. The remaining 3% are divided equally between three tie situations.

See also

R.layout.popup_coin_normal

R.layout.popup_coin_stewie

R.layout.popup_coin_terry

R.layout.popup_coin_vin

Definition at line 24 of file CoinFlipDialog.java.

5.14.2 Constructor & Destructor Documentation

5.14.2.1 CoinFlipDialog()

```
CoinFlipDialog (
            @NonNull Context context)
```

Constructor of a CoinFlipDialog in the given Context.

Parameters

context	Context in which the Dialog is run.

Definition at line 34 of file CoinFlipDialog.java.

5.14.3 Member Function Documentation

5.14.3.1 onCreate()

```
void onCreate (
            Bundle savedInstanceState)  [protected]
```

Initializes layout and sets listeners for each view.
The result of the coin-flip is decided here.

Parameters

savedInstanceState	If this dialog is being reinitialized after the hosting activity was previously shut down, holds the result from the most recent call to onSaveInstanceState, or null if this is the first time.

Reimplemented from OverlayDialog.
Definition at line 48 of file CoinFlipDialog.java.
References RANDOM.

5.14.4 Member Data Documentation

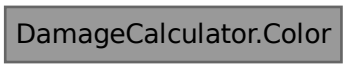
5.14.4.1 RANDOM

`final Random RANDOM = new Random()  [static], [private]`
Random used to decide the result of the coin-flip.
Definition at line 28 of file CoinFlipDialog.java.
Referenced by onCreate().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/CoinFlipDialog.java

5.15 DamageCalculator.Color Enum Reference

An enum containing values representing whether a unit is buffed, de-buffed or not affected by status effects.
Collaboration diagram for DamageCalculator.Color:



Public Attributes

- DEFAULT

  The unit is not affected by status effects.
- BUFFED

  The unit is buffed by status effects.
- DEBUFFED

  The unit is de-buffed by weather effects.

5.15.1 Detailed Description

An enum containing values representing whether a unit is buffed, de-buffed or not affected by status effects.
Definition at line 49 of file DamageCalculator.java.

5.15.2 Member Data Documentation

5.15.2.1 BUFFED

`BUFFED`
The unit is buffed by status effects.
Definition at line 57 of file DamageCalculator.java.

5.15.2.2 DEBUFFED

`DEBUFFED`
The unit is de-buffed by weather effects.
Definition at line 61 of file DamageCalculator.java.

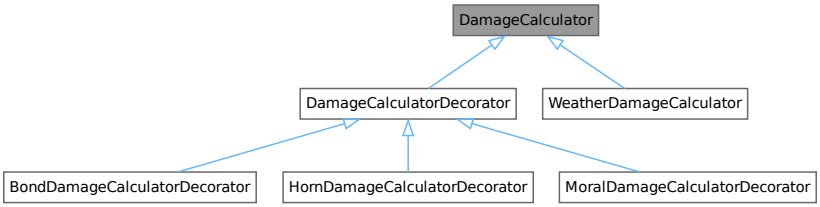
5.15.2.3 DEFAULT

`DEFAULT`
The unit is not affected by status effects.
Definition at line 53 of file DamageCalculator.java.
The documentation for this enum was generated from the following file:

- gwent/domain/damage/DamageCalculator.java

5.16 DamageCalculator Interface Reference

An interface for a class capable of calculating the (de-)buffed damage of a unit for a given UnitEntity.id and UnitEntity.damage.
Inheritance diagram for DamageCalculator:



Collaboration diagram for DamageCalculator:



Classes

- enum Color

  An enum containing values representing whether a unit is buffed, de-buffed or not affected by status effects.

Public Member Functions

- Color isBuffed (int id)

  Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT.
- int calculateDamage (int id, @IntRange(from=0) int damage)

  Calculates the (de-)buffed damage of unit with the given id and the given base-damage.

5.16.1 Detailed Description

An interface for a class capable of calculating the (de-)buffed damage of a unit for a given UnitEntity.id and UnitEntity.damage.
Is used to implement the visitor design pattern, see com.peternaggschga.gwent.data.UnitEntity.calculateDamage()
The respective visitors are created as a decorator hierarchy, see DamageCalculatorBuildDirector.getCalculator().

See also

com.peternaggschga.gwent.data.UnitEntity.calculateDamage(DamageCalculator)

DamageCalculatorBuildDirector.getCalculator(boolean, boolean, Collection)

Definition at line 17 of file DamageCalculator.java.

5.16.2 Member Function Documentation

5.16.2.1 calculateDamage()

```
int calculateDamage (
            int id,
            @IntRange(from=0) int damage)
```

Calculates the (de-)buffed damage of unit with the given id and the given base-damage.
Calculation is defined by the underlying decorator structure implemented via DamageCalculatorDecorator and WeatherDamageCalculator.

Parameters

id	Integer representing the UnitEntity.id of the unit whose (de-)buff damage is calculated.
damage	Integer representing the base-damage of the unit whose (de-)buff damage is calculated.

Returns

Integer representing the (de-)buffed damage of the unit.

See also

DamageCalculatorDecorator

WeatherDamageCalculator

Implemented in BondDamageCalculatorDecorator, HornDamageCalculatorDecorator, MoralDamageCalculatorDecorator, and WeatherDamageCalculator.

5.16.2.2 isBuffed()

```
Color isBuffed (
            int id)
```

Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT.
Units are shown as Color.DEFAULT when they are not affected by any damage changing buffs or de-buffs. Units are shown as Color.DEBUFFED when they are only affected by the weather de-buff. Units are shown as Color.BUFFED when they are affected by any damage-increasing buff.

Parameters

id	Integer representing the UnitEntity.id of the unit buff status is calculated.

Returns

Color representing whether the unit is buffed, de-buffed or not affected.

See also

Color

Implemented in BondDamageCalculatorDecorator, HornDamageCalculatorDecorator, MoralDamageCalculatorDecorator, and WeatherDamageCalculator.
The documentation for this interface was generated from the following file:

- gwent/domain/damage/DamageCalculator.java

5.17 DamageCalculatorBuildDirector Class Reference

A build director responsible for the creation of DamageCalculator instances from sets of units using DamageCalculatorBuilder.
Collaboration diagram for DamageCalculatorBuildDirector:



Static Public Member Functions

- static DamageCalculator getCalculator (boolean weather, boolean horn, @NonNull Collection< UnitEntity > units)

  Creates a DamageCalculator for a row with the given weather status, horn status and units.

Static Private Member Functions

- static void setSquads (@NonNull Collection< UnitEntity > units, @NonNull DamageCalculatorBuilder builder)

  Uses the given units to add a BondDamageCalculatorDecorator to the builder using DamageCalculatorBuilder.setBond().
- static void setMoralBoosts (@NonNull Collection< UnitEntity > units, @NonNull DamageCalculatorBuilder builder)

  Uses the given units to add a MoralDamageCalculatorDecorator to the builder using DamageCalculatorBuilder.setMoral().
- static void setHorns (boolean horn, @NonNull Collection< UnitEntity > units, @NonNull DamageCalculatorBuilder builder)

  Uses the given commander's horn status and the given units to add a HornDamageCalculatorDecorator to the builder using DamageCalculatorBuilder.setHorn().

5.17.1 Detailed Description

A build director responsible for the creation of DamageCalculator instances from sets of units using DamageCalculatorBuilder.
Encapsulates the logic for correct order of decorators.

See also

DamageCalculatorBuilder

Definition at line 21 of file DamageCalculatorBuildDirector.java.

5.17.2 Member Function Documentation

5.17.2.1 getCalculator()

```
DamageCalculator getCalculator (
            boolean weather,
            boolean horn,
            @NonNull Collection< UnitEntity > units)  [static]
```

Creates a DamageCalculator for a row with the given weather status, horn status and units.

Parameters

weather	Boolean defining whether the calculation encompasses the weather debuff.
horn	Boolean defining whether a commander's horn is in the row.
units	Collection of UnitEntity objects that are in the row.

Returns

A DamageCalculator object capable of calculating damage for the given units.

Definition at line 31 of file DamageCalculatorBuildDirector.java.
References Ability.BINDING, DamageCalculatorBuilder.getResult(), Ability.HORN, Ability.MORAL_BOOST, setHorns(), setMoralBoosts(), setSquads(), and DamageCalculatorBuilder.setWeather().
Referenced by DamageCalculatorUseCase.getDamageCalculator().

5.17.2.2 setHorns()

```
void setHorns (
            boolean horn,
            @NonNull Collection< UnitEntity > units,
            @NonNull DamageCalculatorBuilder builder)  [static], [private]
```

Uses the given commander's horn status and the given units to add a HornDamageCalculatorDecorator to the builder using DamageCalculatorBuilder.setHorn().
To accomplish this, the given unit list is converted to a List of the ids of the units with the Ability.HORN Ability (including `null` if horn is `true`).

Parameters

horn	Boolean defining whether a commander's horn is in the row.
units	Collection of UnitEntity objects that are in the row.
builder	DamageCalculatorBuilder where the horn buff is added.

See also

DamageCalculatorBuilder.setHorn(List)

Definition at line 106 of file DamageCalculatorBuildDirector.java.
References UnitEntity.getId(), and Ability.HORN.
Referenced by getCalculator().

5.17.2.3 setMoralBoosts()

```
void setMoralBoosts (
            @NonNull Collection< UnitEntity > units,
            @NonNull DamageCalculatorBuilder builder)  [static], [private]
```

Uses the given units to add a MoralDamageCalculatorDecorator to the builder using DamageCalculatorBuilder.setMoral().
To accomplish this, the given unit list is converted to a List of the ids of the units with the Ability.MORAL_BOOST Ability.

Parameters

units	Collection of UnitEntity objects that are in the row.
builder	DamageCalculatorBuilder where the horn buff is added.

See also

DamageCalculatorBuilder.setMoral(List)

Definition at line 85 of file DamageCalculatorBuildDirector.java.
References UnitEntity.getId(), and Ability.MORAL_BOOST.
Referenced by getCalculator().

5.17.2.4 setSquads()

```
void setSquads (
            @NonNull Collection< UnitEntity > units,
            @NonNull DamageCalculatorBuilder builder)  [static], [private]
```

Uses the given units to add a BondDamageCalculatorDecorator to the builder using DamageCalculatorBuilder.setBond(). To accomplish this, the given unit list is converted to a Map from ids of units with the Ability.BINDING Ability to the respective squad size.

Parameters

units	Collection of UnitEntity objects that are in the row.
builder	DamageCalculatorBuilder where the horn buff is added.

See also

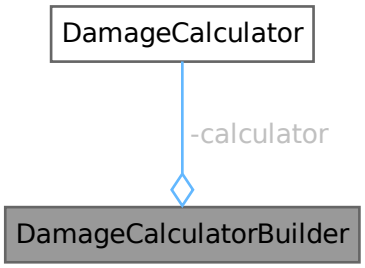
DamageCalculatorBuilder.setBond(Map)

Definition at line 61 of file DamageCalculatorBuildDirector.java.
References Ability.BINDING.
Referenced by getCalculator().
The documentation for this class was generated from the following file:

- gwent/domain/damage/DamageCalculatorBuildDirector.java

5.18 DamageCalculatorBuilder Class Reference

A builder class responsible for creating a decorator structure of DamageCalculator classes with the given parameters.
Collaboration diagram for DamageCalculatorBuilder:



Package Functions

- void setWeather (boolean weather)

Sets the weather debuff for the calculation.

- void setBond (@NonNull Map< Integer, Integer > idToSquad)

  Sets the tight bond buff for the calculation.
- void setMoral (@NonNull List< Integer > unitIds)

  Sets the moral boost buff for the calculation.
- void setHorn (@NonNull List< Integer > unitIds)

  Sets the commander's horn buff for the calculation.
- DamageCalculator getResult ()

  Returns the current state of the built DamageCalculator, i.e.

Private Attributes

- DamageCalculator calculator = new WeatherDamageCalculator(false)

  The DamageCalculator instance that is being built by this DamageCalculatorBuilder.

5.18.1 Detailed Description

A builder class responsible for creating a decorator structure of DamageCalculator classes with the given parameters.
Should only be used by DamageCalculatorBuildDirector. Calls to .setWeather(), .setBond(), .setMoral() or .setHorn() should be in that exact order to create a correct DamageCalculator. Every function call is optional, i.e. every function should be called once or not at all.

See also

DamageCalculatorBuildDirector

DamageCalculator

DamageCalculatorDecorator

Definition at line 19 of file DamageCalculatorBuilder.java.

5.18.2 Member Function Documentation

5.18.2.1 getResult()

```
DamageCalculator getResult ()  [package]
```

Returns the current state of the built DamageCalculator, i.e.
.calculator.

Returns

A DamageCalculator that has been built by this builder.

Definition at line 72 of file DamageCalculatorBuilder.java.
References calculator.
Referenced by DamageCalculatorBuildDirector.getCalculator().

5.18.2.2 setBond()

```
void setBond (
            @NonNull Map< Integer, Integer > idToSquad)  [package]
```

Sets the tight bond buff for the calculation.
Should be called after .setWeather() or never.

Parameters

idToSquad	Map mapping the ids of all units with the com.peternaggschga.gwent.data.Ability.BINDING ability to the respective squad size.

Definition at line 41 of file DamageCalculatorBuilder.java.
References calculator.

5.18.2.3 setHorn()

```
void setHorn (
            @NonNull List< Integer > unitIds)  [package]
```

Sets the commander's horn buff for the calculation.
Should be called after .setMoral() or never.

Parameters

unitIds	List of Integers representing ids of units with the com.peternaggschga.gwent.data.Ability.HORN ability.

Definition at line 62 of file DamageCalculatorBuilder.java.
References calculator.

5.18.2.4 setMoral()

```
void setMoral (
            @NonNull List< Integer > unitIds)  [package]
```

Sets the moral boost buff for the calculation.
Should be called after .setBond() or never.

Parameters

unitIds	List of Integers representing ids of units with the com.peternaggschga.gwent.data.Ability.MORAL_BOOST ability.

Definition at line 51 of file DamageCalculatorBuilder.java.
References calculator.

5.18.2.5 setWeather()

```
void setWeather (
            boolean weather)  [package]
```

Sets the weather debuff for the calculation.
Should be called first or never.

Parameters

weather	Boolean defining whether the weather debuff is active.

Definition at line 32 of file DamageCalculatorBuilder.java.
References calculator.
Referenced by DamageCalculatorBuildDirector.getCalculator().

5.18.3 Member Data Documentation

5.18.3.1 calculator

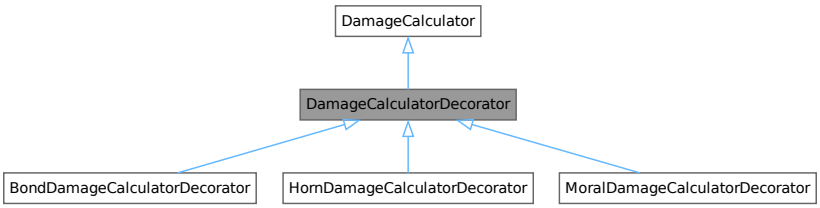
```
DamageCalculator calculator = new WeatherDamageCalculator(false)  [private]
```

The DamageCalculator instance that is being built by this DamageCalculatorBuilder.
Starts as a WeatherDamageCalculator, since this is always at the end of the decorator chain.
Definition at line 25 of file DamageCalculatorBuilder.java.
Referenced by getResult(), setBond(), setHorn(), setMoral(), and setWeather().
The documentation for this class was generated from the following file:

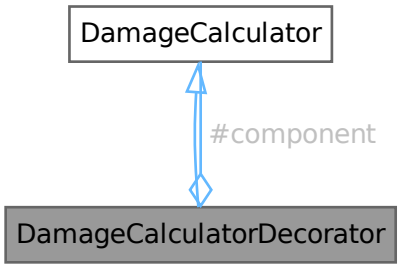
- gwent/domain/damage/DamageCalculatorBuilder.java

5.19 DamageCalculatorDecorator Class Reference

An abstract DamageCalculator that is used as a superclass for the decorator classes implementing the damage calculation.
Inheritance diagram for DamageCalculatorDecorator:



Collaboration diagram for DamageCalculatorDecorator:



Protected Attributes

- final DamageCalculator component

  A DamageCalculator which is decorated by this decorator.

Package Functions

- DamageCalculatorDecorator (@NonNull DamageCalculator component)

  Constructor of a DamageCalculatorDecorator.

Additional Inherited Members

Public Member Functions inherited from DamageCalculator

- Color isBuffed (int id)

  Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT.
- int calculateDamage (int id, @IntRange(from=0) int damage)

  Calculates the (de-)buffed damage of unit with the given id and the given base-damage.

5.19.1 Detailed Description

An abstract DamageCalculator that is used as a superclass for the decorator classes implementing the damage calculation.
Definition at line 9 of file DamageCalculatorDecorator.java.

5.19.2 Constructor & Destructor Documentation

5.19.2.1 DamageCalculatorDecorator()

```
DamageCalculatorDecorator (
            @NonNull DamageCalculator component)  [package]
```

Constructor of a DamageCalculatorDecorator.

Parameters

component	DamageCalculator that is being decorated by this decorator.

Definition at line 20 of file DamageCalculatorDecorator.java.
References component.

5.19.3 Member Data Documentation

5.19.3.1 component

```
final DamageCalculator component  [protected]
```

A DamageCalculator which is decorated by this decorator.
Definition at line 13 of file DamageCalculatorDecorator.java.
Referenced by BondDamageCalculatorDecorator.BondDamageCalculatorDecorator(), DamageCalculatorDecorator(), HornDamageCalculatorDecorator.HornDamageCalculatorDecorator(), MoralDamageCalculatorDecorator.MoralDamageCalculatorDe BondDamageCalculatorDecorator.calculateDamage(), HornDamageCalculatorDecorator.calculateDamage(), MoralDamageCalculatorDecorator.calculateDamage(), BondDamageCalculatorDecorator.isBuffed(), HornDamageCalculatorDecorato and MoralDamageCalculatorDecorator.isBuffed().
The documentation for this class was generated from the following file:

- gwent/domain/damage/DamageCalculatorDecorator.java

5.20 DamageCalculatorUseCase Class Reference

A use case class responsible for creating DamageCalculator objects for rows using DamageCalculatorBuildDirector.
Collaboration diagram for DamageCalculatorUseCase:



Static Public Member Functions

- static Single< DamageCalculator > getDamageCalculator (@NonNull UnitRepository repository, @NonNull RowType row)

  Returns a Single emitting a DamageCalculator object for the given row with properties retrieved from the given UnitRepository.

- static DamageCalculator getDamageCalculator (boolean weather, boolean horn, @NonNull Collection< UnitEntity > units)

  Creates a DamageCalculator for a row with the given weather status, horn status and units.

5.20.1 Detailed Description

A use case class responsible for creating DamageCalculator objects for rows using DamageCalculatorBuildDirector.

See also

DamageCalculatorBuildDirector

DamageCalculator

Definition at line 23 of file DamageCalculatorUseCase.java.

5.20.2 Member Function Documentation

5.20.2.1 getDamageCalculator() [1/2]

```
Single< DamageCalculator > getDamageCalculator (
            @NonNull UnitRepository repository,
            @NonNull RowType row)  [static]
```

Returns a Single emitting a DamageCalculator object for the given row with properties retrieved from the given UnitRepository.

Parameters

repository	UnitRepository used for data collection.
row	RowType defining which row is queried for DamageCalculator generation.

Returns

A Single emitting a DamageCalculator object for the given row.

Definition at line 32 of file DamageCalculatorUseCase.java.
References getDamageCalculator().
Referenced by CardUiStateFactory.createCardUiState(), BurnDialogUseCase.getBurnUnits(), and getDamageCalculator().

5.20.2.2 getDamageCalculator() [2/2]

```
DamageCalculator getDamageCalculator (
            boolean weather,
            boolean horn,
            @NonNull Collection< UnitEntity > units)  [static]
```

Creates a DamageCalculator for a row with the given weather status, horn status and units.
Basically calls DamageCalculatorBuildDirector.getCalculator with the given parameters.

Parameters

weather	Boolean defining whether the calculation encompasses the weather debuff.
horn	Boolean defining whether a commander's horn is in the row.
units	Collection of UnitEntity objects that are in the row.

Returns

A DamageCalculator object capable of calculating damage for the given units.

See also

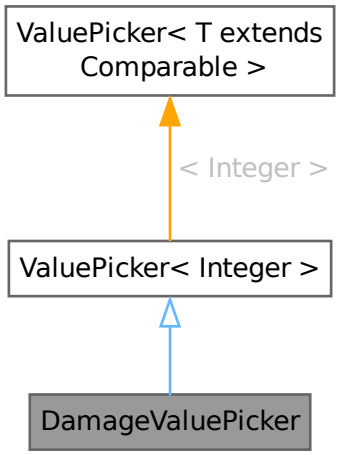
DamageCalculatorBuildDirector.getCalculator(boolean, boolean, Collection)

Definition at line 50 of file DamageCalculatorUseCase.java.
References DamageCalculatorBuildDirector.getCalculator().
The documentation for this class was generated from the following file:

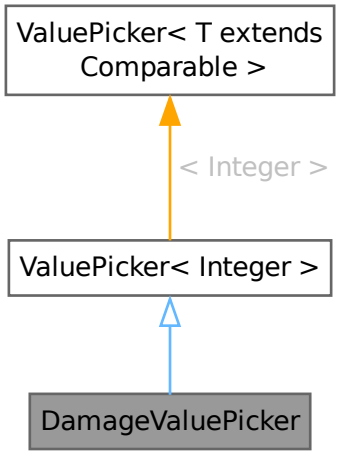
- gwent/domain/cases/DamageCalculatorUseCase.java

5.21 DamageValuePicker Class Reference

A ValuePicker used for selection of damage values for UnitEntity.damage.
Inheritance diagram for DamageValuePicker:



Collaboration diagram for DamageValuePicker:



Protected Member Functions

- String getDisplayString (@NonNull Integer value)

  Returns a localized String representing the given value when .epicValues is false or otherwise the corresponding epic damage value.

Protected Member Functions inherited from ValuePicker< Integer >

- NumberPicker getPicker ()

  Returns the NumberPicker wrapped by this ValuePicker.
- abstract String getDisplayString (@NonNull T value)

  Returns a localized String representing the given value.
- Map< T, Integer > getDisplayIntegers ()

  Returns the displayIntegers map.
- List< T > getSelectableValues ()

  Returns a List of values that can be selected by this picker.

Package Functions

- DamageValuePicker (@NonNull NumberPicker picker)

  Constructor of a DamageValuePicker wrapping the given NumberPicker.
- void setValue (@NonNull @IntRange(from=0, to=NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) Integer value)

  Sets the picker to the given value.
- Integer getValue ()

  Returns the currently selected value.
- void setEpicValues (boolean epicValues)

  Switches the .selectableValues between non-epic and epic damage values.
- void setSelectableValues (@NonNull Collection< Integer > values, @Nullable Integer defaultValue)

  Does nothing.

Package Functions inherited from ValuePicker< Integer >

- ValuePicker (@NonNull NumberPicker picker, @NonNull SortedMap< T, Integer > valueToStringRes)

  Constructor of a ValuePicker wrapping the given NumberPicker and offering the given values.
- void setSelectableValues (@NonNull Collection< T > values, @Nullable T defaultValue)

  Sets selectableValues to the given values and show the given defaultValue.
- void setOnValueChangedListener (@NonNull OnValueChangeListener< T > onValueChangedListener)

  Sets an OnValueChangeListener that is called when the value of the wrapped .picker has changed and stayed the same for 500 ms.
- T getValue ()

  Returns the currently selected value.
- void setValue (@NonNull T value)

  Sets the picker to the given value.

Private Attributes

- boolean epicValues = false

  Boolean defining whether or not this DamageValuePicker shows epic damage values.

Additional Inherited Members

Public Member Functions inherited from ValuePicker< Integer >

- Context getContext ()

  Returns the context the .picker is running in, through which it can access the current theme, resources, etc.

5.21.1 Detailed Description

A ValuePicker used for selection of damage values for UnitEntity.damage.
Allows for switching between epic and normal damage values using .setEpicValues().
Definition at line 25 of file DamageValuePicker.java.

5.21.2 Constructor & Destructor Documentation

5.21.2.1 DamageValuePicker()

```
DamageValuePicker (
            @NonNull NumberPicker picker)  [package]
```

Constructor of a DamageValuePicker wrapping the given NumberPicker.
Calls .setEpicValues() to initialize the NumberPicker with non-epic damage values.

See also

ValuePicker.ValuePicker(NumberPicker, SortedMap)

.setEpicValues(boolean)

Parameters

picker	NumberPicker wrapped by the created DamageValuePicker.

Definition at line 39 of file DamageValuePicker.java.
References ValuePicker< Integer >.picker, and setEpicValues().

5.21.3 Member Function Documentation

5.21.3.1 getDisplayString()

```
String getDisplayString (
            @NonNull Integer value)  [protected]
```

Returns a localized String representing the given value when .epicValues is false or otherwise the corresponding epic damage value.

Parameters

value	Value that should be represented as a String.

Returns

A localized String representing the value.

Exceptions

IllegalStateException	When .epicValues is true and .displayIntegers does not contain a mapping for the given value.

Definition at line 57 of file DamageValuePicker.java.
References epicValues, and ValuePicker< Integer >.getDisplayIntegers().

5.21.3.2 getValue()

```
Integer getValue ()  [package]
```

Returns the currently selected value.

Returns

A value that is selected in .picker.

Definition at line 90 of file DamageValuePicker.java.
References epicValues, and ValuePicker< Integer >.getPicker().

5.21.3.3 setEpicValues()

```
void setEpicValues (
            boolean epicValues)  [package]
```

Switches the .selectableValues between non-epic and epic damage values.
Sets .epicValues.

See also

.epicValues

Parameters

epicValues	Boolean defining whether or not epic values are shown.

Definition at line 100 of file DamageValuePicker.java.
References epicValues, ValuePicker< Integer >.getPicker(), and ValuePicker< Integer >.getSelectableValues().
Referenced by DamageValuePicker().

5.21.3.4 setSelectableValues()

```
void setSelectableValues (
            @NonNull Collection< Integer > values,
            @Nullable Integer defaultValue)  [package]
```

Does nothing.
Overrides parent since .selectableValues should only be modified by .setEpicValues().

Parameters

values	Collection of the new selectable values.
defaultValue	Value that is shown in the beginning. If null, then the first value defined by the Comparable interface is used.

Definition at line 121 of file DamageValuePicker.java.

5.21.3.5 setValue()

```
void setValue (
            @NonNull @IntRange(from=0, to=NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) Integer value)
[package]
```

Sets the picker to the given value.
If .epicValues is true, ValuePicker.setValue() is called. Otherwise the .picker is simply set to the given value.

Parameters

value	Value that the picker is set to.

Exceptions

IllegalStateException	When .epicValues is false and the given value is not in [0,20].

See also

ValuePicker.setValue(Comparable)

Definition at line 74 of file DamageValuePicker.java.
References epicValues, and ValuePicker< Integer >.getPicker().

5.21.4 Member Data Documentation

5.21.4.1 epicValues

```
boolean epicValues = false  [private]
```

Boolean defining whether or not this DamageValuePicker shows epic damage values.

See also

.setEpicValues(boolean)

Definition at line 30 of file DamageValuePicker.java.
Referenced by getDisplayString(), getValue(), setEpicValues(), and setValue().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/addcard/DamageValuePicker.java

5.22 ResetDialogUseCase.DialogType Enum Reference

An Enum defining which form of Dialog should be shown.
Collaboration diagram for ResetDialogUseCase.DialogType:



Public Attributes

- NONE

  No Dialog must be invoked.
- DEFAULT

  A default Dialog asking whether to reset should be invoked.
- MONSTER

  A monster Dialog asking whether to reset and whether to invoke the monster perk, should be invoked.

5.22.1 Detailed Description

An Enum defining which form of Dialog should be shown.

See also

.getDialogType(Context, UnitRepository, Trigger)

Definition at line 148 of file ResetDialogUseCase.java.

5.22.2 Member Data Documentation

5.22.2.1 DEFAULT

`DEFAULT`
A default Dialog asking whether to reset should be invoked.
Definition at line 156 of file ResetDialogUseCase.java.
Referenced by ResetDialogUseCase.getDialogType().

5.22.2.2 MONSTER

`MONSTER`
A monster Dialog asking whether to reset and whether to invoke the monster perk, should be invoked.
Definition at line 161 of file ResetDialogUseCase.java.
Referenced by ResetDialogUseCase.reset().

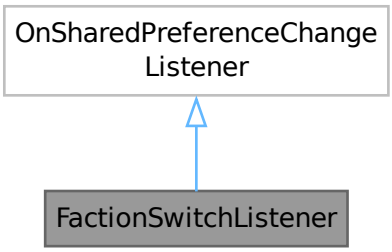
5.22.2.3 NONE

`NONE`
No Dialog must be invoked.
Definition at line 152 of file ResetDialogUseCase.java.
Referenced by ResetDialogUseCase.getDialogType(), and ResetDialogUseCase.reset().
The documentation for this enum was generated from the following file:

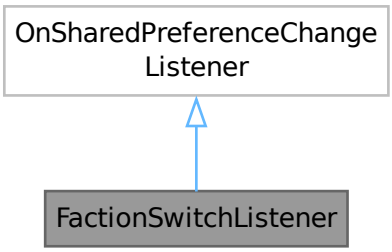
- gwent/domain/cases/ResetDialogUseCase.java

5.23 FactionSwitchListener Class Reference

A class implementing SharedPreference.OnSharedPreferenceChangeListener for the .THEME_PREFERENCE_↩ KEY SharedPreference.
Inheritance diagram for FactionSwitchListener:



Collaboration diagram for FactionSwitchListener:



Public Member Functions

- void onSharedPreferenceChanged (SharedPreferences sharedPreferences, @Nullable String key)

  Called when a shared preference is changed, added, or removed.

Static Public Member Functions

- static FactionSwitchListener getListener (@NonNull Window mainWindow)

  Returns a new FactionSwitchListener instance for the given Window.
- static void setTheme (@NonNull Context context)

  Changes the theme of the content referenced by the given Context depending on the preferences obtainable by said Context.

Static Public Attributes

- static final String THEME_PREFERENCE_KEY = "theme"

  String constant defining the key of the theme SharedPreference.

- static final int THEME_MONSTER = 0

  Integer constant representing the Monster theme.
- static final int THEME_NILFGAARD = 1

  Integer constant representing the Nilfgaard theme.
- static final int THEME_NORTHERN_KINGDOMS = 2

  Integer constant representing the Northern Kingdoms theme.
- static final int THEME_SCOIATAEL = 3

  Integer constant representing the Scoia'tael theme.

Private Member Functions

- FactionSwitchListener (@NonNull List< ImageView > ballViews, @NonNull List< ImageView > cardViews, @NonNull List< TextView > unitViews, @NonNull ImageButton factionButton)

  Constructor of a FactionSwitchListener updating the given View objects when .onSharedPreferenceChanged() is called.

Private Attributes

- final List< ImageView > ballViews

  List of ImageView objects that show a colored ball.
- final List< ImageView > cardViews

  List of ImageView objects that show a card back.
- final List< TextView > unitViews

  List of TextView objects that show the number of units in a certain color.
- final ImageButton factionButton

  ImageButton showing the logo of the current faction.

5.23.1 Detailed Description

A class implementing SharedPreference.OnSharedPreferenceChangeListener for the .THEME_PREFERENCE_↩ KEY SharedPreference.
When this Preference is updated, the theme is switched in an animated way using ImageViewSwitchAnimator.

See also

ImageViewSwitchAnimator

Definition at line 31 of file FactionSwitchListener.java.

5.23.2 Constructor & Destructor Documentation

5.23.2.1 FactionSwitchListener()

```
FactionSwitchListener (
            @NonNull List< ImageView > ballViews,
            @NonNull List< ImageView > cardViews,
            @NonNull List< TextView > unitViews,
            @NonNull ImageButton factionButton)  [private]
```

Constructor of a FactionSwitchListener updating the given View objects when .onSharedPreferenceChanged() is called.
Should only be used by factory method .getListener().

See also

.getListener(Window)

Parameters

ballViews	List of ImageView objects showing a colored ball.
cardViews	List of ImageView objects showing the backside of a card.
unitViews	List of TextView objects showing the number of units.
factionButton	ImageButton that shows the current faction logo.

Definition at line 108 of file FactionSwitchListener.java.
References ballViews, cardViews, factionButton, and unitViews.
Referenced by getListener().

5.23.3 Member Function Documentation

5.23.3.1 getListener()

```
FactionSwitchListener getListener (
            @NonNull Window mainWindow)  [static]
```

Returns a new FactionSwitchListener instance for the given Window.
Factory method of FactionSwitchListener.

Parameters

mainWindow	Window, that is updated by the new FactionSwitchListener.

Returns

A FactionSwitchListener instance for the given Window.

Definition at line 124 of file FactionSwitchListener.java.
References FactionSwitchListener(), ballViews, cardViews, getListener(), and unitViews.
Referenced by getListener(), and MainActivity.onCreate().

5.23.3.2 onSharedPreferenceChanged()

```
void onSharedPreferenceChanged (
            SharedPreferences sharedPreferences,
            @Nullable String key)
```

Called when a shared preference is changed, added, or removed.
Only handles changes when the given key is equal to .THEME_PREFERENCE_KEY. Updates the View objects in .ballViews, .cardViews, .unitViews, and .factionButton using ImageViewSwitchAnimator.

See also

ImageViewSwitchAnimator

Parameters

sharedPreferences	SharedPreferences that received the change.
key	String representing the key of the preference that was changed, added, or removed.

Definition at line 176 of file FactionSwitchListener.java.
References ImageViewSwitchAnimator.animatedSwitch(), ballViews, cardViews, factionButton, THEME_MONSTER, THEME_NILFGAARD, THEME_NORTHERN_KINGDOMS, THEME_PREFERENCE_KEY, THEME_SCOIATAEL, and unitViews.

5.23.3.3 setTheme()

```
void setTheme (
            @NonNull Context context)  [static]
```

Changes the theme of the content referenced by the given Context depending on the preferences obtainable by said Context.

Parameters

context	Context that is used to query the current theme preference and that gets it theme updated accordingly.

Definition at line 148 of file FactionSwitchListener.java.
References THEME_MONSTER, THEME_NILFGAARD, THEME_NORTHERN_KINGDOMS, THEME_PREFERENCE_KEY, and THEME_SCOIATAEL.
Referenced by MainActivity.onCreate(), and SettingsActivity.onCreate().

5.23.4 Member Data Documentation

5.23.4.1 ballViews

```
final List<ImageView> ballViews  [private]
```

List of ImageView objects that show a colored ball.

See also

R.drawable.ball_red

R.drawable.ball_grey

R.drawable.ball_blue

R.drawable.ball_green

Definition at line 66 of file FactionSwitchListener.java.
Referenced by FactionSwitchListener(), getListener(), and onSharedPreferenceChanged().

5.23.4.2 cardViews

```
final List<ImageView> cardViews  [private]
```

List of ImageView objects that show a card back.

See also

R.drawable.card_monster_landscape_free

R.drawable.card_nilfgaard_landscape_free

R.drawable.card_northern_kingdoms_landscape_free

R.drawable.card_scoiatael_landscape_free

Definition at line 76 of file FactionSwitchListener.java.
Referenced by FactionSwitchListener(), getListener(), and onSharedPreferenceChanged().

5.23.4.3 factionButton

```
final ImageButton factionButton  [private]
```

ImageButton showing the logo of the current faction.

See also

R.drawable.icon_round_monster

R.drawable.icon_round_nilfgaard

R.drawable.icon_round_northern_kingdoms

R.drawable.icon_round_scoiatael

Definition at line 96 of file FactionSwitchListener.java.
Referenced by FactionSwitchListener(), and onSharedPreferenceChanged().

5.23.4.4 THEME_MONSTER

```
final int THEME_MONSTER = 0  [static]
```
Integer constant representing the Monster theme.
Definition at line 41 of file FactionSwitchListener.java.
Referenced by onSharedPreferenceChanged(), and setTheme().

5.23.4.5 THEME_NILFGAARD

```
final int THEME_NILFGAARD = 1  [static]
```
Integer constant representing the Nilfgaard theme.
Definition at line 46 of file FactionSwitchListener.java.
Referenced by onSharedPreferenceChanged(), and setTheme().

5.23.4.6 THEME_NORTHERN_KINGDOMS

```
final int THEME_NORTHERN_KINGDOMS = 2  [static]
```
Integer constant representing the Northern Kingdoms theme.
Definition at line 51 of file FactionSwitchListener.java.
Referenced by onSharedPreferenceChanged(), and setTheme().

5.23.4.7 THEME_PREFERENCE_KEY

```
final String THEME_PREFERENCE_KEY = "theme"  [static]
```
String constant defining the key of the theme SharedPreference.
Definition at line 36 of file FactionSwitchListener.java.
Referenced by onSharedPreferenceChanged(), and setTheme().

5.23.4.8 THEME_SCOIATAEL

```
final int THEME_SCOIATAEL = 3  [static]
```
Integer constant representing the Scoia'tael theme.
Definition at line 56 of file FactionSwitchListener.java.
Referenced by onSharedPreferenceChanged(), and setTheme().

5.23.4.9 unitViews

```
final List<TextView> unitViews  [private]
```
List of TextView objects that show the number of units in a certain color.

See also

R.color.color_text_monster

R.color.color_text_nilfgaard

R.color.color_text_northern_kingdoms

R.color.color_text_scoiatael

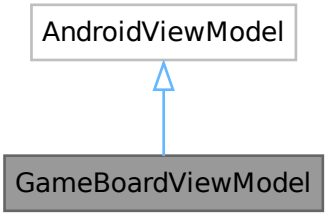
Definition at line 86 of file FactionSwitchListener.java.
Referenced by FactionSwitchListener(), getListener(), and onSharedPreferenceChanged().
The documentation for this class was generated from the following file:

- gwent/ui/main/FactionSwitchListener.java

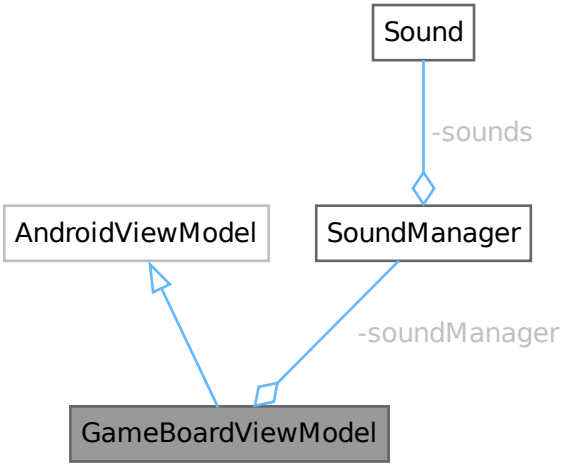
5.24 GameBoardViewModel Class Reference

An AndroidViewModel class responsible for encapsulating and offering state of views in activity_main.xml, i.e., that show the overall game board.

Inheritance diagram for GameBoardViewModel:



Collaboration diagram for GameBoardViewModel:



Public Member Functions

- Flowable< RowUiState > getRowUiState (@NonNull RowType row)

  Returns a Flowable object emitting RowUiState for the given row.
- Flowable< MenuUiState > getMenuUiState ()

  Returns a Flowable object emitting MenuUiState.
- SoundManager getSoundManager ()

  Returns the SoundManager managed by this GameBoardViewModel.
- Completable onWeatherViewPressed (@NonNull RowType row)

  Updates the weather debuff of the given row.
- Completable onHornViewPressed (@NonNull RowType row)

  Updates the horn buff of the given row.
- Completable onResetButtonPressed (@NonNull Context context)

  Triggers a reset and possibly an alert dialog, depending on preferences.

- Completable onFactionSwitchReset (@NonNull Context context)

  Triggers a reset and possibly an alert dialog, depending on preferences.
- Completable onWeatherButtonPressed ()

  Clears all weather effects.
- Completable onBurnButtonPressed (@NonNull Context context)

  Clears units with the most damage.

Static Public Member Functions

- static GameBoardViewModel getModel (@NonNull ViewModelStoreOwner owner, @NonNull UnitRepository repository, @NonNull SoundManager soundManager)

  Factory method of a GameBoardViewModel.

Private Member Functions

- GameBoardViewModel (@NonNull GwentApplication application)

  Constructor of a GameBoardViewModel object.
- Single< UnitRepository > getRepository ()

  Returns the UnitRepository used by the parent GwentApplication.
- Completable reset (@NonNull Context context, @NonNull ResetDialogUseCase.Trigger trigger)

  Triggers a reset and possibly an alert dialog, depending on preferences.

Private Attributes

- final Map< RowType, Flowable< RowUiState > > rowUiStates = new HashMap<>(RowType.↩ values().length)

  A map structure containing the Flowable objects emitting the RowUiState for each row.
- Flowable< MenuUiState > menuUiState = Flowable.empty()

  Flowable emitting the MenuUiState for the right-hand side menu.
- SoundManager soundManager

  SoundManager used to play Sound effects on click events.

Static Private Attributes

- static final ViewModelInitializer< GameBoardViewModel > INITIALIZER

  ViewModelInitializer used by androidx.lifecycle.ViewModelProvider.Factory to instantiate the class.

5.24.1 Detailed Description

An AndroidViewModel class responsible for encapsulating and offering state of views in activity_main.xml, i.e., that show the overall game board.
Click events on the rows and the menu are handled also.
Definition at line 38 of file GameBoardViewModel.java.

5.24.2 Constructor & Destructor Documentation

5.24.2.1 GameBoardViewModel()

```
GameBoardViewModel (
            @NonNull GwentApplication application)  [private]
```

Constructor of a GameBoardViewModel object.
Should only be called in .initializer.

Parameters

application	GwentApplication that uses this AndroidViewModel.

See also

.INITIALIZER

Definition at line 60 of file GameBoardViewModel.java.
Referenced by getModel().

5.24.3 Member Function Documentation

5.24.3.1 getMenuUiState()

```
Flowable< MenuUiState > getMenuUiState ()
```

Returns a Flowable object emitting MenuUiState.

Returns

A Flowable object for the state of the menu.

See also

MenuUiState

.menuUiState

Definition at line 185 of file GameBoardViewModel.java.
References menuUiState.

5.24.3.2 getModel()

```
GameBoardViewModel getModel (
            @NonNull ViewModelStoreOwner owner,
            @NonNull UnitRepository repository,
            @NonNull SoundManager soundManager)  [static]
```

Factory method of a GameBoardViewModel.
Creates a new GameBoardViewModel for the given owner and initializes .rowUiStates and .menuUiState.

Parameters

owner	ViewModelStoreOwner instantiating the GameBoardViewModel.
repository	UnitRepository where Flowables are retrieved.
soundManager	SoundManager used to play Sound effects on click events.

Returns

A new GameBoardViewModel instance.

See also

ViewModelProvider.ViewModelProvider(ViewModelStoreOwner, ViewModelProvider.Factory)

Definition at line 98 of file GameBoardViewModel.java.
References GameBoardViewModel(), INITIALIZER, reset(), rowUiStates, and soundManager.
Referenced by MainActivity.onCreate().

5.24.3.3 getRepository()

```
Single< UnitRepository > getRepository ()  [private]
```

Returns the UnitRepository used by the parent GwentApplication.
Basically a wrapper for GwentApplication.getRepository(Context).

Returns

A Single emitting the UnitRepository instance.

See also

GwentApplication.getRepository(Context)

Definition at line 161 of file GameBoardViewModel.java.
References GwentApplication.getRepository().
Referenced by onHornViewPressed(), onWeatherButtonPressed(), and onWeatherViewPressed().

5.24.3.4 getRowUiState()

```
Flowable< RowUiState > getRowUiState (
            @NonNull RowType row)
```

Returns a Flowable object emitting RowUiState for the given row.

Parameters

row	RowType defining the row for which the state is queried.

Returns

A Flowable object for the state of the given row.

See also

RowUiState

.rowUiStates

Definition at line 173 of file GameBoardViewModel.java.
References rowUiStates.

5.24.3.5 getSoundManager()

```
SoundManager getSoundManager ()
```

Returns the SoundManager managed by this GameBoardViewModel.

Returns

A SoundManager used by this GameBoardViewModel.

Definition at line 195 of file GameBoardViewModel.java.
References soundManager.

5.24.3.6 onBurnButtonPressed()

```
Completable onBurnButtonPressed (
            @NonNull Context context)
```

Clears units with the most damage.
May inflate a warning dialog depending on the user's preferences and a Toast informing the user about the burned units. Should only be called by the View.OnClickListener of the burn button. Plays a matching sound, if units were removed.

Parameters

context	Context

Returns

A Completable tracking operation status.

See also

BurnDialogUseCase.burn(Context, SoundManager)

Definition at line 310 of file GameBoardViewModel.java.
References BurnDialogUseCase.burn(), and soundManager.

5.24.3.7 onFactionSwitchReset()

```
Completable onFactionSwitchReset (
            @NonNull Context context)
```

Triggers a reset and possibly an alert dialog, depending on preferences.
Should only be called when the faction has been switched. Wrapper for .reset(Context, ResetDialogUseCase.Trigger).

Parameters

context	Context object used to acquire SharedPreferences and inflate Dialog views.

Returns

A Completable tracking operation status.

See also

.reset(Context, ResetDialogUseCase.Trigger)

Definition at line 263 of file GameBoardViewModel.java.
References ResetDialogUseCase.Trigger.FACTION_SWITCH, and reset().

5.24.3.8 onHornViewPressed()

```
Completable onHornViewPressed (
            @NonNull RowType row)
```

Updates the horn buff of the given row.
Flips between on and off. Plays a matching sound, if the horn is switched on.

Parameters

row	RowType defining the affected row.

Returns

A Completable tracking operation status.

See also

UnitRepository.switchHorn(RowType)

Definition at line 227 of file GameBoardViewModel.java.
References getRepository(), and soundManager.

5.24.3.9 onResetButtonPressed()

```
Completable onResetButtonPressed (
            @NonNull Context context)
```

Triggers a reset and possibly an alert dialog, depending on preferences.
Should only be called by the button's View.OnClickListener. Wrapper for .reset(Context, ResetDialogUseCase.Trigger).

Parameters

context	Context object used to acquire SharedPreferences and inflate Dialog views.

Returns

A Completable tracking operation status.

See also

.reset(Context, ResetDialogUseCase.Trigger)

ResetDialogUseCase.Trigger.BUTTON_CLICK

Definition at line 250 of file GameBoardViewModel.java.
References ResetDialogUseCase.Trigger.BUTTON_CLICK, and reset().

5.24.3.10 onWeatherButtonPressed()

```
Completable onWeatherButtonPressed ()
```

Clears all weather effects.
Should only be called by the View.OnClickListener of the weather button. Plays a matching sound on completion.

Returns

A Completable tracking operation status.

See also

UnitRepository.clearWeather()

Definition at line 295 of file GameBoardViewModel.java.
References UnitRepository.clearWeather(), and getRepository().

5.24.3.11 onWeatherViewPressed()

```
Completable onWeatherViewPressed (
            @NonNull RowType row)
```

Updates the weather debuff of the given row.
Flips between good and bad weather. Plays a matching sound, if the weather is switched on.

Parameters

row	RowType defining the affected row.

Returns

A Completable tracking operation status.

See also

UnitRepository.switchWeather(RowType)

Definition at line 207 of file GameBoardViewModel.java.
References getRepository(), and soundManager.

5.24.3.12 reset()

```
Completable reset (
            @NonNull Context context,
            @NonNull ResetDialogUseCase.Trigger trigger)  [private]
```

Triggers a reset and possibly an alert dialog, depending on preferences.
Plays a matching sound, if units were removed.

Parameters

context	Context object used to acquire SharedPreferences and inflate Dialog views.
trigger	com.peternaggschga.gwent.domain.cases.ResetDialogUseCase.Trigger defining which action triggered the reset.

Returns

A Completable tracking operation status.

See also

.reset(Context, ResetDialogUseCase.Trigger)

Definition at line 276 of file GameBoardViewModel.java.
References ResetDialogUseCase.reset(), and soundManager.
Referenced by getModel(), onFactionSwitchReset(), and onResetButtonPressed().

5.24.4 Member Data Documentation

5.24.4.1 INITIALIZER

```
final ViewModelInitializer<GameBoardViewModel> INITIALIZER  [static], [private]
```

Initial value:

```
= new ViewModelInitializer<>(
            GameBoardViewModel.class,
            creationExtras -> {
                GwentApplication app = (GwentApplication) creationExtras.get(APPLICATION_KEY);
                assert app != null;
                return new GameBoardViewModel(app);
            })
```

ViewModelInitializer used by androidx.lifecycle.ViewModelProvider.Factory to instantiate the class.

See also

androidx.lifecycle.ViewModelProvider.Factory.from(ViewModelInitializer[])

Definition at line 45 of file GameBoardViewModel.java.
Referenced by getModel().

5.24.4.2 menuUiState

```
Flowable<MenuUiState> menuUiState = Flowable.empty()  [private]
```

Flowable emitting the MenuUiState for the right-hand side menu.
Initialized in .getModel().

See also

.getMenuUiState()

MenuUiState

Definition at line 80 of file GameBoardViewModel.java.
Referenced by getMenuUiState().

5.24.4.3 rowUiStates

```
final Map<RowType, Flowable<RowUiState> > rowUiStates = new HashMap<>(RowType.values().length)
[private]
```

A map structure containing the Flowable objects emitting the RowUiState for each row.
Initialized in .getModel().

See also

.getRowUiState(RowType)

RowUiState

Definition at line 71 of file GameBoardViewModel.java.
Referenced by getModel(), and getRowUiState().

5.24.4.4 soundManager

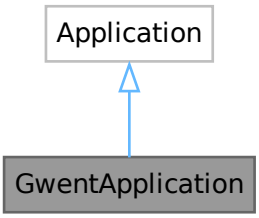
```
SoundManager soundManager  [private]
```

SoundManager used to play Sound effects on click events.
Definition at line 85 of file GameBoardViewModel.java.
Referenced by getModel(), getSoundManager(), onBurnButtonPressed(), onHornViewPressed(), onWeatherViewPressed(), and reset().
The documentation for this class was generated from the following file:

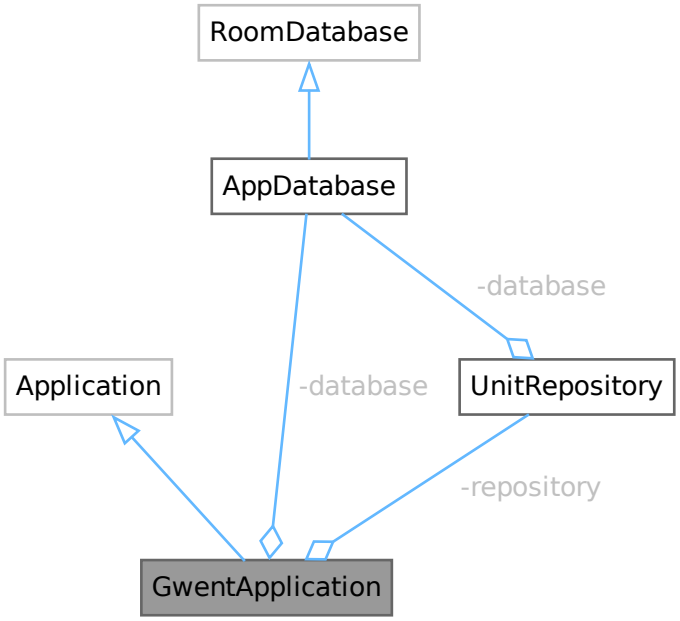
- gwent/ui/main/GameBoardViewModel.java

5.25 GwentApplication Class Reference

An Application encapsulating the Gwent app.
Inheritance diagram for GwentApplication:



Collaboration diagram for GwentApplication:



Public Member Functions

- void onCreate ()

  Called when the application is starting, before any activity, service, or receiver objects (excluding content providers) have been created.
- Single< UnitRepository > getRepository ()

  Returns the repository used to create, read, update and delete game state.

Static Public Member Functions

- static Single< UnitRepository > getRepository (@NonNull Context context)

  Returns a UnitRepository used by the GwentApplication referenced by the given Context.

Private Attributes

- AppDatabase database

  AppDatabase used by this app.

Static Private Attributes

- static UnitRepository repository = null

  UnitRepository used for communication to database.

5.25.1 Detailed Description

An Application encapsulating the Gwent app.
This class is responsible for maintaining a Singleton com.peternaggschga.gwent.data.UnitRepository connected to the com.peternaggschga.gwent.data.AppDatabase of this app.
Definition at line 18 of file GwentApplication.java.

5.25.2 Member Function Documentation

5.25.2.1 getRepository() [1/2]

```
Single< UnitRepository > getRepository ()
```

Returns the repository used to create, read, update and delete game state.
Lazily initializes repository, if not yet done so.

See also

.getRepository(Context)

Returns

A UnitRepository used to access game state.

Definition at line 62 of file GwentApplication.java.
References database, UnitRepository.getRepository(), and repository.

5.25.2.2 getRepository() [2/2]

```
Single< UnitRepository > getRepository (
            @NonNull Context context)  [static]
```

Returns a UnitRepository used by the GwentApplication referenced by the given Context.
Wrapper of getRepository().

See also

.getRepository()

Parameters

context	Context of a GwentApplication.

Returns

A UnitRepository used by the given GwentApplication to access game state.

Definition at line 41 of file GwentApplication.java.
Referenced by CardNumberPickerAdapter.addSelectedUnits(), BurnDialogUseCase.burn(), ShowUnitsDialog.getDialog(), GameBoardViewModel.getRepository(), MainActivity.initializeViewModel(), AddCardDialog.onCreate(), MainActivity.onCreate(), and ResetDialogUseCase.reset().

5.25.2.3 onCreate()

```
void onCreate ()
```

Called when the application is starting, before any activity, service, or receiver objects (excluding content providers) have been created.
Initializes database.
Definition at line 51 of file GwentApplication.java.
References database.

5.25.3 Member Data Documentation

5.25.3.1 database

```
AppDatabase database  [private]
```

AppDatabase used by this app.
Is initialized in onCreate().

See also

.onCreate()

Definition at line 32 of file GwentApplication.java.
Referenced by getRepository(), and onCreate().

5.25.3.2 repository

```
UnitRepository repository = null  [static], [private]
```

UnitRepository used for communication to database.
Is lazily initialized in getRepository() and provided as a Singleton.

See also

.getRepository()

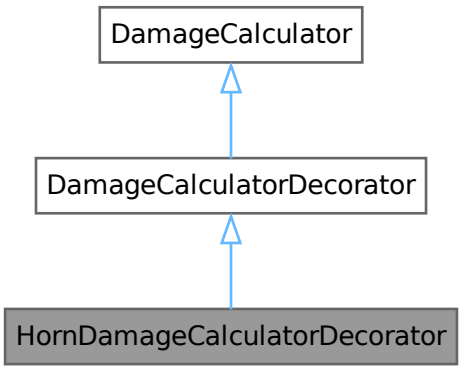
.getRepository(Context)

Definition at line 25 of file GwentApplication.java.
Referenced by getRepository().
The documentation for this class was generated from the following file:

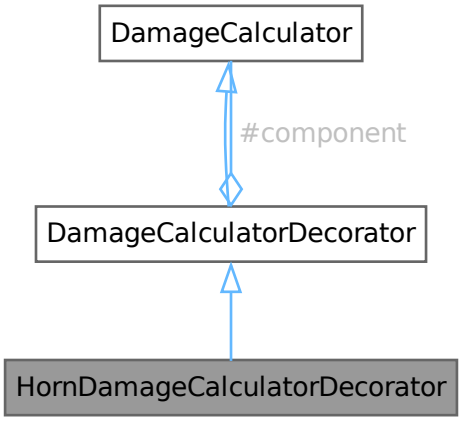
- gwent/GwentApplication.java

5.26 HornDamageCalculatorDecorator Class Reference

A DamageCalculator class responsible for calculating the horn buff if necessary, i.e., if there are units with the com.peternaggschga.gwent.data.Ability.HORN ability that are not the unit the damage is calculated for or if there is a commander's horn in this row.
Inheritance diagram for HornDamageCalculatorDecorator:



Collaboration diagram for HornDamageCalculatorDecorator:



Public Member Functions

- int calculateDamage (int id, @IntRange(from=0) int damage)

  Calculates the (de-)buffed damage of unit with the given id and the given base-damage.
- Color isBuffed (int id)

  Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT.

Package Functions

- HornDamageCalculatorDecorator (@NonNull DamageCalculator component, @NonNull List< Integer > unitIds)

  Constructor of a HornDamageCalculatorDecorator.

Package Functions inherited from DamageCalculatorDecorator

- DamageCalculatorDecorator (@NonNull DamageCalculator component)

  Constructor of a DamageCalculatorDecorator.

Private Member Functions

- boolean doubleDamage (int id)

  Decides whether the unit with the given id is affected by the horn buff.

Private Attributes

- final List< Integer > unitIds

  A List of Integers containing the ids of all units with the com.peternaggschga.gwent.data.Ability.HORN ability.

Additional Inherited Members

Protected Attributes inherited from DamageCalculatorDecorator

- final DamageCalculator component

  A DamageCalculator which is decorated by this decorator.

5.26.1 Detailed Description

A DamageCalculator class responsible for calculating the horn buff if necessary, i.e., if there are units with the com.peternaggschga.gwent.data.Ability.HORN ability that are not the unit the damage is calculated for or if there is a commander's horn in this row.
Acts as a ConcreteDecorator in the implemented decorator pattern and should decorate a WeatherDamageCalculator, a BondDamageCalculatorDecorator or a MoralDamageCalculatorDecorator.
Definition at line 18 of file HornDamageCalculatorDecorator.java.

5.26.2 Constructor & Destructor Documentation

5.26.2.1 HornDamageCalculatorDecorator()

```
HornDamageCalculatorDecorator (
            @NonNull DamageCalculator component,
            @NonNull List< Integer > unitIds)  [package]
```

Constructor of a HornDamageCalculatorDecorator.
Should only be called by DamageCalculatorBuilder. The given damage calculator should be of type WeatherDamageCalculator, BondDamageCalculatorDecorator or MoralDamageCalculatorDecorator for correct damage calculation. If the respective row has a commander's horn, `null` must be an element of the given List.

Parameters

component	DamageCalculator that is being decorated by this decorator.
unitIds	List of Integers representing ids of units with the com.peternaggschga.gwent.data.Ability.HORN ability.

See also

DamageCalculatorBuilder

Definition at line 38 of file HornDamageCalculatorDecorator.java.
References DamageCalculatorDecorator.component, and unitIds.

5.26.3 Member Function Documentation

5.26.3.1 calculateDamage()

```
int calculateDamage (
            int id,
            @IntRange(from=0) int damage)
```

Calculates the (de-)buffed damage of unit with the given id and the given base-damage.
Returns given damage times two if the unit is buffed by a commander's horn.

See also

.doubleDamage(int)

Parameters

id	Integer representing the UnitEntity.id of the unit whose (de-)buff damage is calculated.
damage	Integer representing the base-damage of the unit whose (de-)buff damage is calculated.

Returns

Integer representing the (de-)buffed damage of the unit.

Exceptions

IllegalArgumentException	When damage is negative.

Implements DamageCalculator.
Definition at line 63 of file HornDamageCalculatorDecorator.java.
References DamageCalculatorDecorator.component, and doubleDamage().

5.26.3.2 doubleDamage()

```
boolean doubleDamage (
            int id)  [private]
```

Decides whether the unit with the given id is affected by the horn buff.

Parameters

id	Integer representing the UnitEntity.id of the unit that is being evaluated.

Returns

A Boolean value defining whether the horn buff affects the given unit.

Definition at line 49 of file HornDamageCalculatorDecorator.java.
References unitIds.
Referenced by calculateDamage(), and isBuffed().

5.26.3.3 isBuffed()

```
Color isBuffed (
            int id)
```

Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT. Units are shown as Color.BUFFED when they are affected by a horn buff, otherwise their Color is defined by .component.

Parameters

id	Integer representing the UnitEntity.id of the unit buff status is calculated.

Returns

Color representing whether the unit is buffed, de-buffed or not affected.

See also

Color

.doubleDamage(int)

Implements DamageCalculator.
Definition at line 82 of file HornDamageCalculatorDecorator.java.
References DamageCalculatorDecorator.component, and doubleDamage().

5.26.4 Member Data Documentation

5.26.4.1 unitIds

```
final List<Integer> unitIds  [private]
```

A List of Integers containing the ids of all units with the com.peternaggschga.gwent.data.Ability.HORN ability.
If a commander's horn is in the respective row, .unitIds contains `null`.
Definition at line 24 of file HornDamageCalculatorDecorator.java.
Referenced by HornDamageCalculatorDecorator(), and doubleDamage().
The documentation for this class was generated from the following file:

- gwent/domain/damage/HornDamageCalculatorDecorator.java

5.27 ImageViewSwitchAnimator Class Reference

A class providing functionality for changing the resource shown by an ImageView using a fading animation.
Collaboration diagram for ImageViewSwitchAnimator:



Static Public Member Functions

- static Completable animatedSwitch (@NonNull ImageView view, @DrawableRes int resId)

  Changes the resource of the given ImageView in an animation to the given resource id.

Static Private Member Functions

- static ImageView getOverlayView (@NonNull ImageView template)

  Returns a copy of the given template which is one layer above it.

5.27.1 Detailed Description

A class providing functionality for changing the resource shown by an ImageView using a fading animation.

See also

.animatedSwitch(ImageView, int)

Definition at line 20 of file ImageViewSwitchAnimator.java.

5.27.2 Member Function Documentation

5.27.2.1 animatedSwitch()

```
Completable animatedSwitch (
            @NonNull ImageView view,
            @DrawableRes int resId)  [static]
```

Changes the resource of the given ImageView in an animation to the given resource id.
Animation is realized by creating a new ImageView on top of the given ImageView which is then faded out using the `fade_out` animation provided by Android. The operation is complete when the animation ends.

See also

android.R.anim.fade_out

Parameters

view	ImageView, whose resource is to be changed.
res↩ Id	Integer representing the drawable resource that the given view should show.

Returns

A Completable tracking operation status.

Definition at line 53 of file ImageViewSwitchAnimator.java.
References getOverlayView().
Referenced by MenuUiStateObserver.accept(), RowUiStateObserver.accept(), and FactionSwitchListener.onSharedPreferenceChang

5.27.2.2 getOverlayView()

```
ImageView getOverlayView (
            @NonNull ImageView template)  [static], [private]
```

Returns a copy of the given template which is one layer above it.

Parameters

template	ImageView that is copied.

Returns

An ImageView similar to template but one layer in front of it.

Definition at line 27 of file ImageViewSwitchAnimator.java.
Referenced by animatedSwitch().
The documentation for this class was generated from the following file:

- gwent/ui/main/ImageViewSwitchAnimator.java

5.28 IndicatorManager Class Reference

A class managing the indicator views shown in the bottom bar of the IntroductionActivity.
Collaboration diagram for IndicatorManager:



Package Functions

- IndicatorManager (@NonNull Window window)

Constructor of an IndicatorManager for the given Window.

- void updateIndicators (@IntRange(from=0, to=4) int position)

  Updates the indicator views according to the given new position.

Private Attributes

- final ImageView[ ] indicators = new ImageView[5]

  Array of ImageViews that contains the indicator views.

- int currentView = 0

  Integer referencing the position, i.e.

5.28.1 Detailed Description

A class managing the indicator views shown in the bottom bar of the IntroductionActivity.
The managed views show the user how many pages there are and which one they are on.
Definition at line 15 of file IndicatorManager.java.

5.28.2 Constructor & Destructor Documentation

5.28.2.1 IndicatorManager()

```
IndicatorManager (
            @NonNull Window window)  [package]
```

Constructor of an IndicatorManager for the given Window.
Initializes values in indicators.

Parameters

window	Window containing the indicator views.

See also

Window.findViewById(int)

R.id.introduction_indicator_0

R.id.introduction_indicator_1

R.id.introduction_indicator_2

R.id.introduction_indicator_3

R.id.introduction_indicator_4

Definition at line 43 of file IndicatorManager.java.
References IndicatorManager(), and indicators.
Referenced by IndicatorManager().

5.28.3 Member Function Documentation

5.28.3.1 updateIndicators()

```
void updateIndicators (
            @IntRange(from=0, to=4) int position)  [package]
```

Updates the indicator views according to the given new position.
Sets the currentView to R.drawableindicator_unselected and the view at the new position to R.drawableindicator↩
_selected. Saves the given position in currentView.

Parameters

position	Integer referencing the page represented by the managed indicators.

Exceptions

ArrayIndexOutOfBoundsException	When position is not in [0, 4].

Definition at line 59 of file IndicatorManager.java.
References currentView, and indicators.

5.28.4 Member Data Documentation

5.28.4.1 currentView

```
int currentView = 0  [private]
```

Integer referencing the position, i.e.
the index in indicators, of the view that is currently shown as selected. Is set in updateIndicators(int).

See also

.updateIndicators(int)

Definition at line 30 of file IndicatorManager.java.
Referenced by updateIndicators().

5.28.4.2 indicators

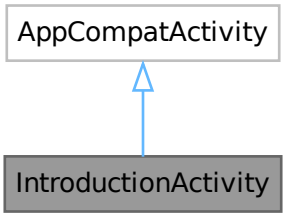
```
final ImageView [] indicators = new ImageView[5]  [private]
```

Array of ImageViews that contains the indicator views.
Definition at line 20 of file IndicatorManager.java.
Referenced by IndicatorManager(), and updateIndicators().
The documentation for this class was generated from the following file:

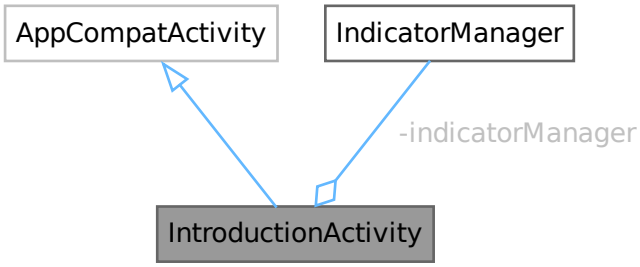
- gwent/ui/introduction/IndicatorManager.java

5.29 IntroductionActivity Class Reference

An AppCompatActivity that gives the user an introduction into the usage of the application.
Inheritance diagram for IntroductionActivity:



Collaboration diagram for IntroductionActivity:



Protected Member Functions

- void onCreate (Bundle savedInstanceState)

  Sets layout to R.layout.activity_introduction, initializes all attributes (viewPager, callback, skipButton, nextButton, finishButton, indicatorManager), sets listeners on the buttons, and creates a new OnBackPressedCallback that switches to the previous page, if it is not the first one.
- void onResume ()

  Refreshes the bottom bar for the currently selected page when the activity resumes.

Private Member Functions

- void updateBottomBar (int position)

  Updates the progress indicators, button visibility, and back-button callback for the selected introduction page.

Private Attributes

- IndicatorManager indicatorManager

  IndicatorManager used to update the progress indicators according to the currently shown page.
- ViewPager2 viewPager

  The ViewPager2 used to display the introduction-fragments.
- OnBackPressedCallback callback

  The OnBackPressedCallback handling presses of the back-button.
- Button skipButton

  The skip button.
- ImageButton nextButton

  The next button.
- Button finishButton

  The finish button.

5.29.1 Detailed Description

An AppCompatActivity that gives the user an introduction into the usage of the application.
Is called when the app is first started (as tracked by the androidx.preference.Preference at key R.stringpreference↩
_first_use_key.
Definition at line 20 of file IntroductionActivity.java.

5.29.2 Member Function Documentation

5.29.2.1 onCreate()

```
void onCreate (
            Bundle savedInstanceState)  [protected]
```

Sets layout to R.layout.activity_introduction, initializes all attributes (viewPager, callback, skipButton, nextButton, finishButton, indicatorManager), sets listeners on the buttons, and creates a new OnBackPressedCallback that switches to the previous page, if it is not the first one.

Parameters

savedInstanceState	If the activity is being re-initialized after previously being shut down then this Bundle contains the data it most recently supplied in onSaveInstanceState. ***Note: Otherwise it is null.***

Called when enabled and the user clicks on the back-button. Switches the ViewPager2 containing the Introduction↩ Fragments to the last position.
Definition at line 63 of file IntroductionActivity.java.
References callback, finishButton, indicatorManager, nextButton, skipButton, and viewPager.

5.29.2.2 onResume()

```
void onResume ()  [protected]
```

Refreshes the bottom bar for the currently selected page when the activity resumes.
Definition at line 116 of file IntroductionActivity.java.
References updateBottomBar(), and viewPager.

5.29.2.3 updateBottomBar()

```
void updateBottomBar (
            int position)  [private]
```

Updates the progress indicators, button visibility, and back-button callback for the selected introduction page.

Parameters

position	Position index of the selected page.

See also

IndicatorManager.updateIndicators(int)

Definition at line 128 of file IntroductionActivity.java.
References callback, finishButton, indicatorManager, nextButton, IntroductionFragment.PAGES_COUNT, and skipButton.
Referenced by onResume().

5.29.3 Member Data Documentation

5.29.3.1 callback

```
OnBackPressedCallback callback  [private]
```

The OnBackPressedCallback handling presses of the back-button.
Definition at line 35 of file IntroductionActivity.java.
Referenced by onCreate(), and updateBottomBar().

5.29.3.2 finishButton

```
Button finishButton  [private]
```

The finish button.
Definition at line 50 of file IntroductionActivity.java.
Referenced by onCreate(), and updateBottomBar().

5.29.3.3 indicatorManager

```
IndicatorManager indicatorManager  [private]
```

IndicatorManager used to update the progress indicators according to the currently shown page.
Is initialized in onCreate(Bundle).
Definition at line 25 of file IntroductionActivity.java.
Referenced by onCreate(), and updateBottomBar().

5.29.3.4 nextButton

```
ImageButton nextButton  [private]
```

The next button.
Definition at line 45 of file IntroductionActivity.java.
Referenced by onCreate(), and updateBottomBar().

5.29.3.5 skipButton

```
Button skipButton  [private]
```

The skip button.
Definition at line 40 of file IntroductionActivity.java.
Referenced by onCreate(), and updateBottomBar().

5.29.3.6 viewPager

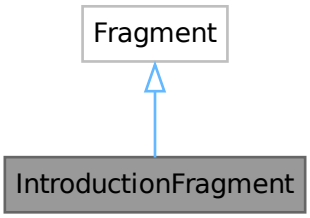
```
ViewPager2 viewPager  [private]
```

The ViewPager2 used to display the introduction-fragments.
Definition at line 30 of file IntroductionActivity.java.
Referenced by onCreate(), and onResume().
The documentation for this class was generated from the following file:

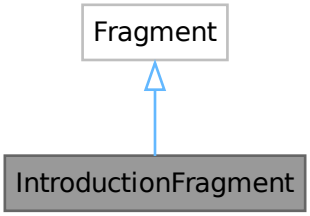
- gwent/ui/introduction/IntroductionActivity.java

5.30 IntroductionFragment Class Reference

A Fragment containing an ImageView and a TextView conveying introductory information.
Inheritance diagram for IntroductionFragment:



Collaboration diagram for IntroductionFragment:



Public Member Functions

- View onCreateView (@NonNull LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState)

  Inflates the R.layoutfragment_introduction layout and sets text and image.

Static Public Member Functions

- static IntroductionFragment newInstance (@IntRange(from=0, to=PAGES_COUNT - 1) int index)

  Creates a new IntroductionFragment showing the page defined by the given index.

Static Public Attributes

- static final int PAGES_COUNT = 5

  Integer constant defining the number of possible pages represented by this fragment.

Static Private Attributes

- static final String ARG_SECTION_NUMBER = "section_number"

  String constant defining the argument key used to communicate the requested page to the fragment.
- static final int[ ] imageIds

  Integer array constant containing the resource ids for all introductory images.
- static final int[ ] stringIds

  Integer array constant containing the resource ids for all introductory texts.

5.30.1 Detailed Description

A Fragment containing an ImageView and a TextView conveying introductory information.
Definition at line 21 of file IntroductionFragment.java.

5.30.2 Member Function Documentation

5.30.2.1 newInstance()

```
IntroductionFragment newInstance (
            @IntRange(from=0, to=PAGES_COUNT - 1) int index)  [static]
```

Creates a new IntroductionFragment showing the page defined by the given index.
Factory method of IntroductionFragment.

Parameters

index	Integer defining the page that the created fragment shows.

Returns

An IntroductionFragment showing the page with the given index.

Exceptions

IllegalArgumentException	When the given index is not in [0, PAGES_COUNT - 1].

Definition at line 50 of file IntroductionFragment.java.
References ARG_SECTION_NUMBER, and PAGES_COUNT.
Referenced by SectionsPagerAdapter.createFragment().

5.30.2.2 onCreateView()

```
View onCreateView (
            @NonNull LayoutInflater inflater,
            ViewGroup container,
            Bundle savedInstanceState)
```

Inflates the R.layoutfragment_introduction layout and sets text and image.

Parameters

inflater	The LayoutInflater object that can be used to inflate any views in the fragment,
container	If non-null, this is the parent view that the fragment's UI should be attached to. The fragment should not add the view itself, but this can be used to generate the Layout↩ Params of the view.
savedInstanceState	If non-null, this fragment is being re-constructed from a previous saved state as given here.

Returns

A View that is the root of the newly inflated layout.

Definition at line 75 of file IntroductionFragment.java.
References ARG_SECTION_NUMBER, imageIds, and stringIds.

5.30.3 Member Data Documentation

5.30.3.1 ARG_SECTION_NUMBER

```
final String ARG_SECTION_NUMBER = "section_number"  [static], [private]
```

String constant defining the argument key used to communicate the requested page to the fragment.
Definition at line 30 of file IntroductionFragment.java.
Referenced by newInstance(), and onCreateView().

5.30.3.2 imageIds

```
final int [] imageIds  [static], [private]
```

Initial value:

```
= {R.drawable.introduction_1, R.drawable.introduction_2,
            R.drawable.introduction_3, R.drawable.introduction_4, R.drawable.introduction_5}
```

Integer array constant containing the resource ids for all introductory images.
Definition at line 34 of file IntroductionFragment.java.
Referenced by onCreateView().

5.30.3.3 PAGES_COUNT

```
final int PAGES_COUNT = 5  [static]
```

Integer constant defining the number of possible pages represented by this fragment.
Is equivalent to the length of stringIds and imageIds.
Definition at line 26 of file IntroductionFragment.java.
Referenced by SectionsPagerAdapter.getItemCount(), newInstance(), and IntroductionActivity.updateBottomBar().

5.30.3.4 stringIds

```
final int [] stringIds  [static], [private]
```

Initial value:

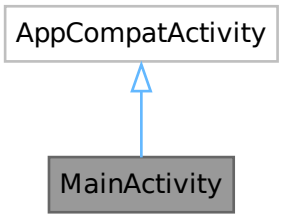
```
= {R.string.introduction_page1, R.string.introduction_page2,
           R.string.introduction_page3, R.string.introduction_page4, R.string.introduction_page5}
```

Integer array constant containing the resource ids for all introductory texts.
Definition at line 39 of file IntroductionFragment.java.
Referenced by onCreateView().
The documentation for this class was generated from the following file:

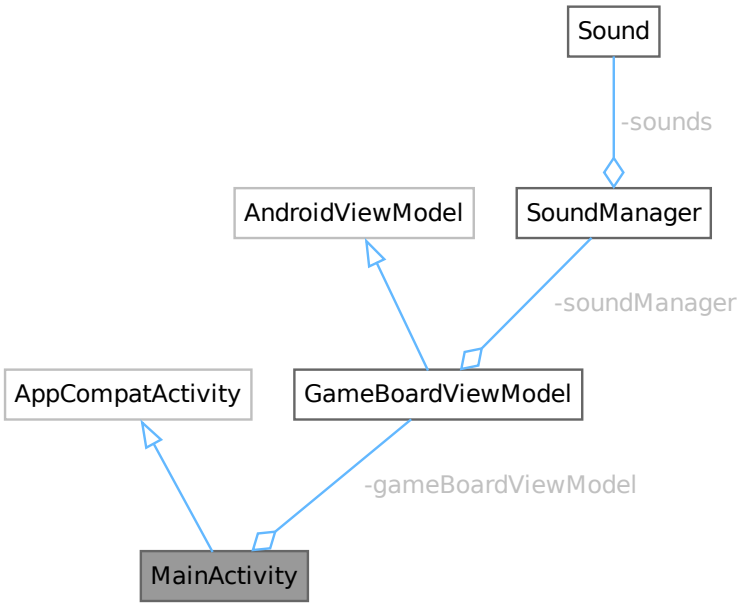
- gwent/ui/introduction/IntroductionFragment.java

5.31 MainActivity Class Reference

An AppCompatActivity that is called on startup and that encapsulates the main view onto the game board.
Inheritance diagram for MainActivity:



Collaboration diagram for MainActivity:



Public Member Functions

- void onWindowFocusChanged (boolean hasFocus)

  Called when the focus of this activities android.view.Window changes.

Protected Member Functions

- void onCreate (@Nullable Bundle savedInstanceState)

  Sets the theme and layout, initializes gameBoardViewModel and factionSwitchListener and sets listeners for some menu buttons.
- void onResume ()

  Called when the application is resumed after a pause or on startup.
- void onDestroy ()

  Called when the activity is destroyed.

Private Member Functions

- void initializeViewModel ()

  Initializes the Views in this activity to communicate with the gameBoardViewModel.
- void inflateFactionPopup ()

  Shows a new ChangeFactionDialog enabling the user to choose the preferred theme.
- void inflateCoinFlipPopup ()

  Shows a new CoinFlipDialog and plays a coin-flip sound using SoundManagerplayCoinSound().

Private Attributes

- final CompositeDisposable disposables = new CompositeDisposable()

CompositeDisposable used to store all io.reactivex.rxjava3.disposables.Disposables, this activity might create to allow for their disposal in onDestroy().

- SharedPreferences.OnSharedPreferenceChangeListener factionSwitchListener

  SharedPreferences.OnSharedPreferenceChangeListener that is called when faction-layout, i.e., the androidx.preference.Preference at the key defined by com.peternaggschga.gwent.ui.main.FactionSwitchListenerTHEME_PREFERENCE_KEY, is changed.

- GameBoardViewModel gameBoardViewModel

  GameBoardViewModel holding the ui state of this activity.

5.31.1 Detailed Description

An AppCompatActivity that is called on startup and that encapsulates the main view onto the game board.
Definition at line 46 of file MainActivity.java.

5.31.2 Member Function Documentation

5.31.2.1 inflateCoinFlipPopup()

```
void inflateCoinFlipPopup () [private]
```

Shows a new CoinFlipDialog and plays a coin-flip sound using SoundManagerplayCoinSound().
Definition at line 274 of file MainActivity.java.
References gameBoardViewModel.
Referenced by onCreate().

5.31.2.2 inflateFactionPopup()

```
void inflateFactionPopup () [private]
```

Shows a new ChangeFactionDialog enabling the user to choose the preferred theme.
The decision is saved in the preference at the key FactionSwitchListenerTHEME_PREFERENCE_KEY and if the user opted to reset on faction switch (i.e., the preference at R.stringpreference_key_faction_reset is true), GameBoardViewModelonFactionSwitchReset(Context) is called.
Definition at line 250 of file MainActivity.java.
References disposables, and gameBoardViewModel.
Referenced by onCreate().

5.31.2.3 initializeViewModel()

```
void initializeViewModel () [private]
```

Initializes the Views in this activity to communicate with the gameBoardViewModel.
Sets android.widget.Button.OnClickListener for the weather-, horn-, and card-views of each row as well as listeners for the reset-, weather- and burn-buttons.
Definition at line 191 of file MainActivity.java.
References disposables, gameBoardViewModel, ShowUnitsDialog.getDialog(), RowUiStateObserver.getObserver(), and GwentApplication.getRepository().
Referenced by onCreate().

5.31.2.4 onCreate()

```
void onCreate (
            @Nullable Bundle savedInstanceState) [protected]
```

Sets the theme and layout, initializes gameBoardViewModel and factionSwitchListener and sets listeners for some menu buttons.
If the application is started for the first time (as tracked by the preference at key R.stringpreference_first_use_key) the IntroductionActivity is called first. The theme is set according to the preference at the key specified by FactionSwitchListenerTHEME_PREFERENCE_KEY. The layout is set to R.layoutactivity_main. android.widget.Button.OnClickListeners are set for the buttons referenced by R.idfactionButton, R.idcoinButton, and R.idsettingsButton.

Parameters

savedInstanceState	If the activity is being re-initialized after previously being shut down then this Bundle contains the data it most recently supplied in onSaveInstanceState. ***Note: Otherwise it is null.***

Definition at line 83 of file MainActivity.java.
References disposables, factionSwitchListener, gameBoardViewModel, FactionSwitchListener.getListener(), GameBoardViewModel.getModel(), GwentApplication.getRepository(), inflateCoinFlipPopup(), inflateFactionPopup(), initializeViewModel(), and FactionSwitchListener.setTheme().

5.31.2.5 onDestroy()

```
void onDestroy () [protected]
```

Called when the activity is destroyed.
Disposes and clears all io.reactivex.rxjava3.disposables.Disposables in disposables.
Definition at line 180 of file MainActivity.java.
References disposables.

5.31.2.6 onResume()

```
void onResume () [protected]
```

Called when the application is resumed after a pause or on startup.
Sets the background image according to the preference at the key referenced by R.stringpreference_key_design.
Definition at line 124 of file MainActivity.java.

5.31.2.7 onWindowFocusChanged()

```
void onWindowFocusChanged (
            boolean hasFocus)
```

Called when the focus of this activities android.view.Window changes.
Hides system ui and sets flags to keep the screen on when the window is in focus for more than 250 ms.

Parameters

hasFocus	Whether the window of this activity has focus.

Definition at line 154 of file MainActivity.java.

5.31.3 Member Data Documentation

5.31.3.1 disposables

```
final CompositeDisposable disposables = new CompositeDisposable() [private]
```

CompositeDisposable used to store all io.reactivex.rxjava3.disposables.Disposables, this activity might create to allow for their disposal in onDestroy().
Definition at line 52 of file MainActivity.java.
Referenced by inflateFactionPopup(), initializeViewModel(), onCreate(), and onDestroy().

5.31.3.2 factionSwitchListener

```
SharedPreferences.OnSharedPreferenceChangeListener factionSwitchListener [private]
```

SharedPreferences.OnSharedPreferenceChangeListener that is called when faction-layout, i.e., the androidx.↩ preference.Preference at the key defined by com.peternaggschga.gwent.ui.main.FactionSwitchListenerTHEME_↩ PREFERENCE_KEY, is changed.
Reference must be kept (even if not used) to avoid garbage collection of the registered listener (see here for more information).
Definition at line 61 of file MainActivity.java.

Referenced by onCreate().

5.31.3.3 gameBoardViewModel

`GameBoardViewModel gameBoardViewModel [private]`
GameBoardViewModel holding the ui state of this activity.
Definition at line 66 of file MainActivity.java.
Referenced by inflateCoinFlipPopup(), inflateFactionPopup(), initializeViewModel(), and onCreate().
The documentation for this class was generated from the following file:

- gwent/ui/main/MainActivity.java

5.32 MenuUiState Class Reference

A data class encapsulating the visible state of the menu in the main view of the application.
Collaboration diagram for MenuUiState:



Public Member Functions

- MenuUiState (@IntRange(from=0) int damage, boolean reset, boolean weather, boolean burn)

  Constructor of a MenuUiState encapsulating the given data.
- int getDamage ()

  Returns the summed-up damage of all units on the game board.
- boolean isReset ()

  Returns whether the reset button is clickable.
- boolean isWeather ()

  Returns whether the weather button is clickable.
- boolean isBurn ()

  Returns whether the burn button is clickable.
- boolean equals (Object o)

  Indicates whether some other object is "equal to" this one.

Private Attributes

- final int damage

  Defines the summed-up damage of all units on the game board.
- final boolean reset

  Defines whether the reset button is clickable.
- final boolean weather

  Defines whether the weather button is clickable.
- final boolean burn

  Defines whether the burn button is clickable.

5.32.1 Detailed Description

A data class encapsulating the visible state of the menu in the main view of the application.
Definition at line 10 of file MenuUiState.java.

5.32.2 Constructor & Destructor Documentation

5.32.2.1 MenuUiState()

```
MenuUiState (
            @IntRange(from=0) int damage,
            boolean reset,
            boolean weather,
            boolean burn)
```

Constructor of a MenuUiState encapsulating the given data.

Parameters

damage	Integer representing the summed-up damage of all units.
reset	Boolean defining whether the reset button is clickable.
weather	Boolean defining whether the weather button is clickable.
burn	Boolean defining whether the burn button is clickable.

Exceptions

IllegalArgumentException	When damage is negative.

Definition at line 41 of file MenuUiState.java.
References MenuUiState(), burn, damage, reset, and weather.
Referenced by MenuUiState(), and equals().

5.32.3 Member Function Documentation

5.32.3.1 equals()

```
boolean equals (
            Object o)
```

Indicates whether some other object is "equal to" this one.

Parameters

o	the reference object with which to compare.

Returns

`true` if this object is the same as the o argument; `false` otherwise.

Definition at line 95 of file MenuUiState.java.
References MenuUiState(), burn, damage, reset, and weather.

5.32.3.2 getDamage()

```
int getDamage ()
```

Returns the summed-up damage of all units on the game board.

Returns

An Integer representing the summed-up damage of all units.

Definition at line 57 of file MenuUiState.java.
References damage, and getDamage().
Referenced by getDamage().

5.32.3.3 isBurn()

```
boolean isBurn ()
```

Returns whether the burn button is clickable.

Returns

A Boolean defining whether the burn button is clickable.

Definition at line 84 of file MenuUiState.java.
References burn.

5.32.3.4 isReset()

```
boolean isReset ()
```

Returns whether the reset button is clickable.

Returns

A Boolean defining whether the reset button is clickable.

Definition at line 66 of file MenuUiState.java.
References reset.

5.32.3.5 isWeather()

```
boolean isWeather ()
```

Returns whether the weather button is clickable.

Returns

A Boolean defining whether the weather button is clickable.

Definition at line 75 of file MenuUiState.java.
References weather.

5.32.4 Member Data Documentation

5.32.4.1 burn

```
final boolean burn  [private]
```

Defines whether the burn button is clickable.
Definition at line 30 of file MenuUiState.java.
Referenced by MenuUiState(), equals(), and isBurn().

5.32.4.2 damage

```
final int damage  [private]
```

Defines the summed-up damage of all units on the game board.
Definition at line 15 of file MenuUiState.java.
Referenced by MenuUiState(), equals(), and getDamage().

5.32.4.3 reset

```
final boolean reset  [private]
```

Defines whether the reset button is clickable.
Definition at line 20 of file MenuUiState.java.
Referenced by MenuUiState(), equals(), and isReset().

5.32.4.4 weather

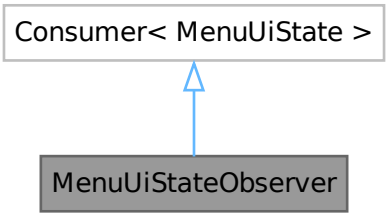
```
final boolean weather  [private]
```

Defines whether the weather button is clickable.
Definition at line 25 of file MenuUiState.java.
Referenced by MenuUiState(), equals(), and isWeather().
The documentation for this class was generated from the following file:

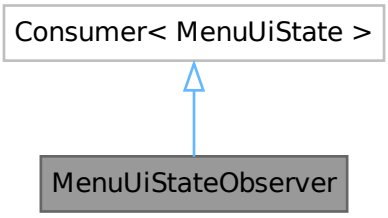
- gwent/ui/main/MenuUiState.java

5.33 MenuUiStateObserver Class Reference

An observer class responsible for updating the menu views when notified, i.e., when a new MenuUiState is produced by the subscribed Flowable.
Inheritance diagram for MenuUiStateObserver:



Collaboration diagram for MenuUiStateObserver:



Public Member Functions

- MenuUiStateObserver (@NonNull TextView damageView, @NonNull ImageButton resetButton, @NonNull ImageButton weatherButton, @NonNull ImageButton burnButton)

  Constructor of a MenuUiStateObserver updating the given views when .accept() is called.
- void accept (@NonNull MenuUiState menuUiState)

  Changes value in damageView as well as appearance and clickable status of resetButton, weatherButton, and burn↩ Button.

Private Attributes

- final TextView damageView

  A TextView showing the user the summed-up damage of all units, i.e., MenuUiState.damage.
- final ImageButton resetButton

  An ImageButton responsible for resetting the whole board, i.e., clearing weather and commander's horn and deleting all units.
- final ImageButton weatherButton

  An ImageButton responsible for clearing all weather effects.
- final ImageButton burnButton

  An ImageButton responsible for deleting the units with the highest damage.

5.33.1 Detailed Description

An observer class responsible for updating the menu views when notified, i.e., when a new MenuUiState is produced by the subscribed Flowable.

See also

MenuUiState

Definition at line 18 of file MenuUiStateObserver.java.

5.33.2 Constructor & Destructor Documentation

5.33.2.1 MenuUiStateObserver()

```
MenuUiStateObserver (
            @NonNull TextView damageView,
            @NonNull ImageButton resetButton,
            @NonNull ImageButton weatherButton,
            @NonNull ImageButton burnButton)
```

Constructor of a MenuUiStateObserver updating the given views when .accept() is called.

Parameters

damageView	TextView showing the summed-up damage of all units.
resetButton	ImageButton used to reset the whole game board.
weatherButton	ImageButton used to reset the weather debuff in all rows.
burnButton	ImageButton used to remove the strongest units from the game board.

Definition at line 55 of file MenuUiStateObserver.java.
References burnButton, damageView, resetButton, and weatherButton.

5.33.3 Member Function Documentation

5.33.3.1 accept()

```
void accept (
            @NonNull MenuUiState menuUiState)
```

Changes value in damageView as well as appearance and clickable status of resetButton, weatherButton, and burnButton.
Appearance changes are animated using ImageViewSwitchAnimator.

Parameters

menuUiState	MenuUiState representing the updated state of the menu.

See also

ImageViewSwitchAnimator

Definition at line 72 of file MenuUiStateObserver.java.
References ImageViewSwitchAnimator.animatedSwitch(), burnButton, damageView, resetButton, and weatherButton.

5.33.4 Member Data Documentation

5.33.4.1 burnButton

```
final ImageButton burnButton  [private]
```

An ImageButton responsible for deleting the units with the highest damage.
Is not clickable and gray when MenuUiState.burn is `false`.
Definition at line 45 of file MenuUiStateObserver.java.
Referenced by MenuUiStateObserver(), and accept().

5.33.4.2 damageView

```
final TextView damageView  [private]
```

A TextView showing the user the summed-up damage of all units, i.e., MenuUiState.damage.
Definition at line 23 of file MenuUiStateObserver.java.
Referenced by MenuUiStateObserver(), and accept().

5.33.4.3 resetButton

```
final ImageButton resetButton  [private]
```

An ImageButton responsible for resetting the whole board, i.e., clearing weather and commander's horn and deleting all units.
Is not clickable and gray when MenuUiState.reset is `false`.
Definition at line 31 of file MenuUiStateObserver.java.
Referenced by MenuUiStateObserver(), and accept().

5.33.4.4 weatherButton

```
final ImageButton weatherButton  [private]
```

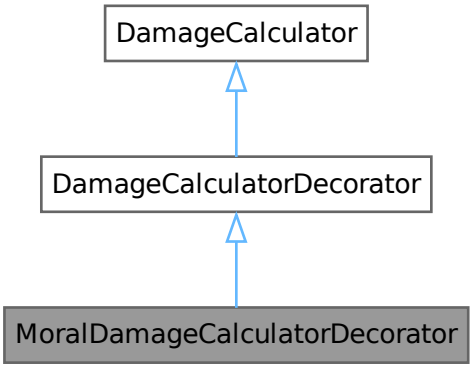
An ImageButton responsible for clearing all weather effects.
Is not clickable and gray when MenuUiState.weather is `false`.
Definition at line 38 of file MenuUiStateObserver.java.
Referenced by MenuUiStateObserver(), and accept().
The documentation for this class was generated from the following file:

- gwent/ui/main/MenuUiStateObserver.java

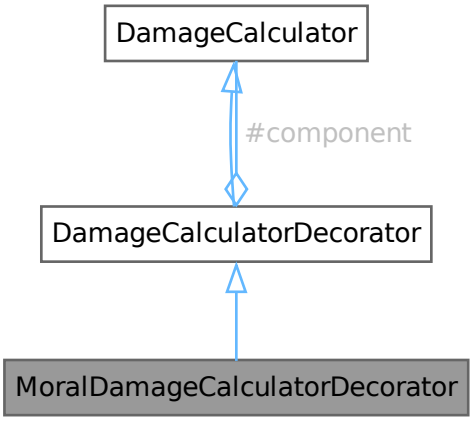
5.34 MoralDamageCalculatorDecorator Class Reference

A DamageCalculator class responsible for calculating the moral buff if necessary, i.e., if there are units with the com.peternaggschga.gwent.data.Ability.MORAL_BOOST ability that are not the unit the damage is calculated for.

Inheritance diagram for MoralDamageCalculatorDecorator:



Collaboration diagram for MoralDamageCalculatorDecorator:



Public Member Functions

- int calculateDamage (int id, @IntRange(from=0) int damage)

  Calculates the (de-)buffed damage of unit with the given id and the given base-damage.
- Color isBuffed (int id)

  Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT.

Package Functions

- MoralDamageCalculatorDecorator (@NonNull DamageCalculator component, @NonNull List< Integer > unitIds)

  Constructor of a MoralDamageCalculatorDecorator.

Package Functions inherited from DamageCalculatorDecorator

- DamageCalculatorDecorator (@NonNull DamageCalculator component)

  Constructor of a DamageCalculatorDecorator.

Private Attributes

- final List< Integer > unitIds

  A List of Integers containing the ids of all units with the com.peternaggschga.gwent.data.Ability.MORAL_BOOST ability.

Additional Inherited Members

Protected Attributes inherited from DamageCalculatorDecorator

- final DamageCalculator component

  A DamageCalculator which is decorated by this decorator.

5.34.1 Detailed Description

A DamageCalculator class responsible for calculating the moral buff if necessary, i.e., if there are units with the com.peternaggschga.gwent.data.Ability.MORAL_BOOST ability that are not the unit the damage is calculated for.
Acts as a ConcreteDecorator in the implemented decorator pattern and should decorate a WeatherDamageCalculator or a BondDamageCalculatorDecorator for correct damage calculation.
Definition at line 17 of file MoralDamageCalculatorDecorator.java.

5.34.2 Constructor & Destructor Documentation

5.34.2.1 MoralDamageCalculatorDecorator()

```
MoralDamageCalculatorDecorator (
            @NonNull DamageCalculator component,
            @NonNull List< Integer > unitIds)  [package]
```

Constructor of a MoralDamageCalculatorDecorator.
Should only be called by DamageCalculatorBuilder. The given damage calculator should be of type WeatherDamageCalculator or BondDamageCalculatorDecorator for correct damage calculation. The given List must not contain `null`.

Parameters

component	DamageCalculator that is being decorated by this decorator.
unitIds	List of Integers representing ids of units with the com.peternaggschga.gwent.data.Ability.MORAL_BOOST ability.

Exceptions

IllegalArgumentException	When unitIds contains null values.

See also

DamageCalculatorBuilder

Definition at line 36 of file MoralDamageCalculatorDecorator.java.
References DamageCalculatorDecorator.component, and unitIds.

5.34.3 Member Function Documentation

5.34.3.1 calculateDamage()

```
int calculateDamage (
            int id,
            @IntRange(from=0) int damage)
```

Calculates the (de-)buffed damage of unit with the given id and the given base-damage.
Returns the given damage plus the number of moral boosts. If the unit itself has the com.peternaggschga.gwent.data.Ability.MORAL_B ability, it is boosted one time less.

Parameters

id	Integer representing the UnitEntity.id of the unit whose (de-)buff damage is calculated.
damage	Integer representing the base-damage of the unit whose (de-)buff damage is calculated.

Returns

Integer representing the (de-)buffed damage of the unit.

Exceptions

IllegalArgumentException	When damage is negative.

Implements DamageCalculator.
Definition at line 56 of file MoralDamageCalculatorDecorator.java.
References DamageCalculatorDecorator.component, and unitIds.

5.34.3.2 isBuffed()

```
Color isBuffed (
            int id)
```

Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT.
Units are shown as Color.BUFFED when they are affected by a moral boost buff, otherwise their Color is defined by .component.

Parameters

id	Integer representing the UnitEntity.id of the unit buff status is calculated.

Returns

Color representing whether the unit is buffed, de-buffed or not affected.

See also

Color

Implements DamageCalculator.
Definition at line 75 of file MoralDamageCalculatorDecorator.java.
References DamageCalculatorDecorator.component, and unitIds.

5.34.4 Member Data Documentation

5.34.4.1 unitIds

```
final List<Integer> unitIds  [private]
```

A List of Integers containing the ids of all units with the com.peternaggschga.gwent.data.Ability.MORAL_BOOST ability.
Definition at line 22 of file MoralDamageCalculatorDecorator.java.
Referenced by MoralDamageCalculatorDecorator(), calculateDamage(), and isBuffed().
The documentation for this class was generated from the following file:

- gwent/domain/damage/MoralDamageCalculatorDecorator.java

5.35 ValuePicker< T extends Comparable >.OnValueChangeListener< T extends Comparable > Interface Template Reference

An interface defining a callback for changes of the current value.
Collaboration diagram for ValuePicker< T extends Comparable >.OnValueChangeListener< T extends Comparable >:



Public Member Functions

- void onValueChange (@NonNull ValuePicker< T > picker, T oldVal, T newVal)

  Called upon a change of the current value.

5.35.1 Detailed Description

An interface defining a callback for changes of the current value.
Definition at line 211 of file ValuePicker.java.

5.35.2 Member Function Documentation

5.35.2.1 onValueChange()

```
void onValueChange (
            @NonNull ValuePicker< T > picker,
            T oldVal,
            T newVal)
```

Called upon a change of the current value.

Parameters

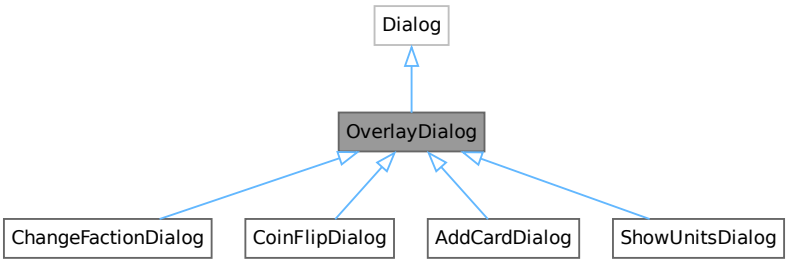
picker	ValuePicker associated with this listener.
oldVal	Value that was previously selected.
newVal	Value that was newly selected.

References ValuePicker< T extends Comparable >.ValuePicker(), and ValuePicker< T extends Comparable >.picker.
The documentation for this interface was generated from the following file:

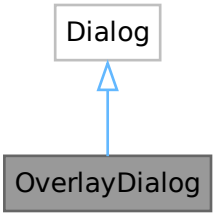
- gwent/ui/dialogs/addcard/ValuePicker.java

5.36 OverlayDialog Class Reference

A Dialog class which is used for popups that are shown on top of the calling Activity.
Inheritance diagram for OverlayDialog:



Collaboration diagram for OverlayDialog:



Protected Member Functions

- OverlayDialog (@NonNull Context context, @LayoutRes int layout, @IdRes int dismissViewId)

  Constructor of an OverlayDialog in the given Context, with the given layout and dismissViewId.
- OverlayDialog (@NonNull Context context, @LayoutRes int layout)

  Constructor of an OverlayDialog in the given Context and with the given layout.
- void onCreate (Bundle savedInstanceState)

  Initializes .layout using .setContentView(int).

Static Package Attributes

- static final int NO_DISMISS_VIEW = -1

  Integer used as .dismissViewId, when the Dialog shouldn't be cancelable by clicking on a View.

Private Attributes

- final int layout

  Integer referencing the layout shown by the Dialog.
- final int dismissViewId

  Integer referencing a view that can be clicked to dismiss the Dialog.

Static Private Attributes

- static final ColorDrawable BACKGROUND = new ColorDrawable(Color.TRANSPARENT)

  ColorDrawable shown as the background of the Dialog (above the calling Activity).

5.36.1 Detailed Description

A Dialog class which is used for popups that are shown on top of the calling Activity.
The Dialog uses the layout specified in .layout with the .BACKGROUND color as the background. The view specified by .dismissViewId may be clicked to dismiss the dialog.
Definition at line 22 of file OverlayDialog.java.

5.36.2 Constructor & Destructor Documentation

5.36.2.1 OverlayDialog() [1/2]

```
OverlayDialog (
            @NonNull Context context,
            @LayoutRes int layout,
            @IdRes int dismissViewId)  [protected]
```

Constructor of an OverlayDialog in the given Context, with the given layout and dismissViewId.
When dismissViewId is equal to .NO_CANCEL_VIEW, the Dialog is not cancelable by clicking on a certain view.

Parameters

context	Context of the created OverlayDialog.
layout	Integer referencing the layout shown by the created OverlayDialog.
dismiss↩ ViewId	Integer referencing the cancel view or .NO_CANCEL_VIEW.

Definition at line 54 of file OverlayDialog.java.
References dismissViewId, and layout.
Referenced by OverlayDialog().

5.36.2.2 OverlayDialog() [2/2]

```
OverlayDialog (
            @NonNull Context context,
            @LayoutRes int layout)  [protected]
```

Constructor of an OverlayDialog in the given Context and with the given layout.
Wrapper of .OverlayDialog(Context, int, int).

See also

.OverlayDialog(Context, int, int)

Parameters

context	Context of the created OverlayDialog.
layout	Integer referencing the layout shown by the created OverlayDialog.

Definition at line 68 of file OverlayDialog.java.
References OverlayDialog(), layout, and NO_DISMISS_VIEW.

5.36.3 Member Function Documentation

5.36.3.1 onCreate()

```
void onCreate (
            Bundle savedInstanceState)  [protected]
```

Initializes .layout using .setContentView(int).
Sets View.OnClickListener canceling the dialog for the View referenced by .dismissViewId if it is set. Switches whether the Dialog is cancelable using .setCancelable().

Parameters

savedInstanceState	If this dialog is being reinitialized after the hosting activity was previously shut down, holds the result from the most recent call to onSaveInstanceState, or null if this is the first time.

See also

.setContentView(int)

.setCancelable(boolean)

Reimplemented in AddCardDialog, ShowUnitsDialog, ChangeFactionDialog, and CoinFlipDialog.
Definition at line 85 of file OverlayDialog.java.
References BACKGROUND, dismissViewId, layout, and NO_DISMISS_VIEW.

5.36.4 Member Data Documentation

5.36.4.1 BACKGROUND

```
final ColorDrawable BACKGROUND = new ColorDrawable(Color.TRANSPARENT)  [static], [private]
```

ColorDrawable shown as the background of the Dialog (above the calling Activity).
Definition at line 33 of file OverlayDialog.java.
Referenced by onCreate().

5.36.4.2 dismissViewId

```
final int dismissViewId  [private]
```

Integer referencing a view that can be clicked to dismiss the Dialog.
Should be equal to .NO_CANCEL_VIEW, when the Dialog is not cancelable.

See also

.NO_DISMISS_VIEW

Definition at line 45 of file OverlayDialog.java.
Referenced by OverlayDialog(), and onCreate().

5.36.4.3 layout

```
final int layout  [private]
```

Integer referencing the layout shown by the Dialog.
Definition at line 38 of file OverlayDialog.java.
Referenced by OverlayDialog(), OverlayDialog(), and onCreate().

5.36.4.4 NO_DISMISS_VIEW

```
final int NO_DISMISS_VIEW = -1  [static], [package]
```

Integer used as .dismissViewId, when the Dialog shouldn't be cancelable by clicking on a View.

See also

.dismissViewId

Definition at line 28 of file OverlayDialog.java.
Referenced by OverlayDialog(), and onCreate().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/OverlayDialog.java

5.37 RemoveUnitsUseCase Class Reference

A use case class responsible for removing units from a UnitRepository.
Collaboration diagram for RemoveUnitsUseCase:



Static Public Member Functions

- static Completable remove (@NonNull Context context, @NonNull UnitRepository repository, @NonNull Collection< UnitEntity > units, @NonNull SoundManager soundManager)

  Removes the given UnitEntity objects from the given UnitRepository.
- static Completable remove (@NonNull Context context, @NonNull UnitRepository repository, int id, @Non↩ Null SoundManager soundManager)

  Removes the unit with the given id from the given UnitRepository.

Static Private Member Functions

- static Dialog getRevengeDialog (@NonNull Context context, @NonNull UnitRepository repository, @NonNull CompletableEmitter emitter, @NonNull Collection< UnitEntity > units, @IntRange(from=1) int revengeUnits, @NonNull SoundManager soundManager)

  Creates a Dialog asking whether the Ability.REVENGE ability should be activated.

5.37.1 Detailed Description

A use case class responsible for removing units from a UnitRepository.
Capable of invoking a Dialog if a UnitEntity with the Ability.REVENGE ability is removed. Should not be used directly by the UI layer.

See also

BurnDialogUseCase

Definition at line 27 of file RemoveUnitsUseCase.java.

5.37.2 Member Function Documentation

5.37.2.1 getRevengeDialog()

```
Dialog getRevengeDialog (
            @NonNull Context context,
```

```
            @NonNull UnitRepository repository,
            @NonNull CompletableEmitter emitter,
            @NonNull Collection< UnitEntity > units,
            @IntRange(from=1) int revengeUnits,
            @NonNull SoundManager soundManager)  [static], [private]
```

Creates a Dialog asking whether the Ability.REVENGE ability should be activated.
The Dialog is created using an RevengeAlertDialogBuilderAdapter.

Parameters

context	Context of the shown Dialog.
repository	UnitRepository where the UnitEntity objects are removed and avengers are inserted.
emitter	CompletableEmitter where CompletableEmitter.onComplete must be called, when the user makes a decision.
units	Collection of UnitEntity objects that are removed.
revengeUnits	Long representing the number of revenge units.
soundManager	SoundManager used when an Avenger is added.

Returns

A Dialog asking whether the Ability.REVENGE ability should be activated.

See also

RevengeAlertDialogBuilderAdapter.insertAvengers(UnitRepository, int, SoundManager)

Definition at line 90 of file RemoveUnitsUseCase.java.
References RevengeAlertDialogBuilderAdapter.insertAvengers(), and RevengeAlertDialogBuilderAdapter.setPositiveCallback().
Referenced by remove().

5.37.2.2 remove() [1/2]

```
Completable remove (
            @NonNull Context context,
            @NonNull UnitRepository repository,
            @NonNull Collection< UnitEntity > units,
            @NonNull SoundManager soundManager)  [static]
```

Removes the given UnitEntity objects from the given UnitRepository.
If a UnitEntity has the Ability.REVENGE ability, a Dialog asking whether the ability should be used is shown.

Parameters

context	Context of the shown Dialog.
repository	UnitRepository where the UnitEntity objects are removed.
units	Collection of UnitEntity objects that are removed.
soundManager	SoundManager used, if an UnitEntity has the Ability.REVENGE ability.

Returns

A Completable tracking operation status.

Exceptions

NullPointerException	When units contains a null value.

See also

.getRevengeDialog(Context, UnitRepository, CompletableEmitter, Collection, int, SoundManager)

UnitRepository.delete(Collection)

Definition at line 43 of file RemoveUnitsUseCase.java.
References getRevengeDialog(), and Ability.REVENGE.
Referenced by ShowUnitsDialog.getDialog().

5.37.2.3 remove() [2/2]

```
Completable remove (
            @NonNull Context context,
            @NonNull UnitRepository repository,
            int id,
            @NonNull SoundManager soundManager)  [static]
```

Removes the unit with the given id from the given UnitRepository.
If the unit has the Ability.REVENGE ability, a Dialog asking whether the ability should be used is shown. Wrapper of .remove(Context, UnitRepository, Collection).

Parameters

context	Context of the shown Dialog.
repository	UnitRepository where the UnitEntity objects are removed.
id	Integer
soundManager	SoundManager used, if an UnitEntity has the Ability.REVENGE ability.

Returns

A Completable tracking operation status.

See also

.remove(Context, UnitRepository, Collection, SoundManager)

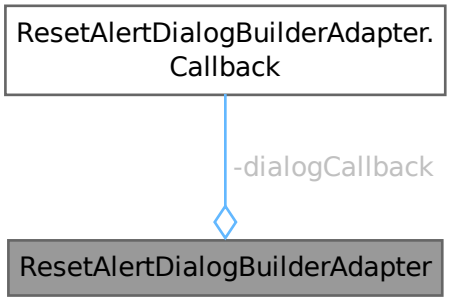
Definition at line 69 of file RemoveUnitsUseCase.java.
The documentation for this class was generated from the following file:

- gwent/domain/cases/RemoveUnitsUseCase.java

5.38 ResetAlertDialogBuilderAdapter Class Reference

An adapter class adapting AlertDialog.Builder to provide an interface for creating an AlertDialog asking the user whether he really wants to reset.

Collaboration diagram for ResetAlertDialogBuilderAdapter:



Classes

- interface Callback

  An interface defining functions to propagate the user's decision back to the creator.

Package Functions

- ResetAlertDialogBuilderAdapter (@NonNull Context context, @NonNull Callback dialogCallback)

  Constructor of a ResetAlertDialogBuilderAdapter.
- AlertDialog create ()

  Creates an AlertDialog with the arguments supplied to this builder.
- ResetAlertDialogBuilderAdapter setTrigger (@NonNull ResetDialogUseCase.Trigger trigger)

  Changes the shown message of the AlertDialog and whether it is cancelable depending on the trigger of the dialog.
- ResetAlertDialogBuilderAdapter setMonsterDialog (boolean monsterDialog)

  Changes the positive button callback depending on whether it is a monsterDialog or not.

Private Attributes

- final AlertDialog.Builder adapteeBuilder

  AlertDialog.Builder that is adapted by this class.
- final Callback dialogCallback

  Callback used to propagate the user's answer to the built AlertDialog back to its creator.

5.38.1 Detailed Description

An adapter class adapting AlertDialog.Builder to provide an interface for creating an AlertDialog asking the user whether he really wants to reset.
Definition at line 16 of file ResetAlertDialogBuilderAdapter.java.

5.38.2 Constructor & Destructor Documentation

5.38.2.1 ResetAlertDialogBuilderAdapter()

```
ResetAlertDialogBuilderAdapter (
            @NonNull Context context,
            @NonNull Callback dialogCallback)  [package]
```

Constructor of a ResetAlertDialogBuilderAdapter.

The given Callback is called when the user responds to the built AlertDialog.

Parameters

context	Context of the built AlertDialog.
dialogCallback	Callback used to propagate the user's answer to the creator.

Definition at line 36 of file ResetAlertDialogBuilderAdapter.java.
References dialogCallback.
Referenced by setMonsterDialog(), and setTrigger().

5.38.3 Member Function Documentation

5.38.3.1 create()

```
AlertDialog create ()  [package]
```

Creates an AlertDialog with the arguments supplied to this builder.
Basically just calls AlertDialog.Builder.create() on .adapteeBuilder.

Returns

The AlertDialog created by the adaptee builder.

See also

AlertDialog.Builder.create()

Definition at line 53 of file ResetAlertDialogBuilderAdapter.java.
References adapteeBuilder.
Referenced by ResetDialogUseCase.reset().

5.38.3.2 setMonsterDialog()

```
ResetAlertDialogBuilderAdapter setMonsterDialog (
            boolean monsterDialog)  [package]
```

Changes the positive button callback depending on whether it is a monsterDialog or not.
When it is a monsterDialog, a checkbox is shown defining whether the perk of monster faction should be activated.

Parameters

monsterDialog	Boolean defining whether a monsterDialog should be shown.

Returns

The ResetAlertDialogBuilderAdapter with the changed trigger.

Definition at line 82 of file ResetAlertDialogBuilderAdapter.java.
References ResetAlertDialogBuilderAdapter(), adapteeBuilder, and dialogCallback.

5.38.3.3 setTrigger()

```
ResetAlertDialogBuilderAdapter setTrigger (
            @NonNull ResetDialogUseCase.Trigger trigger)  [package]
```

Changes the shown message of the AlertDialog and whether it is cancelable depending on the trigger of the dialog.

Parameters

trigger	ResetDialogUseCase.Trigger defining what triggered the reset.

Returns

The ResetAlertDialogBuilderAdapter with the changed trigger.

Definition at line 65 of file ResetAlertDialogBuilderAdapter.java.
References ResetAlertDialogBuilderAdapter(), adapteeBuilder, and ResetDialogUseCase.Trigger.FACTION_SWITCH.
Referenced by ResetDialogUseCase.reset().

5.38.4 Member Data Documentation

5.38.4.1 adapteeBuilder

```
final AlertDialog.Builder adapteeBuilder  [private]
```

AlertDialog.Builder that is adapted by this class.
Definition at line 21 of file ResetAlertDialogBuilderAdapter.java.
Referenced by create(), setMonsterDialog(), and setTrigger().

5.38.4.2 dialogCallback

```
final Callback dialogCallback  [private]
```

Callback used to propagate the user's answer to the built AlertDialog back to its creator.
Definition at line 27 of file ResetAlertDialogBuilderAdapter.java.
Referenced by ResetAlertDialogBuilderAdapter(), and setMonsterDialog().
The documentation for this class was generated from the following file:

- gwent/domain/cases/ResetAlertDialogBuilderAdapter.java

5.39 ResetDialogUseCase Class Reference

A use case class responsible for dispatching a reset call to ResetRepositoryUseCase, possibly after a confirmation by the user obtained from a Dialog.
Collaboration diagram for ResetDialogUseCase:



Classes

- enum Trigger

  An Enum listing the possible triggers of a reset.
- enum DialogType

  An Enum defining which form of Dialog should be shown.

Static Public Member Functions

- static Single< Boolean > reset (@NonNull Context context, @NonNull Trigger trigger, @NonNull SoundManager soundManager)

  Resets the given UnitRepository.

Static Protected Member Functions

- static Single< Boolean > reset (@NonNull Context context, @NonNull UnitRepository repository, @NonNull Trigger trigger, @NonNull SoundManager soundManager)

  Resets the given UnitRepository.

Static Private Member Functions

- static Single< DialogType > getDialogType (@NonNull Context context, @NonNull UnitRepository repository, @NonNull Trigger trigger)

  Returns a DialogType defining which kind of Dialog should be invoked.

5.39.1 Detailed Description

A use case class responsible for dispatching a reset call to ResetRepositoryUseCase, possibly after a confirmation by the user obtained from a Dialog.

See also

ResetRepositoryUseCase

Definition at line 31 of file ResetDialogUseCase.java.

5.39.2 Member Function Documentation

5.39.2.1 getDialogType()

```
Single< DialogType > getDialogType (
            @NonNull Context context,
            @NonNull UnitRepository repository,
            @NonNull Trigger trigger)  [static], [private]
```

Returns a DialogType defining which kind of Dialog should be invoked.

Parameters

context	Context used for retrieval of SharedPreferences.
repository	UnitRepository used to check if a certain DialogType is even necessary.
trigger	Trigger defining what triggered the reset.

Returns

A DialogType defining the kind of Dialog.

See also

DialogType

Definition at line 102 of file ResetDialogUseCase.java.
References ResetDialogUseCase.DialogType.DEFAULT, and ResetDialogUseCase.DialogType.NONE.
Referenced by reset().

5.39.2.2 reset() [1/2]

```
Single< Boolean > reset (
            @NonNull Context context,
            @NonNull Trigger trigger,
            @NonNull SoundManager soundManager)  [static]
```

Resets the given UnitRepository.

May invoke a Dialog asking whether the user really wants to reset depending on the given trigger and warning settings. ResetRepositoryUseCase is used for resetting. Wrapper for .reset(Context, UnitRepository, Trigger).

Parameters

context	Context where a Dialog can be inflated.
trigger	Trigger defining what triggered this reset.
soundManager	SoundManager used, if an UnitEntity has the Ability.REVENGE ability.

Returns

A Single emitting a Boolean defining whether the reset really took place.

See also

.reset(Context, UnitRepository, Trigger, SoundManager)

ResetRepositoryUseCase.reset(Context, UnitRepository, boolean, SoundManager)

Definition at line 47 of file ResetDialogUseCase.java.
References GwentApplication.getRepository(), and reset().
Referenced by reset(), and GameBoardViewModel.reset().

5.39.2.3 reset() [2/2]

```
Single< Boolean > reset (
            @NonNull Context context,
            @NonNull UnitRepository repository,
            @NonNull Trigger trigger,
            @NonNull SoundManager soundManager)  [static], [protected]
```

Resets the given UnitRepository.
May invoke a Dialog asking whether the user really wants to reset depending on the given trigger and warning settings. ResetRepositoryUseCase is used for resetting.

Parameters

context	Context where a Dialog can be inflated.
repository	UnitRepository that is reset.
trigger	Trigger defining what triggered this reset.
soundManager	SoundManager used, if an UnitEntity has the Ability.REVENGE ability.

Returns

A Single emitting a Boolean defining whether the reset really took place.

See also

ResetRepositoryUseCase.reset(Context, UnitRepository, boolean, SoundManager)

Definition at line 66 of file ResetDialogUseCase.java.
References ResetAlertDialogBuilderAdapter.create(), getDialogType(), ResetDialogUseCase.DialogType.MONSTER, ResetDialogUseCase.DialogType.NONE, ResetRepositoryUseCase.reset(), and ResetAlertDialogBuilderAdapter.setTrigger().
The documentation for this class was generated from the following file:

- gwent/domain/cases/ResetDialogUseCase.java

5.40 ResetRepositoryUseCase Class Reference

A use case class responsible for resetting the UnitRepository.
Collaboration diagram for ResetRepositoryUseCase:



Static Public Member Functions

- static Completable reset (@NonNull Context context, @NonNull UnitRepository repository, @NonNull SoundManager soundManager)

  Resets the given UnitRepository.

Static Package Functions

- static Maybe< UnitEntity > reset (@NonNull Context context, @NonNull UnitRepository repository, boolean keepUnit, @NonNull SoundManager soundManager)

  Resets the given UnitRepository and keeps one random unit if keepUnit is true.

Static Private Member Functions

- static Optional< UnitEntity > getRandomUnit (@NonNull List< UnitEntity > units)

  Selects a random unit that is not epic.
- static Dialog getRevengeDialog (@NonNull Context context, @NonNull UnitRepository repository, @NonNull CompletableEmitter emitter, @Nullable UnitEntity keptUnit, @IntRange(from=1) int revengeUnits, @NonNull SoundManager soundManager)

  Creates a Dialog asking whether the Ability.REVENGE ability should be activated.

5.40.1 Detailed Description

A use case class responsible for resetting the UnitRepository.
Capable of invoking a Dialog if a UnitEntity with the Ability.REVENGE ability is removed. Should not be used directly by the UI layer.

See also

ResetDialogUseCase

Definition at line 31 of file ResetRepositoryUseCase.java.

5.40.2 Member Function Documentation

5.40.2.1 getRandomUnit()

```
Optional< UnitEntity > getRandomUnit (
            @NonNull List< UnitEntity > units)  [static], [private]
```

Selects a random unit that is not epic.
If all units are epic or if there are no units at all, an empty Optional is returned.

Parameters

units	List of UnitEntity objects, one of which is selected.

Returns

An Optional containing the selected unit or nothing if no unit could be selected.

Definition at line 71 of file ResetRepositoryUseCase.java.
Referenced by reset().

5.40.2.2 getRevengeDialog()

```
Dialog getRevengeDialog (
            @NonNull Context context,
            @NonNull UnitRepository repository,
            @NonNull CompletableEmitter emitter,
            @Nullable UnitEntity keptUnit,
            @IntRange(from=1) int revengeUnits,
            @NonNull SoundManager soundManager)  [static], [private]
```

Creates a Dialog asking whether the Ability.REVENGE ability should be activated.
The Dialog is created using an RevengeAlertDialogBuilderAdapter.

Parameters

context	Context of the shown Dialog.
repository	UnitRepository where the UnitEntity objects are removed and avengers are inserted.
emitter	CompletableEmitter where CompletableEmitter.onComplete must be called, when the user makes a decision.
keptUnit	UnitEntity that should be kept.
revengeUnits	Long representing the number of revenge units.
soundManager	SoundManager used, if an Avenger is added.

Returns

A Dialog asking whether the Ability.REVENGE ability should be activated.

See also

RevengeAlertDialogBuilderAdapter.insertAvengers(UnitRepository, int, SoundManager)

Definition at line 93 of file ResetRepositoryUseCase.java.
References RevengeAlertDialogBuilderAdapter.insertAvengers(), and RevengeAlertDialogBuilderAdapter.setPositiveCallback().
Referenced by reset().

5.40.2.3 reset() [1/2]

```
Completable reset (
            @NonNull Context context,
            @NonNull UnitRepository repository,
            @NonNull SoundManager soundManager)  [static]
```

Resets the given UnitRepository.
If a removed UnitEntity has the Ability.REVENGE ability, a Dialog asking whether the ability should be used is shown. Wrapper of .reset(Context, UnitRepository, boolean, SoundManager).

Parameters

context	Context of the shown Dialog.
repository	UnitRepository that is being reset.
soundManager	SoundManager used, if an UnitEntity has the Ability.REVENGE ability.

Returns

A Completable tracking operation status.

See also

.reset(Context, UnitRepository, boolean, SoundManager)

Definition at line 123 of file ResetRepositoryUseCase.java.
References reset().

5.40.2.4 reset() [2/2]

```
Maybe< UnitEntity > reset (
            @NonNull Context context,
            @NonNull UnitRepository repository,
            boolean keepUnit,
            @NonNull SoundManager soundManager)  [static], [package]
```

Resets the given UnitRepository and keeps one random unit if keepUnit is true.
If a removed UnitEntity has the Ability.REVENGE ability, a Dialog asking whether the ability should be used is shown.

Parameters

context	Context of the shown Dialog.
repository	UnitRepository that is being reset.
keepUnit	Boolean defining whether a single UnitEntity should be kept.
soundManager	SoundManager used, if an UnitEntity has the Ability.REVENGE ability.

Returns

A Maybe emitting the kept UnitEntity or nothing if keepUnit is false.

See also

.getRevengeDialog(Context, UnitRepository, CompletableEmitter, UnitEntity, int, SoundManager)

UnitRepository.reset(UnitEntity)

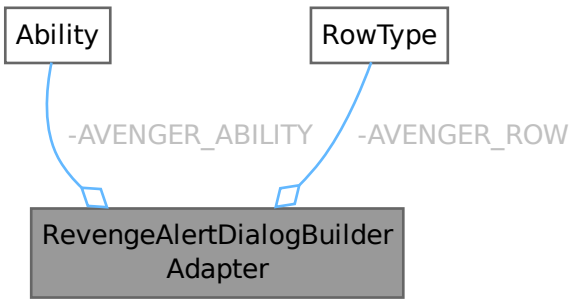
Definition at line 46 of file ResetRepositoryUseCase.java.
References getRandomUnit(), getRevengeDialog(), and Ability.REVENGE.
Referenced by ResetDialogUseCase.reset(), and reset().
The documentation for this class was generated from the following file:

- gwent/domain/cases/ResetRepositoryUseCase.java

5.41 RevengeAlertDialogBuilderAdapter Class Reference

An adapter class adapting AlertDialog.Builder to provide an interface for creating an AlertDialog asking the user whether they want to invoke the Ability.REVENGE ability.

Collaboration diagram for RevengeAlertDialogBuilderAdapter:



Static Public Member Functions

- static Completable insertAvengers (@NonNull UnitRepository repository, @IntRange(from=0) int number↩ OfAvengers, @NonNull SoundManager soundManager)

  Inserts numberOfAvengers avenger units into the given UnitRepository.

Package Functions

- RevengeAlertDialogBuilderAdapter (@NonNull Context context)

  Constructor of a RevengeAlertDialogBuilderAdapter.
- AlertDialog create ()

  Creates an AlertDialog with the arguments supplied to this builder.
- RevengeAlertDialogBuilderAdapter setPositiveCallback (@NonNull DialogInterface.OnClickListener on↩ PositiveButtonClick)

  Adds the given callback to the positive button of the built Dialog.
- RevengeAlertDialogBuilderAdapter setNegativeCallback (@NonNull DialogInterface.OnClickListener on↩ NegativeButtonClick)

  Adds the given callback to the negative button of the built Dialog.

Private Attributes

- final AlertDialog.Builder adapteeBuilder

  AlertDialog.Builder that is adapted by this class.

Static Private Attributes

- static final boolean AVENGER_EPIC = false

  Boolean constant defining whether the default UnitEntity summoned by the Ability.REVENGE ability is epic.
- static final Ability AVENGER_ABILITY = Ability.NONE

  Ability constant defining the Ability of the default UnitEntity summoned by the Ability.REVENGE ability.
- static final int AVENGER_DAMAGE = 8

  Integer constant defining the damage of the default UnitEntity summoned by the Ability.REVENGE ability.
- static final Integer AVENGER_SQUAD = null

  Integer constant defining the squad of the default UnitEntity summoned by the Ability.REVENGE ability.
- static final RowType AVENGER_ROW = RowType.MELEE

  RowType constant defining the row of the default UnitEntity summoned by the Ability.REVENGE ability.

5.41.1 Detailed Description

An adapter class adapting AlertDialog.Builder to provide an interface for creating an AlertDialog asking the user whether they want to invoke the Ability.REVENGE ability.
Definition at line 23 of file RevengeAlertDialogBuilderAdapter.java.

5.41.2 Constructor & Destructor Documentation

5.41.2.1 RevengeAlertDialogBuilderAdapter()

```
RevengeAlertDialogBuilderAdapter (
            @NonNull Context context)  [package]
```

Constructor of a RevengeAlertDialogBuilderAdapter.
Initializes the buttons with empty callbacks.

Parameters

context	Context of the built AlertDialog.

Definition at line 76 of file RevengeAlertDialogBuilderAdapter.java.
Referenced by setNegativeCallback(), and setPositiveCallback().

5.41.3 Member Function Documentation

5.41.3.1 create()

```
AlertDialog create ()  [package]
```

Creates an AlertDialog with the arguments supplied to this builder.
Basically just calls AlertDialog.Builder.create() on .adapteeBuilder.

Returns

The AlertDialog created by this builder.

See also

AlertDialog.Builder.create()

Definition at line 110 of file RevengeAlertDialogBuilderAdapter.java.
References adapteeBuilder.

5.41.3.2 insertAvengers()

```
Completable insertAvengers (
            @NonNull UnitRepository repository,
            @IntRange(from=0) int numberOfAvengers,
            @NonNull SoundManager soundManager)  [static]
```

Inserts numberOfAvengers avenger units into the given UnitRepository.
The inserted UnitEntity objects have the attributes defined in AVENGER_EPIC, AVENGER_DAMAGE, AVENGER↩ _ABILITY, AVENGER_SQUAD, and AVENGER_ROW.

Parameters

repository	UnitRepository where avengers are inserted.
numberOfAvengers	Integer defining how many avengers are inserted.
soundManager	SoundManager used to play the card add sound.

Returns

A Completable tracking operation status.

Definition at line 97 of file RevengeAlertDialogBuilderAdapter.java.
References AVENGER_ABILITY, AVENGER_DAMAGE, AVENGER_EPIC, AVENGER_ROW, and AVENGER_SQUAD.
Referenced by RemoveUnitsUseCase.getRevengeDialog(), and ResetRepositoryUseCase.getRevengeDialog().

5.41.3.3 setNegativeCallback()

```
RevengeAlertDialogBuilderAdapter setNegativeCallback (
            @NonNull DialogInterface.OnClickListener onNegativeButtonClick)  [package]
```

Adds the given callback to the negative button of the built Dialog.

Parameters

onNegativeButtonClick	DialogInterface.OnClickListener that is called, when the negative button is clicked.

Returns

The RevengeAlertDialogBuilder with the updated negative callback.

Definition at line 135 of file RevengeAlertDialogBuilderAdapter.java.
References RevengeAlertDialogBuilderAdapter(), and adapteeBuilder.

5.41.3.4 setPositiveCallback()

```
RevengeAlertDialogBuilderAdapter setPositiveCallback (
            @NonNull DialogInterface.OnClickListener onPositiveButtonClick)  [package]
```

Adds the given callback to the positive button of the built Dialog.
Callback should call insertAvengers().

Parameters

onPositiveButtonClick	DialogInterface.OnClickListener that is called, when the positive button is clicked.

Returns

The RevengeAlertDialogBuilder with the updated positive callback.

See also

.insertAvengers(UnitRepository, int, SoundManager)

Definition at line 123 of file RevengeAlertDialogBuilderAdapter.java.
References RevengeAlertDialogBuilderAdapter(), and adapteeBuilder.
Referenced by RemoveUnitsUseCase.getRevengeDialog(), and ResetRepositoryUseCase.getRevengeDialog().

5.41.4 Member Data Documentation

5.41.4.1 adapteeBuilder

```
final AlertDialog.Builder adapteeBuilder  [private]
```

AlertDialog.Builder that is adapted by this class.
Definition at line 68 of file RevengeAlertDialogBuilderAdapter.java.
Referenced by create(), setNegativeCallback(), and setPositiveCallback().

5.41.4.2 AVENGER_ABILITY

```
final Ability AVENGER_ABILITY = Ability.NONE  [static], [private]
```
Ability constant defining the Ability of the default UnitEntity summoned by the Ability.REVENGE ability.

See also

Ability.REVENGE

Definition at line 38 of file RevengeAlertDialogBuilderAdapter.java.
Referenced by insertAvengers().

5.41.4.3 AVENGER_DAMAGE

```
final int AVENGER_DAMAGE = 8  [static], [private]
```
Integer constant defining the damage of the default UnitEntity summoned by the Ability.REVENGE ability.

See also

Ability.REVENGE

Definition at line 46 of file RevengeAlertDialogBuilderAdapter.java.
Referenced by insertAvengers().

5.41.4.4 AVENGER_EPIC

```
final boolean AVENGER_EPIC = false  [static], [private]
```
Boolean constant defining whether the default UnitEntity summoned by the Ability.REVENGE ability is epic.

See also

Ability.REVENGE

Definition at line 30 of file RevengeAlertDialogBuilderAdapter.java.
Referenced by insertAvengers().

5.41.4.5 AVENGER_ROW

```
final RowType AVENGER_ROW = RowType.MELEE  [static], [private]
```
RowType constant defining the row of the default UnitEntity summoned by the Ability.REVENGE ability.

See also

Ability.REVENGE

Definition at line 62 of file RevengeAlertDialogBuilderAdapter.java.
Referenced by insertAvengers().

5.41.4.6 AVENGER_SQUAD

```
final Integer AVENGER_SQUAD = null  [static], [private]
```
Integer constant defining the squad of the default UnitEntity summoned by the Ability.REVENGE ability.

See also

Ability.REVENGE

Definition at line 54 of file RevengeAlertDialogBuilderAdapter.java.
Referenced by insertAvengers().
The documentation for this class was generated from the following file:

- gwent/domain/cases/RevengeAlertDialogBuilderAdapter.java

5.42 RowDao Interface Reference

An interface defining data access operations related to `rows` table.
Collaboration diagram for RowDao:



Public Member Functions

- Completable insertRow (@NonNull RowEntity row)

  Inserts the given RowEntity into `rows`.
- Completable clearRows ()

  Deletes all RowEntity objects from `rows`.
- Completable updateWeather (@NonNull RowType row)

  Flips RowEntity.weather for the given RowType in `rows`.
- Completable clearWeather ()

  Sets RowEntity.weather to `false` for all RowEntity elements in `rows`.
- Completable updateHorn (@NonNull RowType row)

  Flips RowEntity.horn for the given RowType in `rows`.
- Single< Boolean > isWeather (@NonNull RowType row)

  Fetches RowEntity.weather for the given RowType in `rows`.
- Flowable< Boolean > isWeatherFlowable (@NonNull RowType row)

  Fetches a Flowable of RowEntity.weather for the given RowType in `rows`.
- Single< Boolean > isHorn (@NonNull RowType row)

  Fetches RowEntity.horn for the given RowType in `rows`.
- Flowable< Boolean > isHornFlowable (@NonNull RowType row)

  Fetches a Flowable of RowEntity.horn for the given RowType in `rows`.

5.42.1 Detailed Description

An interface defining data access operations related to `rows` table.
The implementation is generated by the Room framework.
Definition at line 21 of file RowDao.java.

5.42.2 Member Function Documentation

5.42.2.1 clearRows()

```
Completable clearRows ()
```

Deletes all RowEntity objects from `rows`.

Returns

A Completable tracking operation status.

References clearRows().
Referenced by clearRows().

5.42.2.2 clearWeather()

```
Completable clearWeather ()
```

Sets RowEntity.weather to `false` for all RowEntity elements in `rows`.

Returns

A Completable tracking operation status.

References clearWeather().
Referenced by clearWeather().

5.42.2.3 insertRow()

```
Completable insertRow (
            @NonNull RowEntity row)
```

Inserts the given RowEntity into `rows`.
If the same RowType is already in the table, the new insert is ignored.

Parameters

row	RowEntity that is being inserted.

Returns

A Completable tracking operation status.

References insertRow().
Referenced by insertRow().

5.42.2.4 isHorn()

```
Single< Boolean > isHorn (
            @NonNull RowType row)
```

Fetches RowEntity.horn for the given RowType in `rows`.

See also

.isHornFlowable(RowType)

Parameters

row	RowType of the queried RowEntity.

Returns

A Single tracking operation status and returning the value.

References isHorn().
Referenced by isHorn().

5.42.2.5 isHornFlowable()

```
Flowable< Boolean > isHornFlowable (
            @NonNull RowType row)
```

Fetches a Flowable of RowEntity.horn for the given RowType in `rows`.

Parameters

row	RowType of the queried RowEntity.

Returns

A Flowable emitting the values.

See also

.isHorn(RowType)

References isHornFlowable().
Referenced by isHornFlowable().

5.42.2.6 isWeather()

```
Single< Boolean > isWeather (
            @NonNull RowType row)
```

Fetches RowEntity.weather for the given RowType in `rows`.

See also

.isWeatherFlowable(RowType)

Parameters

row	RowType of the queried RowEntity.

Returns

A Single tracking operation status and returning the value.

References isWeather().
Referenced by isWeather().

5.42.2.7 isWeatherFlowable()

```
Flowable< Boolean > isWeatherFlowable (
            @NonNull RowType row)
```

Fetches a Flowable of RowEntity.weather for the given RowType in `rows`.

Parameters

row	RowType of the queried RowEntity.

Returns

A Flowable emitting the values.

See also

.isWeather(RowType)

References isWeatherFlowable().
Referenced by isWeatherFlowable().

5.42.2.8 updateHorn()

```
Completable updateHorn (
            @NonNull RowType row)
```

Flips RowEntity.horn for the given RowType in `rows`.

Parameters

row	RowType of the updated RowEntity.

Returns

A Completable tracking operation status.

References updateHorn().
Referenced by updateHorn().

5.42.2.9 updateWeather()

```
Completable updateWeather (
            @NonNull RowType row)
```

Flips RowEntity.weather for the given RowType in `rows`.

Parameters

row	RowType of the updated RowEntity.

Returns

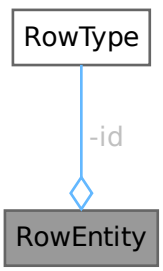
A Completable tracking operation status.

References updateWeather().
Referenced by updateWeather().
The documentation for this interface was generated from the following file:

- gwent/data/RowDao.java

5.43 RowEntity Class Reference

A class representing the state of an attack row, i.e., weather and commanders horn.
Collaboration diagram for RowEntity:



Protected Member Functions

- RowEntity (@NonNull RowType id)

  Constructor of a RowEntity.
- RowType getId ()

  Getter for .id.
- boolean isWeather ()

  Getter for .weather.
- void setWeather (boolean weather)

  Setter for .weather.
- boolean isHorn ()

  Getter for .horn.
- void setHorn (boolean horn)

  Setter for .horn.

Private Attributes

- final RowType id

  Defines the type of the row as one of the values in RowType, thereby limiting the number of different rows to three.
- boolean weather

  Defines whether the weather debuff is present in the represented row, i.e., if set to true, the damage value of units in the represented row is set to 1.
- boolean horn

  Defines whether the commanders horn buff is present in the represented row, i.e., if set to true, the damage value of units in the represented row is doubled.

5.43.1 Detailed Description

A class representing the state of an attack row, i.e., weather and commanders horn.
Is a persistent Entity and is therefore saved in a database table named `rows`.
Definition at line 14 of file RowEntity.java.

5.43.2 Constructor & Destructor Documentation

5.43.2.1 RowEntity()

```
RowEntity (
            @NonNull RowType id)  [protected]
```

Constructor of a RowEntity.
.weather and .horn cannot be set here since a new row must start with the respective default values.

Parameters

id	RowType of the represented row.

Definition at line 47 of file RowEntity.java.
References RowEntity(), and id.
Referenced by RowEntity().

5.43.3 Member Function Documentation

5.43.3.1 getId()

```
RowType getId ()  [protected]
```

Getter for .id.
Only used by Room extension.

Returns

RowType of the represented row.

Definition at line 58 of file RowEntity.java.
References id.

5.43.3.2 isHorn()

```
boolean isHorn ()  [protected]
```

Getter for .horn.
Only used by Room extension.

Returns

Boolean representing the current status of the horn buff.

Definition at line 88 of file RowEntity.java.
References horn.

5.43.3.3 isWeather()

```
boolean isWeather ()  [protected]
```

Getter for .weather.
Only used by Room extension.

Returns

Boolean representing the current status of the weather debuff.

Definition at line 68 of file RowEntity.java.
References weather.

5.43.3.4 setHorn()

```
void setHorn (
            boolean horn)  [protected]
```

Setter for .horn.
Only used by Room extension.

Parameters

horn	Boolean representing the new status of the horn buff.

Definition at line 98 of file RowEntity.java.
References horn.

5.43.3.5 setWeather()

```
void setWeather (
            boolean weather)  [protected]
```

Setter for .weather.
Only used by Room extension.

Parameters

weather	Boolean representing the new status of the weather debuff.

Definition at line 78 of file RowEntity.java.
References weather.

5.43.4 Member Data Documentation

5.43.4.1 horn

`boolean horn [private]`
Defines whether the commanders horn buff is present in the represented row, i.e., if set to true, the damage value of units in the represented row is doubled.
Is set to `false` by default.
Definition at line 38 of file RowEntity.java.
Referenced by isHorn(), and setHorn().

5.43.4.2 id

`final RowType id [private]`
Defines the type of the row as one of the values in RowType, thereby limiting the number of different rows to three.
Primary key of the represented row.
Definition at line 22 of file RowEntity.java.
Referenced by RowEntity(), and getId().

5.43.4.3 weather

`boolean weather [private]`
Defines whether the weather debuff is present in the represented row, i.e., if set to true, the damage value of units in the represented row is set to 1.
Is set to `false` by default.
Definition at line 30 of file RowEntity.java.
Referenced by isWeather(), and setWeather().
The documentation for this class was generated from the following file:

- gwent/data/RowEntity.java

5.44 RowType Enum Reference

An enum listing the different types of attack rows.
Collaboration diagram for RowType:



Public Attributes

- MELEE

  Represents the row containing melee combat units.
- RANGE

  Represents the row containing ranged combat units.
- SIEGE

  Represents the row containing siege combat units.

5.44.1 Detailed Description

An enum listing the different types of attack rows.
Definition at line 6 of file RowType.java.

5.44.2 Member Data Documentation

5.44.2.1 MELEE

`MELEE`
Represents the row containing melee combat units.
Definition at line 10 of file RowType.java.

5.44.2.2 RANGE

`RANGE`
Represents the row containing ranged combat units.
Definition at line 15 of file RowType.java.

5.44.2.3 SIEGE

`SIEGE`
Represents the row containing siege combat units.
Definition at line 20 of file RowType.java.
The documentation for this enum was generated from the following file:

- gwent/data/RowType.java

5.45 RowUiState Class Reference

A data class encapsulating the visible state of a row in the main view of the application.
Collaboration diagram for RowUiState:



Public Member Functions

- RowUiState (@IntRange(from=0) int damage, boolean weather, boolean horn, @IntRange(from=0) int units)

  Constructor of a RowUiState encapsulating the given data.
- int getDamage ()

  Returns the summed-up damage of all units in this row.
- boolean isWeather ()

  Returns whether the weather debuff is active in this row.
- boolean isHorn ()

  Returns whether the commander's horn buff is active in this row.
- int getUnits ()

  Returns the number of units in this row.
- boolean equals (Object o)

  Indicates whether some other object is "equal to" this one.

Private Attributes

- final int damage

  Defines the summed-up damage of all units in this row.
- final boolean weather

  Defines whether the weather debuff is active in this row.
- final boolean horn

  Defines whether the commander's horn buff is active in this row.
- final int units

  Defines the number of units in this row.

5.45.1 Detailed Description

A data class encapsulating the visible state of a row in the main view of the application.
Definition at line 10 of file RowUiState.java.

5.45.2 Constructor & Destructor Documentation

5.45.2.1 RowUiState()

```
RowUiState (
            @IntRange(from=0) int damage,
            boolean weather,
            boolean horn,
            @IntRange(from=0) int units)
```

Constructor of a RowUiState encapsulating the given data.

Parameters

damage	Integer representing the summed-up damage of all units.
weather	Boolean defining whether the weather debuff is active.
horn	Boolean defining whether the commander's horn buff is active.
units	Integer representing the number of units.

Exceptions

IllegalArgumentException	When damage or units is negative.

Definition at line 41 of file RowUiState.java.
References RowUiState(), damage, horn, units, and weather.
Referenced by RowUiState(), and equals().

5.45.3 Member Function Documentation

5.45.3.1 equals()

```
boolean equals (
            Object o)
```

Indicates whether some other object is "equal to" this one.

Parameters

o	the reference object with which to compare.

Returns

`true` if this object is the same as the o argument; `false` otherwise.

Definition at line 95 of file RowUiState.java.
References RowUiState(), damage, horn, units, and weather.

5.45.3.2 getDamage()

`int getDamage ()`
Returns the summed-up damage of all units in this row.

Returns

An Integer representing the summed-up damage of all units.

Definition at line 59 of file RowUiState.java.
References damage, and getDamage().
Referenced by getDamage().

5.45.3.3 getUnits()

`int getUnits ()`
Returns the number of units in this row.

Returns

An Integer representing the number of units in this row.

Definition at line 84 of file RowUiState.java.
References getUnits(), and units.
Referenced by getUnits().

5.45.3.4 isHorn()

`boolean isHorn ()`
Returns whether the commander's horn buff is active in this row.

Returns

A Boolean whether the commander's horn buff is active.

Definition at line 75 of file RowUiState.java.
References horn.

5.45.3.5 isWeather()

`boolean isWeather ()`
Returns whether the weather debuff is active in this row.

Returns

A Boolean defining whether the weather debuff is active.

Definition at line 67 of file RowUiState.java.
References weather.

5.45.4 Member Data Documentation

5.45.4.1 damage

`final int damage [private]`
Defines the summed-up damage of all units in this row.
Definition at line 15 of file RowUiState.java.
Referenced by RowUiState(), equals(), and getDamage().

5.45.4.2 horn

`final boolean horn [private]`
Defines whether the commander's horn buff is active in this row.
Definition at line 25 of file RowUiState.java.
Referenced by RowUiState(), equals(), and isHorn().

5.45.4.3 units

`final int units [private]`
Defines the number of units in this row.
Definition at line 31 of file RowUiState.java.
Referenced by RowUiState(), equals(), and getUnits().

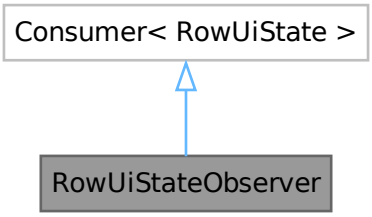
5.45.4.4 weather

`final boolean weather [private]`
Defines whether the weather debuff is active in this row.
Definition at line 20 of file RowUiState.java.
Referenced by RowUiState(), equals(), and isWeather().
The documentation for this class was generated from the following file:

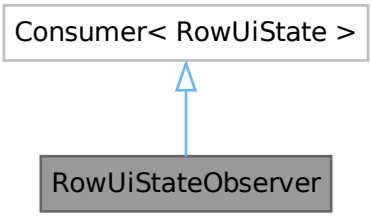
- gwent/ui/main/RowUiState.java

5.46 RowUiStateObserver Class Reference

An observer class responsible for updating the views of the row defined in .row when notified, i.e., when a new RowUiState is produced the observed Flowable.
Inheritance diagram for RowUiStateObserver:



Collaboration diagram for RowUiStateObserver:



Public Member Functions

- void accept (@NonNull RowUiState rowUiState)

  Changes values in .damageView and .unitView as well as the image in .weatherView and .hornView.

Static Public Member Functions

- static RowUiStateObserver getObserver (@NonNull RowType row, @NonNull TextView damageView, @NonNull ImageView weatherView, @NonNull ImageView hornView, @NonNull TextView unitView)

  Returns a new RowUiStateObserver for the given row updating the given views.

Private Member Functions

- RowUiStateObserver (@NonNull TextView damageView, @NonNull ImageView weatherView, @Drawable↩ Res int weatherResource, @NonNull ImageView hornView, @NonNull TextView unitView)

  Constructor of a RowUiStateObserver updating the given views when .onChanged() is called.

Private Attributes

- final TextView damageView

  A TextView showing the user the summed-up damage of all units in this row, i.e., RowUiState.damage.
- final ImageView weatherView

  An ImageView responsible for showing the current state of the weather debuff in this row.
- final int weatherResource

  An Integer representing the drawable resource shown by .weatherView when the weather debuff is active, i.e., when RowUiState.weather is `true`*.*
- final ImageView hornView

  An ImageView responsible for showing the current state of the commander's horn buff in this row.
- final TextView unitView

  A TextView showing the number of units in this row, i.e., RowUiState.units.

5.46.1 Detailed Description

An observer class responsible for updating the views of the row defined in .row when notified, i.e., when a new RowUiState is produced the observed Flowable.

See also

RowUiState

Definition at line 22 of file RowUiStateObserver.java.

5.46.2 Constructor & Destructor Documentation

5.46.2.1 RowUiStateObserver()

```
RowUiStateObserver (
            @NonNull TextView damageView,
            @NonNull ImageView weatherView,
            @DrawableRes int weatherResource,
            @NonNull ImageView hornView,
            @NonNull TextView unitView)  [private]
```

Constructor of a RowUiStateObserver updating the given views when .onChanged() is called.
Should only be called by .getObserver().

Parameters

damageView	TextView showing the summed-up damage of all units in the observed row.
weatherView	ImageView showing the current state of the weather debuff of the observed row.
weatherResource	Integer representing the drawable resource shown when the weather debuff is active.
hornView	ImageView showing the current state of the commander's horn buff of the observed row.
unitView	TextView showing the number of units in the observed row.

See also

.getObserver(RowType, TextView, ImageView, ImageView, TextView)

Definition at line 74 of file RowUiStateObserver.java.
References damageView, hornView, unitView, weatherResource, and weatherView.
Referenced by getObserver().

5.46.3 Member Function Documentation

5.46.3.1 accept()

```
void accept (
            @NonNull RowUiState rowUiState)
```

Changes values in .damageView and .unitView as well as the image in .weatherView and .hornView.
Image switches are animated using ImageViewSwitchAnimator.

Parameters

rowUiState	RowUiState representing the updated state of the row.

See also

ImageViewSwitchAnimator

Definition at line 127 of file RowUiStateObserver.java.
References ImageViewSwitchAnimator.animatedSwitch(), damageView, hornView, unitView, weatherResource, and weatherView.

5.46.3.2 getObserver()

```
RowUiStateObserver getObserver (
            @NonNull RowType row,
            @NonNull TextView damageView,
            @NonNull ImageView weatherView,
```

```
@NonNull ImageView hornView,
@NonNull TextView unitView)  [static]
```

Returns a new RowUiStateObserver for the given row updating the given views.
Factory method for RowUiStateObserver. .weatherResource is defined according to the given RowType.

Parameters

row	RowType defining which row is being observed.
damageView	TextView showing the summed-up damage of all units in the observed row.
weatherView	ImageView showing the current state of the weather debuff of the observed row.
hornView	ImageView showing the current state of the commander's horn buff of the observed row.
unitView	TextView showing the number of units in the observed row.

Returns

A RowUiStateObserver updating the given views.

See also

.RowUiStateObserver(TextView, ImageView, int, ImageView, TextView)

Definition at line 99 of file RowUiStateObserver.java.
References RowUiStateObserver(), damageView, getObserver(), hornView, unitView, weatherResource, and weatherView.
Referenced by getObserver(), and MainActivity.initializeViewModel().

5.46.4 Member Data Documentation

5.46.4.1 damageView

```
final TextView damageView  [private]
```

A TextView showing the user the summed-up damage of all units in this row, i.e., RowUiState.damage.
Definition at line 28 of file RowUiStateObserver.java.
Referenced by RowUiStateObserver(), accept(), and getObserver().

5.46.4.2 hornView

```
final ImageView hornView  [private]
```

An ImageView responsible for showing the current state of the commander's horn buff in this row.
Is gray when RowUiState.horn is `false`.

See also

.weatherResource

Definition at line 55 of file RowUiStateObserver.java.
Referenced by RowUiStateObserver(), accept(), and getObserver().

5.46.4.3 unitView

```
final TextView unitView  [private]
```

A TextView showing the number of units in this row, i.e., RowUiState.units.
Definition at line 61 of file RowUiStateObserver.java.
Referenced by RowUiStateObserver(), accept(), and getObserver().

5.46.4.4 weatherResource

```
final int weatherResource  [private]
```

An Integer representing the drawable resource shown by .weatherView when the weather debuff is active, i.e., when RowUiState.weather is `true`.

See also

.weatherView

Definition at line 46 of file RowUiStateObserver.java.
Referenced by RowUiStateObserver(), accept(), and getObserver().

5.46.4.5 weatherView

`final ImageView weatherView [private]`
An ImageView responsible for showing the current state of the weather debuff in this row.
Displays the resource in .weatherResource when RowUiState.weather is `true`.

See also

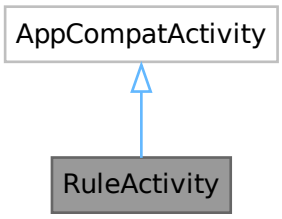
.weatherResource

Definition at line 37 of file RowUiStateObserver.java.
Referenced by RowUiStateObserver(), accept(), and getObserver().
The documentation for this class was generated from the following file:

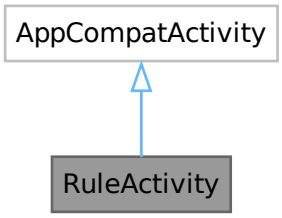
- gwent/ui/main/RowUiStateObserver.java

5.47 RuleActivity Class Reference

An AppCompatActivity used to present the rules of the game.
Inheritance diagram for RuleActivity:



Collaboration diagram for RuleActivity:



Public Member Functions

- void onCreate (Bundle savedInstanceState)

  Initializes the content of the selected rule.
- boolean onOptionsItemSelected (@NonNull MenuItem item)

  Called whenever a MenuItem in the options menu is selected.

Static Public Attributes

- static final String INTENT_EXTRA_TAG = "rule_section"

  String constant defining the identifier where the requested RuleSection is provided in the calling android.content.↩ Intent.

5.47.1 Detailed Description

An AppCompatActivity used to present the rules of the game.
Can be accessed from the SettingsActivity. When called, a selected RuleSection must always be given through the calling android.content.Intent at the key defined in INTENT_EXTRA_TAG.
Definition at line 23 of file RuleActivity.java.

5.47.2 Member Function Documentation

5.47.2.1 onCreate()

```
void onCreate (
            Bundle savedInstanceState)
```

Initializes the content of the selected rule.

Parameters

savedInstanceState	If the activity is being re-initialized after previously being shut down then this Bundle contains the data it most recently supplied in onSaveInstanceState. ***Note: Otherwise it is null.***

Definition at line 38 of file RuleActivity.java.
References INTENT_EXTRA_TAG.

5.47.2.2 onOptionsItemSelected()

```
boolean onOptionsItemSelected (
            @NonNull MenuItem item)
```

Called whenever a MenuItem in the options menu is selected.
Returns to the calling android.app.Activity when the android.R.idhome item was selected.

Parameters

item	MenuItem that was selected.

Returns

Boolean defining whether the call has been handled.

Definition at line 99 of file RuleActivity.java.

5.47.3 Member Data Documentation

5.47.3.1 INTENT_EXTRA_TAG

```
final String INTENT_EXTRA_TAG = "rule_section"  [static]
```

String constant defining the identifier where the requested RuleSection is provided in the calling android.content.↩ Intent.
Definition at line 28 of file RuleActivity.java.
Referenced by SettingsRuleFragment.getSectionClickListener(), and onCreate().
The documentation for this class was generated from the following file:

- gwent/ui/settings/RuleActivity.java

5.48 RuleSection Enum Reference

An Enum used to discern the different possible sections shown by RuleActivity.
Collaboration diagram for RuleSection:



Public Attributes

- GENERAL

  Represents a section containing general background-information on the game Gwent.
- COURSE

  Represents a section containing information on the course of a game of Gwent.
- FACTIONS

  Represents a section containing information on the factions playable in Gwent.
- COMMANDER

  Represents a section containing information on the available commanders of the factions and their abilities.
- CARDS

  Represents a section containing information on the different types of cards in Gwent.
- CARD_ABILITIES

  Represents a section containing information on the possible abilities of cards in Gwent.
- SPECIAL_CARDS

  Represents a section containing information on special cards available in Gwent.

5.48.1 Detailed Description

An Enum used to discern the different possible sections shown by RuleActivity.
Definition at line 6 of file RuleSection.java.

5.48.2 Member Data Documentation

5.48.2.1 CARD_ABILITIES

```
CARD_ABILITIES
```

Represents a section containing information on the possible abilities of cards in Gwent.
Definition at line 35 of file RuleSection.java.

5.48.2.2 CARDS

CARDS
Represents a section containing information on the different types of cards in Gwent.
Definition at line 30 of file RuleSection.java.

5.48.2.3 COMMANDER

COMMANDER
Represents a section containing information on the available commanders of the factions and their abilities.
Definition at line 25 of file RuleSection.java.

5.48.2.4 COURSE

COURSE
Represents a section containing information on the course of a game of Gwent.
Definition at line 15 of file RuleSection.java.

5.48.2.5 FACTIONS

FACTIONS
Represents a section containing information on the factions playable in Gwent.
Definition at line 20 of file RuleSection.java.

5.48.2.6 GENERAL

GENERAL
Represents a section containing general background-information on the game Gwent.
Definition at line 10 of file RuleSection.java.

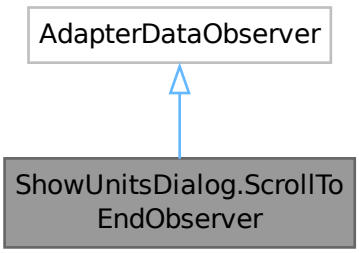
5.48.2.7 SPECIAL_CARDS

SPECIAL_CARDS
Represents a section containing information on special cards available in Gwent.
Definition at line 40 of file RuleSection.java.
The documentation for this enum was generated from the following file:

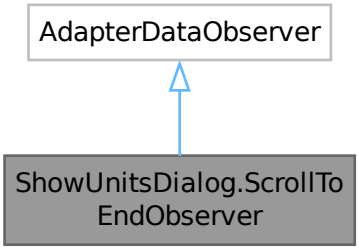
- gwent/ui/settings/RuleSection.java

5.49 ShowUnitsDialog.ScrollToEndObserver Class Reference

Smoothly scrolls to the last item in the RecyclerView to show the user that an item has been inserted.
Inheritance diagram for ShowUnitsDialog.ScrollToEndObserver:



Collaboration diagram for ShowUnitsDialog.ScrollToEndObserver:



Public Member Functions

- void onItemRangeInserted (int positionStart, int itemCount)

5.49.1 Detailed Description

Smoothly scrolls to the last item in the RecyclerView to show the user that an item has been inserted.
Definition at line 168 of file ShowUnitsDialog.java.

5.49.2 Member Function Documentation

5.49.2.1 onItemRangeInserted()

```
void onItemRangeInserted (
            int positionStart,
            int itemCount)
```

Parameters

positionStart	Integer defining the first position from where new items are inserted.
itemCount	Integer defining how many items have been inserted.

See also

RecyclerView.smoothScrollToPosition(int)

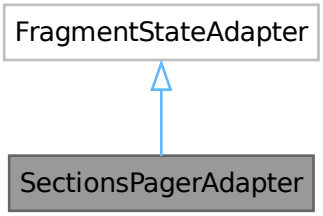
Definition at line 175 of file ShowUnitsDialog.java.
References ShowUnitsDialog.unitRecyclerView.
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/cards/ShowUnitsDialog.java

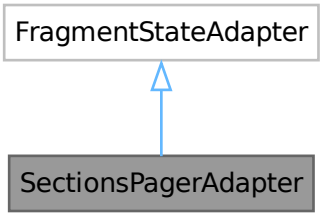
5.50 SectionsPagerAdapter Class Reference

A FragmentStateAdapter that returns an IntroductionFragment corresponding to one of the sections/tabs/pages.

Inheritance diagram for SectionsPagerAdapter:



Collaboration diagram for SectionsPagerAdapter:



Public Member Functions

- SectionsPagerAdapter (@NonNull FragmentActivity fragmentActivity)

  Constructor of a SectionsPagerAdapter for the given FragmentActivity.
- int getItemCount ()

  Returns the total number of items in the data set held by the adapter.
- Fragment createFragment (int position)

  Provides a new IntroductionFragment associated with the specified position.

5.50.1 Detailed Description

A FragmentStateAdapter that returns an IntroductionFragment corresponding to one of the sections/tabs/pages.
Definition at line 12 of file SectionsPagerAdapter.java.

5.50.2 Constructor & Destructor Documentation

5.50.2.1 SectionsPagerAdapter()

```
SectionsPagerAdapter (
            @NonNull FragmentActivity fragmentActivity)
```

Constructor of a SectionsPagerAdapter for the given FragmentActivity.

Parameters

fragmentActivity	FragmentActivity that uses this adapter.

Definition at line 18 of file SectionsPagerAdapter.java.

5.50.3 Member Function Documentation

5.50.3.1 createFragment()

```
Fragment createFragment (
            int position)
```

Provides a new IntroductionFragment associated with the specified position.

Parameters

position	Integer defining the page that is queried.

Returns

A Fragment that will be shown at the specified position.

Definition at line 39 of file SectionsPagerAdapter.java.
References IntroductionFragment.newInstance().

5.50.3.2 getItemCount()

```
int getItemCount ()
```

Returns the total number of items in the data set held by the adapter.
Essentially queries IntroductionFragmentPAGES_COUNT.

Returns

An Integer defining the number of pages.

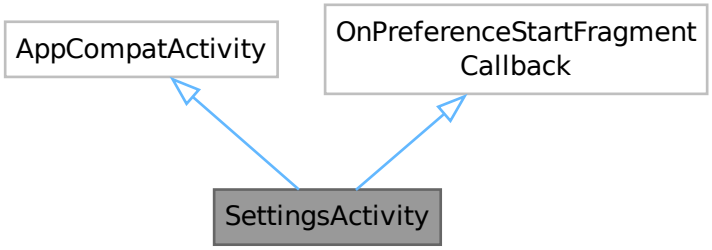
Definition at line 28 of file SectionsPagerAdapter.java.
References IntroductionFragment.PAGES_COUNT.
The documentation for this class was generated from the following file:

- gwent/ui/introduction/SectionsPagerAdapter.java

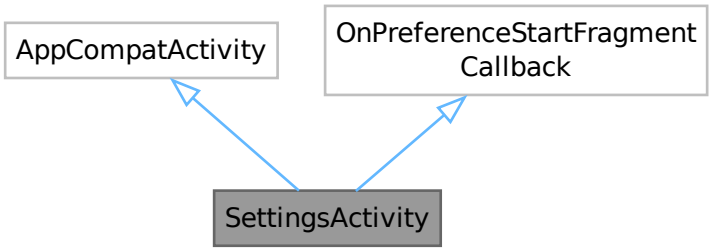
5.51 SettingsActivity Class Reference

An AppCompatActivity implementing PreferenceFragmentCompat.OnPreferenceStartFragmentCallback that is used by the user to manage the SharedPreferences of the application.

Inheritance diagram for SettingsActivity:



Collaboration diagram for SettingsActivity:



Public Member Functions

- boolean onOptionsItemSelected (@NonNull MenuItem item)

  Called whenever a MenuItem in the options menu is selected.
- boolean onPreferenceStartFragment (@NonNull PreferenceFragmentCompat caller, @NonNull Preference pref)

  Called when the user has clicked on a Preference that has a Fragment class name associated with it.

Protected Member Functions

- void onCreate (@Nullable Bundle savedInstanceState)

  Initializes layout and ActionBar as well as creates and displays a new SettingsHeaderFragment.
- void onSaveInstanceState (@NonNull Bundle outState)

  Saves the currently visible Fragment into the given Bundle.

Static Private Attributes

- static final String CURRENT_FRAGMENT_KEY = "currentFragment"

  String constant defining the key in savedInstanceState Bundles where the last shown Fragment is saved.

5.51.1 Detailed Description

An AppCompatActivity implementing PreferenceFragmentCompat.OnPreferenceStartFragmentCallback that is used by the user to manage the SharedPreferences of the application.
Definition at line 24 of file SettingsActivity.java.

5.51.2 Member Function Documentation

5.51.2.1 onCreate()

```
void onCreate (
            @Nullable Bundle savedInstanceState)  [protected]
```

Initializes layout and ActionBar as well as creates and displays a new SettingsHeaderFragment.

Parameters

savedInstanceState	If the activity is being re-initialized after previously being shut down then this Bundle contains the data it most recently supplied in onSaveInstanceState. ***Note: Otherwise it is null.***

Definition at line 38 of file SettingsActivity.java.
References CURRENT_FRAGMENT_KEY, and FactionSwitchListener.setTheme().

5.51.2.2 onOptionsItemSelected()

```
boolean onOptionsItemSelected (
            @NonNull MenuItem item)
```

Called whenever a MenuItem in the options menu is selected.
Returns to the calling android.app.Activity when the android.R.idhome item was selected.

Parameters

item	MenuItem that was selected.

Returns

Boolean defining whether the call has been handled.

Definition at line 85 of file SettingsActivity.java.

5.51.2.3 onPreferenceStartFragment()

```
boolean onPreferenceStartFragment (
            @NonNull PreferenceFragmentCompat caller,
            @NonNull Preference pref)
```

Called when the user has clicked on a Preference that has a Fragment class name associated with it.
Switches to an instance of the given Fragment.

Parameters

caller	PreferenceFragmentCompat requesting navigation.
pref	Preference requesting the Fragment.

Returns

Boolean defining whether the Fragment creation has been handled.

Definition at line 102 of file SettingsActivity.java.

5.51.2.4 onSaveInstanceState()

```
void onSaveInstanceState (
            @NonNull Bundle outState)  [protected]
```

Saves the currently visible Fragment into the given Bundle.

Parameters

outState	Bundle in which the currently used Fragment is saved.

Definition at line 66 of file SettingsActivity.java.
References CURRENT_FRAGMENT_KEY.

5.51.3 Member Data Documentation

5.51.3.1 CURRENT_FRAGMENT_KEY

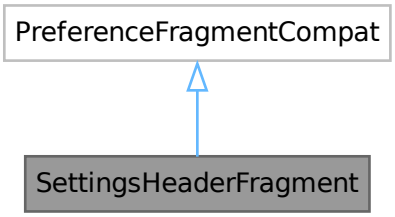
```
final String CURRENT_FRAGMENT_KEY = "currentFragment"  [static], [private]
```

String constant defining the key in savedInstanceState Bundles where the last shown Fragment is saved.
Definition at line 29 of file SettingsActivity.java.
Referenced by onCreate(), and onSaveInstanceState().
The documentation for this class was generated from the following file:

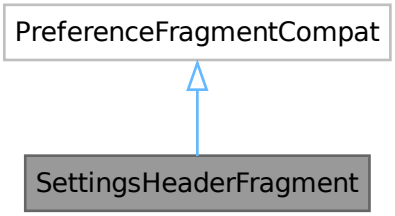
- gwent/ui/settings/SettingsActivity.java

5.52 SettingsHeaderFragment Class Reference

A PreferenceFragmentCompat class encapsulating the main preference screen, i.e., the Preferences defined in R.xmlheader_preferences.
Inheritance diagram for SettingsHeaderFragment:



Collaboration diagram for SettingsHeaderFragment:



Public Member Functions

- void onCreatePreferences (Bundle savedInstanceState, String rootKey)

  Called during onCreate(Bundle) to supply the preferences for this fragment.

5.52.1 Detailed Description

A PreferenceFragmentCompat class encapsulating the main preference screen, i.e., the Preferences defined in R.xmlheader_preferences.
Definition at line 18 of file SettingsHeaderFragment.java.

5.52.2 Member Function Documentation

5.52.2.1 onCreatePreferences()

```
void onCreatePreferences (
            Bundle savedInstanceState,
            String rootKey)
```

Called during onCreate(Bundle) to supply the preferences for this fragment.
Sets shown Preferences from R.xmlheader_preferences and registers an androidx.preference.Preference.On↩ PreferenceClickListener on the Preference at R.stringpreference_key_introduction to start a new IntroductionActivity.

Parameters

savedInstanceState	If the fragment is being re-created from a previous saved state, this is the state.
rootKey	If non-null, this preference fragment should be rooted at the androidx.preference.↩ PreferenceScreen with this key.

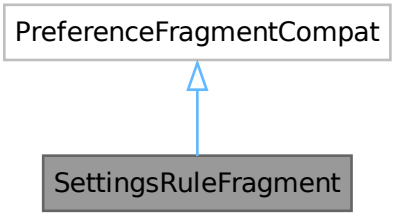
Definition at line 31 of file SettingsHeaderFragment.java.
The documentation for this class was generated from the following file:

- gwent/ui/settings/SettingsHeaderFragment.java

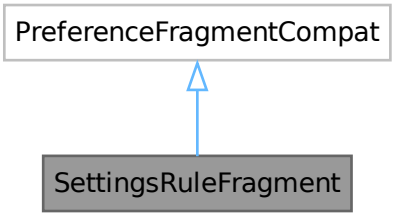
5.53 SettingsRuleFragment Class Reference

A PreferenceFragmentCompat class encapsulating the rule preference screen, i.e., the rule sections defined in R.xmlrule_preferences.

Inheritance diagram for SettingsRuleFragment:



Collaboration diagram for SettingsRuleFragment:



Public Member Functions

- void onCreatePreferences (Bundle savedInstanceState, String rootKey)

  Called during onCreate(Bundle) to supply the preferences for this fragment.

Private Member Functions

- Preference.OnPreferenceClickListener getSectionClickListener (@NonNull RuleSection section)

  Creates an Preference.OnPreferenceClickListener that starts a new RuleActivity for the given RuleSection using an Intent.

5.53.1 Detailed Description

A PreferenceFragmentCompat class encapsulating the rule preference screen, i.e., the rule sections defined in R.xmlrule_preferences.
Definition at line 31 of file SettingsRuleFragment.java.

5.53.2 Member Function Documentation

5.53.2.1 getSectionClickListener()

```
Preference.OnPreferenceClickListener getSectionClickListener (
            @NonNull RuleSection section)  [private]
```

Creates an Preference.OnPreferenceClickListener that starts a new RuleActivity for the given RuleSection using an Intent.
The Intent provides the requested RuleSection to the RuleActivity using IntentputExtra(String, with RuleActivity↩ INTENT_EXTRA_TAG as a tag.

Parameters

section	RuleSection that is requested.

Returns

An Preference.OnPreferenceClickListener calling a RuleActivity.

Definition at line 43 of file SettingsRuleFragment.java.
References getSectionClickListener(), and RuleActivity.INTENT_EXTRA_TAG.
Referenced by getSectionClickListener(), and onCreatePreferences().

5.53.2.2 onCreatePreferences()

```
void onCreatePreferences (
            Bundle savedInstanceState,
            String rootKey)
```

Called during onCreate(Bundle) to supply the preferences for this fragment.
Sets shown Preferences from R.xmlrule_preferences. Also provides each element with an Preference.On↩ PreferenceClickListener that starts a new RuleActivity for the respective RuleSection.

Parameters

savedInstanceState	If the fragment is being re-created from a previous saved state, this is the state.
rootKey	If non-null, this preference fragment should be rooted at the androidx.preference.↩ PreferenceScreen with this key.

See also

.getSectionClickListener(RuleSection)

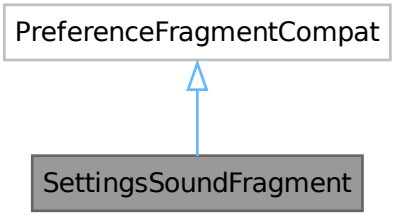
Definition at line 66 of file SettingsRuleFragment.java.
References getSectionClickListener().
The documentation for this class was generated from the following file:

- gwent/ui/settings/SettingsRuleFragment.java

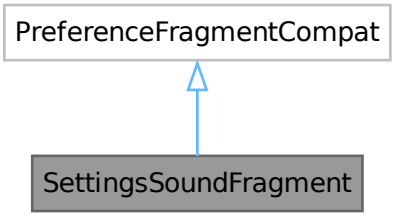
5.54 SettingsSoundFragment Class Reference

A PreferenceFragmentCompat class encapsulating the sound preference screen, i.e., the Preferences defined in R.xmlsound_preferences.

Inheritance diagram for SettingsSoundFragment:



Collaboration diagram for SettingsSoundFragment:



Public Member Functions

- void onCreatePreferences (Bundle savedInstanceState, String rootKey)

  Called during onCreate(Bundle) to supply the preferences for this fragment.

5.54.1 Detailed Description

A PreferenceFragmentCompat class encapsulating the sound preference screen, i.e., the Preferences defined in R.xmlsound_preferences.
Definition at line 16 of file SettingsSoundFragment.java.

5.54.2 Member Function Documentation

5.54.2.1 onCreatePreferences()

```
void onCreatePreferences (
            Bundle savedInstanceState,
            String rootKey)
```

Called during onCreate(Bundle) to supply the preferences for this fragment.
Sets shown Preferences from R.xmlsound_preferences.

Parameters

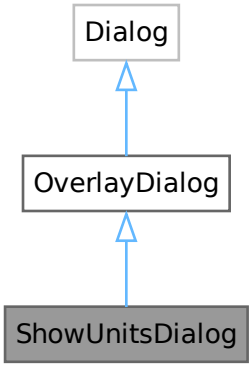
savedInstanceState	If the fragment is being re-created from a previous saved state, this is the state.
rootKey	If non-null, this preference fragment should be rooted at the androidx.preference.↩ PreferenceScreen with this key.

Definition at line 27 of file SettingsSoundFragment.java.
The documentation for this class was generated from the following file:

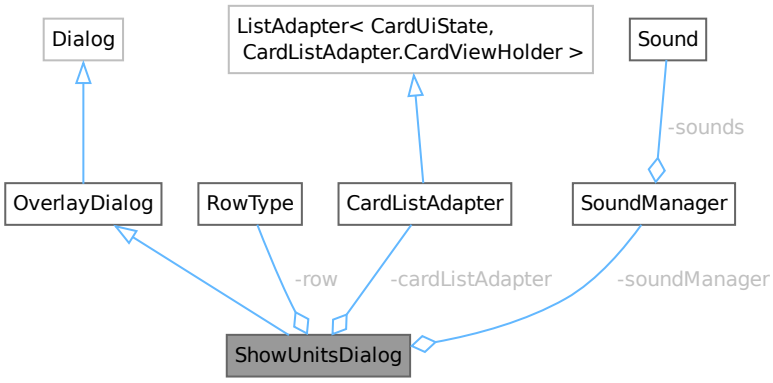
- gwent/ui/settings/SettingsSoundFragment.java

5.55 ShowUnitsDialog Class Reference

An OverlayDialog used to list the units of a certain row and enabling the user to copy, add, or delete new units.
Inheritance diagram for ShowUnitsDialog:



Collaboration diagram for ShowUnitsDialog:



Classes

- class ScrollToEndObserver

Smoothly scrolls to the last item in the RecyclerView to show the user that an item has been inserted.

Public Member Functions

- RowType getRow ()

  Returns the row this ShowUnitsDialog is representing.

Static Public Member Functions

- static Single< ShowUnitsDialog > getDialog (@NonNull Context context, @NonNull RowType row, @Non↩ Null SoundManager soundManager)

  Creates a new ShowUnitsDialog in the given Context and for the given row.

Protected Member Functions

- void onCreate (Bundle savedInstanceState)

  Initializes the RecyclerView by connecting it to the .cardListAdapter and sets View.OnClickListener for each button.

Protected Member Functions inherited from OverlayDialog

- OverlayDialog (@NonNull Context context, @LayoutRes int layout, @IdRes int dismissViewId)

  Constructor of an OverlayDialog in the given Context, with the given layout and dismissViewId.
- OverlayDialog (@NonNull Context context, @LayoutRes int layout)

  Constructor of an OverlayDialog in the given Context and with the given layout.

Private Member Functions

- ShowUnitsDialog (@NonNull Context context, @NonNull RowType row, @NonNull CardListAdapter cardListAdapter, @NonNull SoundManager soundManager)

  Constructor of a ShowUnitsDialog shown in the given Context and with the given CardListAdapter.

Private Attributes

- final RowType row

  RowType defining which row is represented by this Dialog.
- final CardListAdapter cardListAdapter

  CardListAdapter offering a list of CardUiState objects to the UI.
- final SoundManager soundManager

  SoundManager used to play a Sound whenever a card is added or removed.
- final CompositeDisposable disposables = new CompositeDisposable()

  CompositeDisposable keeping track of all subscriptions to observables made by this class.
- RecyclerView unitRecyclerView

  RecyclerView presenting the units provided by the cardListAdapter.

Additional Inherited Members

Static Package Attributes inherited from OverlayDialog

- static final int NO_DISMISS_VIEW = -1

  Integer used as .dismissViewId, when the Dialog shouldn't be cancelable by clicking on a View.

5.55.1 Detailed Description

An OverlayDialog used to list the units of a certain row and enabling the user to copy, add, or delete new units.

See also

CardListAdapter

Definition at line 31 of file ShowUnitsDialog.java.

5.55.2 Constructor & Destructor Documentation

5.55.2.1 ShowUnitsDialog()

```
ShowUnitsDialog (
            @NonNull Context context,
            @NonNull RowType row,
            @NonNull CardListAdapter cardListAdapter,
            @NonNull SoundManager soundManager)  [private]
```

Constructor of a ShowUnitsDialog shown in the given Context and with the given CardListAdapter.

Parameters

context	Context this Dialog is shown in.
row	RowType defining which row all shown units belong to.
cardListAdapter	CardListAdapter providing an always up-to-date list of CardUiState objects for a certain row.
soundManager	SoundManager used to play sounds when cards are added or removed.

Definition at line 78 of file ShowUnitsDialog.java.
References cardListAdapter, row, and soundManager.
Referenced by getDialog(), and onCreate().

5.55.3 Member Function Documentation

5.55.3.1 getDialog()

```
Single< ShowUnitsDialog > getDialog (
            @NonNull Context context,
            @NonNull RowType row,
            @NonNull SoundManager soundManager)  [static]
```

Creates a new ShowUnitsDialog in the given Context and for the given row.

Parameters

context	Context the Dialog is shown in.
row	RowType defining the row that is represented by this Dialog.
soundManager	SoundManager used to play sounds when cards are added or removed.

Returns

A Single emitting the created ShowUnitsDialog.

Definition at line 94 of file ShowUnitsDialog.java.
References ShowUnitsDialog(), disposables, GwentApplication.getRepository(), UnitEntity.isEpic(), RemoveUnitsUseCase.remove(), row, and soundManager.
Referenced by MainActivity.initializeViewModel().

5.55.3.2 getRow()

```
RowType getRow ()
```

Returns the row this ShowUnitsDialog is representing.

Returns

A RowType defining the represented row.

See also

.row

Definition at line 191 of file ShowUnitsDialog.java.
References row.

5.55.3.3 onCreate()

```
void onCreate (
            Bundle savedInstanceState)  [protected]
```

Initializes the RecyclerView by connecting it to the .cardListAdapter and sets View.OnClickListener for each button. Also registers a RecyclerView.AdapterDataObserver responsible for scrolling to the end of the RecyclerView whenever an item is being inserted.

See also

CardListAdapter.registerAdapterDataObserver(RecyclerView.AdapterDataObserver)

Parameters

savedInstanceState	If this dialog is being reinitialized after the hosting activity was previously shut down, holds the result from the most recent call to onSaveInstanceState, or null if this is the first time.

Reimplemented from OverlayDialog.
Definition at line 142 of file ShowUnitsDialog.java.
References ShowUnitsDialog(), cardListAdapter, disposables, soundManager, and unitRecyclerView.

5.55.4 Member Data Documentation

5.55.4.1 cardListAdapter

```
final CardListAdapter cardListAdapter  [private]
```

CardListAdapter offering a list of CardUiState objects to the UI.
Definition at line 44 of file ShowUnitsDialog.java.
Referenced by ShowUnitsDialog(), and onCreate().

5.55.4.2 disposables

```
final CompositeDisposable disposables = new CompositeDisposable()  [private]
```

CompositeDisposable keeping track of all subscriptions to observables made by this class.
Is being disposed in an android.content.DialogInterface.OnDismissListener that is set in .onCreate().

See also

android.content.DialogInterface.OnDismissListener

.onCreate(Bundle)

Definition at line 63 of file ShowUnitsDialog.java.
Referenced by getDialog(), and onCreate().

5.55.4.3 row

```
final RowType row  [private]
```

RowType defining which row is represented by this Dialog.

See also

.getRow()

Definition at line 38 of file ShowUnitsDialog.java.
Referenced by ShowUnitsDialog(), getDialog(), and getRow().

5.55.4.4 soundManager

```
final SoundManager soundManager  [private]
```

SoundManager used to play a Sound whenever a card is added or removed.

See also

SoundManager.playCardAddSound(RowType, boolean)

SoundManager.playCardRemovedSound()

Definition at line 53 of file ShowUnitsDialog.java.
Referenced by ShowUnitsDialog(), getDialog(), and onCreate().

5.55.4.5 unitRecyclerView

```
RecyclerView unitRecyclerView  [private]
```

RecyclerView presenting the units provided by the cardListAdapter.
Definition at line 68 of file ShowUnitsDialog.java.
Referenced by onCreate(), and ShowUnitsDialog.ScrollToEndObserver.onItemRangeInserted().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/cards/ShowUnitsDialog.java

5.56 Sound Class Reference

A data class encapsulating information on a certain sound effect, e.g., its id in a SoundPool and whether it is activated in settings.
Collaboration diagram for Sound:



Package Functions

- String getPreferenceKey ()

  Returns the key where this Sound is activated in the SharedPreferences.
- int getSoundId ()

  Returns the id of this sound effect at the SoundPool.
- boolean isActivated ()

  Returns whether the sound is activated in the settings.
- void setActivated (boolean activated)

  Changes the value of .activated to the given Boolean.
- void setActivated (@NonNull SharedPreferences sharedPreferences)

  Changes the value of .activated based on the given SharedPreferences.

Static Package Functions

- static Sound createSound (@NonNull Context context, @StringRes int preferenceRes, @NonNull SoundPool soundPool, @RawRes int resId)

  Returns a new Sound object encapsulating data for a newly registered sound effect.

Private Member Functions

- Sound (@NonNull String preferenceKey, int soundId, boolean activated)

  Constructor of a Sound encapsulating the given values.

Private Attributes

- final String preferenceKey

  String representing a key of the default SharedPreferences.
- final int soundId

  Integer representing the id of the registered sound at a SoundPool.
- boolean activated

  Boolean defining whether the sound is muted in the settings.

5.56.1 Detailed Description

A data class encapsulating information on a certain sound effect, e.g., its id in a SoundPool and whether it is activated in settings.
Definition at line 20 of file Sound.java.

5.56.2 Constructor & Destructor Documentation

5.56.2.1 Sound()

```
Sound (
            @NonNull String preferenceKey,
            int soundId,
            boolean activated)  [private]
```

Constructor of a Sound encapsulating the given values.
Should only be called by factory method .createSound()

Parameters

preferenceKey	String representing the key where the activated status is defined in SharedPreferences.
soundId	Integer representing the id of the registered sound at a SoundPool.
activated	Boolean defining whether the sound is activated in the settings.

See also

.createSound(Context, int, SoundPool, int)

Definition at line 48 of file Sound.java.
References activated, preferenceKey, and soundId.
Referenced by createSound().

5.56.3 Member Function Documentation

5.56.3.1 createSound()

```
Sound createSound (
            @NonNull Context context,
            @StringRes int preferenceRes,
            @NonNull SoundPool soundPool,
            @RawRes int resId)  [static], [package]
```

Returns a new Sound object encapsulating data for a newly registered sound effect.

Factory method for the Sound class. Gets .preferenceKey from the given Context. Registers the sound with the given resId at the given SoundPool. Decides the value of .activated from SharedPreferences obtained using the given Context.

Parameters

context	Context of the application where this Sound is used.
preferenceRes	Integer referencing the String resource of the preference key for this Sound.
soundPool	SoundPool where the Sound is registered.
resId	Integer referencing the raw resource of the Sound.

Returns

A Sound object referencing the newly registered sound effect.

Definition at line 69 of file Sound.java.
References Sound(), activated, createSound(), preferenceKey, and soundId.
Referenced by SoundManager.SoundManager(), and createSound().

5.56.3.2 getPreferenceKey()

```
String getPreferenceKey ()  [package]
```

Returns the key where this Sound is activated in the SharedPreferences.

Returns

A String referencing the activation option.

Definition at line 86 of file Sound.java.
References preferenceKey.

5.56.3.3 getSoundId()

```
int getSoundId ()  [package]
```

Returns the id of this sound effect at the SoundPool.

Returns

An Integer referencing the sound effect at the SoundPool.

Definition at line 95 of file Sound.java.
References soundId.

5.56.3.4 isActivated()

```
boolean isActivated ()  [package]
```

Returns whether the sound is activated in the settings.

Returns

A Boolean defining whether the Sound is activated.

Definition at line 104 of file Sound.java.
References activated.

5.56.3.5 setActivated() [1/2]

```
void setActivated (
            @NonNull SharedPreferences sharedPreferences)  [package]
```

Changes the value of .activated based on the given SharedPreferences.

Only queries the boolean at key .preferenceKey, not whether all sounds are deactivated!

Parameters

sharedPreferences	SharedPreferences containing information on the activated sounds.

Definition at line 124 of file Sound.java.
References activated, preferenceKey, and setActivated().

5.56.3.6 setActivated() [2/2]

```
void setActivated (
            boolean activated)  [package]
```

Changes the value of .activated to the given Boolean.

Parameters

activated	Boolean defining whether the Sound is activated.

See also

.setActivated(SharedPreferences)

Definition at line 114 of file Sound.java.
References activated.
Referenced by setActivated().

5.56.4 Member Data Documentation

5.56.4.1 activated

```
boolean activated  [private]
```

Boolean defining whether the sound is muted in the settings.
Definition at line 37 of file Sound.java.
Referenced by Sound(), createSound(), isActivated(), setActivated(), and setActivated().

5.56.4.2 preferenceKey

```
final String preferenceKey  [private]
```

String representing a key of the default SharedPreferences.
The referenced preference decides whether the sound is activated (assuming that sounds are not completely disabled).
Definition at line 27 of file Sound.java.
Referenced by Sound(), createSound(), getPreferenceKey(), and setActivated().

5.56.4.3 soundId

```
final int soundId  [private]
```

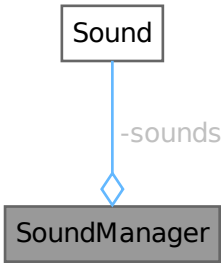
Integer representing the id of the registered sound at a SoundPool.
Definition at line 32 of file Sound.java.
Referenced by Sound(), createSound(), and getSoundId().
The documentation for this class was generated from the following file:

- gwent/ui/sounds/Sound.java

5.57 SoundManager Class Reference

A class responsible for initializing and playing sounds when they are enabled.

Collaboration diagram for SoundManager:



Public Member Functions

- SoundManager (@NonNull Context context)

  Constructor of a SoundManager in the given Context.
- void playSound (@IntRange(from=SOUND_WEATHER_GOOD, to=SOUND_COIN) int soundId)

  Plays the given Sound if it is .activated.
- void playClearWeatherSound ()

  Plays the clear weather sound.
- void playWeatherSound (@NonNull RowType row)

  Plays the weather sound of the given row.
- void playHornSound ()

  Plays the horn sound.
- void playCardAddSound (@NonNull RowType row, boolean epic)

  Plays the card add sound for the given row.
- void playCardRemovedSound ()

  Plays the reset sound.
- void playResetSound ()

  Plays the reset sound.
- void playBurnSound ()

  Plays the burn sound.
- void playCoinSound ()

  Plays the coin sound.

Static Public Attributes

- static final int SOUND_WEATHER_GOOD = 0

  Constant Integer representing the clear weather sound.
- static final int SOUND_WEATHER_FROST = 1

  Constant Integer representing the frost weather sound.
- static final int SOUND_WEATHER_FOG = 2

  Constant Integer representing the fog weather sound.
- static final int SOUND_WEATHER_RAIN = 3

  Constant Integer representing the rain weather sound.
- static final int SOUND_HORN = 4

  Constant Integer representing the horn sound.

- static final int SOUND_CARDS_EPIC = 5

  Constant Integer representing the epic unit sound.
- static final int SOUND_CARDS_MELEE = 6

  Constant Integer representing the melee unit sound.
- static final int SOUND_CARDS_RANGE = 7

  Constant Integer representing the range unit sound.
- static final int SOUND_CARDS_SIEGE = 8

  Constant Integer representing the siege unit sound.
- static final int SOUND_RESET = 9

  Constant Integer representing the reset sound.
- static final int SOUND_BURN = 10

  Constant Integer representing the scorch sound.
- static final int SOUND_COIN = 11

  Constant Integer representing the coin-flip sound.

Private Attributes

- final Sound[ ] sounds = new Sound[SOUND_COIN + 1]

  Array of Sound objects representing the different sound effects.
- final SoundPool soundPool

  SoundPool where sound effects are registered.
- final SharedPreferences.OnSharedPreferenceChangeListener changeListener

  OnSharedPreferenceChangeListener that is registered for the default SharedPreferences and updates the Sound.↩ active attributes of all .sounds when a preference change occurs.

5.57.1 Detailed Description

A class responsible for initializing and playing sounds when they are enabled.

See also

Sound

Definition at line 23 of file SoundManager.java.

5.57.2 Constructor & Destructor Documentation

5.57.2.1 SoundManager()

```
SoundManager (
            @NonNull Context context)
```

Constructor of a SoundManager in the given Context.
Creates a new .soundPool and registers .sounds using Sound.createSound(). Also registers a new .changeListener that updates the Sound.activated status when sound settings are updated.

Parameters

context	Context, this SoundManager is used in.

See also

Sound.createSound(Context, int, SoundPool, int)

Definition at line 156 of file SoundManager.java.
References SoundManager(), changeListener, Sound.createSound(), SOUND_BURN, SOUND_CARDS_EPIC, SOUND_CARDS_MELEE, SOUND_CARDS_RANGE, SOUND_CARDS_SIEGE, SOUND_COIN, SOUND_HORN, SOUND_RESET, SOUND_WEATHER_FOG, SOUND_WEATHER_FROST, SOUND_WEATHER_GOOD, SOUND_WEATHER_RAIN, soundPool, and sounds.
Referenced by SoundManager().

5.57.3 Member Function Documentation

5.57.3.1 playBurnSound()

```
void playBurnSound ()
```

Plays the burn sound.
Wrapper for .playSound().

See also

.playSound(int)

Definition at line 311 of file SoundManager.java.
References playSound(), and SOUND_BURN.

5.57.3.2 playCardAddSound()

```
void playCardAddSound (
            @NonNull RowType row,
            boolean epic)
```

Plays the card add sound for the given row.
If the unit is epic, the epic sound is played. Wrapper for .playSound().

Parameters

row	RowType referencing the row for which the sound should be played.
epic	Boolean defining whether the added unit is epic.

See also

.playSound(int)

Definition at line 268 of file SoundManager.java.
References playSound(), SOUND_CARDS_EPIC, SOUND_CARDS_MELEE, SOUND_CARDS_RANGE, and SOUND_CARDS_SIEGE.

5.57.3.3 playCardRemovedSound()

```
void playCardRemovedSound ()
```

Plays the reset sound.
Wrapper for .playSound().

See also

.playSound(int)

Definition at line 291 of file SoundManager.java.
References playSound(), and SOUND_RESET.

5.57.3.4 playClearWeatherSound()

```
void playClearWeatherSound ()
```

Plays the clear weather sound.
Wrapper for .playSound().

See also

.playSound(int)

Definition at line 225 of file SoundManager.java.
References playSound(), and SOUND_WEATHER_GOOD.

5.57.3.5 playCoinSound()

```
void playCoinSound ()
```

Plays the coin sound.
Wrapper for .playSound().

See also

.playSound(int)

Definition at line 320 of file SoundManager.java.
References playSound(), and SOUND_COIN.

5.57.3.6 playHornSound()

```
void playHornSound ()
```

Plays the horn sound.
Wrapper for .playSound().

See also

.playSound(int)

Definition at line 255 of file SoundManager.java.
References playSound(), and SOUND_HORN.

5.57.3.7 playResetSound()

```
void playResetSound ()
```

Plays the reset sound.
Wrapper for .playSound().

See also

.playSound(int)

Definition at line 301 of file SoundManager.java.
References playSound(), and SOUND_RESET.

5.57.3.8 playSound()

```
void playSound (
            @IntRange(from=SOUND_WEATHER_GOOD, to=SOUND_COIN) int soundId)
```

Plays the given Sound if it is .activated.
Given soundId must be either .SOUND_WEATHER_GOOD, .SOUND_WEATHER_FROST, .SOUND_WEATHER_FOG, .SOUND_WEATHER_RAIN, .SOUND_HORN, .SOUND_CARDS_EPIC, .SOUND_CARDS_MELEE, .SOUND_CARDS_RANGE, .SOUND_CARDS_SIEGE, .SOUND_RESET, .SOUND_BURN, or .SOUND_COIN, i.e., $0 \leq soundId \leq 11$ must be true. Alternately, .playClearWeatherSound(), .playWeatherSound(), .playHornSound(), .playCardAddSound(), .playCardRemovedSound(), .playResetSound(), .playBurnSound(), or .playCoinSound() may be used.

Parameters

soundId	Integer representing the Sound that should be played.

Exceptions

IndexOutOfBoundsException	When $0 \leq soundId \leq 11$ is not true.

Definition at line 213 of file SoundManager.java.
References SOUND_COIN, SOUND_WEATHER_GOOD, soundPool, and sounds.
Referenced by playBurnSound(), playCardAddSound(), playCardRemovedSound(), playClearWeatherSound(), playCoinSound(), playHornSound(), playResetSound(), and playWeatherSound().

5.57.3.9 playWeatherSound()

```
void playWeatherSound (
            @NonNull RowType row)
```

Plays the weather sound of the given row.
Wrapper for .playSound().

Parameters

row	RowType referencing the row for which the sound should be played.

See also

.playSound(int)

Definition at line 236 of file SoundManager.java.
References playSound(), SOUND_WEATHER_FOG, SOUND_WEATHER_FROST, and SOUND_WEATHER_RAIN.

5.57.4 Member Data Documentation

5.57.4.1 changeListener

```
final SharedPreferences.OnSharedPreferenceChangeListener changeListener  [private]
```

OnSharedPreferenceChangeListener that is registered for the default SharedPreferences and updates the Sound.↩ active attributes of all .sounds when a preference change occurs.
Reference must be kept (even if not used) to avoid garbage collection of the registered listener (see here for more information).
Definition at line 145 of file SoundManager.java.
Referenced by SoundManager().

5.57.4.2 SOUND_BURN

```
final int SOUND_BURN = 10  [static]
```

Constant Integer representing the scorch sound.
Index of the respective Sound object in .sounds.

See also

.playBurnSound()

Definition at line 110 of file SoundManager.java.
Referenced by SoundManager(), and playBurnSound().

5.57.4.3 SOUND_CARDS_EPIC

```
final int SOUND_CARDS_EPIC = 5  [static]
```

Constant Integer representing the epic unit sound.
Index of the respective Sound object in .sounds.

See also

.playCardAddSound(RowType, boolean)

Definition at line 70 of file SoundManager.java.
Referenced by SoundManager(), and playCardAddSound().

5.57.4.4 SOUND_CARDS_MELEE

```
final int SOUND_CARDS_MELEE = 6  [static]
```

Constant Integer representing the melee unit sound.
Index of the respective Sound object in .sounds.

See also

.playCardAddSound(RowType, boolean)

Definition at line 78 of file SoundManager.java.
Referenced by SoundManager(), and playCardAddSound().

5.57.4.5 SOUND_CARDS_RANGE

```
final int SOUND_CARDS_RANGE = 7  [static]
```

Constant Integer representing the range unit sound.
Index of the respective Sound object in .sounds.

See also

.playCardAddSound(RowType, boolean)

Definition at line 86 of file SoundManager.java.
Referenced by SoundManager(), and playCardAddSound().

5.57.4.6 SOUND_CARDS_SIEGE

```
final int SOUND_CARDS_SIEGE = 8  [static]
```

Constant Integer representing the siege unit sound.
Index of the respective Sound object in .sounds.

See also

.playCardAddSound(RowType, boolean)

Definition at line 94 of file SoundManager.java.
Referenced by SoundManager(), and playCardAddSound().

5.57.4.7 SOUND_COIN

```
final int SOUND_COIN = 11  [static]
```

Constant Integer representing the coin-flip sound.
Index of the respective Sound object in .sounds.

See also

.playCoinSound()

Definition at line 118 of file SoundManager.java.
Referenced by SoundManager(), playCoinSound(), and playSound().

5.57.4.8 SOUND_HORN

```
final int SOUND_HORN = 4  [static]
```

Constant Integer representing the horn sound.
Index of the respective Sound object in .sounds.

See also

.playHornSound()

Definition at line 62 of file SoundManager.java.
Referenced by SoundManager(), and playHornSound().

5.57.4.9 SOUND_RESET

`final int SOUND_RESET = 9 [static]`
Constant Integer representing the reset sound.
Index of the respective Sound object in .sounds.

See also

.playResetSound()

Definition at line 102 of file SoundManager.java.
Referenced by SoundManager(), playCardRemovedSound(), and playResetSound().

5.57.4.10 SOUND_WEATHER_FOG

`final int SOUND_WEATHER_FOG = 2 [static]`
Constant Integer representing the fog weather sound.
Index of the respective Sound object in .sounds.

See also

.playCardAddSound(RowType, boolean)

Definition at line 46 of file SoundManager.java.
Referenced by SoundManager(), and playWeatherSound().

5.57.4.11 SOUND_WEATHER_FROST

`final int SOUND_WEATHER_FROST = 1 [static]`
Constant Integer representing the frost weather sound.
Index of the respective Sound object in .sounds.

See also

.playCardAddSound(RowType, boolean)

Definition at line 38 of file SoundManager.java.
Referenced by SoundManager(), and playWeatherSound().

5.57.4.12 SOUND_WEATHER_GOOD

`final int SOUND_WEATHER_GOOD = 0 [static]`
Constant Integer representing the clear weather sound.
Index of the respective Sound object in .sounds.

See also

.playClearWeatherSound()

Definition at line 30 of file SoundManager.java.
Referenced by SoundManager(), playClearWeatherSound(), and playSound().

5.57.4.13 SOUND_WEATHER_RAIN

`final int SOUND_WEATHER_RAIN = 3 [static]`
Constant Integer representing the rain weather sound.
Index of the respective Sound object in .sounds.

See also

.playCardAddSound(RowType, boolean)

Definition at line 54 of file SoundManager.java.
Referenced by SoundManager(), and playWeatherSound().

5.57.4.14 soundPool

```
final SoundPool soundPool  [private]
```

SoundPool where sound effects are registered.
Definition at line 134 of file SoundManager.java.
Referenced by SoundManager(), and playSound().

5.57.4.15 sounds

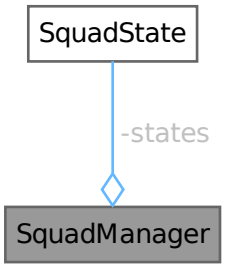
```
final Sound [] sounds = new Sound[SOUND_COIN + 1]  [private]
```

Array of Sound objects representing the different sound effects.
Indices are the public constants defined here, i.e., .SOUND_WEATHER_GOOD, .SOUND_WEATHER_FROST, .SOUND_WEATHER_FOG, .SOUND_WEATHER_RAIN, .SOUND_HORN, .SOUND_CARDS_EPIC, .SOUND_↩ CARDS_MELEE, .SOUND_CARDS_RANGE, .SOUND_CARDS_SIEGE, .SOUND_RESET, .SOUND_BURN, and .SOUND_COIN.
Definition at line 128 of file SoundManager.java.
Referenced by SoundManager(), and playSound().
The documentation for this class was generated from the following file:

- gwent/ui/sounds/SoundManager.java

5.58 SquadManager Class Reference

A class used by the CardNumberPickerAdapter to determine the first squad with members or set the default damage according to the squads base damage.
Collaboration diagram for SquadManager:



Static Public Attributes

- static final int MAX_NR_SQUADS = 3

  Integer constant defining how many different squads there may be per row.

Package Functions

- SquadManager (@NonNull List< UnitEntity > units)

  Constructor of a new SquadManager storing information on the squads of the given units.
- int getFirstSquadWithMembers ()

  Returns the lowest squad number referring to a squad that has members.
- void onSquadChanged (@IntRange(from=1, to=MAX_NR_SQUADS) int newVal, @NonNull DamageValuePicker picker)

  Shows a Toast with information about the selected squad and sets the given picker to the SquadState.memberBaseDamage of said squad.

Private Attributes

- final SquadState[ ] states = new SquadState[MAX_NR_SQUADS]

  Array of SquadState objects containing one SquadState for each possible squad.

5.58.1 Detailed Description

A class used by the CardNumberPickerAdapter to determine the first squad with members or set the default damage according to the squads base damage.

See also

CardNumberPickerAdapter

Definition at line 24 of file SquadManager.java.

5.58.2 Constructor & Destructor Documentation

5.58.2.1 SquadManager()

```
SquadManager (
            @NonNull List< UnitEntity > units)  [package]
```

Constructor of a new SquadManager storing information on the squads of the given units.

Parameters

units	List of UnitEntity objects that is used to fetch squad information.

Definition at line 41 of file SquadManager.java.
References Ability.BINDING, SquadState.getState(), MAX_NR_SQUADS, and states.

5.58.3 Member Function Documentation

5.58.3.1 getFirstSquadWithMembers()

```
int getFirstSquadWithMembers ()  [package]
```

Returns the lowest squad number referring to a squad that has members.
If no squad has members, i.e., there are no units with the Ability.BINDING ability, 1 is returned.

Returns

An Integer referencing to the first squad with members.

Definition at line 55 of file SquadManager.java.
References SquadState.getSquadNumber(), SquadState.hasMembers(), and states.

5.58.3.2 onSquadChanged()

```
void onSquadChanged (
            @IntRange(from=1, to=MAX_NR_SQUADS) int newVal,
            @NonNull DamageValuePicker picker)  [package]
```

Shows a Toast with information about the selected squad and sets the given picker to the SquadState.memberBaseDamage of said squad.

Parameters

newVal	Integer representing the newly selected squad.
picker	DamageValuePicker that is updated.

Exceptions

IllegalArgumentException	When newVal is not between 1 and .MAX_NR_SQUADS.

See also

SquadState.getMemberBaseDamage()

Definition at line 72 of file SquadManager.java.
References SquadState.getMemberBaseDamage(), SquadState.getSquadMembers(), SquadState.getSquadNumber(), SquadState.hasMembers(), MAX_NR_SQUADS, and states.

5.58.4 Member Data Documentation

5.58.4.1 MAX_NR_SQUADS

```
final int MAX_NR_SQUADS = 3  [static]
```

Integer constant defining how many different squads there may be per row.
Definition at line 28 of file SquadManager.java.
Referenced by CardNumberPickerAdapter.CardNumberPickerAdapter(), SquadManager(), and onSquadChanged().

5.58.4.2 states

```
final SquadState [] states = new SquadState[MAX_NR_SQUADS]  [private]
```

Array of SquadState objects containing one SquadState for each possible squad.
Definition at line 34 of file SquadManager.java.
Referenced by SquadManager(), getFirstSquadWithMembers(), and onSquadChanged().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/addcard/SquadManager.java

5.59 SquadState Class Reference

A data class encapsulating information about the squad defined by .squadNumber, i.e., how many .squadMembers are there and what is the .memberBaseDamage of units in this squad.
Collaboration diagram for SquadState:



Package Functions

- boolean hasMembers ()

  Returns whether the represented squad has members, i.e., whether .squadMembers is greater than 0.
- int getSquadNumber ()

  Returns the number of the represented squad.
- int getSquadMembers ()

  Returns the number of members in the represented squad.
- int getMemberBaseDamage ()

  Returns the base damage of members in the represented squad.

Static Package Functions

- static SquadState getState (@IntRange(from=1, to=MAX_NR_SQUADS) int squadNumber, @NonNull List< UnitEntity > units)

  Creates a new SquadState representing the squad with the given .squadNumber.

Private Member Functions

- SquadState (@IntRange(from=1, to=MAX_NR_SQUADS) int squadNumber, @IntRange(from=0) int squadMembers, @IntRange(from=0, to=NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int memberBaseDamage)

  Constructor of a SquadState with the given .squadNumber, .squadMembers, and .memberBaseDamage.

Private Attributes

- final int squadNumber

  Integer containing the number of the represented squad.
- final int squadMembers

  Integer representing the number of members in the squad.
- final int memberBaseDamage

  Integer containing the base damage of members of this squad.

5.59.1 Detailed Description

A data class encapsulating information about the squad defined by .squadNumber, i.e., how many .squadMembers are there and what is the .memberBaseDamage of units in this squad.

See also

SquadManager

Definition at line 19 of file SquadState.java.

5.59.2 Constructor & Destructor Documentation

5.59.2.1 SquadState()

```
SquadState (
            @IntRange(from=1, to=MAX_NR_SQUADS) int squadNumber,
            @IntRange(from=0) int squadMembers,
            @IntRange(from=0, to=NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int memberBaseDamage)
[private]
```

Constructor of a SquadState with the given .squadNumber, .squadMembers, and .memberBaseDamage.
Should only be used by .getState().

Parameters

squadNumber	Integer containing the number of the represented squad.
squadMembers	Integer representing the number of members in the squad.
memberBaseDamage	Integer containing the base damage of members of this squad.

See also

.getState(int, List)

Exceptions

IllegalArgumentException	When one of the parameters doesn't meet its IntRange constraint.

Definition at line 55 of file SquadState.java.
References SquadState(), memberBaseDamage, squadMembers, and squadNumber.
Referenced by SquadState(), and getState().

5.59.3 Member Function Documentation

5.59.3.1 getMemberBaseDamage()

`int getMemberBaseDamage () [package]`
Returns the base damage of members in the represented squad.

See also

.memberBaseDamage

Returns

An Integer defining the base damage of units in the represented squad.

Definition at line 127 of file SquadState.java.
References getMemberBaseDamage(), and memberBaseDamage.
Referenced by getMemberBaseDamage(), and SquadManager.onSquadChanged().

5.59.3.2 getSquadMembers()

`int getSquadMembers () [package]`
Returns the number of members in the represented squad.

See also

.squadMembers

Returns

An Integer defining how many members are in the represented squad.

Definition at line 117 of file SquadState.java.
References getSquadMembers(), and squadMembers.
Referenced by getSquadMembers(), and SquadManager.onSquadChanged().

5.59.3.3 getSquadNumber()

`int getSquadNumber () [package]`
Returns the number of the represented squad.

See also

.squadNumber

Returns

An Integer defining which squad is represented.

Definition at line 107 of file SquadState.java.
References getSquadNumber(), and squadNumber.
Referenced by SquadManager.getFirstSquadWithMembers(), getSquadNumber(), and SquadManager.onSquadChanged().

5.59.3.4 getState()

```
SquadState getState (
            @IntRange(from=1, to=MAX_NR_SQUADS) int squadNumber,
            @NonNull List< UnitEntity > units)  [static], [package]
```

Creates a new SquadState representing the squad with the given .squadNumber.
Information for .squadMembers and .memberBaseDamage is retrieved from the given List of UnitEntity objects.

Parameters

squadNumber	Integer containing the number of the represented squad.
units	List of UnitEntity objects used to count squad-members.

Returns

A SquadState object that is newly created from the given List of units.

Exceptions

IllegalArgumentException	When the given squad number is not between 1 and .MAX_NR_SQUADS.

Definition at line 80 of file SquadState.java.
References SquadState(), UnitEntity.getDamage(), and squadNumber.
Referenced by SquadManager.SquadManager().

5.59.3.5 hasMembers()

```
boolean hasMembers ()  [package]
```

Returns whether the represented squad has members, i.e., whether .squadMembers is greater than 0.

Returns

A Boolean defining whether the represented squad has members or not.

Definition at line 97 of file SquadState.java.
References squadMembers.
Referenced by SquadManager.getFirstSquadWithMembers(), and SquadManager.onSquadChanged().

5.59.4 Member Data Documentation

5.59.4.1 memberBaseDamage

```
final int memberBaseDamage  [private]
```

Integer containing the base damage of members of this squad.
If members have different UnitEntity.damage values, it is undefined which of them is picked. If .squadMembers is 0, .memberBaseDamage defaults to 5.

See also

.getMemberBaseDamage()

Definition at line 43 of file SquadState.java.
Referenced by SquadState(), and getMemberBaseDamage().

5.59.4.2 squadMembers

```
final int squadMembers  [private]
```

Integer representing the number of members in the squad.

See also

.getSquadMembers()

Definition at line 34 of file SquadState.java.
Referenced by SquadState(), getSquadMembers(), and hasMembers().

5.59.4.3 squadNumber

```
final int squadNumber  [private]
```

Integer containing the number of the represented squad.
Equivalent to UnitEntity.squad of the units in the squad.

See also

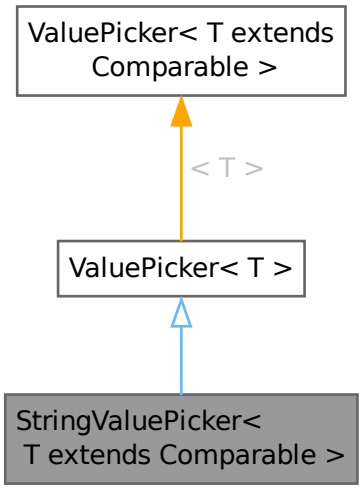
.getSquadNumber()

Definition at line 27 of file SquadState.java.
Referenced by SquadState(), getSquadNumber(), and getState().
The documentation for this class was generated from the following file:

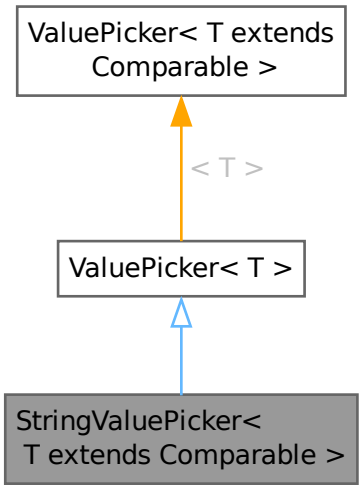
- gwent/ui/dialogs/addcard/SquadState.java

5.60 StringValuePicker< T extends Comparable > Class Template Reference

A ValuePicker displaying a resource String for each element in selectableValues.
Inheritance diagram for StringValuePicker< T extends Comparable >:



Collaboration diagram for StringValuePicker< T extends Comparable >:



Protected Member Functions

- String getDisplayString (@NonNull T value)

  Returns a localized String representing the given value by looking up the respective String id in displayIntegers.

Protected Member Functions inherited from ValuePicker< T >

- NumberPicker getPicker ()

  Returns the NumberPicker wrapped by this ValuePicker.
- abstract String getDisplayString (@NonNull T value)

  Returns a localized String representing the given value.
- Map< T, Integer > getDisplayIntegers ()

  Returns the displayIntegers map.
- List< T > getSelectableValues ()

  Returns a List of values that can be selected by this picker.

Package Functions

- StringValuePicker (@NonNull NumberPicker picker, @NonNull SortedMap< T, Integer > valueToStringRes)

  Constructor of a StringValuePicker wrapping the given NumberPicker and offering the given values.
- StringValuePicker (@NonNull NumberPicker picker, @NonNull SortedMap< T, Integer > valueToStringRes, @Nullable T defaultValue)

  Constructor of a StringValuePicker wrapping the given NumberPicker, offering the given values and showing the given defaultValue.

Package Functions inherited from ValuePicker< T >

- ValuePicker (@NonNull NumberPicker picker, @NonNull SortedMap< T, Integer > valueToStringRes)

  Constructor of a ValuePicker wrapping the given NumberPicker and offering the given values.
- void setSelectableValues (@NonNull Collection< T > values, @Nullable T defaultValue)

  Sets selectableValues to the given values and show the given defaultValue.

- void setOnValueChangedListener (@NonNull OnValueChangeListener< T > onValueChangedListener)

  Sets an OnValueChangeListener that is called when the value of the wrapped .picker has changed and stayed the same for 500 ms.
- T getValue ()

  Returns the currently selected value.
- void setValue (@NonNull T value)

  Sets the picker to the given value.

Additional Inherited Members

Public Member Functions inherited from ValuePicker< T >

- Context getContext ()

  Returns the context the .picker is running in, through which it can access the current theme, resources, etc.

5.60.1 Detailed Description

A ValuePicker displaying a resource String for each element in selectableValues.
The resource String is retrieved from displayIntegers by using the associated Integer as the String id in getDisplayString().

See also

getDisplayString(Comparable)

Definition at line 17 of file StringValuePicker.java.

5.60.2 Constructor & Destructor Documentation

5.60.2.1 StringValuePicker() [1/2]

```
StringValuePicker (
            @NonNull NumberPicker picker,
            @NonNull SortedMap< T, Integer > valueToStringRes)  [package]
```

Constructor of a StringValuePicker wrapping the given NumberPicker and offering the given values.
Wrapper around .StringValuePicker(NumberPicker, SortedMap, Comparable) with null as defaultValue.

Parameters

picker	NumberPicker wrapped by the created StringValuePicker.
valueToStringRes	SortedMap with all selectableValues as keys and the respective displayIntegers (String ids) as value.

Exceptions

IllegalArgumentException	When valueToStringRes is empty.

See also

.StringValuePicker(NumberPicker, SortedMap, Comparable)

Definition at line 28 of file StringValuePicker.java.
References StringValuePicker(), and ValuePicker< T >.picker.
Referenced by StringValuePicker().

5.60.2.2 StringValuePicker() [2/2]

```
StringValuePicker (
            @NonNull NumberPicker picker,
            @NonNull SortedMap< T, Integer > valueToStringRes,
            @Nullable T defaultValue)  [package]
```

Constructor of a StringValuePicker wrapping the given NumberPicker, offering the given values and showing the given defaultValue.
Matches super-constructor ValuePicker(NumberPicker, SortedMap, Comparable).

See also

.StringValuePicker(NumberPicker, SortedMap)

Parameters

picker	NumberPicker wrapped by the created StringValuePicker.
valueToStringRes	SortedMap with all selectableValues as keys and the respective displayIntegers (String ids) as value.
defaultValue	Value that is shown in the beginning. If null, then the first value defined by the Comparable interface is used.

Exceptions

IllegalArgumentException	When valueToStringRes is empty. or when defaultValue is not null but not contained in valueToStringRes.

Definition at line 45 of file StringValuePicker.java.
References ValuePicker< T >.picker.

5.60.3 Member Function Documentation

5.60.3.1 getDisplayString()

```
String getDisplayString (
            @NonNull T value)  [protected]
```

Returns a localized String representing the given value by looking up the respective String id in displayIntegers.

Parameters

value	Value that should be represented as a String.

Returns

A localized String representing the value.

Exceptions

IllegalStateException	When displayIntegers does not contain a String id for the given value.

Definition at line 58 of file StringValuePicker.java.
References ValuePicker< T >.getContext(), and ValuePicker< T >.getDisplayIntegers().
The documentation for this class was generated from the following file:

- gwent/ui/dialogs/addcard/StringValuePicker.java

5.61 ResetDialogUseCase.Trigger Enum Reference

An Enum listing the possible triggers of a reset.
Collaboration diagram for ResetDialogUseCase.Trigger:



Public Attributes

- BUTTON_CLICK

  Represents, that a reset was triggered by a click on the reset button.
- FACTION_SWITCH

  Represents that a reset was triggered by a faction switch.

5.61.1 Detailed Description

An Enum listing the possible triggers of a reset.
Definition at line 130 of file ResetDialogUseCase.java.

5.61.2 Member Data Documentation

5.61.2.1 BUTTON_CLICK

`BUTTON_CLICK`
Represents, that a reset was triggered by a click on the reset button.
Definition at line 134 of file ResetDialogUseCase.java.
Referenced by GameBoardViewModel.onResetButtonPressed().

5.61.2.2 FACTION_SWITCH

`FACTION_SWITCH`
Represents that a reset was triggered by a faction switch.
Only relevant if faction reset is activated, i.e., the preference at the key referenced by R.stringpreference_key_↩ faction_reset is true.
Definition at line 140 of file ResetDialogUseCase.java.
Referenced by GameBoardViewModel.onFactionSwitchReset(), and ResetAlertDialogBuilderAdapter.setTrigger().
The documentation for this enum was generated from the following file:

- gwent/domain/cases/ResetDialogUseCase.java

5.62 UnitDao Interface Reference

An interface defining data access operations related to `units` table.

Collaboration diagram for UnitDao:



Public Member Functions

- Completable insertUnit (@NonNull UnitEntity unit)

  Inserts the given UnitEntity into `units`.
- Completable insertUnit (boolean epic, @IntRange(from=0, to=UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int damage, @NonNull Ability ability, @IntRange(from=1) @Nullable Integer squad, @NonNull RowType row)

  Inserts a UnitEntity with the given values into `units`.
- Completable deleteUnit (int id)

  Deletes the UnitEntity object with the given id from `units`.
- Completable deleteUnits (@NonNull Collection< UnitEntity > units)

  Deletes the given UnitEntity objects from `units`.
- Single< UnitEntity > getUnit (int id)

  Fetches the UnitEntity object with the given id from `units`.
- Single< List< UnitEntity > > getUnits (@NonNull RowType row)

  Fetches all UnitEntity objects from `units` *in the given row.*
- Flowable< List< UnitEntity > > getUnitsFlowable (@NonNull RowType row)

  Fetches a Flowable for all UnitEntity objects from `units` *in the given row.*
- Single< List< UnitEntity > > getUnits ()

  Fetches all UnitEntity objects from `units`.
- Single< Integer > countUnits (@NonNull RowType row)

  Counts UnitEntity objects in `units` *in the given row.*
- Single< Integer > countUnits ()

  Counts UnitEntity objects in `units`.
- Flowable< Boolean > hasNonEpicUnitsFlowable ()

  Fetches a Flowable of Booleans defining whether `units` *contains units that are not epic.*

5.62.1 Detailed Description

An interface defining data access operations related to `units` table.
The implementation is generated by the Room framework.
Definition at line 27 of file UnitDao.java.

5.62.2 Member Function Documentation

5.62.2.1 countUnits() [1/2]

```
Single< Integer > countUnits ()
```

Counts UnitEntity objects in `units`.

Returns

A Single tracking operation status and returning the value.

References countUnits().

5.62.2.2 countUnits() [2/2]

```
Single< Integer > countUnits (
            @NonNull RowType row)
```

Counts UnitEntity objects in `units` in the given row.

Parameters

row	RowType defining the UnitEntity.row foreign key.

Returns

A Single tracking operation status and returning the value.

References countUnits().
Referenced by countUnits(), and countUnits().

5.62.2.3 deleteUnit()

```
Completable deleteUnit (
            int id)
```

Deletes the UnitEntity object with the given id from `units`.

Parameters

id	Integer representing the UnitEntity.id of the deleted UnitEntity.

Returns

A Completable tracking operation status.

References deleteUnit().
Referenced by deleteUnit().

5.62.2.4 deleteUnits()

```
Completable deleteUnits (
            @NonNull Collection< UnitEntity > units)
```

Deletes the given UnitEntity objects from `units`.

Parameters

units	List of UnitEntity that are being inserted.

Returns

A Completable tracking operation status.

References deleteUnits().
Referenced by deleteUnits().

5.62.2.5 getUnit()

```
Single< UnitEntity > getUnit (
            int id)
```

Fetches the UnitEntity object with the given id from `units`.

Parameters

id	Integer representing the UnitEntity.id of the requested UnitEntity.

Returns

A Single tracking operation status and returning the value.

References getUnit().
Referenced by getUnit().

5.62.2.6 getUnits() [1/2]

```
Single< List< UnitEntity > > getUnits ()
```

Fetches all UnitEntity objects from `units`.

Returns

A Single tracking operation status and returning the value.

References getUnits().

5.62.2.7 getUnits() [2/2]

```
Single< List< UnitEntity > > getUnits (
            @NonNull RowType row)
```

Fetches all UnitEntity objects from `units` in the given row.

See also

.getUnitsFlowable(RowType)

Parameters

row	RowType defining the UnitEntity.row foreign key.

Returns

A Single tracking operation status and returning the value.

References getUnits().
Referenced by getUnits(), and getUnits().

5.62.2.8 getUnitsFlowable()

```
Flowable< List< UnitEntity > > getUnitsFlowable (
            @NonNull RowType row)
```

Fetches a Flowable for all UnitEntity objects from `units` in the given row.

Parameters

row	RowType defining the UnitEntity.row foreign key.

Returns

A Flowable emitting the values.

See also

.getUnits(RowType)

References getUnitsFlowable().
Referenced by getUnitsFlowable().

5.62.2.9 hasNonEpicUnitsFlowable()

```
Flowable< Boolean > hasNonEpicUnitsFlowable ()
```

Fetches a Flowable of Booleans defining whether `units` contains units that are not epic.

Returns

A Flowable emitting the values.

References hasNonEpicUnitsFlowable().
Referenced by hasNonEpicUnitsFlowable().

5.62.2.10 insertUnit() [1/2]

```
Completable insertUnit (
            @NonNull UnitEntity unit)
```

Inserts the given UnitEntity into `units`.
If the same UnitEntity.id is already in the table, the new insert is ignored.

Parameters

unit	UnitEntity that is being inserted.

Returns

A Completable tracking operation status.

References insertUnit().
Referenced by insertUnit(), and insertUnit().

5.62.2.11 insertUnit() [2/2]

```
Completable insertUnit (
            boolean epic,
            @IntRange(from=0, to=UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int damage,
            @NonNull Ability ability,
            @IntRange(from=1) @Nullable Integer squad,
            @NonNull RowType row)
```

Inserts a UnitEntity with the given values into `units`.

Parameters

epic	Boolean defining whether the unit is epic.
damage	Non-Negative value defining the base-damage of the unit.
ability	Ability defining the ability of the unit.
squad	Non-Negative value defining the squad of units, if UnitEntity.ability is Ability.BINDING, or `null`.
row	RowType defining the UnitEntity.row foreign key.

Returns

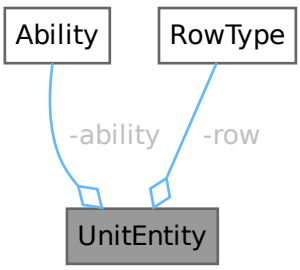
A Completable tracking operation status.

References insertUnit().
The documentation for this interface was generated from the following file:

- gwent/data/UnitDao.java

5.63 UnitEntity Class Reference

A class representing a card on the game board.
Collaboration diagram for UnitEntity:



Public Member Functions

- int calculateDamage (@NonNull DamageCalculator calculator)

  Calculates the damage of this unit when (de-)buffed.
- Color isBuffed (@NonNull DamageCalculator calculator)

  Returns whether this unit is (de-)buffed.
- String toString (@NonNull Context context)

  Returns a string representation of this unit.
- int getId ()

  Getter for .id.
- boolean isEpic ()

  Getter for .epic.
- int getDamage ()

  Getter for .damage.
- Ability getAbility ()

  Getter for .ability.
- Integer getSquad ()

  Getter for .squad.
- RowType getRow ()

  Getter for .row.

Static Public Member Functions

- static String collectionToString (@NonNull Context context, @NonNull Collection< UnitEntity > units)

  Creates a String containing the descriptions of all units in the given collection, separated by commas.

Static Public Attributes

- static final Integer[ ] EPIC_DAMAGE_VALUES = new Integer[ ]{0, 7, 8, 10, 11, 15}

  Integer array containing every damage value an epic unit may have.
- static final int NON_EPIC_DAMAGE_VALUES_UPPER_BOUND = 20

  Integer defining what the maximum damage of a non-epic UnitEntity can be.

Package Functions

- UnitEntity (boolean epic, @IntRange(from=0) int damage, @NonNull Ability ability, @IntRange(from=1) @Nullable Integer squad, @NonNull RowType row)

  Constructor of a UnitEntity.
- void setId (int id)

  Setter for .id.
- void setEpic (boolean epic)

  Setter for .epic.
- void setDamage (@IntRange(from=0) int damage)

  Setter for .damage.
- void setAbility (@NonNull Ability ability)

  Setter for .ability.
- void setSquad (@IntRange(from=1) @Nullable Integer squad)

  Setter for .squad.
- void setRow (@NonNull RowType row)

  Setter for .row.

Private Attributes

- int id

  Defines the primary key of the represented unit.
- boolean epic

  Defines whether the represented card is epic.
- int damage

  Defines the base-damage of the represented card.
- Ability ability

  Defines the Ability of the represented card.
- Integer squad

  Defines the squad the unit belongs to if .ability is Ability.BINDING.
- RowType row

  Defines the attack row the card lies in.

5.63.1 Detailed Description

A class representing a card on the game board.
Is a persistent Entity and is therefore saved in a database table named `units`.
Definition at line 36 of file UnitEntity.java.

5.63.2 Constructor & Destructor Documentation

5.63.2.1 UnitEntity()

```
UnitEntity (
            boolean epic,
            @IntRange(from=0) int damage,
            @NonNull Ability ability,
```

```
            @IntRange(from=1) @Nullable Integer squad,
            @NonNull RowType row)  [package]
```

Constructor of a UnitEntity.
.id may not be set here since the value is generated automatically.

Parameters

epic	Boolean representing whether card is .epic.
damage	Non-negative value representing the .damage of the card.
ability	Ability representing the .ability of the card.
squad	Integer representing the .squad of a card that has the Ability.BINDING .ability.
row	RowType representing the combat type of the card.

Exceptions

IllegalArgumentException	When damage is less than zero or if ability is Ability.BINDING and squad is null or less than one or if ability is not Ability.BINDING and squad is not null.

Definition at line 105 of file UnitEntity.java.
References ability, Ability.BINDING, damage, epic, row, and squad.
Referenced by collectionToString().

5.63.3 Member Function Documentation

5.63.3.1 calculateDamage()

```
int calculateDamage (
            @NonNull DamageCalculator calculator)
```

Calculates the damage of this unit when (de-)buffed.
Returns .damage if .epic is true. Otherwise, the damage is calculated through the given DamageCalculator, which follows the visitor pattern.

Parameters

calculator	DamageCalculator visitor used for damage calculation.

Returns

An Integer representing the units (de-)buffed damage.

See also

.getDamage()

Definition at line 132 of file UnitEntity.java.
References damage, and epic.

5.63.3.2 collectionToString()

```
String collectionToString (
            @NonNull Context context,
            @NonNull Collection< UnitEntity > units)  [static]
```

Creates a String containing the descriptions of all units in the given collection, separated by commas.

If n units have the same description, instead of printing the same description n times, "n×" is printed in front of the description. Unit descriptions are created using .toString(Context). Ordering of units in the given Collection is not guaranteed to be kept.

Parameters

context	Context used to acquire String resources.
units	Collection of UnitEntity objects that should be in the created String.

Returns

A String containing the description of all units.

Exceptions

IllegalArgumentException	When units collection is empty.

See also

.toString(Context)

Definition at line 166 of file UnitEntity.java.
References UnitEntity().
Referenced by BurnDialogUseCase.burn().

5.63.3.3 getAbility()

```
Ability getAbility ()
```

Getter for .ability.

Returns

Ability representing the units' ability.

Definition at line 315 of file UnitEntity.java.
References ability.
Referenced by toString().

5.63.3.4 getDamage()

```
int getDamage ()
```

Getter for .damage.

Returns

Integer representing the card's base-damage.

See also

.calculateDamage(DamageCalculator)

Definition at line 291 of file UnitEntity.java.
References damage.
Referenced by SquadState.getState(), and toString().

5.63.3.5 getId()

```
int getId ()
```
Getter for .id.

Returns

Integer representing the units' id.

Definition at line 252 of file UnitEntity.java.
References id.
Referenced by UnitRepository.getUnitsFlowable(), DamageCalculatorBuildDirector.setHorns(), and DamageCalculatorBuildDirector.se

5.63.3.6 getRow()

```
RowType getRow ()
```
Getter for .row.

Returns

RowType representing the units combat row.

Definition at line 362 of file UnitEntity.java.
References row.
Referenced by toString().

5.63.3.7 getSquad()

```
Integer getSquad ()
```
Getter for .squad.

Returns

Integer representing the units' squad if .ability is Ability.BINDING or `null`.

Definition at line 335 of file UnitEntity.java.
References squad.
Referenced by toString().

5.63.3.8 isBuffed()

```
Color isBuffed (
            @NonNull DamageCalculator calculator)
```
Returns whether this unit is (de-)buffed.
Returns Color.DEFAULT if .epic is true. Otherwise, whether the unit is (de-)buffed is calculated through the given DamageCalculator, which follows the visitor pattern.

Parameters

calculator	DamageCalculator visitor used for damage calculation.

Returns

A Color representing the units (de-)buffed damage.

See also

.getDamage()

Definition at line 147 of file UnitEntity.java.
References epic.

5.63.3.9 isEpic()

```
boolean isEpic ()
```

Getter for .epic.

Returns

Boolean representing whether the card is epic.

Definition at line 271 of file UnitEntity.java.
References epic.
Referenced by ShowUnitsDialog.getDialog(), and toString().

5.63.3.10 setAbility()

```
void setAbility (
            @NonNull Ability ability)  [package]
```

Setter for .ability.
Only used by Room extension.

Parameters

ability	Ability representing the units' ability.

Definition at line 325 of file UnitEntity.java.
References ability.

5.63.3.11 setDamage()

```
void setDamage (
            @IntRange(from=0) int damage)  [package]
```

Setter for .damage.
Only used by Room extension.

Parameters

damage	Integer representing the card's base-damage.

Exceptions

IllegalArgumentException	When damage is less than zero.

Definition at line 302 of file UnitEntity.java.
References damage.

5.63.3.12 setEpic()

```
void setEpic (
            boolean epic)  [package]
```

Setter for .epic.
Only used by Room extension.

Parameters

epic	Boolean representing whether the card is epic.

Definition at line 281 of file UnitEntity.java.
References epic.

5.63.3.13 setId()

```
void setId (
            int id)  [package]
```

Setter for .id.
Only used by Room extension.

Parameters

id	Integer representing the units' id.

Definition at line 262 of file UnitEntity.java.
References id.

5.63.3.14 setRow()

```
void setRow (
            @NonNull RowType row)  [package]
```

Setter for .row.
Only used by Room extension.

Parameters

row	RowType representing the units combat row.

Definition at line 372 of file UnitEntity.java.
References row.

5.63.3.15 setSquad()

```
void setSquad (
            @IntRange(from=1) @Nullable Integer squad)  [package]
```

Setter for .squad.
Only used by Room extension.

Parameters

squad	Integer representing the units' squad if .ability is Ability.BINDING or `null`.

Exceptions

IllegalArgumentException	When .ability is Ability.BINDING and squad is null or less than 1 or if .ability is not Ability.BINDING and squad is not null.

Definition at line 346 of file UnitEntity.java.
References ability, Ability.BINDING, and squad.

5.63.3.16 toString()

```
String toString (
            @NonNull Context context)
```

Returns a string representation of this unit.
The representation contains information on each field of this class, i.e., .row, .epic, .damage, .ability, and .squad.

Parameters

context	Context used to acquire String resources.

Returns

A string representing the unit.

Definition at line 210 of file UnitEntity.java.
References ability, epic, getAbility(), getDamage(), getRow(), getSquad(), isEpic(), row, and squad.

5.63.4 Member Data Documentation

5.63.4.1 ability

```
Ability ability  [private]
```

Defines the Ability of the represented card.
Is set to Ability.NONE by default. Must not be `null`.
Definition at line 75 of file UnitEntity.java.
Referenced by UnitEntity(), getAbility(), setAbility(), setSquad(), and toString().

5.63.4.2 damage

```
int damage  [private]
```

Defines the base-damage of the represented card.
Must be non-negative.
Definition at line 55 of file UnitEntity.java.
Referenced by UnitEntity(), calculateDamage(), getDamage(), and setDamage().

5.63.4.3 epic

```
boolean epic  [private]
```

Defines whether the represented card is epic.
Is set to `false` by default.
Definition at line 49 of file UnitEntity.java.
Referenced by UnitEntity(), calculateDamage(), isBuffed(), isEpic(), setEpic(), and toString().

5.63.4.4 EPIC_DAMAGE_VALUES

```
final Integer [] EPIC_DAMAGE_VALUES = new Integer[]{0, 7, 8, 10, 11, 15}  [static]
```

Integer array containing every damage value an epic unit may have.
Definition at line 61 of file UnitEntity.java.
Referenced by CardNumberPickerAdapter.addSelectedUnits(), and UnitRepository.insertUnit().

5.63.4.5 id

```
int id  [private]
```

Defines the primary key of the represented unit.
Is generated automatically on insert.
Definition at line 42 of file UnitEntity.java.
Referenced by getId(), and setId().

5.63.4.6 NON_EPIC_DAMAGE_VALUES_UPPER_BOUND

```
final int NON_EPIC_DAMAGE_VALUES_UPPER_BOUND = 20  [static]
```

Integer defining what the maximum damage of a non-epic UnitEntity can be.

Definition at line 66 of file UnitEntity.java.
Referenced by UnitRepository.insertUnit().

5.63.4.7 row

`RowType row [private]`
Defines the attack row the card lies in.
Must not be `null`.
Definition at line 92 of file UnitEntity.java.
Referenced by UnitEntity(), getRow(), setRow(), and toString().

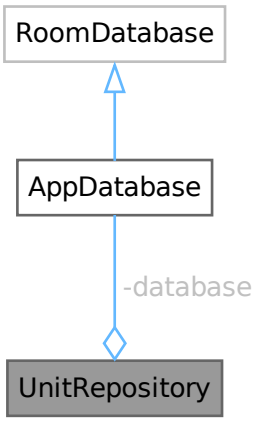
5.63.4.8 squad

`Integer squad [private]`
Defines the squad the unit belongs to if .ability is Ability.BINDING.
If .ability is anything else, this value is `null`.
Definition at line 84 of file UnitEntity.java.
Referenced by UnitEntity(), getSquad(), setSquad(), and toString().
The documentation for this class was generated from the following file:

- gwent/data/UnitEntity.java

5.64 UnitRepository Class Reference

A facade class managing public access to the data layer.
Collaboration diagram for UnitRepository:



Public Member Functions

- Completable reset ()

  Resets the board by removing all units and resetting row status.
- Completable reset (@Nullable UnitEntity keptUnit)

  Resets the board by removing all units but the given one and resetting row status.
- Completable insertUnit (boolean epic, @IntRange(from=0, to=UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int damage, @NonNull Ability ability, @IntRange(from=1) @Nullable Integer squad, @NonNull RowType row, @IntRange(from=0) int number)

Adds a number of units with the given stats to the given row.

- Completable switchWeather (@NonNull RowType row)

  Flips RowEntity.weather of the given attack row.
- Single< Boolean > isWeather (@NonNull RowType row)

  Returns the value of RowEntity.weather for the given attack row.
- Flowable< Boolean > isWeatherFlowable (@NonNull RowType row)

  Returns a Flowable emitting the latest value of RowEntity.weather for the given attack row.
- Completable clearWeather ()

  Sets RowEntity.weather to `false` for all attack rows.
- Completable switchHorn (@NonNull RowType row)

  Flips RowEntity.horn of the given attack row.
- Single< Boolean > isHorn (@NonNull RowType row)

  Returns the value of RowEntity.horn for the given attack row.
- Flowable< Boolean > isHornFlowable (@NonNull RowType row)

  Returns a Flowable emitting the values of RowEntity.horn for the given attack row.
- Completable delete (@NonNull Collection< UnitEntity > units)

  Removes the given units from the game.
- Completable delete (int id)

  Removes the unit with the given id from the game.
- Completable copy (int id)

  Copies the unit with the given id.
- Single< Integer > countUnits (@NonNull RowType row)

  Counts the units in the given attack row.
- Single< Integer > countUnits ()

  Counts the units in all attack rows.
- Single< UnitEntity > getUnit (int id)

  Returns the unit with the given id.
- Single< List< UnitEntity > > getUnits (@NonNull RowType row)

  Returns the units in the given attack row.
- Flowable< List< UnitEntity > > getUnitsFlowable (@NonNull RowType row)

  Returns a Flowable emitting the units in the given attack row.
- Single< List< UnitEntity > > getUnits ()

  Returns the units in the given attack row.
- Flowable< Boolean > hasNonEpicUnitsFlowable ()

  Returns a Flowable emitting Boolean values defining whether there are units in the game that are not epic.

Static Public Member Functions

- static Single< UnitRepository > getRepository (@NonNull AppDatabase database)

  Factory method for the UnitRepository class.

Private Member Functions

- UnitRepository (@NonNull AppDatabase database)

  Constructor of a UnitRepository.
- Completable initializeRows ()

  Adds one attack row for each RowType.
- Completable insertUnit (boolean epic, @IntRange(from=0, to=UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int damage, @NonNull Ability ability, @IntRange(from=1) @Nullable Integer squad, @NonNull RowType row)

  Adds a unit with the given stats to the given row.

Private Attributes

- final AppDatabase database

  Defines the AppDatabase that is used as a data source by this repository.

5.64.1 Detailed Description

A facade class managing public access to the data layer.
The contained functions mostly redirect requests to package-private DAO methods in RowDao and UnitDao. Some functions implement slightly more complex behavior by chaining multiple DAO calls, e.g., .reset().
Definition at line 25 of file UnitRepository.java.

5.64.2 Constructor & Destructor Documentation

5.64.2.1 UnitRepository()

```
UnitRepository (
            @NonNull AppDatabase database)  [private]
```

Constructor of a UnitRepository.
Depends on the given AppDatabase as a data source. Should only be called by .getRepository().

Parameters

database	AppDatabase that is injected for the repository.

See also

.getRepository(AppDatabase)

Definition at line 41 of file UnitRepository.java.
References database.
Referenced by getRepository().

5.64.3 Member Function Documentation

5.64.3.1 clearWeather()

```
Completable clearWeather ()
```

Sets RowEntity.weather to `false` for all attack rows.

Returns

A Completable tracking operation status.

Definition at line 208 of file UnitRepository.java.
References database.
Referenced by GameBoardViewModel.onWeatherButtonPressed().

5.64.3.2 copy()

```
Completable copy (
            int id)
```

Copies the unit with the given id.

Parameters

id	Integer representing the unit that should be copied.

Returns

A Completable tracking operation status.

Definition at line 277 of file UnitRepository.java.
References getUnit(), and insertUnit().

5.64.3.3 countUnits() [1/2]

```
Single< Integer > countUnits ()
```

Counts the units in all attack rows.

Returns

A Single tracking operation status and returning the value.

See also

.countUnits(RowType)

Definition at line 302 of file UnitRepository.java.
References database.

5.64.3.4 countUnits() [2/2]

```
Single< Integer > countUnits (
            @NonNull RowType row)
```

Counts the units in the given attack row.

Parameters

row	RowEntity.id where the units are counted.

Returns

A Single tracking operation status and returning the value.

See also

.countUnits()

Definition at line 291 of file UnitRepository.java.
References database.

5.64.3.5 delete() [1/2]

```
Completable delete (
            @NonNull Collection< UnitEntity > units)
```

Removes the given units from the game.

Parameters

units	List of units to be removed.

Returns

A Completable tracking operation status.

Definition at line 256 of file UnitRepository.java.
References database.

5.64.3.6 delete() [2/2]

```
Completable delete (
            int id)
```

Removes the unit with the given id from the game.

Parameters

id	Integer representing the unit that should be deleted.

Returns

A Completable tracking operation status.

Definition at line 267 of file UnitRepository.java.
References database.

5.64.3.7 getRepository()

```
Single< UnitRepository > getRepository (
            @NonNull AppDatabase database)  [static]
```

Factory method for the UnitRepository class.
Creates a new UnitRepository managing the given AppDatabase. Also initializes one RowEntity per RowType using .initializeRows().

See also

.initializeRows()

Parameters

database	AppDatabase managed and initialized by the returned UnitRepository.

Returns

A Single emitting the created UnitRepository when initialization is finished.

Definition at line 54 of file UnitRepository.java.
References UnitRepository(), database, and initializeRows().
Referenced by GwentApplication.getRepository().

5.64.3.8 getUnit()

```
Single< UnitEntity > getUnit (
            int id)
```

Returns the unit with the given id.

Parameters

id	Integer representing the queried unit.

Returns

A Single tracking operation status and returning the value.

Definition at line 313 of file UnitRepository.java.
References database.
Referenced by copy().

5.64.3.9 getUnits() [1/2]

```
Single< List< UnitEntity > > getUnits ()
```

Returns the units in the given attack row.

Returns

A Single tracking operation status and returning the value.

See also

.getUnits(RowType)

Definition at line 361 of file UnitRepository.java.
References database.

5.64.3.10 getUnits() [2/2]

```
Single< List< UnitEntity > > getUnits (
            @NonNull RowType row)
```

Returns the units in the given attack row.

See also

.getUnitsFlowable(RowType)

Parameters

row	RowEntity.id where the units have been placed.

Returns

A Single tracking operation status and returning the value.

See also

.getUnits()

Definition at line 325 of file UnitRepository.java.
References database.
Referenced by AddCardDialog.onCreate().

5.64.3.11 getUnitsFlowable()

```
Flowable< List< UnitEntity > > getUnitsFlowable (
            @NonNull RowType row)
```

Returns a Flowable emitting the units in the given attack row.

Parameters

row	RowEntity.id where the units have been placed.

Returns

A Flowable emitting the values.

See also

.getUnits(RowType)

.getUnits()

Definition at line 338 of file UnitRepository.java.
References database, and UnitEntity.getId().

5.64.3.12 hasNonEpicUnitsFlowable()

```
Flowable< Boolean > hasNonEpicUnitsFlowable ()
```

Returns a Flowable emitting Boolean values defining whether there are units in the game that are not epic.

Returns

A Flowable emitting the values.

Definition at line 370 of file UnitRepository.java.
References database.

5.64.3.13 initializeRows()

```
Completable initializeRows ()  [private]
```

Adds one attack row for each RowType.
If an attack row already exists, it is not inserted again.

Returns

A Completable tracking operation status.

Definition at line 69 of file UnitRepository.java.
References database.
Referenced by getRepository(), and reset().

5.64.3.14 insertUnit() [1/2]

```
Completable insertUnit (
            boolean epic,
            @IntRange(from=0, to=UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int damage,
            @NonNull Ability ability,
            @IntRange(from=1) @Nullable Integer squad,
            @NonNull RowType row)  [private]
```

Adds a unit with the given stats to the given row.

Parameters

epic	Boolean representing whether card is .epic.
damage	Non-negative value representing the .damage of the card.
ability	Ability representing the .ability of the card.
squad	Integer representing the .squad of a card that has the Ability.BINDING .ability.
row	RowType representing the combat type of the card.

Returns

A Completable tracking operation status.

Exceptions

IllegalArgumentException	When damage is less than zero or if ability is Ability.BINDING and squad is null or less than zero or if ability is not Ability.BINDING and squad is not null.

Definition at line 119 of file UnitRepository.java.
References Ability.BINDING, database, UnitEntity.EPIC_DAMAGE_VALUES, and UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER
Referenced by copy(), and insertUnit().

5.64.3.15 insertUnit() [2/2]

```
Completable insertUnit (
            boolean epic,
            @IntRange(from=0, to=UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int damage,
            @NonNull Ability ability,
            @IntRange(from=1) @Nullable Integer squad,
            @NonNull RowType row,
            @IntRange(from=0) int number)
```

Adds a number of units with the given stats to the given row.
Essentially calls .insertUnit(boolean, int, Ability, Integer, RowType) multiple times.

Parameters

epic	Boolean representing whether card is .epic.
damage	Non-negative value representing the .damage of the card.
ability	Ability representing the .ability of the card.
squad	Integer representing the .squad of a card that has the Ability.BINDING .ability.
row	RowType representing the combat type of the card.
number	Integer representing the number of units to be added.

Returns

A Completable tracking operation status.

See also

.insertUnit(boolean, int, Ability, Integer, RowType)

Exceptions

IllegalArgumentException	When damage is less than zero or if ability is Ability.BINDING and squad is null or less than zero or if ability is not Ability.BINDING and squad is not null.

Definition at line 153 of file UnitRepository.java.
References insertUnit().

5.64.3.16 isHorn()

```
Single< Boolean > isHorn (
            @NonNull RowType row)
```

Returns the value of RowEntity.horn for the given attack row.

See also

.isHornFlowable(RowType)

Parameters

row	RowEntity.id where the horn status is queried.

Returns

A Single tracking operation status and returning the value.

Definition at line 229 of file UnitRepository.java.
References database.

5.64.3.17 isHornFlowable()

```
Flowable< Boolean > isHornFlowable (
            @NonNull RowType row)
```

Returns a Flowable emitting the values of RowEntity.horn for the given attack row.

Parameters

row	RowEntity.id where the horn status is queried.

Returns

A Flowable emitting the values.

See also

.isHorn(RowType)

Definition at line 241 of file UnitRepository.java.
References database.

5.64.3.18 isWeather()

```
Single< Boolean > isWeather (
            @NonNull RowType row)
```

Returns the value of RowEntity.weather for the given attack row.

See also

.isWeatherFlowable(RowType)

Parameters

row	RowEntity.id where the weather is queried.

Returns

A Single tracking operation status and returning the value.

Definition at line 181 of file UnitRepository.java.
References database.

5.64.3.19 isWeatherFlowable()

```
Flowable< Boolean > isWeatherFlowable (
            @NonNull RowType row)
```

Returns a Flowable emitting the latest value of RowEntity.weather for the given attack row.

Parameters

row	RowEntity.id where the weather is queried.

Returns

A Flowable emitting the values.

See also

.isWeather(RowType)

Definition at line 193 of file UnitRepository.java.
References database.

5.64.3.20 reset() [1/2]

```
Completable reset ()
```

Resets the board by removing all units and resetting row status.
Resetting row status is equivalent to removing the old rows and calling .initializeRows(). Method is a wrapper for .reset(UnitEntity).

Returns

A Completable tracking operation status.

See also

.reset(UnitEntity)

Definition at line 86 of file UnitRepository.java.
References reset().
Referenced by reset().

5.64.3.21 reset() [2/2]

```
Completable reset (
            @Nullable UnitEntity keptUnit)
```

Resets the board by removing all units but the given one and resetting row status.
Resetting row status is equivalent to removing the old rows and calling .initializeRows().

Parameters

keptUnit	UnitEntity that should be kept.

Returns

A Completable tracking operation status.

See also

.initializeRows()

Definition at line 99 of file UnitRepository.java.
References database, and initializeRows().

5.64.3.22 switchHorn()

```
Completable switchHorn (
            @NonNull RowType row)
```

Flips RowEntity.horn of the given attack row.

Parameters

row	RowEntity.id where the horn status should be updated.

Returns

A Completable tracking operation status.

Definition at line 218 of file UnitRepository.java.
References database.

5.64.3.23 switchWeather()

```
Completable switchWeather (
            @NonNull RowType row)
```

Flips RowEntity.weather of the given attack row.

Parameters

row	RowEntity.id where the weather should be updated.

Returns

A Completable tracking operation status.

Definition at line 170 of file UnitRepository.java.
References database.

5.64.4 Member Data Documentation

5.64.4.1 database

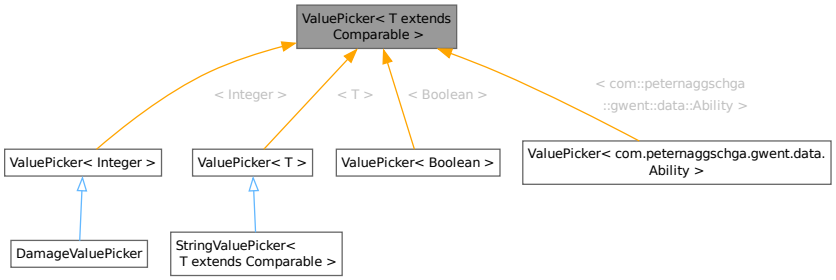
```
final AppDatabase database  [private]
```

Defines the AppDatabase that is used as a data source by this repository.
Is provided by dependency injection in .UnitRepository().
Definition at line 31 of file UnitRepository.java.
Referenced by UnitRepository(), clearWeather(), countUnits(), countUnits(), delete(), delete(), getRepository(), getUnit(), getUnits(), getUnits(), getUnitsFlowable(), hasNonEpicUnitsFlowable(), initializeRows(), insertUnit(), isHorn(), isHornFlowable(), isWeather(), isWeatherFlowable(), reset(), switchHorn(), and switchWeather().
The documentation for this class was generated from the following file:

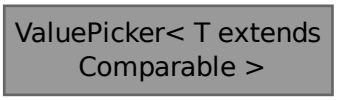
- gwent/data/UnitRepository.java

5.65 ValuePicker< T extends Comparable > Class Template Reference

A generic abstract wrapper class around NumberPicker used to select values of arbitrary types extending Comparable.
Inheritance diagram for ValuePicker< T extends Comparable >:



Collaboration diagram for ValuePicker< T extends Comparable >:



Classes

- interface OnValueChangeListener

  An interface defining a callback for changes of the current value.

Public Member Functions

- Context getContext ()

  Returns the context the .picker is running in, through which it can access the current theme, resources, etc.

Protected Member Functions

- NumberPicker getPicker ()

  Returns the NumberPicker wrapped by this ValuePicker.
- abstract String getDisplayString (@NonNull T value)

  Returns a localized String representing the given value.
- Map< T, Integer > getDisplayIntegers ()

  Returns the displayIntegers map.
- List< T > getSelectableValues ()

  Returns a List of values that can be selected by this picker.

Package Functions

- ValuePicker (@NonNull NumberPicker picker, @NonNull SortedMap< T, Integer > valueToStringRes)

  Constructor of a ValuePicker wrapping the given NumberPicker and offering the given values.
- ValuePicker (@NonNull NumberPicker picker, @NonNull SortedMap< T, Integer > valueToStringRes, @Nullable T defaultValue)

  Constructor of a ValuePicker wrapping the given NumberPicker, offering the given values and showing the given defaultValue.
- void setSelectableValues (@NonNull Collection< T > values, @Nullable T defaultValue)

  Sets selectableValues to the given values and show the given defaultValue.
- void setOnValueChangedListener (@NonNull OnValueChangeListener< T > onValueChangedListener)

  Sets an OnValueChangeListener that is called when the value of the wrapped .picker has changed and stayed the same for 500 ms.
- T getValue ()

  Returns the currently selected value.
- void setValue (@NonNull T value)

  Sets the picker to the given value.
- void setSelectableValues (@NonNull Collection< T > values)

  Sets selectableValues to the given values.

Private Attributes

- final NumberPicker picker

  NumberPicker that is wrapped by this class, used to select the values.
- final Map< T, Integer > displayIntegers

  Map mapping each of the selectableValues to an Integer that may be used to select the displayed value Strings in implementations of getDisplayString().
- final List< T > selectableValues

  List of values that are selectable using this ValuePicker.

5.65.1 Detailed Description

A generic abstract wrapper class around NumberPicker used to select values of arbitrary types extending Comparable.
Must be subclassed to provide the displayed String values using getDisplayString(). The default case, which is implemented in StringValuePicker, is that displayIntegers maps the selectableValues to a String resource id.

See also

NumberPicker

StringValuePicker

Definition at line 29 of file ValuePicker.java.

5.65.2 Constructor & Destructor Documentation

5.65.2.1 ValuePicker() [1/2]

```
ValuePicker (
            @NonNull NumberPicker picker,
            @NonNull SortedMap< T, Integer > valueToStringRes)  [package]
```

Constructor of a ValuePicker wrapping the given NumberPicker and offering the given values.
Wrapper around ValuePicker(NumberPicker, SortedMap, Comparable) with null as defaultValue.

See also

ValuePicker(NumberPicker, SortedMap, Comparable)

Parameters

picker	NumberPicker wrapped by the created ValuePicker.
valueToStringRes	SortedMap with all selectableValues as keys and the respective displayIntegers as value.

Exceptions

IllegalArgumentException	When valueToStringRes is empty.

Definition at line 65 of file ValuePicker.java.
References picker.
Referenced by ValuePicker< T extends Comparable >.OnValueChangeListener< T extends Comparable >.onValueChange(), and setOnValueChangedListener().

5.65.2.2 ValuePicker() [2/2]

```
ValuePicker (
            @NonNull NumberPicker picker,
```

```
@NonNull SortedMap< T, Integer > valueToStringRes,
@Nullable T defaultValue)  [package]
```

Constructor of a ValuePicker wrapping the given NumberPicker, offering the given values and showing the given defaultValue.

See also

ValuePicker(NumberPicker, SortedMap)

Parameters

picker	NumberPicker wrapped by the created ValuePicker.
valueToStringRes	SortedMap with all selectableValues as keys and the respective displayIntegers as value.
defaultValue	Value that is shown in the beginning. If null, then the first value defined by the Comparable interface is used.

Exceptions

IllegalArgumentException	When valueToStringRes is empty. or when defaultValue is not null but not contained in valueToStringRes.

Definition at line 81 of file ValuePicker.java.
References displayIntegers, picker, selectableValues, and setSelectableValues().

5.65.3 Member Function Documentation

5.65.3.1 getContext()

```
Context getContext ()
```

Returns the context the .picker is running in, through which it can access the current theme, resources, etc.

Returns

A Context object of the wrapped .picker.

Definition at line 204 of file ValuePicker.java.
References picker.

5.65.3.2 getDisplayIntegers()

```
Map< T, Integer > getDisplayIntegers ()  [protected]
```

Returns the displayIntegers map.

Returns

A Map from values to Integer objects representing information on their String representation.

See also

displayIntegers

Definition at line 229 of file ValuePicker.java.
References displayIntegers.

5.65.3.3 getDisplayString()

```
abstract String getDisplayString (
            @NonNull T value)  [abstract], [protected]
```

Returns a localized String representing the given value.

Parameters

value	Value that should be represented as a String.

Returns

A localized String representing the value.

Referenced by setSelectableValues().

5.65.3.4 getPicker()

```
NumberPicker getPicker ()  [protected]
```

Returns the NumberPicker wrapped by this ValuePicker.

Returns

A NumberPicker wrapped in this ValuePicker.

See also

.picker

Definition at line 143 of file ValuePicker.java.
References picker.

5.65.3.5 getSelectableValues()

```
List< T > getSelectableValues ()  [protected]
```

Returns a List of values that can be selected by this picker.

Returns

A List of values that are selectable.

See also

selectableValues

Definition at line 240 of file ValuePicker.java.
References selectableValues.

5.65.3.6 getValue()

```
T getValue ()  [package]
```

Returns the currently selected value.

Returns

A value that is selected in .picker.

Definition at line 171 of file ValuePicker.java.
References picker, and selectableValues.

5.65.3.7 setOnValueChangedListener()

```
void setOnValueChangedListener (
            @NonNull OnValueChangeListener< T > onValueChangedListener)  [package]
```

Sets an OnValueChangeListener that is called when the value of the wrapped .picker has changed and stayed the same for 500 ms.
The delayed NumberPicker.OnValueChangeListener that is generated from the given OnValueChangeListener using CardNumberPickerAdapter.getDelayedOnValueChangeListener().

See also

OnValueChangeListener

CardNumberPickerAdapter.getDelayedOnValueChangeListener(NumberPicker.OnValueChangeListener)

Parameters

onValueChangedListener	OnValueChangedListener that is to be executed when the value changes.

Definition at line 157 of file ValuePicker.java.
References ValuePicker(), CardNumberPickerAdapter.getDelayedOnValueChangeListener(), picker, and selectableValues.

5.65.3.8 setSelectableValues() [1/2]

```
void setSelectableValues (
            @NonNull Collection< T > values)  [package]
```

Sets selectableValues to the given values.
Every given value must be in the key-set of displayIntegers. Wrapper of setSelectableValues(Collection, Comparable).

Parameters

values	Collection of the new selectable values.

Exceptions

IllegalArgumentException	When values is empty.
IllegalStateException	When values contains a value not present in the key-set of displayIntegers.

See also

setSelectableValues(Collection, Comparable)

Definition at line 254 of file ValuePicker.java.
References setSelectableValues().

5.65.3.9 setSelectableValues() [2/2]

```
void setSelectableValues (
            @NonNull Collection< T > values,
            @Nullable T defaultValue)  [package]
```

Sets selectableValues to the given values and show the given defaultValue.
Every given value must be in the key-set of displayIntegers.

See also

setSelectableValues(Collection)

Parameters

values	Collection of the new selectable values.

defaultValue	Value that is shown in the beginning. If null, then the first value defined by the Comparable interface is used.

Exceptions

IllegalArgumentException	When values is empty or displayIntegers does not contain defaultValue when defaultValue is not null.
IllegalStateException	When values contains a value not present in the key-set of displayIntegers.

Definition at line 107 of file ValuePicker.java.
References displayIntegers, getDisplayString(), picker, selectableValues, and setValue().
Referenced by ValuePicker(), and setSelectableValues().

5.65.3.10 setValue()

```
void setValue (
            @NonNull T value)  [package]
```

Sets the picker to the given value.

Parameters

value	Value that the picker is set to.

Exceptions

IllegalStateException	When selectableValues does not contain the given value.

Definition at line 180 of file ValuePicker.java.
References picker, and selectableValues.
Referenced by setSelectableValues().

5.65.4 Member Data Documentation

5.65.4.1 displayIntegers

```
final Map<T, Integer> displayIntegers  [private]
```

Map mapping each of the selectableValues to an Integer that may be used to select the displayed value Strings in implementations of getDisplayString().

See also

getDisplayIntegers()

Definition at line 46 of file ValuePicker.java.
Referenced by ValuePicker(), getDisplayIntegers(), and setSelectableValues().

5.65.4.2 picker

```
final NumberPicker picker  [private]
```

NumberPicker that is wrapped by this class, used to select the values.

See also

getValue()

setValue(Comparable)

getPicker()

Definition at line 38 of file ValuePicker.java.
Referenced by ValuePicker(), ValuePicker(), getContext(), getPicker(), getValue(), ValuePicker< T extends Comparable >.OnValueCh
setOnValueChangedListener(), setSelectableValues(), and setValue().

5.65.4.3 selectableValues

```
final List<T> selectableValues  [private]
```

List of values that are selectable using this ValuePicker.

See also

setSelectableValues(Collection)

setSelectableValues(Collection, Comparable)

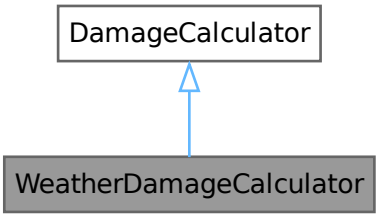
getSelectableValues()

Definition at line 55 of file ValuePicker.java.
Referenced by ValuePicker(), getSelectableValues(), getValue(), setOnValueChangedListener(), setSelectableValues(),
and setValue().
The documentation for this class was generated from the following file:

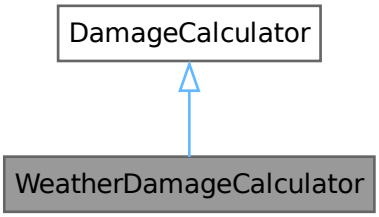
- gwent/ui/dialogs/addcard/ValuePicker.java

5.66 WeatherDamageCalculator Class Reference

A DamageCalculator class responsible for calculating the weather debuff if necessary, i.e., if .weather is true.
Inheritance diagram for WeatherDamageCalculator:



Collaboration diagram for WeatherDamageCalculator:



Public Member Functions

- int calculateDamage (int id, @IntRange(from=0) int damage)

  Calculates the (de-)buffed damage of unit with the given id and the given base-damage.
- Color isBuffed (int id)

  Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT.

Package Functions

- WeatherDamageCalculator (boolean weather)

  Constructor of a WeatherDamageCalculator.

Private Attributes

- final boolean weather

  Defines whether the weather debuff should be applied when calling .calculateDamage().

5.66.1 Detailed Description

A DamageCalculator class responsible for calculating the weather debuff if necessary, i.e., if .weather is true.
Acts as the ConcreteComponent in the implemented decorator pattern and is therefore always at the end of the decorator chain.
Definition at line 14 of file WeatherDamageCalculator.java.

5.66.2 Constructor & Destructor Documentation

5.66.2.1 WeatherDamageCalculator()

```
WeatherDamageCalculator (
            boolean weather)  [package]
```

Constructor of a WeatherDamageCalculator.
Should only be called by DamageCalculatorBuilder.

Parameters

weather	Boolean defining whether the weather debuff should be applied.

See also

DamageCalculatorBuilder

Definition at line 27 of file WeatherDamageCalculator.java.
References weather.

5.66.3 Member Function Documentation

5.66.3.1 calculateDamage()

```
int calculateDamage (
            int id,
            @IntRange(from=0) int damage)
```

Calculates the (de-)buffed damage of unit with the given id and the given base-damage.
Returns the given damage if .weather is false or 1.

Parameters

id	Integer representing the UnitEntity.id of the unit whose (de-)buff damage is calculated.
damage	Integer representing the base-damage of the unit whose (de-)buff damage is calculated.

Returns

Integer representing the (de-)buffed damage of the unit.

Exceptions

IllegalArgumentException	When damage is negative.

Implements DamageCalculator.
Definition at line 41 of file WeatherDamageCalculator.java.
References weather.

5.66.3.2 isBuffed()

```
Color isBuffed (
            int id)
```

Calculates whether the unit with the given id is shown as Color.BUFFED, Color.DEBUFFED, or Color.DEFAULT.
Returns Color.DEBUFFED if .weather is true or else Color.DEFAULT.

Parameters

id	Integer representing the UnitEntity.id of the unit buff status is calculated.

Returns

Color representing whether the unit is buffed, de-buffed or not affected.

See also

Color

Implements DamageCalculator.
Definition at line 61 of file WeatherDamageCalculator.java.
References weather.

5.66.4 Member Data Documentation

5.66.4.1 weather

```
final boolean weather  [private]
```

Defines whether the weather debuff should be applied when calling .calculateDamage().
Definition at line 18 of file WeatherDamageCalculator.java.
Referenced by WeatherDamageCalculator(), calculateDamage(), and isBuffed().
The documentation for this class was generated from the following file:

- gwent/domain/damage/WeatherDamageCalculator.java

Chapter 6

File Documentation

6.1 Ability.java

```
00001 package com.peternaggschga.gwent.data;
00002
00006 public enum Ability {
00010     NONE,
00011
00015     HORN,
00016
00020     REVENGE,
00021
00025     BINDING,
00026
00030     MORAL_BOOST
00031 }
```

6.2 AppDatabase.java

```
00001 package com.peternaggschga.gwent.data;
00002
00003 import androidx.room3.Database;
00004 import androidx.room3.RoomDatabase;
00005
00010 @Database(entities = {UnitEntity.class, RowEntity.class}, version = 1)
00011 public abstract class AppDatabase extends RoomDatabase {
00017     abstract UnitDao units();
00018
00024     abstract RowDao rows();
00025 }
```

6.3 RowDao.java

```
00001 package com.peternaggschga.gwent.data;
00002
00003 import androidx.annotation.NonNull;
00004 import androidx.room3.Dao;
00005 import androidx.room3.DaoReturnTypeConverters;
00006 import androidx.room3.Insert;
00007 import androidx.room3.OnConflictStrategy;
00008 import androidx.room3.Query;
00009 import androidx.room3.rxjava3.RxDaoReturnTypeConverters;
00010
00011 import io.reactivex.rxjava3.core.Completable;
00012 import io.reactivex.rxjava3.core.Flowable;
00013 import io.reactivex.rxjava3.core.Single;
00014
00019 @Dao
00020 @DaoReturnTypeConverters(RxDaoReturnTypeConverters.class)
00021 public interface RowDao {
00029     @Insert(onConflict = OnConflictStrategy.IGNORE)
00030     Completable insertRow(@NonNull RowEntity row);
00031
00037     @Query("DELETE FROM rows")
00038     Completable clearRows();
00039
00046     @Query("UPDATE rows SET weather = NOT weather WHERE id = :row")
00047     Completable updateWeather(@NonNull RowType row);
```

```
00048
00054     @Query("UPDATE rows SET weather = 0")
00055     Completable clearWeather();
00056
00063     @Query("UPDATE rows SET horn = NOT horn WHERE id = :row")
00064     Completable updateHorn(@NonNull RowType row);
00065
00072     @Query("SELECT weather FROM rows WHERE id = :row")
00073     Single<Boolean> isWeather(@NonNull RowType row);
00074
00082     @Query("SELECT weather FROM rows WHERE id = :row")
00083     Flowable<Boolean> isWeatherFlowable(@NonNull RowType row);
00084
00091     @Query("SELECT horn FROM rows WHERE id = :row")
00092     Single<Boolean> isHorn(@NonNull RowType row);
00093
00101     @Query("SELECT horn FROM rows WHERE id = :row")
00102     Flowable<Boolean> isHornFlowable(@NonNull RowType row);
00103 }
```

6.4 RowEntity.java

```
00001 package com.peternaggschga.gwent.data;
00002
00003 import androidx.annotation.NonNull;
00004 import androidx.room3.ColumnInfo;
00005 import androidx.room3.Entity;
00006 import androidx.room3.PrimaryKey;
00007
00012 @Entity(tableName = "rows")
00013 @SuppressWarnings("unused")
00014 public class RowEntity {
00020     @PrimaryKey
00021     @NonNull
00022     private final RowType id;
00023
00029     @ColumnInfo(defaultValue = "false")
00030     private boolean weather;
00031
00037     @ColumnInfo(defaultValue = "false")
00038     private boolean horn;
00039
00047     protected RowEntity(@NonNull RowType id) {
00048         this.id = id;
00049     }
00050
00057     @NonNull
00058     protected RowType getId() {
00059         return id;
00060     }
00061
00068     protected boolean isWeather() {
00069         return weather;
00070     }
00071
00078     protected void setWeather(boolean weather) {
00079         this.weather = weather;
00080     }
00081
00088     protected boolean isHorn() {
00089         return horn;
00090     }
00091
00098     protected void setHorn(boolean horn) {
00099         this.horn = horn;
00100     }
00101 }
```

6.5 RowType.java

```
00001 package com.peternaggschga.gwent.data;
00002
00006 public enum RowType {
00010     MELEE,
00011
00015     RANGE,
00016
00020     SIEGE
00021 }
```

6.6 UnitDao.java

```
00001 package com.peternaggschga.gwent.data;
00002
00003 import androidx.annotation.IntRange;
00004 import androidx.annotation.NonNull;
00005 import androidx.annotation.Nullable;
00006 import androidx.room3.Dao;
00007 import androidx.room3.DaoReturnTypeConverters;
00008 import androidx.room3.Delete;
00009 import androidx.room3.Insert;
00010 import androidx.room3.OnConflictStrategy;
00011 import androidx.room3.Query;
00012 import androidx.room3.rxjava3.RxDaoReturnTypeConverters;
00013
00014 import java.util.Collection;
00015 import java.util.List;
00016
00017 import io.reactivex.rxjava3.core.Completable;
00018 import io.reactivex.rxjava3.core.Flowable;
00019 import io.reactivex.rxjava3.core.Single;
00020
00025 @Dao
00026 @DaoReturnTypeConverters(RxDaoReturnTypeConverters.class)
00027 public interface UnitDao {
00035     @Insert(onConflict = OnConflictStrategy.IGNORE)
00036     Completable insertUnit(@NonNull UnitEntity unit);
00037
00049     @Query("INSERT INTO units (epic, damage, ability, squad, `row`) VALUES (:epic, :damage, :ability,
      :squad, :row)")
00050     Completable insertUnit(boolean epic, @IntRange(from = 0, to =
      UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int damage, @NonNull Ability ability, @IntRange(from =
      1) @Nullable Integer squad, @NonNull RowType row);
00051
00057     @Query("DELETE FROM units WHERE `id` = :id")
00058     Completable deleteUnit(int id);
00059
00066     @Delete
00067     Completable deleteUnits(@NonNull Collection<UnitEntity> units);
00068
00074     @Query("SELECT * FROM units WHERE `id` = :id")
00075     Single<UnitEntity> getUnit(int id);
00076
00083     @Query("SELECT * FROM units WHERE `row` = :row")
00084     @NonNull
00085     Single<List<UnitEntity>> getUnits(@NonNull RowType row);
00086
00094     @Query("SELECT * FROM units WHERE `row` = :row")
00095     @NonNull
00096     Flowable<List<UnitEntity>> getUnitsFlowable(@NonNull RowType row);
00097
00102     @Query("SELECT * FROM units")
00103     @NonNull
00104     Single<List<UnitEntity>> getUnits();
00105
00111     @Query("SELECT COUNT(*) FROM units WHERE `row` = :row")
00112     Single<Integer> countUnits(@NonNull RowType row);
00113
00119     @Query("SELECT COUNT(*) FROM units")
00120     Single<Integer> countUnits();
00121
00127     @Query("SELECT COUNT(*) > 0 FROM units WHERE epic = 0")
00128     Flowable<Boolean> hasNonEpicUnitsFlowable();
00129 }
```

6.7 UnitEntity.java

```
00001 package com.peternaggschga.gwent.data;
00002
00003 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color;
00004 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color.DEFAULT;
00005
00006 import android.content.Context;
00007
00008 import androidx.annotation.IntRange;
00009 import androidx.annotation.NonNull;
00010 import androidx.annotation.Nullable;
00011 import androidx.room3.ColumnInfo;
00012 import androidx.room3.Entity;
00013 import androidx.room3.ForeignKey;
00014 import androidx.room3.PrimaryKey;
00015
00016 import com.peternaggschga.gwent.R;
```

```
00017 import com.peternaggschga.gwent.domain.damage.DamageCalculator;
00018
00019 import java.util.ArrayList;
00020 import java.util.Collection;
00021 import java.util.HashMap;
00022 import java.util.Iterator;
00023 import java.util.List;
00024 import java.util.Map;
00025
00030 @Entity(tableName = "units", foreignKeys = {
00031         @ForeignKey(entity = RowEntity.class,
00032                 parentColumns = "id",
00033                 childColumns = "row",
00034                 onDelete = ForeignKey.CASCADE)})
00035 @SuppressWarnings("unused")
00036 public class UnitEntity {
00041     @PrimaryKey(autoGenerate = true)
00042     private int id;
00043
00048     @ColumnInfo(defaultValue = "false")
00049     private boolean epic;
00050
00055     private int damage;
00056
00060     @NonNull
00061     public static final Integer[] EPIC_DAMAGE_VALUES = new Integer[]{0, 7, 8, 10, 11, 15};
00062
00066     public static final int NON_EPIC_DAMAGE_VALUES_UPPER_BOUND = 20;
00067
00073     @ColumnInfo(defaultValue = "NONE")
00074     @NonNull
00075     private Ability ability;
00076
00081     @ColumnInfo(defaultValue = "NULL")
00082     @IntRange(from = 1)
00083     @Nullable
00084     private Integer squad;
00085
00090     @ColumnInfo(index = true)
00091     @NonNull
00092     private RowType row;
00093
00105     UnitEntity(boolean epic, @IntRange(from = 0) int damage, @NonNull Ability ability, @IntRange(from
      = 1) @Nullable Integer squad, @NonNull RowType row) {
00106         if (damage < 0) {
00107             throw new IllegalArgumentException("Damage must be greater or equal to 0 but is " + damage
      + ".");
00108         }
00109         if (ability != Ability.BINDING && squad != null) {
00110             throw new IllegalArgumentException("Squad must be null or ability must be BINDING but
      squad is " + squad + " and ability is " + ability + ".");
00111         }
00112         if (ability == Ability.BINDING && (squad == null || squad < 1)) {
00113             throw new IllegalArgumentException("Squad must not be null or less than one but squad is "
      + squad + ".");
00114         }
00115         this.epic = epic;
00116         this.damage = damage;
00117         this.ability = ability;
00118         this.squad = squad;
00119         this.row = row;
00120     }
00121
00132     public int calculateDamage(@NonNull DamageCalculator calculator) {
00133         return epic ? damage : calculator.calculateDamage(id, damage);
00134     }
00135
00146     @NonNull
00147     public Color isBuffed(@NonNull DamageCalculator calculator) {
00148         return epic ? DEFAULT : calculator.isBuffed(id);
00149     }
00150
00165     @NonNull
00166     public static String collectionToString(@NonNull Context context, @NonNull Collection<UnitEntity>
      units) {
00167         if (units.isEmpty()) {
00168             throw new IllegalArgumentException("Units collection must not be empty.");
00169         }
00170
00171         Map<String, Integer> descriptionStrings = new HashMap<>(units.size());
00172         for (UnitEntity unit : units) {
00173             String description = unit.toString(context);
00174             descriptionStrings.merge(description, 1, Integer::sum);
00175         }
00176
00177         List<String> descriptions = new ArrayList<>(descriptionStrings.size());
00178         if (descriptionStrings.size() < units.size()) {
```

```
00179             descriptionStrings.forEach((key, value) ->
00180                     descriptions.add(context.getString(R.string.unit_toString_multiplicity, value,
      key)));
00181         } else {
00182             descriptions.addAll(descriptionStrings.keySet());
00183         }
00184 
00185         Iterator<String> descriptionIterator = descriptions.iterator();
00186         String[] result = {descriptionIterator.next()};
00187         if (descriptionIterator.hasNext()) {
00188             result[0] = context.getString(R.string.unit_collection_toString_accumulation_word,
00189                             descriptionIterator.next(),
00190                             result[0])
00191                     .trim();
00192         }
00193         descriptionIterator.forEachRemaining(description ->
00194                 result[0] = context.getString(R.string.unit_collection_toString_accumulation_symbol,
00195                                 description,
00196                                 result[0])
00197                         .trim());
00198         return result[0];
00199     }
00200 
00209     @NonNull
00210     public String toString(@NonNull Context context) {
00211         String row;
00212         switch (getRow()) {
00213             case RANGE:
00214                 row = context.getString(R.string.unit_toString_range);
00215                 break;
00216             case SIEGE:
00217                 row = context.getString(R.string.unit_toString_siege);
00218                 break;
00219             case MELEE:
00220             default:
00221                 row = context.getString(R.string.unit_toString_melee);
00222         }
00223         String epic = isEpic() ? context.getString(R.string.unit_toString_epic) :
      context.getString(R.string.unit_toString_unit);
00224         String ability;
00225         String squad = "";
00226         switch (getAbility()) {
00227             case HORN:
00228                 ability = context.getString(R.string.add_picker_ability_horn);
00229                 break;
00230             case BINDING:
00231                 ability = context.getString(R.string.add_picker_ability_binding);
00232                 squad = context.getString(R.string.unit_toString_squad, getSquad());
00233                 break;
00234             case MORAL_BOOST:
00235                 ability = context.getString(R.string.add_picker_ability_moralBoost);
00236                 break;
00237             case REVENGE:
00238                 ability = context.getString(R.string.add_picker_ability_revenge);
00239                 break;
00240             case NONE:
00241             default:
00242                 ability = context.getString(R.string.unit_toString_ability_none);
00243         }
00244         return context.getString(R.string.unit_toString, row, epic, getDamage(), ability,
      squad).trim();
00245     }
00246 
00252     public int getId() {
00253         return id;
00254     }
00255 
00262     void setId(int id) {
00263         this.id = id;
00264     }
00265 
00271     public boolean isEpic() {
00272         return epic;
00273     }
00274 
00281     void setEpic(boolean epic) {
00282         this.epic = epic;
00283     }
00284 
00291     public int getDamage() {
00292         return damage;
00293     }
00294 
00302     void setDamage(@IntRange(from = 0) int damage) {
00303         if (damage < 0) {
00304             throw new IllegalArgumentException("Damage must not be less than zero but is " + damage +
      ".");
```

```
00305         }
00306         this.damage = damage;
00307     }
00308
00314     @NonNull
00315     public Ability getAbility() {
00316         return ability;
00317     }
00318
00325     void setAbility(@NonNull Ability ability) {
00326         this.ability = ability;
00327     }
00328
00334     @Nullable
00335     public Integer getSquad() {
00336         return squad;
00337     }
00338
00346     void setSquad(@IntRange(from = 1) @Nullable Integer squad) {
00347         if (ability != Ability.BINDING && squad != null) {
00348             throw new IllegalArgumentException("Squad must be null or ability must be BINDING but
      squad is " + squad + " and ability is " + ability + ".");
00349         }
00350         if (ability == Ability.BINDING && (squad == null || squad < 1)) {
00351             throw new IllegalArgumentException("Squad must not be null or less than one but squad is "
      + squad + ".");
00352         }
00353         this.squad = squad;
00354     }
00355
00361     @NonNull
00362     public RowType getRow() {
00363         return row;
00364     }
00365
00372     void setRow(@NonNull RowType row) {
00373         this.row = row;
00374     }
00375 }
```

6.8 UnitRepository.java

```
00001 package com.peternaggschga.gwent.data;
00002
00003 import androidx.annotation.IntRange;
00004 import androidx.annotation.NonNull;
00005 import androidx.annotation.Nullable;
00006
00007 import java.util.Arrays;
00008 import java.util.Collection;
00009 import java.util.Iterator;
00010 import java.util.List;
00011 import java.util.concurrent.TimeUnit;
00012
00013 import io.reactivex.rxjava3.android.schedulers.AndroidSchedulers;
00014 import io.reactivex.rxjava3.core.Completable;
00015 import io.reactivex.rxjava3.core.Flowable;
00016 import io.reactivex.rxjava3.core.Single;
00017 import io.reactivex.rxjava3.schedulers.Schedulers;
00018
00025 public class UnitRepository {
00030     @NonNull
00031     private final AppDatabase database;
00032
00041     private UnitRepository(@NonNull AppDatabase database) {
00042         this.database = database;
00043     }
00044
00053     @NonNull
00054     public static Single<UnitRepository> getRepository(@NonNull AppDatabase database) {
00055         UnitRepository repository = new UnitRepository(database);
00056         return repository.initializeRows()
00057                 .andThen(Single.just(repository))
00058                 .subscribeOn(Schedulers.io())
00059                 .observeOn(AndroidSchedulers.mainThread());
00060     }
00061
00068     @NonNull
00069     private Completable initializeRows() {
00070         Completable result = Completable.complete();
00071         for (RowType row : RowType.values()) {
00072             result = result.andThen(database.rows().insertRow(new RowEntity(row)));
00073         }
00074         return result;
```

```
00075     }
00076
00085     @NonNull
00086     public Completable reset() {
00087         return reset(null);
00088     }
00089
00098     @NonNull
00099     public Completable reset(@Nullable UnitEntity keptUnit) {
00100         Completable result = database.rows().clearRows().andThen(initializeRows());
00101         if (keptUnit != null) {
00102             result = result.andThen(database.units().insertUnit(keptUnit));
00103         }
00104         return result.subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00105     }
00106
00118     @NonNull
00119     private Completable insertUnit(boolean epic, @IntRange(from = 0, to =
    UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int damage, @NonNull Ability ability,
00120                                    @IntRange(from = 1) @Nullable Integer squad, @NonNull RowType row)
    {
00121         if (damage < 0 || damage > UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) {
00122             throw new IllegalArgumentException("Damage be between 0 and " +
    UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND + " but is " + damage + ".");
00123         }
00124         if (epic && Arrays.stream(UnitEntity.EPIC_DAMAGE_VALUES).noneMatch(integer -> integer ==
    damage)) {
00125             throw new IllegalArgumentException("Damage of epic units must be in " +
    Arrays.toString(UnitEntity.EPIC_DAMAGE_VALUES) + " but is " + damage + ".");
00126         }
00127         if (ability != Ability.BINDING && squad != null) {
00128             throw new IllegalArgumentException("Squad must be null or ability must be BINDING but
    squad is " + squad + " and ability is " + ability + ".");
00129         }
00130         if (ability == Ability.BINDING && (squad == null || squad < 1)) {
00131             throw new IllegalArgumentException("Squad must not be null or less than one but squad is "
    + squad + ".");
00132         }
00133         return database.units().insertUnit(epic, damage, ability, squad, row)
00134                 .subscribeOn(Schedulers.io())
00135                 .observeOn(AndroidSchedulers.mainThread());
00136     }
00137
00152     @NonNull
00153     public Completable insertUnit(boolean epic, @IntRange(from = 0, to =
    UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int damage, @NonNull Ability ability,
00154                                   @IntRange(from = 1) @Nullable Integer squad, @NonNull RowType row,
00155                                   @IntRange(from = 0) int number) {
00156         Completable result = Completable.complete();
00157         for (int i = 0; i < number; i++) {
00158             result = result.andThen(insertUnit(epic, damage, ability, squad, row));
00159         }
00160         return result.subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00161     }
00162
00169     @NonNull
00170     public Completable switchWeather(@NonNull RowType row) {
00171         return
    database.rows().updateWeather(row).subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00172     }
00173
00180     @NonNull
00181     public Single<Boolean> isWeather(@NonNull RowType row) {
00182         return
    database.rows().isWeather(row).subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00183     }
00184
00192     @NonNull
00193     public Flowable<Boolean> isWeatherFlowable(@NonNull RowType row) {
00194         return database.rows()
00195                 .isWeatherFlowable(row)
00196                 .onBackpressureLatest()
00197                 .distinctUntilChanged()
00198                 .subscribeOn(Schedulers.io())
00199                 .observeOn(AndroidSchedulers.mainThread());
00200     }
00201
00207     @NonNull
00208     public Completable clearWeather() {
00209         return
    database.rows().clearWeather().subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00210     }
00211
00217     @NonNull
00218     public Completable switchHorn(@NonNull RowType row) {
00219         return
    database.rows().updateHorn(row).subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
```

```
00220     }
00221
00228     @NonNull
00229     public Single<Boolean> isHorn(@NonNull RowType row) {
00230         return
      database.rows().isHorn(row).subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00231     }
00232
00240     @NonNull
00241     public Flowable<Boolean> isHornFlowable(@NonNull RowType row) {
00242         return database.rows()
00243                 .isHornFlowable(row)
00244                 .onBackpressureLatest()
00245                 .distinctUntilChanged()
00246                 .subscribeOn(Schedulers.io())
00247                 .observeOn(AndroidSchedulers.mainThread());
00248     }
00249
00255     @NonNull
00256     public Completable delete(@NonNull Collection<UnitEntity> units) {
00257         return
      database.units().deleteUnits(units).subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00258     }
00259
00266     @NonNull
00267     public Completable delete(int id) {
00268         return
      database.units().deleteUnit(id).subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00269     }
00270
00276     @NonNull
00277     public Completable copy(int id) {
00278         return getUnit(id).flatMapCompletable(unit ->
00279                         insertUnit(unit.isEpic(), unit.getDamage(), unit.getAbility(),
      unit.getSquad(), unit.getRow()))
00280                 .subscribeOn(Schedulers.io())
00281                 .observeOn(AndroidSchedulers.mainThread());
00282     }
00283
00290     @NonNull
00291     public Single<Integer> countUnits(@NonNull RowType row) {
00292         return
      database.units().countUnits(row).subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00293     }
00294
00301     @NonNull
00302     public Single<Integer> countUnits() {
00303         return
      database.units().countUnits().subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00304     }
00305
00312     @NonNull
00313     public Single<UnitEntity> getUnit(int id) {
00314         return
      database.units().getUnit(id).subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00315     }
00316
00324     @NonNull
00325     public Single<List<UnitEntity» getUnits(@NonNull RowType row) {
00326         return
      database.units().getUnits(row).subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00327     }
00328
00337     @NonNull
00338     public Flowable<List<UnitEntity» getUnitsFlowable(@NonNull RowType row) {
00339         return database.units()
00340                 .getUnitsFlowable(row)
00341                 .onBackpressureLatest()
00342                 .debounce(10, TimeUnit.MILLISECONDS)
00343                 .distinctUntilChanged((list1, list2) -> {
00344                     if (list1.size() != list2.size()) {
00345                         return false;
00346                     }
00347                     Iterator<UnitEntity> list1Iterator = list1.iterator();
00348                     return list2.stream().map(UnitEntity::getId).allMatch(id ->
      id.equals(list1Iterator.next().getId()));
00349                 })
00350                 .subscribeOn(Schedulers.io())
00351                 .observeOn(AndroidSchedulers.mainThread());
00352     }
00353
00360     @NonNull
00361     public Single<List<UnitEntity» getUnits() {
00362         return
      database.units().getUnits().subscribeOn(Schedulers.io()).observeOn(AndroidSchedulers.mainThread());
00363     }
00364
```

```
00369     @NonNull
00370     public Flowable<Boolean> hasNonEpicUnitsFlowable() {
00371         return database.units()
00372                 .hasNonEpicUnitsFlowable()
00373                 .onBackpressureLatest()
00374                 .debounce(10, TimeUnit.MILLISECONDS)
00375                 .distinctUntilChanged()
00376                 .subscribeOn(Schedulers.io())
00377                 .observeOn(AndroidSchedulers.mainThread());
00378     }
00379 }
```

6.9 BurnDialogUseCase.java

```
00001 package com.peternaggschga.gwent.domain.cases;
00002
00003 import android.content.Context;
00004
00005 import androidx.annotation.NonNull;
00006 import androidx.appcompat.app.AlertDialog;
00007
00008 import com.peternaggschga.gwent.GwentApplication;
00009 import com.peternaggschga.gwent.R;
00010 import com.peternaggschga.gwent.data.RowType;
00011 import com.peternaggschga.gwent.data.UnitEntity;
00012 import com.peternaggschga.gwent.data.UnitRepository;
00013 import com.peternaggschga.gwent.domain.damage.DamageCalculator;
00014 import com.peternaggschga.gwent.ui.sounds.SoundManager;
00015
00016 import java.util.ArrayList;
00017 import java.util.Collection;
00018 import java.util.HashMap;
00019 import java.util.List;
00020 import java.util.Map;
00021 import java.util.Objects;
00022
00023 import io.reactivex.rxjava3.core.Single;
00024
00028 public class BurnDialogUseCase {
00036     @NonNull
00037     private static Single<List<UnitEntity>> getBurnUnits(@NonNull UnitRepository repository) {
00038         return repository.getUnits()
00039                 .flatMap(units -> {
00040                     if (units.isEmpty()) {
00041                         return Single.just(units);
00042                     }
00043
00044                     Single<Map<RowType, DamageCalculator>> calculators = Single.just(new
      HashMap<>(RowType.values().length));
00045                     for (RowType row : RowType.values()) {
00046                         calculators =
      calculators.zipWith(DamageCalculatorUseCase.getDamageCalculator(repository, row), (calculatorMap,
      damageCalculator) -> {
00047                             calculatorMap.put(row, damageCalculator);
00048                             return calculatorMap;
00049                         });
00050                     }
00051
00052                     return calculators.map(damageCalculators -> {
00053                         List<UnitEntity> maxDamageUnits = new ArrayList<>(units.size());
00054                         final int[] maxDamage = {0};
00055                         units.stream()
00056                                 .filter(unit -> !unit.isEpic())
00057                                 .forEach(unit -> {
00058                                     int damage =
      unit.calculateDamage(Objects.requireNonNull(damageCalculators.get(unit.getRow())));
00059                                     if (damage > maxDamage[0]) {
00060                                         maxDamage[0] = damage;
00061                                         maxDamageUnits.clear();
00062                                         maxDamageUnits.add(unit);
00063                                     } else if (damage == maxDamage[0]) {
00064                                         maxDamageUnits.add(unit);
00065                                     }
00066                                 });
00067                         return maxDamageUnits;
00068                     });
00069                 });
00070     }
00071
00084     @NonNull
00085     public static Single<Boolean> burn(@NonNull Context context, @NonNull SoundManager soundManager) {
00086         return GwentApplication.getRepository(context).flatMap(repository -> burn(context, repository,
      soundManager));
00087     }
```

```
00088
00100     @NonNull
00101     protected static Single<Boolean> burn(@NonNull Context context, @NonNull UnitRepository
      repository, @NonNull SoundManager soundManager) {
00102         return getBurnUnits(repository).flatMap(units -> {
00103             if (units.isEmpty()) {
00104                 return Single.just(false);
00105             }
00106             return Single.create(emitter -> new AlertDialog.Builder(context)
00107                     .setIconAttribute(android.R.attr.alertDialogIcon)
00108                     .setTitle(R.string.alertDialog_burn_title)
00109                     .setMessage(context.getString(R.string.alertDialog_burn_msg,
      UnitEntity.collectionToString(context, units)))
00110                     .setNegativeButton(R.string.alertDialog_burn_negative, (dialog, which) ->
      dialog.cancel())
00111                     .setPositiveButton(R.string.alertDialog_burn_positive, (dialog, which) -> {
00112                         // noinspection CheckResult, ResultOfMethodCallIgnored
00113                         RemoveUnitsUseCase.remove(context, repository, units,
      soundManager).subscribe(() -> emitter.onSuccess(true));
00114                     })
00115                     .setCancelable(true)
00116                     .setOnCancelListener(dialog -> emitter.onSuccess(false))
00117                     .create()
00118                     .show());
00119         });
00120     }
00121 }
```

6.10 DamageCalculatorUseCase.java

```
00001 package com.peternaggschga.gwent.domain.cases;
00002
00003 import android.util.Pair;
00004
00005 import androidx.annotation.NonNull;
00006
00007 import com.peternaggschga.gwent.data.RowType;
00008 import com.peternaggschga.gwent.data.UnitEntity;
00009 import com.peternaggschga.gwent.data.UnitRepository;
00010 import com.peternaggschga.gwent.domain.damage.DamageCalculator;
00011 import com.peternaggschga.gwent.domain.damage.DamageCalculatorBuildDirector;
00012
00013 import java.util.Collection;
00014
00015 import io.reactivex.rxjava3.core.Single;
00016
00023 public class DamageCalculatorUseCase {
00031     @NonNull
00032     public static Single<DamageCalculator> getDamageCalculator(@NonNull UnitRepository repository,
      @NonNull RowType row) {
00033         return repository.isWeather(row)
00034                 .zipWith(repository.isHorn(row), Pair::create)
00035                 .zipWith(repository.getUnits(row), (weatherHorn, units) ->
00036                         getDamageCalculator(weatherHorn.first, weatherHorn.second, units));
00037     }
00038
00049     @NonNull
00050     public static DamageCalculator getDamageCalculator(boolean weather, boolean horn, @NonNull
      Collection<UnitEntity> units) {
00051         return DamageCalculatorBuildDirector.getCalculator(weather, horn, units);
00052     }
00053 }
```

6.11 RemoveUnitsUseCase.java

```
00001 package com.peternaggschga.gwent.domain.cases;
00002
00003 import android.app.Dialog;
00004 import android.content.Context;
00005
00006 import androidx.annotation.IntRange;
00007 import androidx.annotation.NonNull;
00008
00009 import com.peternaggschga.gwent.data.Ability;
00010 import com.peternaggschga.gwent.data.UnitEntity;
00011 import com.peternaggschga.gwent.data.UnitRepository;
00012 import com.peternaggschga.gwent.ui.sounds.SoundManager;
00013
00014 import java.util.Collection;
00015 import java.util.Collections;
00016
```

```
00017 import io.reactivex.rxjava3.core.Completable;
00018 import io.reactivex.rxjava3.core.CompletableEmitter;
00019
00027 public class RemoveUnitsUseCase {
00042     @NonNull
00043     public static Completable remove(@NonNull Context context, @NonNull UnitRepository repository,
00044                                      @NonNull Collection<UnitEntity> units, @NonNull SoundManager
      soundManager) {
00045         long revengeUnits = units.stream()
00046                 .filter(unit -> unit.getAbility() == Ability.REVENGE)
00047                 .count();
00048         if (revengeUnits == 0) {
00049             return repository.delete(units);
00050         }
00051         return Completable.create(emitter ->
00052                 getRevengeDialog(context, repository, emitter, units, (int) revengeUnits,
      soundManager).show()
00053         );
00054     }
00055
00069     public static Completable remove(@NonNull Context context, @NonNull UnitRepository repository, int
      id, @NonNull SoundManager soundManager) {
00070         return repository.getUnit(id)
00071                 .flatMapCompletable(unitEntity ->
00072                         remove(context, repository, Collections.singletonList(unitEntity),
      soundManager));
00073     }
00074
00089     @NonNull
00090     private static Dialog getRevengeDialog(@NonNull Context context, @NonNull UnitRepository
      repository,
00091                                            @NonNull CompletableEmitter emitter, @NonNull
      Collection<UnitEntity> units,
00092                                            @IntRange(from = 1) int revengeUnits, @NonNull SoundManager
      soundManager) {
00093         return new RevengeAlertDialogBuilderAdapter(context)
00094                 .setPositiveCallback((dialog, which) -> {
00095                     // noinspection CheckResult, ResultOfMethodCallIgnored
00096                     repository.delete(units)
00097                             .andThen(RevengeAlertDialogBuilderAdapter.insertAvengers(repository,
      revengeUnits, soundManager))
00098                             .subscribe(emitter::onComplete);
00099                 })
00100                 .setNegativeCallback(((dialog, which) -> {
00101                     // noinspection CheckResult, ResultOfMethodCallIgnored
00102                     repository.delete(units).subscribe(emitter::onComplete);
00103                 }))
00104                 .create();
00105     }
00106 }
```

6.12 ResetAlertDialogBuilderAdapter.java

```
00001 package com.peternaggschga.gwent.domain.cases;
00002
00003 import android.content.Context;
00004 import android.view.View;
00005 import android.widget.CheckBox;
00006
00007 import androidx.annotation.NonNull;
00008 import androidx.appcompat.app.AlertDialog;
00009
00010 import com.peternaggschga.gwent.R;
00011
00016 class ResetAlertDialogBuilderAdapter {
00020     @NonNull
00021     private final AlertDialog.Builder adapteeBuilder;
00022
00026     @NonNull
00027     private final Callback dialogCallback;
00028
00036     ResetAlertDialogBuilderAdapter(@NonNull Context context, @NonNull Callback dialogCallback) {
00037         this.adapteeBuilder = new AlertDialog.Builder(context)
00038                 .setIconAttribute(android.R.attr.alertDialogIcon)
00039                 .setTitle(R.string.alertDialog_reset_title)
00040                 .setOnCancelListener(dialog -> dialogCallback.reset(false))
00041                 .setNegativeButton(R.string.alertDialog_reset_negative, (dialog, which) ->
      dialog.cancel());
00042         this.dialogCallback = dialogCallback;
00043     }
00044
00052     @NonNull
00053     AlertDialog create() {
00054         return adapteeBuilder.create();
```

```
00055     }
00056
00064     @NonNull
00065     ResetAlertDialogBuilderAdapter setTrigger(@NonNull ResetDialogUseCase.Trigger trigger) {
00066         adapteeBuilder.setMessage((trigger != ResetDialogUseCase.Trigger.FACTION_SWITCH) ?
00067                         R.string.alertDialog_reset_msg_default :
00068                         R.string.alertDialog_reset_msg_faction_switch)
00069                 .setCancelable(trigger != ResetDialogUseCase.Trigger.FACTION_SWITCH);
00070         return this;
00071     }
00072
00081     @NonNull
00082     ResetAlertDialogBuilderAdapter setMonsterDialog(boolean monsterDialog) {
00083         if (monsterDialog) {
00084             View checkBoxView = View.inflate(adapteeBuilder.getContext(),
      R.layout.alertdialog_checkbox, null);
00085             adapteeBuilder.setView(checkBoxView)
00086                     .setPositiveButton(R.string.alertDialog_reset_positive, (dialog, which) -> {
00087                         CheckBox checkBox = checkBoxView.findViewById(R.id.alertDialog_checkbox);
00088                         dialogCallback.reset(true, checkBox.isChecked());
00089                     });
00090         } else {
00091             adapteeBuilder.setPositiveButton(R.string.alertDialog_reset_positive, ((dialog, which) ->
      dialogCallback.reset(true)));
00092         }
00093         return this;
00094     }
00095
00099     interface Callback {
00108         void reset(boolean resetDecision, boolean keepUnit);
00109
00118         default void reset(boolean resetDecision) {
00119             reset(resetDecision, false);
00120         }
00121     }
00122 }
```

6.13 ResetDialogUseCase.java

```
00001 package com.peternaggschga.gwent.domain.cases;
00002
00003 import static com.peternaggschga.gwent.ui.main.FactionSwitchListener.THEME_MONSTER;
00004 import static com.peternaggschga.gwent.ui.main.FactionSwitchListener.THEME_PREFERENCE_KEY;
00005 import static com.peternaggschga.gwent.ui.main.FactionSwitchListener.THEME_SCOIATAEL;
00006
00007 import android.content.Context;
00008 import android.content.SharedPreferences;
00009 import android.widget.Toast;
00010
00011 import androidx.annotation.NonNull;
00012 import androidx.preference.PreferenceManager;
00013
00014 import com.peternaggschga.gwent.GwentApplication;
00015 import com.peternaggschga.gwent.R;
00016 import com.peternaggschga.gwent.data.RowType;
00017 import com.peternaggschga.gwent.data.UnitRepository;
00018 import com.peternaggschga.gwent.ui.sounds.SoundManager;
00019
00020 import java.util.Arrays;
00021 import java.util.stream.Collectors;
00022
00023 import io.reactivex.rxjava3.core.Single;
00024
00031 public class ResetDialogUseCase {
00046     @NonNull
00047     public static Single<Boolean> reset(@NonNull Context context, @NonNull Trigger trigger, @NonNull
      SoundManager soundManager) {
00048         return GwentApplication.getRepository(context)
00049                 .flatMap(repository -> reset(context, repository, trigger, soundManager));
00050     }
00051
00065     @NonNull
00066     protected static Single<Boolean> reset(@NonNull Context context, @NonNull UnitRepository
      repository,
00067                                            @NonNull Trigger trigger, @NonNull SoundManager
      soundManager) {
00068         return getDialogType(context, repository, trigger).flatMap(dialogType -> {
00069             if (dialogType == DialogType.NONE) {
00070                 return ResetRepositoryUseCase.reset(context, repository,
      soundManager).andThen(Single.just(true));
00071             }
00072             return Single.create(emitter -> new ResetAlertDialogBuilderAdapter(context,
      (resetDecision, keepUnit) -> {
00073                 if (!resetDecision) {
```

```
00074                     emitter.onSuccess(false);
00075                     return;
00076                 }
00077                 // noinspection CheckResult, ResultOfMethodCallIgnored
00078                 ResetRepositoryUseCase.reset(context, repository, keepUnit, soundManager)
00079                         .doAfterTerminate(() -> emitter.onSuccess(true))
00080                         .subscribe(unit ->
00081                                 Toast.makeText(context,
00082 
      context.getString(R.string.alertDialog_factionreset_monster_toast_keep, unit.toString(context)),
00083                                                 Toast.LENGTH_LONG)
00084                                         .show());
00085             }).setTrigger(trigger)
00086                     .setMonsterDialog(dialogType == DialogType.MONSTER)
00087                     .create()
00088                     .show());
00089         });
00090     }
00091 
00101     @NonNull
00102     private static Single<DialogType> getDialogType(@NonNull Context context, @NonNull UnitRepository
      repository,
00103                                                     @NonNull Trigger trigger) {
00104         return Single.concat(Arrays.stream(RowType.values()).map(row ->
00105                         repository.isWeather(row)
00106                                 .concatWith(repository.isHorn(row))
00107                                 .any(state -> state)
00108                 ).collect(Collectors.toList())).any(state -> state)
00109                 .zipWith(repository.getUnits(), (statusEffects, units) -> {
00110                     SharedPreferences preferences =
      PreferenceManager.getDefaultSharedPreferences(context);
00111                     boolean monsterDialog = trigger != Trigger.FACTION_SWITCH;
00112                     monsterDialog &= preferences.getInt(THEME_PREFERENCE_KEY, THEME_SCOIATAEL) ==
      THEME_MONSTER;
00113                     monsterDialog &= units.stream().anyMatch(unit -> !unit.isEpic());
00114                     if (monsterDialog) {
00115                         return DialogType.MONSTER;
00116                     }
00117                     boolean defaultWarning = statusEffects || !units.isEmpty();
00118                     defaultWarning &=
      preferences.getBoolean(context.getString(R.string.preference_key_warning),
00119                             context.getResources().getBoolean(R.bool.warning_preference_default));
00120                     if (defaultWarning) {
00121                         return DialogType.DEFAULT;
00122                     }
00123                     return DialogType.NONE;
00124                 });
00125     }
00126 
00130     public enum Trigger {
00134         BUTTON_CLICK,
00140         FACTION_SWITCH
00141     }
00142 
00148     private enum DialogType {
00152         NONE,
00156         DEFAULT,
00161         MONSTER
00162     }
00163 }
```

6.14 ResetRepositoryUseCase.java

```
00001 package com.peternaggschga.gwent.domain.cases;
00002 
00003 import android.app.Dialog;
00004 import android.content.Context;
00005 
00006 import androidx.annotation.IntRange;
00007 import androidx.annotation.NonNull;
00008 import androidx.annotation.Nullable;
00009 
00010 import com.peternaggschga.gwent.data.Ability;
00011 import com.peternaggschga.gwent.data.UnitEntity;
00012 import com.peternaggschga.gwent.data.UnitRepository;
00013 import com.peternaggschga.gwent.ui.sounds.SoundManager;
00014 
00015 import java.util.List;
00016 import java.util.Optional;
00017 import java.util.Random;
00018 import java.util.stream.Collectors;
00019 
00020 import io.reactivex.rxjava3.core.Completable;
00021 import io.reactivex.rxjava3.core.CompletableEmitter;
```

```
00022 import io.reactivex.rxjava3.core.Maybe;
00023
00031 class ResetRepositoryUseCase {
00045     @NonNull
00046     static Maybe<UnitEntity> reset(@NonNull Context context, @NonNull UnitRepository repository,
00047                                    boolean keepUnit, @NonNull SoundManager soundManager) {
00048         return repository.getUnits()
00049                 .flatMapMaybe(units -> {
00050                     Optional<UnitEntity> keptUnit = keepUnit ? getRandomUnit(units) :
      Optional.empty();
00051                     long revengeUnits = units.stream()
00052                             .filter(unit -> unit.getAbility() == Ability.REVENGE)
00053                             .count() - (keptUnit.isPresent() && keptUnit.get().getAbility() ==
      Ability.REVENGE ? 1 : 0);
00054                     Completable resultAction = (revengeUnits == 0) ?
00055                             repository.reset(keptUnit.orElse(null)) :
00056                             Completable.create(emitter ->
00057                                     getRevengeDialog(context, repository, emitter,
      keptUnit.orElse(null), (int) revengeUnits, soundManager).show()
00058                             );
00059                     return resultAction.andThen(Maybe.fromOptional(keptUnit));
00060                 });
00061     }
00062
00070     @NonNull
00071     private static Optional<UnitEntity> getRandomUnit(@NonNull List<UnitEntity> units) {
00072         units = units.stream()
00073                 .filter(unit -> !unit.isEpic())
00074                 .collect(Collectors.toList());
00075         return units.isEmpty() ? Optional.empty() : Optional.of(units.get(new
      Random().nextInt(units.size())));
00076     }
00077
00092     @NonNull
00093     private static Dialog getRevengeDialog(@NonNull Context context, @NonNull UnitRepository
      repository,
00094                                            @NonNull CompletableEmitter emitter, @Nullable UnitEntity
      keptUnit,
00095                                            @IntRange(from = 1) int revengeUnits, @NonNull SoundManager
      soundManager) {
00096         return new RevengeAlertDialogBuilderAdapter(context)
00097                 .setPositiveCallback((dialogInterface, which) -> {
00098                     // noinspection CheckResult, ResultOfMethodCallIgnored
00099                     repository.reset(keptUnit)
00100                             .andThen(RevengeAlertDialogBuilderAdapter.insertAvengers(repository,
      revengeUnits, soundManager))
00101                             .subscribe(emitter::onComplete);
00102                 })
00103                 .setNegativeCallback(((dialog, which) -> {
00104                     // noinspection CheckResult, ResultOfMethodCallIgnored
00105                     repository.reset(keptUnit).subscribe(emitter::onComplete);
00106                 }))
00107                 .create();
00108     }
00109
00122     @NonNull
00123     public static Completable reset(@NonNull Context context, @NonNull UnitRepository repository,
      @NonNull SoundManager soundManager) {
00124         return Completable.fromMaybe(reset(context, repository, false, soundManager));
00125     }
00126 }
```

6.15 RevengeAlertDialogBuilderAdapter.java

```
00001 package com.peternaggschga.gwent.domain.cases;
00002
00003 import android.content.Context;
00004 import android.content.DialogInterface;
00005
00006 import androidx.annotation.IntRange;
00007 import androidx.annotation.NonNull;
00008 import androidx.appcompat.app.AlertDialog;
00009
00010 import com.peternaggschga.gwent.R;
00011 import com.peternaggschga.gwent.data.Ability;
00012 import com.peternaggschga.gwent.data.RowType;
00013 import com.peternaggschga.gwent.data.UnitRepository;
00014 import com.peternaggschga.gwent.ui.sounds.SoundManager;
00015
00016 import io.reactivex.rxjava3.core.Completable;
00017
00023 class RevengeAlertDialogBuilderAdapter {
00030     private static final boolean AVENGER_EPIC = false;
00031
```

```
00038     private static final Ability AVENGER_ABILITY = Ability.NONE;
00039
00046     private static final int AVENGER_DAMAGE = 8;
00047
00054     private static final Integer AVENGER_SQUAD = null;
00055
00062     private static final RowType AVENGER_ROW = RowType.MELEE;
00063
00067     @NonNull
00068     private final AlertDialog.Builder adapteeBuilder;
00069
00076     RevengeAlertDialogBuilderAdapter(@NonNull Context context) {
00077         this.adapteeBuilder = new AlertDialog.Builder(context)
00078                 .setIconAttribute(android.R.attr.alertDialogIcon)
00079                 .setTitle(R.string.alertDialog_revenge_title)
00080                 .setMessage(R.string.alertDialog_revenge_msg)
00081                 .setCancelable(false)
00082                 .setPositiveButton(R.string.alertDialog_revenge_positive, (dialog, which) ->
      dialog.cancel())
00083                 .setNegativeButton(R.string.alertDialog_revenge_negative, (dialog, which) ->
      dialog.cancel());
00084     }
00085
00096     @NonNull
00097     public static Completable insertAvengers(@NonNull UnitRepository repository, @IntRange(from = 0)
      int numberOfAvengers, @NonNull SoundManager soundManager) {
00098         return repository.insertUnit(AVENGER_EPIC, AVENGER_DAMAGE, AVENGER_ABILITY, AVENGER_SQUAD,
      AVENGER_ROW, numberOfAvengers)
00099                 .doOnComplete(() -> soundManager.playCardAddSound(AVENGER_ROW, AVENGER_EPIC));
00100     }
00101
00109     @NonNull
00110     AlertDialog create() {
00111         return adapteeBuilder.create();
00112     }
00113
00122     @NonNull
00123     RevengeAlertDialogBuilderAdapter setPositiveCallback(@NonNull DialogInterface.OnClickListener
      onPositiveButtonClick) {
00124         adapteeBuilder.setPositiveButton(R.string.alertDialog_revenge_positive,
      onPositiveButtonClick);
00125         return this;
00126     }
00127
00134     @NonNull
00135     RevengeAlertDialogBuilderAdapter setNegativeCallback(@NonNull DialogInterface.OnClickListener
      onNegativeButtonClick) {
00136         adapteeBuilder.setNegativeButton(R.string.alertDialog_revenge_negative,
      onNegativeButtonClick);
00137         return this;
00138     }
00139 }
```

6.16 BondDamageCalculatorDecorator.java

```
00001 package com.peternaggschga.gwent.domain.damage;
00002
00003 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color.BUFFED;
00004
00005 import androidx.annotation.IntRange;
00006 import androidx.annotation.NonNull;
00007
00008 import java.util.Map;
00009 import java.util.Objects;
00010
00017 class BondDamageCalculatorDecorator extends DamageCalculatorDecorator {
00022     private final Map<Integer, Integer> idToSquadSize;
00023
00035     BondDamageCalculatorDecorator(@NonNull DamageCalculator component, @NonNull Map<Integer, Integer>
      idToSquadSize) {
00036         super(component);
00037         if (idToSquadSize.values().stream().anyMatch(integer -> integer == null || integer < 1)) {
00038             throw new IllegalArgumentException("Map idToSquadSize must not contain non-positive or
      null values.");
00039         }
00040         this.idToSquadSize = idToSquadSize;
00041     }
00042
00053     @Override
00054     public int calculateDamage(int id, @IntRange(from = 0) int damage) {
00055         if (damage < 0) {
00056             throw new IllegalArgumentException("Damage must be greater or equal to 0.");
00057         }
00058         return Objects.requireNonNull(idToSquadSize.getOrDefault(id, 1)) *
```

```
    component.calculateDamage(id, damage);
00059     }
00060
00071     @Override
00072     public Color isBuffed(int id) {
00073         return (Objects.requireNonNull(idToSquadSize.getOrDefault(id, 0)) > 1)
00074                 ? BUFFED
00075                 : component.isBuffed(id);
00076     }
00077 }
```

6.17 DamageCalculator.java

```
00001 package com.peternaggschga.gwent.domain.damage;
00002
00003 import androidx.annotation.IntRange;
00004
00005 import java.util.Collection;
00006
00017 public interface DamageCalculator {
00030     Color isBuffed(int id);
00031
00043     int calculateDamage(int id, @IntRange(from = 0) int damage);
00044
00049     enum Color {
00053         DEFAULT,
00057         BUFFED,
00061         DEBUFFED
00062     }
00063 }
```

6.18 DamageCalculatorBuildDirector.java

```
00001 package com.peternaggschga.gwent.domain.damage;
00002
00003 import androidx.annotation.NonNull;
00004
00005 import com.peternaggschga.gwent.data.Ability;
00006 import com.peternaggschga.gwent.data.UnitEntity;
00007
00008 import java.util.Collection;
00009 import java.util.HashMap;
00010 import java.util.List;
00011 import java.util.Map;
00012 import java.util.Objects;
00013 import java.util.stream.Collectors;
00014
00021 public class DamageCalculatorBuildDirector {
00030     @NonNull
00031     public static DamageCalculator getCalculator(boolean weather, boolean horn, @NonNull
    Collection<UnitEntity> units) {
00032         DamageCalculatorBuilder builder = new DamageCalculatorBuilder();
00033         builder.setWeather(weather);
00034
00035         if (units.stream().anyMatch(unit -> unit.getAbility() == Ability.BINDING)) {
00036             setSquads(units, builder);
00037         }
00038
00039         if (units.stream().anyMatch(unit -> unit.getAbility() == Ability.MORAL_BOOST)) {
00040             setMoralBoosts(units, builder);
00041         }
00042
00043         if (horn || units.stream().anyMatch(unit -> unit.getAbility() == Ability.HORN)) {
00044             setHorns(horn, units, builder);
00045         }
00046
00047         return builder.getResult();
00048     }
00049
00061     private static void setSquads(@NonNull Collection<UnitEntity> units, @NonNull
    DamageCalculatorBuilder builder) {
00062         List<UnitEntity> bindingUnits = units.stream().filter(unit -> unit.getAbility() ==
    Ability.BINDING).collect(Collectors.toList());
00063         Map<Integer, Integer> squadToSquadSize = new HashMap<>();
00064         for (UnitEntity unit : bindingUnits) {
00065             squadToSquadSize.putIfAbsent(unit.getSquad(), 0);
00066             squadToSquadSize.put(unit.getSquad(),
    Objects.requireNonNull(squadToSquadSize.get(unit.getSquad())) + 1);
00067         }
00068         Map<Integer, Integer> idToSquadSize = new HashMap<>();
00069         for (UnitEntity unit : bindingUnits) {
```

```
00070             idToSquadSize.put(unit.getId(), squadToSquadSize.get(unit.getSquad()));
00071         }
00072         builder.setBond(idToSquadSize);
00073     }
00074
00085     private static void setMoralBoosts(@NonNull Collection<UnitEntity> units, @NonNull
      DamageCalculatorBuilder builder) {
00086         List<Integer> unitIds = units.stream()
00087                 .filter(unit -> unit.getAbility() == Ability.MORAL_BOOST)
00088                 .map(UnitEntity::getId)
00089                 .collect(Collectors.toList());
00090         builder.setMoral(unitIds);
00091     }
00092
00106     private static void setHorns(boolean horn, @NonNull Collection<UnitEntity> units, @NonNull
      DamageCalculatorBuilder builder) {
00107         List<Integer> unitIds = units.stream()
00108                 .filter(unit -> unit.getAbility() == Ability.HORN)
00109                 .map(UnitEntity::getId)
00110                 .collect(Collectors.toList());
00111         if (horn) {
00112             unitIds.add(null);
00113         }
00114         builder.setHorn(unitIds);
00115     }
00116 }
```

6.19 DamageCalculatorBuilder.java

```
00001 package com.peternaggschga.gwent.domain.damage;
00002
00003 import androidx.annotation.NonNull;
00004
00005 import java.util.List;
00006 import java.util.Map;
00007
00019 class DamageCalculatorBuilder {
00024     @NonNull
00025     private DamageCalculator calculator = new WeatherDamageCalculator(false);
00026
00032     void setWeather(boolean weather) {
00033         calculator = new WeatherDamageCalculator(weather);
00034     }
00035
00041     void setBond(@NonNull Map<Integer, Integer> idToSquad) {
00042         calculator = new BondDamageCalculatorDecorator(calculator, idToSquad);
00043     }
00044
00051     void setMoral(@NonNull List<Integer> unitIds) {
00052         calculator = new MoralDamageCalculatorDecorator(calculator, unitIds);
00053     }
00054
00062     void setHorn(@NonNull List<Integer> unitIds) {
00063         calculator = new HornDamageCalculatorDecorator(calculator, unitIds);
00064     }
00065
00071     @NonNull
00072     DamageCalculator getResult() {
00073         return calculator;
00074     }
00075 }
```

6.20 DamageCalculatorDecorator.java

```
00001 package com.peternaggschga.gwent.domain.damage;
00002
00003 import androidx.annotation.NonNull;
00004
00009 abstract class DamageCalculatorDecorator implements DamageCalculator {
00013     protected final DamageCalculator component;
00014
00020     DamageCalculatorDecorator(@NonNull DamageCalculator component) {
00021         this.component = component;
00022     }
00023 }
```

6.21 HornDamageCalculatorDecorator.java

```
00001 package com.peternaggschga.gwent.domain.damage;
00002
00003 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color.BUFFED;
00004
00005 import androidx.annotation.IntRange;
00006 import androidx.annotation.NonNull;
00007
00008 import java.util.List;
00009
00018 class HornDamageCalculatorDecorator extends DamageCalculatorDecorator {
00024     private final List<Integer> unitIds;
00025
00038     HornDamageCalculatorDecorator(@NonNull DamageCalculator component, @NonNull List<Integer> unitIds)
      {
00039         super(component);
00040         this.unitIds = unitIds;
00041     }
00042
00049     private boolean doubleDamage(int id) {
00050         return unitIds.contains(null) || !(unitIds.isEmpty() || unitIds.contains(id)) ||
      unitIds.size() > 1;
00051     }
00052
00062     @Override
00063     public int calculateDamage(int id, @IntRange(from = 0) int damage) {
00064         if (damage < 0) {
00065             throw new IllegalArgumentException("Damage must be greater or equal to 0 but is " + damage
      + ".");
00066         }
00067         return (doubleDamage(id) ? 2 : 1) * component.calculateDamage(id, damage);
00068     }
00069
00081     @Override
00082     public Color isBuffed(int id) {
00083         return doubleDamage(id) ? BUFFED : component.isBuffed(id);
00084     }
00085 }
```

6.22 MoralDamageCalculatorDecorator.java

```
00001 package com.peternaggschga.gwent.domain.damage;
00002
00003 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color.BUFFED;
00004
00005 import androidx.annotation.IntRange;
00006 import androidx.annotation.NonNull;
00007
00008 import java.util.List;
00009
00017 class MoralDamageCalculatorDecorator extends DamageCalculatorDecorator {
00022     private final List<Integer> unitIds;
00023
00036     MoralDamageCalculatorDecorator(@NonNull DamageCalculator component, @NonNull List<Integer>
      unitIds) {
00037         super(component);
00038         if (unitIds.contains(null)) {
00039             throw new IllegalArgumentException("List<Integer> unitIds must not contain null values.");
00040         }
00041         this.unitIds = unitIds;
00042     }
00043
00055     @Override
00056     public int calculateDamage(int id, @IntRange(from = 0) int damage) {
00057         if (damage < 0) {
00058             throw new IllegalArgumentException("Damage must be greater or equal to 0 but is " + damage
      + ".");
00059         }
00060         int componentDamage = component.calculateDamage(id, damage) + unitIds.size();
00061         return unitIds.contains(id) ? componentDamage - 1 : componentDamage;
00062     }
00063
00074     @Override
00075     public Color isBuffed(int id) {
00076         return (!unitIds.isEmpty() && (!unitIds.contains(id) || (unitIds.size() > 1)))
00077                 ? BUFFED
00078                 : component.isBuffed(id);
00079     }
00080 }
```

6.23 WeatherDamageCalculator.java

```
00001 package com.peternaggschga.gwent.domain.damage;
00002
00003 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color.DEBUFFED;
00004 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color.DEFAULT;
00005
00006 import androidx.annotation.IntRange;
00007
00014 class WeatherDamageCalculator implements DamageCalculator {
00018     private final boolean weather;
00019
00027     WeatherDamageCalculator(boolean weather) {
00028         this.weather = weather;
00029     }
00030
00040     @Override
00041     public int calculateDamage(int id, @IntRange(from = 0) int damage) {
00042         if (damage < 0) {
00043             throw new IllegalArgumentException("Damage must not be less than zero but is " + damage +
      ".");
00044         }
00045         if (damage == 0) {
00046             return damage;
00047         }
00048         return weather ? 1 : damage;
00049     }
00050
00060     @Override
00061     public Color isBuffed(int id) {
00062         return weather ? DEBUFFED : DEFAULT;
00063     }
00064 }
```

6.24 GwentApplication.java

```
00001 package com.peternaggschga.gwent;
00002
00003 import android.app.Application;
00004 import android.content.Context;
00005
00006 import androidx.annotation.NonNull;
00007 import androidx.room3.Room;
00008
00009 import com.peternaggschga.gwent.data.AppDatabase;
00010 import com.peternaggschga.gwent.data.UnitRepository;
00011
00012 import io.reactivex.rxjava3.core.Single;
00013
00018 public class GwentApplication extends Application {
00025     private static UnitRepository repository = null;
00032     private AppDatabase database;
00033
00041     public static Single<UnitRepository> getRepository(@NonNull Context context) {
00042         return ((GwentApplication) context.getApplicationContext()).getRepository();
00043     }
00044
00050     @Override
00051     public void onCreate() {
00052         super.onCreate();
00053         database = Room.databaseBuilder(this, AppDatabase.class, "database").build();
00054     }
00055
00062     public Single<UnitRepository> getRepository() {
00063         if (repository != null) {
00064             return Single.just(repository);
00065         }
00066         return UnitRepository.getRepository(database).doOnSuccess(unitRepository -> repository =
      unitRepository);
00067     }
00068 }
```

6.25 AddCardDialog.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.addcard;
00002
00003 import android.content.Context;
00004 import android.os.Bundle;
00005
00006 import androidx.annotation.NonNull;
```

```
00007 import androidx.annotation.Nullable;
00008
00009 import com.peternaggschga.gwent.GwentApplication;
00010 import com.peternaggschga.gwent.R;
00011 import com.peternaggschga.gwent.data.RowType;
00012 import com.peternaggschga.gwent.data.UnitRepository;
00013 import com.peternaggschga.gwent.ui.dialogs.OverlayDialog;
00014 import com.peternaggschga.gwent.ui.dialogs.cards.ShowUnitsDialog;
00015 import com.peternaggschga.gwent.ui.sounds.SoundManager;
00016
00017 import io.reactivex.rxjava3.disposables.CompositeDisposable;
00018
00022 public class AddCardDialog extends OverlayDialog {
00026     @NonNull
00027     private final RowType row;
00028
00034     @NonNull
00035     private final SoundManager soundManager;
00036
00046     @NonNull
00047     private final CompositeDisposable disposables = new CompositeDisposable();
00048
00056     @Nullable
00057     private CardNumberPickerAdapter pickerAdapter = null;
00058
00067     public AddCardDialog(@NonNull ShowUnitsDialog caller, @NonNull SoundManager soundManager) {
00068         this(caller.getContext(), caller.getRow(), soundManager);
00069
00070         setOnDismissListener(dialog -> {
00071             disposables.dispose();
00072             caller.show();
00073         });
00074     }
00075
00084     public AddCardDialog(@NonNull Context context, @NonNull RowType row, @NonNull SoundManager
      soundManager) {
00085         super(context, R.layout.popup_add_card, R.id.popup_add_card_cancel_button);
00086         this.row = row;
00087         this.soundManager = soundManager;
00088
00089         setOnDismissListener(dialog -> disposables.dispose());
00090     }
00091
00101     @Override
00102     protected void onCreate(Bundle savedInstanceState) {
00103         super.onCreate(savedInstanceState);
00104
00105         if (pickerAdapter == null) {
00106             disposables.add(
00107                     GwentApplication.getRepository(getContext())
00108                             .flatMap(UnitRepository::getUnits)
00109                             .map(SquadManager::new)
00110                             .map(squadManager ->
00111                                     new CardNumberPickerAdapter(findViewById(R.id.card_layout),
      squadManager))
00112                             .subscribe(cardNumberPickerAdapter -> pickerAdapter =
      cardNumberPickerAdapter)
00113             );
00114         }
00115
00116         findViewById(R.id.popup_add_card_save_button).setOnClickListener(v -> {
00117             if (pickerAdapter != null) {
00118                 disposables.add(pickerAdapter.addSelectedUnits(row).subscribe(epic -> {
00119                     soundManager.playCardAddSound(row, epic);
00120                     dismiss();
00121                 }));
00122             }
00123         });
00124     }
00125 }
```

6.26 CardNumberPickerAdapter.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.addcard;
00002
00003 import android.os.Handler;
00004 import android.os.Looper;
00005 import android.view.View;
00006 import android.view.ViewGroup;
00007 import android.widget.NumberPicker;
00008
00009 import androidx.annotation.NonNull;
00010
00011 import com.peternaggschga.gwent.GwentApplication;
```

```
00012 import com.peternaggschga.gwent.R;
00013 import com.peternaggschga.gwent.data.Ability;
00014 import com.peternaggschga.gwent.data.RowType;
00015 import com.peternaggschga.gwent.data.UnitEntity;
00016
00017 import org.jetbrains.annotations.Contract;
00018
00019 import java.util.Arrays;
00020 import java.util.List;
00021 import java.util.Objects;
00022 import java.util.SortedMap;
00023 import java.util.TreeMap;
00024
00025 import io.reactivex.rxjava3.core.Single;
00026
00033 class CardNumberPickerAdapter {
00037     @NonNull
00038     private static final List<Ability> EPIC_UNIT_ABILITIES = Arrays.asList(Ability.NONE, Ability.HORN,
      Ability.MORAL_BOOST);
00039
00043     @NonNull
00044     private final ValuePicker<Boolean> epicPicker;
00045
00049     @NonNull
00050     private final DamageValuePicker damagePicker;
00051
00057     @NonNull
00058     private final ValuePicker<Ability> abilityPicker;
00059
00065     @NonNull
00066     private final NumberPicker squadPicker;
00067
00071     @NonNull
00072     private final SquadManager squadManager;
00073
00077     @NonNull
00078     private final NumberPicker numberPicker;
00079
00089     CardNumberPickerAdapter(@NonNull ViewGroup pickerGroup, @NonNull SquadManager squadManager) {
00090         this.squadManager = squadManager;
00091
00092         SortedMap<Boolean, Integer> epicStringResources = new TreeMap<>();
00093         epicStringResources.put(false, R.string.add_picker_epic_normal);
00094         epicStringResources.put(true, R.string.add_picker_epic_epic);
00095         epicPicker = new
      StringValuePicker<>(pickerGroup.findViewById(R.id.popup_add_card_epic_picker),
00096                 epicStringResources,
00097                 false);
00098
00099         damagePicker = new
      DamageValuePicker(pickerGroup.findViewById(R.id.popup_add_card_dmg_picker));
00100
00101         SortedMap<Ability, Integer> abilityStringResources = new TreeMap<>();
00102         abilityStringResources.put(Ability.NONE, R.string.add_picker_ability_default);
00103         abilityStringResources.put(Ability.HORN, R.string.add_picker_ability_horn);
00104         abilityStringResources.put(Ability.REVENGE, R.string.add_picker_ability_revenge);
00105         abilityStringResources.put(Ability.BINDING, R.string.add_picker_ability_binding);
00106         abilityStringResources.put(Ability.MORAL_BOOST, R.string.add_picker_ability_moralBoost);
00107         abilityPicker = new
      StringValuePicker<>(pickerGroup.findViewById(R.id.popup_add_card_ability_picker),
00108                 abilityStringResources,
00109                 Ability.NONE);
00110
00111         squadPicker = pickerGroup.findViewById(R.id.popup_add_card_binding_picker);
00112         squadPicker.setMinValue(1);
00113         squadPicker.setMaxValue(SquadManager.MAX_NR_SQUADS);
00114
00115         numberPicker = pickerGroup.findViewById(R.id.popup_add_card_number_picker);
00116         numberPicker.setMinValue(1);
00117         numberPicker.setMaxValue(10);
00118
00119         epicPicker.setOnValueChangedListener((picker, oldVal, newVal) -> {
00120             damagePicker.setEpicValues(newVal);
00121             abilityPicker.setSelectableValues(newVal ? EPIC_UNIT_ABILITIES :
      Arrays.asList(Ability.values()));
00122         });
00123
00124         abilityPicker.setOnValueChangedListener((picker, oldVal, newVal) -> {
00125             if (newVal == Ability.BINDING) {
00126                 squadPicker.setVisibility(View.VISIBLE);
00127                 squadPicker.setValue(squadManager.getFirstSquadWithMembers());
00128                 squadManager.onSquadChanged(squadManager.getFirstSquadWithMembers(), damagePicker);
00129             } else {
00130                 squadPicker.setVisibility(View.GONE);
00131             }
00132         });
00133
```

```
00134         squadPicker.setOnValueChangedListener(
00135                 getDelayedOnValueChangeListener((picker, oldVal, newVal) -> {
00136                     if (newVal == picker.getValue()) {
00137                         squadManager.onSquadChanged(newVal, damagePicker);
00138                     }
00139                 })
00140         );
00141     }
00142 
00150     @NonNull
00151     @Contract(pure = true)
00152     static NumberPicker.OnValueChangeListener getDelayedOnValueChangeListener(@NonNull
      NumberPicker.OnValueChangeListener originalListener) {
00153         return (picker, oldVal, newVal) -> new Handler(Looper.getMainLooper()).postDelayed(() -> {
00154             if (newVal == picker.getValue()) {
00155                 originalListener.onValueChange(picker, oldVal, newVal);
00156             }
00157         }, 500);
00158     }
00159 
00165     @NonNull
00166     Single<Boolean> addSelectedUnits(@NonNull RowType row) {
00167         // handle delayed events due to delayed OnValueChangedListeners
00168         // delayed changes in damagePicker after hero selection
00169         int damage;
00170         if (epicPicker.getValue()) {
00171             damage = Arrays.stream(UnitEntity.EPIC_DAMAGE_VALUES).anyMatch(integer ->
      Objects.equals(integer, damagePicker.getValue())) ? damagePicker.getValue() :
      UnitEntity.EPIC_DAMAGE_VALUES[3];
00172         } else {
00173             damage = damagePicker.getValue();
00174         }
00175 
00176         // delayed changes in squadPicker after squad selection
00177         Integer squad;
00178         if (abilityPicker.getValue() == Ability.BINDING) {
00179             squad = squadPicker.getVisibility() == View.VISIBLE ? squadPicker.getValue() :
      squadManager.getFirstSquadWithMembers();
00180         } else {
00181             squad = null;
00182         }
00183         return GwentApplication.getRepository(numberPicker.getContext())
00184                 .flatMapCompletable(repository ->
00185                         repository.insertUnit(epicPicker.getValue(),
00186                                 damage,
00187                                 abilityPicker.getValue(),
00188                                 squad,
00189                                 row,
00190                                 numberPicker.getValue()))
00191                 .andThen(Single.just(epicPicker.getValue()));
00192     }
00193 }
```

6.27 DamageValuePicker.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.addcard;
00002 
00003 
00004 import static com.peternaggschga.gwent.data.UnitEntity.EPIC_DAMAGE_VALUES;
00005 import static com.peternaggschga.gwent.data.UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND;
00006 
00007 import android.widget.NumberPicker;
00008 
00009 import androidx.annotation.IntRange;
00010 import androidx.annotation.NonNull;
00011 import androidx.annotation.Nullable;
00012 
00013 import java.util.Arrays;
00014 import java.util.Collection;
00015 import java.util.Map;
00016 import java.util.SortedMap;
00017 import java.util.TreeMap;
00018 import java.util.stream.Collectors;
00019 import java.util.stream.IntStream;
00020 
00025 class DamageValuePicker extends ValuePicker<Integer> {
00030     private boolean epicValues = false;
00031 
00039     DamageValuePicker(@NonNull NumberPicker picker) {
00040         super(picker, Arrays.stream(EPIC_DAMAGE_VALUES).collect(
00041                 TreeMap::new,
00042                 (map, integer) -> map.put(integer, integer),
00043                 Map::putAll)
00044         );
```

```
00045         setEpicValues(false);
00046     }
00047
00055     @Override
00056     @NonNull
00057     protected String getDisplayString(@NonNull Integer value) {
00058         if (epicValues && !getDisplayIntegers().containsKey(value)) {
00059             throw new IllegalStateException("When epicValues is true, displayIntegers must contain the
      given key but does not contain " + value + ".");
00060         }
00061         return epicValues ? String.valueOf(getDisplayIntegers().get(value)) : String.valueOf(value);
00062     }
00063
00073     @Override
00074     void setValue(@NonNull @IntRange(from = 0, to = NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) Integer value)
      {
00075         if (epicValues) {
00076             super.setValue(value);
00077         } else if (value < 0 || value > NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) {
00078             throw new IllegalStateException("Value must be in [0, " +
      NON_EPIC_DAMAGE_VALUES_UPPER_BOUND + "] but is " + value + ".");
00079         } else {
00080             getPicker().setValue(value);
00081         }
00082     }
00083
00088     @NonNull
00089     @Override
00090     Integer getValue() {
00091         return epicValues ? super.getValue() : getPicker().getValue();
00092     }
00093
00100     void setEpicValues(boolean epicValues) {
00101         this.epicValues = epicValues;
00102
00103         if (epicValues) {
00104             super.setSelectableValues(Arrays.asList(EPIC_DAMAGE_VALUES), EPIC_DAMAGE_VALUES[3]);
00105             return;
00106         }
00107         getSelectableValues().clear();
00108         getSelectableValues().addAll(IntStream.rangeClosed(0,
      NON_EPIC_DAMAGE_VALUES_UPPER_BOUND).boxed().collect(Collectors.toList()));
00109         getPicker().setDisplayedValues(null);
00110         getPicker().setMaxValue(NON_EPIC_DAMAGE_VALUES_UPPER_BOUND);
00111         getPicker().setValue(5);
00112     }
00113
00120     @Override
00121     void setSelectableValues(@NonNull Collection<Integer> values, @Nullable Integer defaultValue) {
00122     }
00123 }
```

6.28 SquadManager.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.addcard;
00002
00003
00004
00005 import android.content.Context;
00006 import android.widget.Toast;
00007
00008 import androidx.annotation.IntRange;
00009 import androidx.annotation.NonNull;
00010
00011 import com.peternaggschga.gwent.R;
00012 import com.peternaggschga.gwent.data.Ability;
00013 import com.peternaggschga.gwent.data.UnitEntity;
00014
00015 import java.util.Arrays;
00016 import java.util.List;
00017 import java.util.stream.Collectors;
00018
00024 class SquadManager {
00028     public static final int MAX_NR_SQUADS = 3;
00029
00033     @NonNull
00034     private final SquadState[] states = new SquadState[MAX_NR_SQUADS];
00035
00041     SquadManager(@NonNull List<UnitEntity> units) {
00042         units = units.stream()
00043                 .filter(unit -> unit.getAbility() == Ability.BINDING)
00044                 .collect(Collectors.toList());
00045         for (int i = 0; i < MAX_NR_SQUADS; i++) {
00046             states[i] = SquadState.getState(i + 1, units);
```

```
00047         }
00048     }
00049
00055     int getFirstSquadWithMembers() {
00056         return Arrays.stream(states)
00057                 .filter(SquadState::hasMembers)
00058                 .findFirst()
00059                 .map(SquadState::getSquadNumber)
00060                 .orElse(1);
00061     }
00062
00072     void onSquadChanged(@IntRange(from = 1, to = MAX_NR_SQUADS) int newVal, @NonNull DamageValuePicker
      picker) {
00073         if (newVal < 1 || newVal > MAX_NR_SQUADS) {
00074             throw new IllegalArgumentException("NewVal must be in [1, " + MAX_NR_SQUADS + "] but is "
      + newVal + ".");
00075         }
00076         SquadState squad = states[newVal - 1];
00077
00078         Context context = picker.getContext();
00079         Toast.makeText(context,
00080                 context.getString(R.string.popUp_add_card_binding_count, squad.getSquadNumber(),
      squad.getSquadMembers()),
00081                 Toast.LENGTH_SHORT).show();
00082
00083         if (squad.hasMembers()) {
00084             picker.setValue(squad.getMemberBaseDamage());
00085         }
00086     }
00087 }
```

6.29 SquadState.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.addcard;
00002
00003 import static com.peternaggschga.gwent.data.UnitEntity.NON_EPIC_DAMAGE_VALUES_UPPER_BOUND;
00004 import static com.peternaggschga.gwent.ui.dialogs.addcard.SquadManager.MAX_NR_SQUADS;
00005
00006 import androidx.annotation.IntRange;
00007 import androidx.annotation.NonNull;
00008
00009 import com.peternaggschga.gwent.data.UnitEntity;
00010
00011 import java.util.List;
00012 import java.util.stream.Collectors;
00013
00019 class SquadState {
00026     @IntRange(from = 1, to = MAX_NR_SQUADS)
00027     private final int squadNumber;
00028
00033     @IntRange(from = 0)
00034     private final int squadMembers;
00035
00042     @IntRange(from = 0, to = NON_EPIC_DAMAGE_VALUES_UPPER_BOUND)
00043     private final int memberBaseDamage;
00044
00055     private SquadState(@IntRange(from = 1, to = MAX_NR_SQUADS) int squadNumber, @IntRange(from = 0)
      int squadMembers,
00056                        @IntRange(from = 0, to = NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) int
      memberBaseDamage) {
00057         if (squadNumber < 1 || squadNumber > MAX_NR_SQUADS) {
00058             throw new IllegalArgumentException("SquadNumber must be in [1, " + MAX_NR_SQUADS + "] but
      is " + squadNumber + ".");
00059         }
00060         if (squadMembers < 0) {
00061             throw new IllegalArgumentException("SquadMembers must be non-negative but is " +
      squadMembers + ".");
00062         }
00063         if (memberBaseDamage < 0 || memberBaseDamage > NON_EPIC_DAMAGE_VALUES_UPPER_BOUND) {
00064             throw new IllegalArgumentException("MemberBaseDamage must be in [0, " +
      NON_EPIC_DAMAGE_VALUES_UPPER_BOUND + "] but is " + memberBaseDamage + ".");
00065         }
00066         this.squadNumber = squadNumber;
00067         this.squadMembers = squadMembers;
00068         this.memberBaseDamage = memberBaseDamage;
00069     }
00070
00079     @NonNull
00080     static SquadState getState(@IntRange(from = 1, to = MAX_NR_SQUADS) int squadNumber,
00081                                @NonNull List<UnitEntity> units) {
00082         if (squadNumber < 1 || squadNumber > MAX_NR_SQUADS) {
00083             throw new IllegalArgumentException("SquadNumber must be in [1, " + MAX_NR_SQUADS + "] but
      is " + squadNumber + ".");
00084         }
```

```
00085         units = units.stream()
00086                 .filter(unit -> unit.getSquad() != null && unit.getSquad() == squadNumber)
00087                 .collect(Collectors.toList());
00088         return new SquadState(squadNumber,
00089                 units.size(),
00090                 units.stream().findAny().map(UnitEntity::getDamage).orElse(5));
00091     }
00092
00097     boolean hasMembers() {
00098         return squadMembers > 0;
00099     }
00100
00106     @IntRange(from = 1, to = MAX_NR_SQUADS)
00107     int getSquadNumber() {
00108         return squadNumber;
00109     }
00110
00116     @IntRange(from = 0)
00117     int getSquadMembers() {
00118         return squadMembers;
00119     }
00120
00126     @IntRange(from = 0, to = NON_EPIC_DAMAGE_VALUES_UPPER_BOUND)
00127     int getMemberBaseDamage() {
00128         return memberBaseDamage;
00129     }
00130 }
```

6.30 StringValuePicker.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.addcard;
00002
00003
00004
00005 import android.widget.NumberPicker;
00006
00007 import androidx.annotation.NonNull;
00008 import androidx.annotation.Nullable;
00009
00010 import java.util.SortedMap;
00011
00017 class StringValuePicker<T extends Comparable> extends ValuePicker<T> {
00027     @SuppressWarnings("unused")
00028     StringValuePicker(@NonNull NumberPicker picker, @NonNull SortedMap<T, Integer> valueToStringRes) {
00029         this(picker, valueToStringRes, null);
00030     }
00031
00045     StringValuePicker(@NonNull NumberPicker picker, @NonNull SortedMap<T, Integer> valueToStringRes,
      @Nullable T defaultValue) {
00046         super(picker, valueToStringRes, defaultValue);
00047     }
00048
00056     @Override
00057     @NonNull
00058     protected String getDisplayString(@NonNull T value) {
00059         Integer resId;
00060         if ((resId = getDisplayIntegers().get(value)) == null) {
00061             throw new IllegalStateException("Value must be key in displayIntegers but is " + value +
      ".");
00062         }
00063         return getContext().getString(resId);
00064     }
00065 }
```

6.31 ValuePicker.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.addcard;
00002
00003
00004
00005 import android.content.Context;
00006 import android.widget.NumberPicker;
00007
00008 import androidx.annotation.NonNull;
00009 import androidx.annotation.Nullable;
00010 import androidx.annotation.UiContext;
00011
00012 import java.util.ArrayList;
00013 import java.util.Collection;
00014 import java.util.HashMap;
00015 import java.util.List;
```

```
00016 import java.util.Map;
00017 import java.util.SortedMap;
00018
00029 abstract class ValuePicker<T extends Comparable> {
00037     @NonNull
00038     private final NumberPicker picker;
00039
00045     @NonNull
00046     private final Map<T, Integer> displayIntegers;
00047
00054     @NonNull
00055     private final List<T> selectableValues;
00056
00065     ValuePicker(@NonNull NumberPicker picker, @NonNull SortedMap<T, Integer> valueToStringRes) {
00066         this(picker, valueToStringRes, null);
00067     }
00068
00081     ValuePicker(@NonNull NumberPicker picker, @NonNull SortedMap<T, Integer> valueToStringRes,
00082                 @Nullable T defaultValue) {
00083         if (valueToStringRes.isEmpty()) {
00084             throw new IllegalArgumentException("SortedMap<T, Integer> valueToStringRes must not be
      empty.");
00085         }
00086         if (defaultValue != null && !valueToStringRes.containsKey(defaultValue)) {
00087             throw new IllegalArgumentException("DefaultValue must be null or a key in the given
      SortedMap.");
00088         }
00089         this.picker = picker;
00090         displayIntegers = new HashMap<>(valueToStringRes);
00091         selectableValues = new ArrayList<>(valueToStringRes.size());
00092
00093         picker.setMinValue(0);
00094         setSelectableValues(valueToStringRes.keySet(), defaultValue);
00095     }
00096
00107     void setSelectableValues(@NonNull Collection<T> values, @Nullable T defaultValue) {
00108         if (values.isEmpty()) {
00109             throw new IllegalArgumentException("Collection<T> values must not be empty.");
00110         }
00111         if (!displayIntegers.keySet().containsAll(values)) {
00112             throw new IllegalStateException("DisplayIntegers must contain all given values.");
00113         }
00114         if (defaultValue != null && !displayIntegers.containsKey(defaultValue)) {
00115             throw new IllegalArgumentException("DefaultValue must be null or a key in the given
      SortedMap.");
00116         }
00117
00118         picker.setDisplayedValues(null);
00119         picker.setValue(0);
00120         picker.setMaxValue(values.size() - 1);
00121
00122         selectableValues.clear();
00123         String[] displayValues = new String[values.size()];
00124         int i = 0;
00125         for (T value : values) {
00126             selectableValues.add(value);
00127             displayValues[i++] = getDisplayString(value);
00128         }
00129
00130         if (defaultValue != null) {
00131             setValue(defaultValue);
00132         }
00133         picker.setDisplayedValues(displayValues);
00134     }
00135
00142     @NonNull
00143     protected NumberPicker getPicker() {
00144         return picker;
00145     }
00146
00157     void setOnValueChangedListener(@NonNull OnValueChangeListener<T> onValueChangedListener) {
00158         picker.setOnValueChangedListener(
00159                 CardNumberPickerAdapter.getDelayedOnValueChangeListener((picker, oldVal, newVal) ->
00160                         onValueChangedListener.onValueChange(ValuePicker.this,
00161                                 selectableValues.get(oldVal),
00162                                 selectableValues.get(newVal)))
00163         );
00164     }
00165
00170     @NonNull
00171     T getValue() {
00172         return selectableValues.get(picker.getValue());
00173     }
00174
00180     void setValue(@NonNull T value) {
00181         int position;
00182         if ((position = selectableValues.indexOf(value)) < 0) {
```

```
00183             throw new IllegalStateException("SelectableValues must contain the given value.");
00184         }
00185         picker.setValue(position);
00186     }
00187
00194     @NonNull
00195     protected abstract String getDisplayString(@NonNull T value);
00196
00203     @UiContext
00204     public Context getContext() {
00205         return picker.getContext();
00206     }
00207
00211     interface OnValueChangeListener<T extends Comparable> {
00219         void onValueChange(@NonNull ValuePicker<T> picker, T oldVal, T newVal);
00220     }
00221
00228     @NonNull
00229     protected Map<T, Integer> getDisplayIntegers() {
00230         return displayIntegers;
00231     }
00232
00239     @NonNull
00240     protected List<T> getSelectableValues() {
00241         return selectableValues;
00242     }
00243
00254     void setSelectableValues(@NonNull Collection<T> values) {
00255         setSelectableValues(values, null);
00256     }
00257 }
```

6.32 CardListAdapter.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.cards;
00002
00003 import static androidx.recyclerview.widget.RecyclerView.NO_ID;
00004
00005 import android.view.LayoutInflater;
00006 import android.view.View;
00007 import android.view.ViewGroup;
00008 import android.widget.ImageView;
00009 import android.widget.TextView;
00010
00011 import androidx.annotation.IntRange;
00012 import androidx.annotation.NonNull;
00013 import androidx.core.util.Consumer;
00014 import androidx.recyclerview.widget.DiffUtil;
00015 import androidx.recyclerview.widget.ListAdapter;
00016 import androidx.recyclerview.widget.RecyclerView;
00017
00018 import com.peternaggschga.gwent.R;
00019
00028 class CardListAdapter extends ListAdapter<CardUiState, CardListAdapter.CardViewHolder> {
00038     @NonNull
00039     private final Consumer<Integer> onCopy;
00040
00049     @NonNull
00050     private final Consumer<Integer> onRemove;
00051
00065     CardListAdapter(@NonNull Consumer<Integer> onCopy, @NonNull Consumer<Integer> onRemove) {
00066         super(CardUiState.DIFF_CALLBACK);
00067         this.onCopy = onCopy;
00068         this.onRemove = onRemove;
00069         setHasStableIds(true);
00070     }
00071
00079     @NonNull
00080     @Override
00081     public CardViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
00082         View cardView = LayoutInflater.from(parent.getContext()).inflate(R.layout.card, parent,
      false);
00083         return new CardViewHolder(cardView, onCopy, onRemove);
00084     }
00085
00092     @Override
00093     public void onBindViewHolder(@NonNull CardViewHolder holder, @IntRange(from = 0) int position) {
00094         CardUiState item = getItem(position);
00095         holder.getDamageView().setText(item.getDamageString());
00096         holder.getDamageView().setBackgroundResource(item.getDamageBackgroundImageId());
00097         holder.getDamageView().setTextColor(item.getDamageTextColor());
00098
00099         if (item.showAbility()) {
00100             holder.getAbilityView().setImageResource(item.getAbilityImageId());
```

```
00101             holder.getAbilityView().setVisibility(View.VISIBLE);
00102         } else {
00103             holder.getAbilityView().setVisibility(View.GONE);
00104         }
00105 
00106         if (item.showSquad()) {
00107             holder.getBindingView().setText(item.getSquadString());
00108             holder.getBindingView().setVisibility(View.VISIBLE);
00109         } else {
00110             holder.getBindingView().setVisibility(View.GONE);
00111         }
00112 
00113         holder.setItemId(item.getUnitId());
00114     }
00115 
00121     @Override
00122     public long getItemId(@IntRange(from = 0) int position) {
00123         return getItem(position).getUnitId();
00124     }
00125 
00131     static class CardViewHolder extends RecyclerView.ViewHolder {
00136         @NonNull
00137         private final TextView damageView;
00143         @NonNull
00144         private final ImageView abilityView;
00151         @NonNull
00152         private final TextView bindingView;
00161         private int itemId = (int) NO_ID;
00162 
00173         CardViewHolder(@NonNull View itemView,
00174                        @NonNull Consumer<Integer> onCopy, @NonNull Consumer<Integer> onRemove) {
00175             super(itemView);
00176 
00177             damageView = itemView.findViewById(R.id.damageView);
00178             abilityView = itemView.findViewById(R.id.abilityView);
00179             bindingView = itemView.findViewById(R.id.bindingView);
00180 
00181             itemView.findViewById(R.id.copyButton).setOnClickListener(v -> {
00182                 if (itemId <= NO_ID) {
00183                     throw new IllegalArgumentException("ItemId must be greater than NO_ID (" + NO_ID +
      ") but is " + itemId + ".");
00184                 }
00185                 onCopy.accept(itemId);
00186             });
00187             itemView.findViewById(R.id.deleteButton).setOnClickListener(v -> {
00188                 if (itemId <= NO_ID) {
00189                     throw new IllegalArgumentException("ItemId must be greater than NO_ID (" + NO_ID +
      ") but is " + itemId + ".");
00190                 }
00191                 onRemove.accept(itemId);
00192             });
00193         }
00194 
00200         @NonNull
00201         TextView getDamageView() {
00202             return damageView;
00203         }
00204 
00210         @NonNull
00211         ImageView getAbilityView() {
00212             return abilityView;
00213         }
00214 
00220         @NonNull
00221         TextView getBindingView() {
00222             return bindingView;
00223         }
00224 
00232         void setItemId(int itemId) {
00233             if (itemId <= NO_ID) {
00234                 throw new IllegalArgumentException("ItemId must be greater than NO_ID (-1) but is " +
      itemId + ".");
00235             }
00236             this.itemId = itemId;
00237         }
00238     }
00239 }
```

6.33 CardUiState.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.cards;
00002 
00003 
00004 
```

```
00005 import androidx.annotation.ColorInt;
00006 import androidx.annotation.DrawableRes;
00007 import androidx.annotation.IntRange;
00008 import androidx.annotation.NonNull;
00009 import androidx.annotation.Nullable;
00010 import androidx.recyclerview.widget.DiffUtil;
00011
00012 import com.peternaggschga.gwent.data.UnitEntity;
00013 import com.peternaggschga.gwent.domain.damage.DamageCalculator;
00014
00015 import java.util.Objects;
00016
00024 public class CardUiState {
00032     @NonNull
00033     public static final DiffUtil.ItemCallback<CardUiState> DIFF_CALLBACK = new
      DiffUtil.ItemCallback<CardUiState>() {
00034         @Override
00035         public boolean areItemsTheSame(@NonNull CardUiState oldItem, @NonNull CardUiState newItem) {
00036             return oldItem.unitId == newItem.unitId;
00037         }
00038
00039         @Override
00040         public boolean areContentsTheSame(@NonNull CardUiState oldItem, @NonNull CardUiState newItem)
      {
00041             return Objects.equals(oldItem, newItem);
00042         }
00043     };
00044
00050     public static final int UNUSED = -1;
00051
00057     private final int unitId;
00058
00063     @DrawableRes
00064     private final int damageBackgroundImageId;
00065
00071     @NonNull
00072     private final String damageString;
00073
00078     @ColorInt
00079     private final int damageTextColor;
00080
00087     @DrawableRes
00088     private final int abilityImageId;
00089
00094     @NonNull
00095     private final String squadString;
00096
00108     public CardUiState(int unitId, @DrawableRes int damageBackgroundImageId, @IntRange(from = UNUSED)
      int damage,
00109                        @ColorInt int damageTextColor, @DrawableRes int abilityImageId,
00110                        @Nullable @IntRange(from = 1) Integer squad) {
00111         if (damage < UNUSED) {
00112             throw new IllegalArgumentException("Damage must be greater or equal to UNUSED (" + UNUSED
      + ") but is " + damage + ".");
00113         }
00114         if (squad != null && squad < 1) {
00115             throw new IllegalArgumentException("Squad must not be null or less than 1 but is " + squad
      + ".");
00116         }
00117         this.unitId = unitId;
00118         this.damageBackgroundImageId = damageBackgroundImageId;
00119         this.damageString = damage == UNUSED ? "" : String.valueOf(damage);
00120         this.damageTextColor = damageTextColor;
00121         this.abilityImageId = abilityImageId;
00122         this.squadString = squad == null ? "" : String.valueOf(squad);
00123     }
00124
00130     public boolean showAbility() {
00131         return abilityImageId != UNUSED;
00132     }
00133
00139     public boolean showSquad() {
00140         return showAbility() && !squadString.isEmpty();
00141     }
00142
00148     public int getUnitId() {
00149         return unitId;
00150     }
00151
00157     @DrawableRes
00158     public int getDamageBackgroundImageId() {
00159         return damageBackgroundImageId;
00160     }
00161
00167     @NonNull
00168     public String getDamageString() {
00169         return damageString;
```

```
00170     }
00171
00177     @ColorInt
00178     public int getDamageTextColor() {
00179         return damageTextColor;
00180     }
00181
00188     @DrawableRes
00189     public int getAbilityImageId() {
00190         return abilityImageId;
00191     }
00192
00199     @NonNull
00200     public String getSquadString() {
00201         return squadString;
00202     }
00203
00211     @Override
00212     public boolean equals(Object o) {
00213         if (this == o) return true;
00214         if (!(o instanceof CardUiState)) return false;
00215         CardUiState that = (CardUiState) o;
00216         return damageBackgroundImageId == that.damageBackgroundImageId
00217                 && damageTextColor == that.damageTextColor
00218                 && abilityImageId == that.abilityImageId
00219                 && Objects.equals(damageString, that.damageString)
00220                 && Objects.equals(squadString, that.squadString);
00221     }
00222 }
```

6.34 CardUiStateFactory.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.cards;
00002
00003 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color;
00004 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color.BUFFED;
00005 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color.DEBUFFED;
00006 import static com.peternaggschga.gwent.domain.damage.DamageCalculator.Color.DEFAULT;
00007
00008 import android.content.Context;
00009
00010 import androidx.annotation.NonNull;
00011
00012 import com.peternaggschga.gwent.R;
00013 import com.peternaggschga.gwent.data.UnitEntity;
00014 import com.peternaggschga.gwent.domain.cases.DamageCalculatorUseCase;
00015 import com.peternaggschga.gwent.domain.damage.DamageCalculator;
00016
00017 import java.util.Collection;
00018 import java.util.HashMap;
00019 import java.util.List;
00020 import java.util.Map;
00021 import java.util.Objects;
00022 import java.util.stream.Collectors;
00023
00028 public class CardUiStateFactory {
00033     private final boolean weather;
00034
00039     private final boolean horn;
00040
00045     @NonNull
00046     private final Map<Color, Integer> damageTextColors = new HashMap<>(Color.values().length);
00047
00058     public CardUiStateFactory(@NonNull Context context, boolean weather, boolean horn) {
00059         this.weather = weather;
00060         this.horn = horn;
00061         damageTextColors.put(DEFAULT, context.getColor(R.color.color_damage_textColor));
00062         damageTextColors.put(BUFFED, context.getColor(R.color.color_damage_textColor_buffed));
00063         damageTextColors.put(DEBUFFED, context.getColor(R.color.color_damage_textColor_debuffed));
00064     }
00065
00073     @NonNull
00074     public List<CardUiState> createCardUiState(@NonNull Collection<UnitEntity> units) {
00075         DamageCalculator calculator = DamageCalculatorUseCase.getDamageCalculator(weather, horn,
      units);
00076         return units.stream()
00077                 .map(unit -> createCardUiState(unit, calculator))
00078                 .collect(Collectors.toList());
00079     }
00080
00088     @NonNull
00089     public CardUiState createCardUiState(@NonNull UnitEntity unit, @NonNull DamageCalculator
      calculator) {
00090         int damageBackgroundId = R.drawable.icon_damage_background;
```

```
00091         int damage = unit.calculateDamage(calculator);
00092
00093         if (unit.isEpic()) {
00094             switch (damage) {
00095                 case 7:
00096                     damageBackgroundId = R.drawable.icon_epic_damage_7;
00097                     break;
00098                 case 8:
00099                     damageBackgroundId = R.drawable.icon_epic_damage_8;
00100                    break;
00101                case 10:
00102                    damageBackgroundId = R.drawable.icon_epic_damage_10;
00103                    break;
00104                case 11:
00105                    damageBackgroundId = R.drawable.icon_epic_damage_11;
00106                    break;
00107                case 15:
00108                    damageBackgroundId = R.drawable.icon_epic_damage_15;
00109                    break;
00110                case 0:
00111                default:
00112                    damageBackgroundId = R.drawable.icon_epic_damage_0;
00113            }
00114            damage = CardUiState.UNUSED;
00115        }
00116
00117        int damageColor = Objects.requireNonNull(damageTextColors.get(unit.isBuffed(calculator)));
00118
00119        int abilityImage;
00120        switch (unit.getAbility()) {
00121            case HORN:
00122                abilityImage = R.drawable.icon_horn;
00123                break;
00124            case REVENGE:
00125                abilityImage = R.drawable.icon_revenge;
00126                break;
00127            case BINDING:
00128                abilityImage = R.drawable.icon_binding;
00129                break;
00130            case MORAL_BOOST:
00131                abilityImage = R.drawable.icon_moral_boost;
00132                break;
00133            case NONE:
00134            default:
00135                abilityImage = CardUiState.UNUSED;
00136        }
00137
00138        return new CardUiState(unit.getId(), damageBackgroundId, damage, damageColor, abilityImage,
      unit.getSquad());
00139    }
00140 }
```

6.35 ShowUnitsDialog.java

```
00001 package com.peternaggschga.gwent.ui.dialogs.cards;
00002
00003 import android.content.Context;
00004 import android.os.Bundle;
00005 import android.util.Log;
00006
00007 import androidx.annotation.NonNull;
00008 import androidx.recyclerview.widget.DefaultItemAnimator;
00009 import androidx.recyclerview.widget.LinearLayoutManager;
00010 import androidx.recyclerview.widget.RecyclerView;
00011
00012 import com.peternaggschga.gwent.GwentApplication;
00013 import com.peternaggschga.gwent.R;
00014 import com.peternaggschga.gwent.data.RowType;
00015 import com.peternaggschga.gwent.data.UnitEntity;
00016 import com.peternaggschga.gwent.domain.cases.RemoveUnitsUseCase;
00017 import com.peternaggschga.gwent.ui.dialogs.OverlayDialog;
00018 import com.peternaggschga.gwent.ui.dialogs.addcard.AddCardDialog;
00019 import com.peternaggschga.gwent.ui.sounds.SoundManager;
00020
00021 import java.util.Objects;
00022
00023 import io.reactivex.rxjava3.core.Single;
00024 import io.reactivex.rxjava3.disposables.CompositeDisposable;
00025
00031 public class ShowUnitsDialog extends OverlayDialog {
00037     @NonNull
00038     private final RowType row;
00039
00043     @NonNull
```

```
00044     private final CardListAdapter cardListAdapter;
00045
00052     @NonNull
00053     private final SoundManager soundManager;
00054
00062     @NonNull
00063     private final CompositeDisposable disposables = new CompositeDisposable();
00064
00068     private RecyclerView unitRecyclerView;
00069
00078     private ShowUnitsDialog(@NonNull Context context, @NonNull RowType row,
00079                             @NonNull CardListAdapter cardListAdapter, @NonNull SoundManager
      soundManager) {
00080         super(context, R.layout.popup_cards, R.id.popup_cards_cancel_button);
00081         this.row = row;
00082         this.cardListAdapter = cardListAdapter;
00083         this.soundManager = soundManager;
00084     }
00085
00093     @NonNull
00094     public static Single<ShowUnitsDialog> getDialog(@NonNull Context context, @NonNull RowType row,
      @NonNull SoundManager soundManager) {
00095         return GwentApplication.getRepository(context)
00096                 .flatMap(repository -> repository.isWeather(row)
00097                         .zipWith(repository.isHorn(row), (weather, horn) ->
00098                                 new CardUiStateFactory(context, weather, horn))
00099                         .map(factory -> {
00100                             CompositeDisposable initialDisposables = new CompositeDisposable();
00101                             CardListAdapter adapter = new CardListAdapter(
00102                                     id -> initialDisposables.add(
00103                                             repository
00104                                                     .copy(id)
00105                                                     .andThen(repository.getUnit(id))
00106                                                     .map(UnitEntity::isEpic)
00107                                                     .subscribe(epic ->
      soundManager.playCardAddSound(row, epic))
00108                                     ),
00109                                     id -> initialDisposables.add(
00110                                             RemoveUnitsUseCase.remove(context, repository, id,
      soundManager)
00111                                                     .subscribe(soundManager::playCardRemovedSound,
00112                                                             throwable ->
      Log.e(ShowUnitsDialog.class.getSimpleName(),
00113                                                                     "There has been an error with the
      removal of a unit. " +
00114                                                                             "A reason might be tapping
      delete buttons too fast!",
00115                                                                     throwable))
00116                                     )
00117                             );
00118                             initialDisposables.add(
00119                                     repository.getUnitsFlowable(row)
00120                                             .map(factory::createCardUiState)
00121                                             .subscribe(adapter::submitList)
00122                             );
00123                             ShowUnitsDialog result = new ShowUnitsDialog(context, row, adapter,
      soundManager);
00124                             result.disposables.add(initialDisposables);
00125                             return result;
00126                         })
00127                 );
00128     }
00129
00141     @Override
00142     protected void onCreate(Bundle savedInstanceState) {
00143         super.onCreate(savedInstanceState);
00144
00145         if (unitRecyclerView == null) {
00146             LinearLayoutManager llm = new LinearLayoutManager(getContext());
00147             llm.setOrientation(RecyclerView.HORIZONTAL);
00148             unitRecyclerView = findViewById(R.id.cards_list);
00149             ((DefaultItemAnimator)
      Objects.requireNonNull(unitRecyclerView.getItemAnimator())).setSupportsChangeAnimations(false);
00150             unitRecyclerView.setLayoutManager(llm);
00151             unitRecyclerView.setAdapter(cardListAdapter);
00152
00153             cardListAdapter.registerAdapterDataObserver(new ScrollToEndObserver());
00154         }
00155
00156         findViewById(R.id.popup_cards_add_button).setOnClickListener(v -> {
00157             hide();
00158             new AddCardDialog(ShowUnitsDialog.this, soundManager).show();
00159         });
00160
00161         setOnDismissListener(dialog -> disposables.dispose());
00162     }
00163
```

```
00168     private class ScrollToEndObserver extends RecyclerView.AdapterDataObserver {
00174         @Override
00175         public void onItemRangeInserted(int positionStart, int itemCount) {
00176             super.onItemRangeInserted(positionStart, itemCount);
00177             int scrollPosition = positionStart + itemCount - 1;
00178             if (scrollPosition >= 0) {
00179                 unitRecyclerView.smoothScrollToPosition(scrollPosition);
00180             }
00181         }
00182     }
00183
00190     @NonNull
00191     public RowType getRow() {
00192         return row;
00193     }
00194 }
```

6.36 ChangeFactionDialog.java

```
00001 package com.peternaggschga.gwent.ui.dialogs;
00002
00003 import static com.peternaggschga.gwent.ui.main.FactionSwitchListener.THEME_MONSTER;
00004 import static com.peternaggschga.gwent.ui.main.FactionSwitchListener.THEME_NILFGAARD;
00005 import static com.peternaggschga.gwent.ui.main.FactionSwitchListener.THEME_NORTHERN_KINGDOMS;
00006 import static com.peternaggschga.gwent.ui.main.FactionSwitchListener.THEME_SCOIATAEL;
00007
00008 import android.content.Context;
00009 import android.os.Bundle;
00010 import android.view.View;
00011
00012 import androidx.annotation.IntRange;
00013 import androidx.annotation.NonNull;
00014
00015 import com.peternaggschga.gwent.R;
00016
00020 public class ChangeFactionDialog extends OverlayDialog {
00024     private final Callback callback;
00025
00033     public ChangeFactionDialog(@NonNull Context context, @NonNull Callback callback) {
00034         super(context, R.layout.popup_faction, R.id.factionBackground);
00035         this.callback = callback;
00036     }
00037
00046     @Override
00047     protected void onCreate(Bundle savedInstanceState) {
00048         super.onCreate(savedInstanceState);
00049
00050         findViewById(R.id.monsterCardView).setOnClickListener(getOnThemeClickListener(THEME_MONSTER));
00051         findViewById(R.id.monsterButton).setOnClickListener(getOnThemeClickListener(THEME_MONSTER));
00052
00053
      findViewById(R.id.nilfgaardCardView).setOnClickListener(getOnThemeClickListener(THEME_NILFGAARD));
00054
      findViewById(R.id.nilfgaardButton).setOnClickListener(getOnThemeClickListener(THEME_NILFGAARD));
00055
00056
      findViewById(R.id.northernKingdomsCardView).setOnClickListener(getOnThemeClickListener(THEME_NORTHERN_KINGDOMS));
00057
      findViewById(R.id.northernKingdomsButton).setOnClickListener(getOnThemeClickListener(THEME_NORTHERN_KINGDOMS));
00058
00059
      findViewById(R.id.scoiataelCardView).setOnClickListener(getOnThemeClickListener(THEME_SCOIATAEL));
00060
      findViewById(R.id.scoiataelButton).setOnClickListener(getOnThemeClickListener(THEME_SCOIATAEL));
00061     }
00062
00072     @NonNull
00073     private View.OnClickListener getOnThemeClickListener(@IntRange(from = THEME_MONSTER, to =
      THEME_SCOIATAEL) int theme) {
00074         return v -> {
00075             cancel();
00076             callback.onThemeSelect(theme);
00077         };
00078     }
00079
00084     public interface Callback {
00090         void onThemeSelect(@IntRange(from = THEME_MONSTER, to = THEME_SCOIATAEL) int theme);
00091     }
00092 }
```

6.37 CoinFlipDialog.java

```
00001 package com.peternaggschga.gwent.ui.dialogs;
00002
00003 import android.content.Context;
00004 import android.os.Bundle;
00005 import android.widget.ImageView;
00006 import android.widget.TextView;
00007
00008 import androidx.annotation.NonNull;
00009
00010 import com.peternaggschga.gwent.R;
00011
00012 import java.util.Random;
00013
00024 public class CoinFlipDialog extends OverlayDialog {
00028     private static final Random RANDOM = new Random();
00029
00034     public CoinFlipDialog(@NonNull Context context) {
00035         super(context, R.layout.popup_coin_normal, R.id.coinflipBackground);
00036     }
00037
00047     @Override
00048     protected void onCreate(Bundle savedInstanceState) {
00049         super.onCreate(savedInstanceState);
00050
00051         switch (RANDOM.nextInt(100)) {
00052             case 0:
00053                 setContentView(R.layout.popup_coin_stewie);
00054                 break;
00055             case 1:
00056                 setContentView(R.layout.popup_coin_terry);
00057                 break;
00058             case 2:
00059                 setContentView(R.layout.popup_coin_vin);
00060                 break;
00061             default:
00062                 if (RANDOM.nextBoolean()) {
00063                     ((ImageView)
      findViewById(R.id.popup_coin_normal_coinView)).setImageResource(R.drawable.coin_lose);
00064                     ((TextView)
      findViewById(R.id.popup_coin_normal_textView)).setText(R.string.popUp_coin_normal_lose);
00065                 }
00066         }
00067     }
00068 }
```

6.38 OverlayDialog.java

```
00001 package com.peternaggschga.gwent.ui.dialogs;
00002
00003 import android.app.Dialog;
00004 import android.content.Context;
00005 import android.graphics.Color;
00006 import android.graphics.drawable.ColorDrawable;
00007 import android.os.Bundle;
00008 import android.view.ViewGroup;
00009 import android.view.Window;
00010
00011 import androidx.annotation.IdRes;
00012 import androidx.annotation.LayoutRes;
00013 import androidx.annotation.NonNull;
00014
00015 import java.util.Objects;
00016
00022 public class OverlayDialog extends Dialog {
00028     static final int NO_DISMISS_VIEW = -1;
00032     @NonNull
00033     private static final ColorDrawable BACKGROUND = new ColorDrawable(Color.TRANSPARENT);
00037     @LayoutRes
00038     private final int layout;
00044     @IdRes
00045     private final int dismissViewId;
00046
00054     protected OverlayDialog(@NonNull Context context, @LayoutRes int layout, @IdRes int dismissViewId)
      {
00055         super(context);
00056         this.layout = layout;
00057         this.dismissViewId = dismissViewId;
00058     }
00059
00067     @SuppressWarnings("unused")
00068     protected OverlayDialog(@NonNull Context context, @LayoutRes int layout) {
```

```
00069         this(context, layout, NO_DISMISS_VIEW);
00070     }
00071
00084     @Override
00085     protected void onCreate(Bundle savedInstanceState) {
00086         super.onCreate(savedInstanceState);
00087
00088         setContentView(layout);
00089
00090         Window window = Objects.requireNonNull(getWindow());
00091         window.setLayout(ViewGroup.LayoutParams.MATCH_PARENT, ViewGroup.LayoutParams.MATCH_PARENT);
00092         window.setBackgroundDrawable(BACKGROUND);
00093
00094         if (dismissViewId != NO_DISMISS_VIEW) {
00095             findViewById(dismissViewId).setOnClickListener(v -> dismiss());
00096         }
00097         setCancelable(true);
00098     }
00099 }
```

6.39 IndicatorManager.java

```
00001 package com.peternaggschga.gwent.ui.introduction;
00002
00003 import android.view.Window;
00004 import android.widget.ImageView;
00005
00006 import androidx.annotation.IntRange;
00007 import androidx.annotation.NonNull;
00008
00009 import com.peternaggschga.gwent.R;
00010
00015 class IndicatorManager {
00019     @NonNull
00020     private final ImageView[] indicators = new ImageView[5];
00021
00029     @IntRange(from = 0, to = 4)
00030     private int currentView = 0;
00031
00043     IndicatorManager(@NonNull Window window) {
00044         indicators[0] = window.findViewById(R.id.introduction_indicator_0);
00045         indicators[1] = window.findViewById(R.id.introduction_indicator_1);
00046         indicators[2] = window.findViewById(R.id.introduction_indicator_2);
00047         indicators[3] = window.findViewById(R.id.introduction_indicator_3);
00048         indicators[4] = window.findViewById(R.id.introduction_indicator_4);
00049     }
00050
00059     void updateIndicators(@IntRange(from = 0, to = 4) int position) {
00060         indicators[currentView].setImageResource(R.drawable.indicator_unselected);
00061         indicators[position].setImageResource(R.drawable.indicator_selected);
00062         currentView = position;
00063     }
00064 }
```

6.40 IntroductionActivity.java

```
00001 package com.peternaggschga.gwent.ui.introduction;
00002
00003 import android.os.Bundle;
00004 import android.view.View;
00005 import android.widget.Button;
00006 import android.widget.ImageButton;
00007
00008 import androidx.activity.OnBackPressedCallback;
00009 import androidx.appcompat.app.AppCompatActivity;
00010 import androidx.preference.PreferenceManager;
00011 import androidx.viewpager2.widget.ViewPager2;
00012
00013 import com.peternaggschga.gwent.R;
00014
00020 public class IntroductionActivity extends AppCompatActivity {
00025     private IndicatorManager indicatorManager;
00026
00030     private ViewPager2 viewPager;
00031
00035     private OnBackPressedCallback callback;
00036
00040     private Button skipButton;
00041
00045     private ImageButton nextButton;
00046
```

```
00050     private Button finishButton;
00051
00062     @Override
00063     protected void onCreate(Bundle savedInstanceState) {
00064         super.onCreate(savedInstanceState);
00065
00066         setContentView(R.layout.activity_introduction);
00067
00068         if (indicatorManager == null) {
00069             indicatorManager = new IndicatorManager(getWindow());
00070         }
00071
00072         viewPager = findViewById(R.id.introduction_viewPager);
00073         callback = new OnBackPressedCallback(false) {
00078             @Override
00079             public void handleOnBackPressed() {
00080                 viewPager.setCurrentItem(viewPager.getCurrentItem() - 1);
00081             }
00082         };
00083         getOnBackPressedDispatcher().addCallback(callback);
00084
00085         View.OnClickListener onFinish = view -> {
00086             PreferenceManager.getDefaultSharedPreferences(getApplicationContext())
00087                     .edit()
00088                     .putBoolean(getString(R.string.preference_first_use_key), false)
00089                     .apply();
00090             callback.setEnabled(false);
00091             getOnBackPressedDispatcher().onBackPressed();
00092         };
00093
00094         skipButton = findViewById(R.id.introduction_button_skip);
00095         nextButton = findViewById(R.id.introduction_button_next);
00096         finishButton = findViewById(R.id.introduction_button_finish);
00097
00098         skipButton.setOnClickListener(onFinish);
00099         nextButton.setOnClickListener(v -> viewPager.setCurrentItem(viewPager.getCurrentItem() + 1));
00100         finishButton.setOnClickListener(onFinish);
00101
00102         viewPager.setAdapter(new SectionsPagerAdapter(this));
00103         viewPager.registerOnPageChangeCallback(new ViewPager2.OnPageChangeCallback() {
00104             // Keep the bottom bar in sync with the selected page.
00105             @Override
00106             public void onPageSelected(int position) {
00107                 updateBottomBar(position);
00108             }
00109         });
00110     }
00111
00115     @Override
00116     protected void onResume() {
00117         super.onResume();
00118         updateBottomBar(viewPager.getCurrentItem());
00119     }
00120
00128     private void updateBottomBar(int position) {
00129         indicatorManager.updateIndicators(position);
00130         skipButton.setVisibility(position == IntroductionFragment.PAGES_COUNT - 1 ? View.GONE :
      View.VISIBLE);
00131         nextButton.setVisibility(position == IntroductionFragment.PAGES_COUNT - 1 ? View.GONE :
      View.VISIBLE);
00132         finishButton.setVisibility(position == IntroductionFragment.PAGES_COUNT - 1 ? View.VISIBLE :
      View.GONE);
00133         callback.setEnabled(position != 0);
00134     }
00135 }
```

6.41 IntroductionFragment.java

```
00001 package com.peternaggschga.gwent.ui.introduction;
00002
00003 import android.os.Bundle;
00004 import android.text.Html;
00005 import android.text.method.LinkMovementMethod;
00006 import android.view.LayoutInflater;
00007 import android.view.View;
00008 import android.view.ViewGroup;
00009 import android.widget.ImageView;
00010 import android.widget.TextView;
00011
00012 import androidx.annotation.IntRange;
00013 import androidx.annotation.NonNull;
00014 import androidx.fragment.app.Fragment;
00015
00016 import com.peternaggschga.gwent.R;
```

```
00017
00021 public class IntroductionFragment extends Fragment {
00026     public static final int PAGES_COUNT = 5;
00030     private static final String ARG_SECTION_NUMBER = "section_number";
00034     private static final int[] imageIds = {R.drawable.introduction_1, R.drawable.introduction_2,
00035             R.drawable.introduction_3, R.drawable.introduction_4, R.drawable.introduction_5};
00039     private static final int[] stringIds = {R.string.introduction_page1, R.string.introduction_page2,
00040             R.string.introduction_page3, R.string.introduction_page4, R.string.introduction_page5};
00041
00049     @NonNull
00050     public static IntroductionFragment newInstance(@IntRange(from = 0, to = PAGES_COUNT - 1) int
      index) {
00051         if (index < 0 || index >= PAGES_COUNT) {
00052             throw new IllegalArgumentException("Index must be in [0, PAGES_COUNT - 1] but is " + index
      + ".");
00053         }
00054         Bundle arguments = new Bundle();
00055         arguments.putInt(ARG_SECTION_NUMBER, index);
00056         IntroductionFragment fragment = new IntroductionFragment();
00057         fragment.setArguments(arguments);
00058         return fragment;
00059     }
00060
00074     @Override
00075     public View onCreateView(@NonNull LayoutInflater inflater, ViewGroup container, Bundle
      savedInstanceState) {
00076         int page = getArguments() != null ? getArguments().getInt(ARG_SECTION_NUMBER) : 0;
00077
00078         View root = inflater.inflate(R.layout.fragment_introduction, container, false);
00079
00080         TextView textView = root.findViewById(R.id.introduction_textView);
00081         textView.setText(Html.fromHtml(getString(stringIds[page]), Html.FROM_HTML_MODE_LEGACY));
00082         textView.setMovementMethod(LinkMovementMethod.getInstance());
00083
00084         ((ImageView) root.findViewById(R.id.introduction_imageView)).setImageResource(imageIds[page]);
00085         return root;
00086     }
00087 }
```

6.42 SectionsPagerAdapter.java

```
00001 package com.peternaggschga.gwent.ui.introduction;
00002
00003 import androidx.annotation.NonNull;
00004 import androidx.fragment.app.Fragment;
00005 import androidx.fragment.app.FragmentActivity;
00006 import androidx.viewpager2.adapter.FragmentStateAdapter;
00007
00012 public class SectionsPagerAdapter extends FragmentStateAdapter {
00018     public SectionsPagerAdapter(@NonNull FragmentActivity fragmentActivity) {
00019         super(fragmentActivity);
00020     }
00021
00027     @Override
00028     public int getItemCount() {
00029         return IntroductionFragment.PAGES_COUNT;
00030     }
00031
00037     @NonNull
00038     @Override
00039     public Fragment createFragment(int position) {
00040         return IntroductionFragment.newInstance(position);
00041     }
00042 }
```

6.43 FactionSwitchListener.java

```
00001 package com.peternaggschga.gwent.ui.main;
00002
00003 import android.content.Context;
00004 import android.content.SharedPreferences;
00005 import android.content.res.TypedArray;
00006 import android.os.Build;
00007 import android.view.Window;
00008 import android.widget.ImageButton;
00009 import android.widget.ImageView;
00010 import android.widget.TextView;
00011
00012 import androidx.annotation.NonNull;
00013 import androidx.annotation.Nullable;
00014 import androidx.constraintlayout.widget.ConstraintLayout;
```

```
00015 import androidx.preference.PreferenceManager;
00016
00017 import com.peternaggschga.gwent.R;
00018
00019 import org.jetbrains.annotations.Contract;
00020
00021 import java.util.ArrayList;
00022 import java.util.List;
00023
00031 public class FactionSwitchListener implements SharedPreferences.OnSharedPreferenceChangeListener {
00035     @NonNull
00036     public static final String THEME_PREFERENCE_KEY = "theme";
00037
00041     public static final int THEME_MONSTER = 0;
00042
00046     public static final int THEME_NILFGAARD = 1;
00047
00051     public static final int THEME_NORTHERN_KINGDOMS = 2;
00052
00056     public static final int THEME_SCOIATAEL = 3;
00057
00065     @NonNull
00066     private final List<ImageView> ballViews;
00067
00075     @NonNull
00076     private final List<ImageView> cardViews;
00077
00085     @NonNull
00086     private final List<TextView> unitViews;
00087
00095     @NonNull
00096     private final ImageButton factionButton;
00097
00108     private FactionSwitchListener(@NonNull List<ImageView> ballViews, @NonNull List<ImageView>
      cardViews,
00109                                   @NonNull List<TextView> unitViews, @NonNull ImageButton
      factionButton) {
00110         this.ballViews = ballViews;
00111         this.cardViews = cardViews;
00112         this.unitViews = unitViews;
00113         this.factionButton = factionButton;
00114     }
00115
00122     @NonNull
00123     @Contract("_ -> new")
00124     public static FactionSwitchListener getListener(@NonNull Window mainWindow) {
00125         List<ImageView> ballViews = new ArrayList<>(4);
00126         List<ImageView> cardViews = new ArrayList<>(3);
00127         List<TextView> unitViews = new ArrayList<>(3);
00128
00129         for (int id : new int[]{R.id.firstRow, R.id.secondRow, R.id.thirdRow}) {
00130             ConstraintLayout rowLayout = mainWindow.findViewById(id);
00131
00132             ballViews.add(rowLayout.findViewById(R.id.pointBall));
00133             cardViews.add(rowLayout.findViewById(R.id.cardsImage));
00134             unitViews.add(rowLayout.findViewById(R.id.cardCountView));
00135         }
00136
00137         ballViews.add(mainWindow.findViewById(R.id.overallPointBall));
00138
00139         return new FactionSwitchListener(ballViews, cardViews, unitViews,
      mainWindow.findViewById(R.id.factionButton));
00140     }
00141
00148     public static void setTheme(@NonNull Context context) {
00149         switch (PreferenceManager.getDefaultSharedPreferences(context)
00150                 .getInt(THEME_PREFERENCE_KEY, THEME_SCOIATAEL)) {
00151             case THEME_MONSTER:
00152                 context.setTheme(R.style.MonsterTheme);
00153                 break;
00154             case THEME_NILFGAARD:
00155                 context.setTheme(R.style.NilfgaardTheme);
00156                 break;
00157             case THEME_NORTHERN_KINGDOMS:
00158                 context.setTheme(R.style.NorthernKingdomsTheme);
00159                 break;
00160             case THEME_SCOIATAEL:
00161             default:
00162                 context.setTheme(R.style.ScoiataelTheme);
00163         }
00164     }
00165
00175     @Override
00176     public void onSharedPreferenceChanged(SharedPreferences sharedPreferences, @Nullable String key) {
00177         if (!THEME_PREFERENCE_KEY.equals(key)) {
00178             return;
00179         }
```

```
00180
00181         Context context = factionButton.getContext();
00182
00183         switch (sharedPreferences.getInt(key, THEME_SCOIATAEL)) {
00184             case THEME_MONSTER:
00185                 context.setTheme(R.style.MonsterTheme);
00186                 break;
00187             case THEME_NILFGAARD:
00188                 context.setTheme(R.style.NilfgaardTheme);
00189                 break;
00190             case THEME_NORTHERN_KINGDOMS:
00191                 context.setTheme(R.style.NorthernKingdomsTheme);
00192                 break;
00193             case THEME_SCOIATAEL:
00194             default:
00195                 context.setTheme(R.style.ScoiataelTheme);
00196         }
00197
00198         int ballImageRes;
00199         int cardImageRes;
00200         int unitNumberTextColor;
00201         int factionButtonImageRes;
00202         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
00203             try (TypedArray attributeValues =
    context.getTheme().obtainStyledAttributes(R.styleable.theme)) {
00204                 ballImageRes = attributeValues.getResourceId(R.styleable.theme_point_ball_mipmap,
    R.drawable.ball_green);
00205                 cardImageRes = attributeValues.getResourceId(R.styleable.theme_card_view_mipmap,
    R.drawable.card_scoiatael_landscape_free);
00206                 unitNumberTextColor = attributeValues.getColor(R.styleable.theme_colorPrimary,
    context.getColor(R.color.color_text_scoiatael));
00207                 factionButtonImageRes =
    attributeValues.getResourceId(R.styleable.theme_android_alertDialogIcon,
    R.drawable.icon_round_scoiatael);
00208             }
00209         } else {
00210             //noinspection resource
00211             TypedArray attributeValues = context.getTheme().obtainStyledAttributes(R.styleable.theme);
00212             ballImageRes = attributeValues.getResourceId(R.styleable.theme_point_ball_mipmap,
    R.drawable.ball_green);
00213             cardImageRes = attributeValues.getResourceId(R.styleable.theme_card_view_mipmap,
    R.drawable.card_scoiatael_landscape_free);
00214             unitNumberTextColor = attributeValues.getColor(R.styleable.theme_colorPrimary,
    context.getColor(R.color.color_text_scoiatael));
00215             factionButtonImageRes =
    attributeValues.getResourceId(R.styleable.theme_android_alertDialogIcon,
    R.drawable.icon_round_scoiatael);
00216             attributeValues.recycle();
00217         }
00218
00219         ballViews.forEach(view -> ImageViewSwitchAnimator.animatedSwitch(view,
    ballImageRes).subscribe());
00220         cardViews.forEach(view -> ImageViewSwitchAnimator.animatedSwitch(view,
    cardImageRes).subscribe());
00221         unitViews.forEach(textView -> textView.setTextColor(unitNumberTextColor));
00222         ImageViewSwitchAnimator.animatedSwitch(factionButton, factionButtonImageRes).subscribe();
00223     }
00224 }
```

6.44 GameBoardViewModel.java

```
00001 package com.peternaggschga.gwent.ui.main;
00002
00003 import static androidx.lifecycle.ViewModelProvider.AndroidViewModelFactory.APPLICATION_KEY;
00004
00005 import android.content.Context;
00006
00007 import androidx.annotation.NonNull;
00008 import androidx.lifecycle.AndroidViewModel;
00009 import androidx.lifecycle.ViewModelProvider;
00010 import androidx.lifecycle.ViewModelStoreOwner;
00011 import androidx.lifecycle.viewmodel.ViewModelInitializer;
00012
00013 import com.peternaggschga.gwent.GwentApplication;
00014 import com.peternaggschga.gwent.data.RowType;
00015 import com.peternaggschga.gwent.data.UnitRepository;
00016 import com.peternaggschga.gwent.domain.cases.BurnDialogUseCase;
00017 import com.peternaggschga.gwent.domain.cases.DamageCalculatorUseCase;
00018 import com.peternaggschga.gwent.domain.cases.ResetDialogUseCase;
00019 import com.peternaggschga.gwent.domain.damage.DamageCalculator;
00020 import com.peternaggschga.gwent.ui.sounds.SoundManager;
00021
00022 import java.util.HashMap;
00023 import java.util.Map;
```

```
00024 import java.util.Objects;
00025 import java.util.concurrent.TimeUnit;
00026
00027 import io.reactivex.rxjava3.android.schedulers.AndroidSchedulers;
00028 import io.reactivex.rxjava3.core.Completable;
00029 import io.reactivex.rxjava3.core.Flowable;
00030 import io.reactivex.rxjava3.core.Single;
00031 import io.reactivex.rxjava3.schedulers.Schedulers;
00032
00038 public class GameBoardViewModel extends AndroidViewModel {
00044     @NonNull
00045     private static final ViewModelInitializer<GameBoardViewModel> INITIALIZER = new
      ViewModelInitializer<>(
00046             GameBoardViewModel.class,
00047             creationExtras -> {
00048                 GwentApplication app = (GwentApplication) creationExtras.get(APPLICATION_KEY);
00049                 assert app != null;
00050                 return new GameBoardViewModel(app);
00051             });
00052
00060     private GameBoardViewModel(@NonNull GwentApplication application) {
00061         super(application);
00062     }
00063
00070     @NonNull
00071     private final Map<RowType, Flowable<RowUiState>> rowUiStates = new
      HashMap<>(RowType.values().length);
00072
00079     @NonNull
00080     private Flowable<MenuUiState> menuUiState = Flowable.empty();
00081
00085     private SoundManager soundManager;
00086
00097     @NonNull
00098     public static GameBoardViewModel getModel(@NonNull ViewModelStoreOwner owner,
00099                                               @NonNull UnitRepository repository,
00100                                               @NonNull SoundManager soundManager) {
00101         GameBoardViewModel result = new ViewModelProvider(owner,
      ViewModelProvider.Factory.from(INITIALIZER))
00102                 .get(GameBoardViewModel.class);
00103
00104         for (RowType row : RowType.values()) {
00105             result.rowUiStates.put(row,
00106                     Flowable.combineLatest(repository.isWeatherFlowable(row),
00107                             repository.isHornFlowable(row),
00108                             repository.getUnitsFlowable(row),
00109                             (weather, horn, units) -> {
00110                                 DamageCalculator calculator =
      DamageCalculatorUseCase.getDamageCalculator(weather, horn, units);
00111                                 int damage = units.stream()
00112                                         .map(unitEntity -> unitEntity.calculateDamage(calculator))
00113                                         .reduce(0, Integer::sum);
00114                                 return new RowUiState(damage, weather, horn, units.size());
00115                             })
00116                             .distinctUntilChanged()
00117                             .onBackpressureLatest()
00118                             .debounce(10, TimeUnit.MILLISECONDS)
00119                             .subscribeOn(Schedulers.io())
00120                             .observeOn(AndroidSchedulers.mainThread())
00121             );
00122         }
00123
00124         Flowable<MenuUiState> combinedRowStates = Flowable.combineLatest(result.rowUiStates.values(),
      (Object[] rowUiStates) -> {
00125             int damage = 0;
00126             boolean reset = false;
00127             boolean weather = false;
00128             for (Object state : rowUiStates) {
00129                 RowUiState rowUiState = (RowUiState) state;
00130                 damage += rowUiState.getDamage();
00131                 reset |= rowUiState.isHorn() || rowUiState.getUnits() != 0;
00132                 weather |= rowUiState.isWeather();
00133             }
00134             reset |= weather;
00135             return new MenuUiState(damage, reset, weather, false);
00136         }).distinctUntilChanged().onBackpressureLatest();
00137
00138         result.menuUiState = Flowable.combineLatest(combinedRowStates,
00139                         repository.hasNonEpicUnitsFlowable(),
00140                         (menuUiState, hasNonEpicUnits) -> new MenuUiState(menuUiState.getDamage(),
00141                                 menuUiState.isReset(),
00142                                 menuUiState.isWeather(),
00143                                 hasNonEpicUnits))
00144                 .distinctUntilChanged()
00145                 .onBackpressureLatest()
00146                 .debounce(10, TimeUnit.MILLISECONDS)
00147                 .subscribeOn(Schedulers.io())
```

```
00148                 .observeOn(AndroidSchedulers.mainThread());
00149
00150         result.soundManager = soundManager;
00151         return result;
00152     }
00153
00161     private Single<UnitRepository> getRepository() {
00162         return GwentApplication.getRepository(getApplication());
00163     }
00164
00172     @NonNull
00173     public Flowable<RowUiState> getRowUiState(@NonNull RowType row) {
00174         return Objects.requireNonNull(rowUiStates.get(row));
00175     }
00176
00184     @NonNull
00185     public Flowable<MenuUiState> getMenuUiState() {
00186         return menuUiState;
00187     }
00188
00194     @NonNull
00195     public SoundManager getSoundManager() {
00196         return soundManager;
00197     }
00198
00206     @NonNull
00207     public Completable onWeatherViewPressed(@NonNull RowType row) {
00208         return getRepository()
00209                 .flatMap(repository -> repository
00210                         .switchWeather(row)
00211                         .andThen(repository.isWeather(row)))
00212                 .doOnSuccess(weather -> {
00213                     if (weather) {
00214                         soundManager.playWeatherSound(row);
00215                     }
00216                 })
00217                 .ignoreElement();
00218     }
00219
00227     public Completable onHornViewPressed(@NonNull RowType row) {
00228         return getRepository()
00229                 .flatMap(repository -> repository
00230                         .switchHorn(row)
00231                         .andThen(repository.isHorn(row)))
00232                 .doOnSuccess(horn -> {
00233                     if (horn) {
00234                         soundManager.playHornSound();
00235                     }
00236                 })
00237                 .ignoreElement();
00238     }
00239
00249     @NonNull
00250     public Completable onResetButtonPressed(@NonNull Context context) {
00251         return reset(context, ResetDialogUseCase.Trigger.BUTTON_CLICK);
00252     }
00253
00262     @NonNull
00263     public Completable onFactionSwitchReset(@NonNull Context context) {
00264         return reset(context, ResetDialogUseCase.Trigger.FACTION_SWITCH);
00265     }
00266
00275     @NonNull
00276     private Completable reset(@NonNull Context context, @NonNull ResetDialogUseCase.Trigger trigger) {
00277         return ResetDialogUseCase
00278                 .reset(context, trigger, soundManager)
00279                 .doOnSuccess(playSound -> {
00280                     if (playSound) {
00281                         soundManager.playResetSound();
00282                     }
00283                 })
00284                 .ignoreElement();
00285     }
00286
00294     @NonNull
00295     public Completable onWeatherButtonPressed() {
00296         return
      getRepository().flatMapCompletable(UnitRepository::clearWeather).doOnComplete(soundManager::playClearWeatherSound);
00297     }
00298
00309     @NonNull
00310     public Completable onBurnButtonPressed(@NonNull Context context) {
00311         return BurnDialogUseCase
00312                 .burn(context, soundManager)
00313                 .doOnSuccess(playSound -> {
00314                     if (playSound) {
00315                         soundManager.playBurnSound();
```

```
00316                     }
00317                 })
00318                 .ignoreElement();
00319     }
00320 }
```

6.45 ImageViewSwitchAnimator.java

```
00001 package com.peternaggschga.gwent.ui.main;
00002
00003 import android.view.ViewGroup;
00004 import android.view.animation.Animation;
00005 import android.view.animation.AnimationUtils;
00006 import android.widget.ImageView;
00007
00008 import androidx.annotation.DrawableRes;
00009 import androidx.annotation.NonNull;
00010 import androidx.constraintlayout.widget.ConstraintLayout;
00011
00012 import io.reactivex.rxjava3.core.Completable;
00013
00020 public class ImageViewSwitchAnimator {
00026     @NonNull
00027     private static ImageView getOverlayView(@NonNull ImageView template) {
00028         ImageView result = new ImageView(template.getContext());
00029         ViewGroup.LayoutParams oldParams = template.getLayoutParams();
00030         ConstraintLayout.LayoutParams newParams = new ConstraintLayout.LayoutParams(oldParams.width,
      oldParams.height);
00031         newParams.topToTop = template.getId();
00032         newParams.endToEnd = template.getId();
00033         newParams.startToStart = template.getId();
00034         newParams.bottomToBottom = template.getId();
00035         result.setLayoutParams(newParams);
00036         result.setImageDrawable(template.getDrawable());
00037         result.setScaleType(template.getScaleType());
00038         result.setTranslationZ(template.getTranslationZ() + 1);
00039         return result;
00040     }
00041
00052     @NonNull
00053     public static Completable animatedSwitch(@NonNull ImageView view, @DrawableRes int resId) {
00054         return Completable.create(emitter -> {
00055             Animation fadeAnimation = AnimationUtils.loadAnimation(view.getContext(),
      android.R.anim.fade_out);
00056             ImageView foregroundView = getOverlayView(view);
00057             ViewGroup parent = (ViewGroup) view.getParent();
00058
00059             fadeAnimation.setAnimationListener(new Animation.AnimationListener() {
00060                 @Override
00061                 public void onAnimationStart(Animation animation) {
00062                     view.setImageResource(resId);
00063                 }
00064
00065                 @Override
00066                 public void onAnimationEnd(Animation animation) {
00067                     parent.removeView(foregroundView);
00068                     emitter.onComplete();
00069                 }
00070
00071                 @Override
00072                 public void onAnimationRepeat(Animation animation) {
00073                 }
00074             });
00075
00076             parent.addView(foregroundView);
00077             foregroundView.startAnimation(fadeAnimation);
00078         });
00079     }
00080 }
```

6.46 MainActivity.java

```
00001 package com.peternaggschga.gwent.ui.main;
00002
00003 import static com.peternaggschga.gwent.ui.main.FactionSwitchListener.THEME_PREFERENCE_KEY;
00004 import static com.peternaggschga.gwent.ui.main.FactionSwitchListener.THEME_SCOIATAEL;
00005
00006 import android.app.Dialog;
00007 import android.content.Context;
00008 import android.content.Intent;
00009 import android.content.SharedPreferences;
```

```
00010 import android.os.Bundle;
00011 import android.os.Handler;
00012 import android.os.Looper;
00013 import android.view.View;
00014 import android.view.WindowManager;
00015 import android.widget.ImageButton;
00016 import android.widget.ImageView;
00017
00018 import androidx.annotation.NonNull;
00019 import androidx.annotation.Nullable;
00020 import androidx.appcompat.app.AppCompatActivity;
00021 import androidx.constraintlayout.widget.ConstraintLayout;
00022 import androidx.core.view.WindowCompat;
00023 import androidx.core.view.WindowInsetsCompat;
00024 import androidx.core.view.WindowInsetsControllerCompat;
00025 import androidx.preference.PreferenceManager;
00026
00027 import com.peternaggschga.gwent.GwentApplication;
00028 import com.peternaggschga.gwent.R;
00029 import com.peternaggschga.gwent.data.RowType;
00030 import com.peternaggschga.gwent.ui.dialogs.ChangeFactionDialog;
00031 import com.peternaggschga.gwent.ui.dialogs.CoinFlipDialog;
00032 import com.peternaggschga.gwent.ui.dialogs.OverlayDialog;
00033 import com.peternaggschga.gwent.ui.dialogs.addcard.AddCardDialog;
00034 import com.peternaggschga.gwent.ui.dialogs.cards.ShowUnitsDialog;
00035 import com.peternaggschga.gwent.ui.introduction.IntroductionActivity;
00036 import com.peternaggschga.gwent.ui.settings.SettingsActivity;
00037 import com.peternaggschga.gwent.ui.sounds.SoundManager;
00038
00039 import io.reactivex.rxjava3.core.Single;
00040 import io.reactivex.rxjava3.disposables.CompositeDisposable;
00041 import io.reactivex.rxjava3.functions.Function;
00042
00046 public class MainActivity extends AppCompatActivity {
00051     @NonNull
00052     private final CompositeDisposable disposables = new CompositeDisposable();
00053
00061     private SharedPreferences.OnSharedPreferenceChangeListener factionSwitchListener;
00062
00066     private GameBoardViewModel gameBoardViewModel;
00067
00082     @Override
00083     protected void onCreate(@Nullable Bundle savedInstanceState) {
00084         super.onCreate(savedInstanceState);
00085
00086         SharedPreferences preferences = PreferenceManager.getDefaultSharedPreferences(this);
00087
00088         if (preferences.getBoolean(getString(R.string.preference_first_use_key), true)) {
00089             startActivity(new Intent(this, IntroductionActivity.class));
00090         }
00091
00092         FactionSwitchListener.setTheme(this);
00093         setContentView(R.layout.activity_main);
00094
00095         if (gameBoardViewModel == null) {
00096             disposables.add(
00097                     GwentApplication.getRepository(this)
00098                             .map(repository ->
00099                                     GameBoardViewModel.getModel(MainActivity.this,
00100                                             repository,
00101                                             new SoundManager(this)))
00102                             .subscribe(gameBoardViewModel -> {
00103                                 this.gameBoardViewModel = gameBoardViewModel;
00104                                 initializeViewModel();
00105                             })
00106             );
00107         }
00108
00109         if (factionSwitchListener == null) {
00110             factionSwitchListener = FactionSwitchListener.getListener(getWindow());
00111             preferences.registerOnSharedPreferenceChangeListener(factionSwitchListener);
00112         }
00113
00114         findViewById(R.id.factionButton).setOnClickListener(v -> inflateFactionPopup());
00115         findViewById(R.id.coinButton).setOnClickListener(v -> inflateCoinFlipPopup());
00116         findViewById(R.id.settingsButton).setOnClickListener(v -> startActivity(new
      Intent(MainActivity.this, SettingsActivity.class)));
00117     }
00118
00123     @Override
00124     protected void onResume() {
00125         super.onResume();
00126         // set background image according to preferences
00127         int backgroundImageKey = Integer.parseInt(
00128                 PreferenceManager.getDefaultSharedPreferences(this)
00129                         .getString(getString(R.string.preference_key_design),
00130                                 getString(R.string.design_preference_default))
```

```
00131         );
00132         ImageView backgroundImage = findViewById(R.id.backgroundImageView);
00133         if (backgroundImageKey != 0) {
00134             backgroundImage.setVisibility(View.VISIBLE);
00135             backgroundImage.setImageResource(new int[]{
00136                     R.drawable.background_geralt,
00137                     R.drawable.background_ciri,
00138                     R.drawable.background_jaskier,
00139                     R.drawable.background_yennefer,
00140                     R.drawable.background_eredin
00141             }[backgroundImageKey - 1]);
00142         } else {
00143             backgroundImage.setVisibility(View.INVISIBLE);
00144         }
00145     }
00146
00153     @Override
00154     public void onWindowFocusChanged(boolean hasFocus) {
00155         super.onWindowFocusChanged(hasFocus);
00156
00157         if (hasFocus) {
00158             new Handler(Looper.getMainLooper()).postDelayed(() -> {
00159                 if (!hasWindowFocus()) {
00160                     return;
00161                 }
00162
00163                 // hide system UI
00164                 WindowInsetsControllerCompat windowController =
      WindowCompat.getInsetsController(getWindow(), getWindow().getDecorView());
00165                 windowController.hide(WindowInsetsCompat.Type.systemBars());
00166                 windowController.hide(WindowInsetsCompat.Type.tappableElement());
00167
      windowController.setSystemBarsBehavior(WindowInsetsControllerCompat.BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE);
00168
00169                 // keep screen on
00170                 getWindow().addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON);
00171             }, 250);
00172         }
00173     }
00174
00179     @Override
00180     protected void onDestroy() {
00181         super.onDestroy();
00182         disposables.dispose();
00183         disposables.clear();
00184     }
00185
00191     private void initializeViewModel() {
00192         int[] rowIds = {R.id.firstRow, R.id.secondRow, R.id.thirdRow};
00193         for (int rowId = 0; rowId < rowIds.length; rowId++) {
00194             RowType row = RowType.values()[rowId];
00195             ConstraintLayout rowLayout = findViewById(rowIds[rowId]);
00196
00197             ImageView weather = rowLayout.findViewById(R.id.weatherView);
00198             ImageView horn = rowLayout.findViewById(R.id.hornView);
00199             ConstraintLayout cards = rowLayout.findViewById(R.id.cardView);
00200
00201             weather.setOnClickListener(v ->
00202                     disposables.add(gameBoardViewModel.onWeatherViewPressed(row).subscribe()));
00203             horn.setOnClickListener(v ->
00204                     disposables.add(gameBoardViewModel.onHornViewPressed(row).subscribe()));
00205
00206             cards.setOnClickListener(v -> disposables.add(
00207                     GwentApplication.getRepository(this)
00208                             .flatMap(repository -> repository.countUnits(row))
00209                             .map(count -> count == 0)
00210                             .flatMap((Function<Boolean, Single<? extends OverlayDialog>>) rowEmpty ->
00211                                     rowEmpty
00212                                             ? Single.just(new AddCardDialog(MainActivity.this, row,
      gameBoardViewModel.getSoundManager()))
00213                                             : ShowUnitsDialog.getDialog(MainActivity.this, row,
      gameBoardViewModel.getSoundManager()))
00214                             .subscribe(Dialog::show)
00215             ));
00216
00217             final RowUiStateObserver observer = RowUiStateObserver.getObserver(row,
00218                     rowLayout.findViewById(R.id.pointView),
00219                     weather,
00220                     horn,
00221                     rowLayout.findViewById(R.id.cardCountView));
00222             disposables.add(gameBoardViewModel.getRowUiState(row).subscribe(observer));
00223         }
00224
00225         ImageButton reset = findViewById(R.id.resetButton);
00226         ImageButton weather = findViewById(R.id.weatherButton);
00227         ImageButton burn = findViewById(R.id.burnButton);
00228
```

```
00229         final MenuUiStateObserver observer = new
      MenuUiStateObserver(findViewById(R.id.overallPointView),
00230                 reset,
00231                 weather,
00232                 burn);
00233         disposables.add(gameBoardViewModel.getMenuUiState().subscribe(observer));
00234
00235         reset.setOnClickListener(v ->
00236                 disposables.add(gameBoardViewModel.onResetButtonPressed(this).subscribe()));
00237         weather.setOnClickListener(v ->
00238                 disposables.add(gameBoardViewModel.onWeatherButtonPressed().subscribe()));
00239         burn.setOnClickListener(v ->
00240
      disposables.add(gameBoardViewModel.onBurnButtonPressed(MainActivity.this).subscribe()));
00241     }
00242
00250     private void inflateFactionPopup() {
00251         new ChangeFactionDialog(this, theme -> {
00252             SharedPreferences preferences = PreferenceManager.getDefaultSharedPreferences(this);
00253
00254             if (preferences.getInt(THEME_PREFERENCE_KEY, THEME_SCOIATAEL) == theme) {
00255                 return;
00256             }
00257
00258             boolean resetOnFactionSwitch = preferences.getBoolean(
00259                     getString(R.string.preference_key_faction_reset),
00260                     getResources().getBoolean(R.bool.faction_reset_preference_default)
00261             );
00262             if (resetOnFactionSwitch) {
00263                 disposables.add(gameBoardViewModel.onFactionSwitchReset(this).subscribe());
00264             }
00265             preferences.edit()
00266                     .putInt(THEME_PREFERENCE_KEY, theme)
00267                     .apply();
00268         }).show();
00269     }
00270
00274     private void inflateCoinFlipPopup() {
00275         new CoinFlipDialog(this).show();
00276         gameBoardViewModel.getSoundManager().playCoinSound();
00277     }
00278 }
```

6.47 MenuUiState.java

```
00001 package com.peternaggschga.gwent.ui.main;
00002
00003
00004
00005 import androidx.annotation.IntRange;
00006
00010 public class MenuUiState {
00014     @IntRange(from = 0)
00015     private final int damage;
00016
00020     private final boolean reset;
00021
00025     private final boolean weather;
00026
00030     private final boolean burn;
00031
00041     public MenuUiState(@IntRange(from = 0) int damage, boolean reset, boolean weather, boolean burn) {
00042         if (damage < 0) {
00043             throw new IllegalArgumentException("Damage must be greater or equal to 0 but is " + damage
      + ".");
00044         }
00045         this.damage = damage;
00046         this.reset = reset;
00047         this.weather = weather;
00048         this.burn = burn;
00049     }
00050
00056     @IntRange(from = 0)
00057     public int getDamage() {
00058         return damage;
00059     }
00060
00066     public boolean isReset() {
00067         return reset;
00068     }
00069
00075     public boolean isWeather() {
00076         return weather;
00077     }
```

```
00078
00084     public boolean isBurn() {
00085         return burn;
00086     }
00087
00094     @Override
00095     public boolean equals(Object o) {
00096         if (this == o) return true;
00097         if (!(o instanceof MenuUiState)) return false;
00098         MenuUiState that = (MenuUiState) o;
00099         return damage == that.damage && reset == that.reset && weather == that.weather && burn ==
      that.burn;
00100     }
00101 }
```

6.48 MenuUiStateObserver.java

```
00001 package com.peternaggschga.gwent.ui.main;
00002
00003 import android.widget.ImageButton;
00004 import android.widget.TextView;
00005
00006 import androidx.annotation.NonNull;
00007
00008 import com.peternaggschga.gwent.R;
00009
00010 import io.reactivex.rxjava3.functions.Consumer;
00011
00018 public class MenuUiStateObserver implements Consumer<MenuUiState> {
00022     @NonNull
00023     private final TextView damageView;
00024
00030     @NonNull
00031     private final ImageButton resetButton;
00032
00037     @NonNull
00038     private final ImageButton weatherButton;
00039
00044     @NonNull
00045     private final ImageButton burnButton;
00046
00055     public MenuUiStateObserver(@NonNull TextView damageView, @NonNull ImageButton resetButton,
00056                                @NonNull ImageButton weatherButton, @NonNull ImageButton burnButton) {
00057         this.damageView = damageView;
00058         this.resetButton = resetButton;
00059         this.weatherButton = weatherButton;
00060         this.burnButton = burnButton;
00061     }
00062
00071     @Override
00072     public void accept(@NonNull MenuUiState menuUiState) {
00073         damageView.setText(String.valueOf(menuUiState.getDamage()));
00074         resetButton.setClickable(menuUiState.isReset());
00075         ImageViewSwitchAnimator.animatedSwitch(resetButton, menuUiState.isReset() ?
      R.drawable.icon_reset : R.drawable.icon_reset_grey)
00076                 .subscribe();
00077         weatherButton.setClickable(menuUiState.isWeather());
00078         ImageViewSwitchAnimator.animatedSwitch(weatherButton, menuUiState.isWeather() ?
      R.drawable.icon_weather : R.drawable.icon_weather_grey)
00079                 .subscribe();
00080         burnButton.setClickable(menuUiState.isBurn());
00081         ImageViewSwitchAnimator.animatedSwitch(burnButton, menuUiState.isBurn() ? R.drawable.icon_burn
      : R.drawable.icon_burn_grey)
00082                 .subscribe();
00083     }
00084 }
```

6.49 RowUiState.java

```
00001 package com.peternaggschga.gwent.ui.main;
00002
00003
00004
00005 import androidx.annotation.IntRange;
00006
00010 public class RowUiState {
00014     @IntRange(from = 0)
00015     private final int damage;
00016
00020     private final boolean weather;
00021
```

```
00025     private final boolean horn;
00026
00030     @IntRange(from = 0)
00031     private final int units;
00032
00041     public RowUiState(@IntRange(from = 0) int damage, boolean weather, boolean horn, @IntRange(from =
      0) int units) {
00042         if (damage < 0) {
00043             throw new IllegalArgumentException("Damage must be greater or equal to 0 but is " + damage
      + ".");
00044         }
00045         if (units < 0) {
00046             throw new IllegalArgumentException("Units must be greater or equal to 0 but is " + units +
      ".");
00047         }
00048         this.damage = damage;
00049         this.weather = weather;
00050         this.horn = horn;
00051         this.units = units;
00052     }
00053
00058     @IntRange(from = 0)
00059     public int getDamage() {
00060         return damage;
00061     }
00062
00067     public boolean isWeather() {
00068         return weather;
00069     }
00070
00075     public boolean isHorn() {
00076         return horn;
00077     }
00078
00083     @IntRange(from = 0)
00084     public int getUnits() {
00085         return units;
00086     }
00087
00094     @Override
00095     public boolean equals(Object o) {
00096         if (this == o) return true;
00097         if (!(o instanceof RowUiState)) return false;
00098         RowUiState state = (RowUiState) o;
00099         return damage == state.damage && weather == state.weather && horn == state.horn && units ==
      state.units;
00100     }
00101 }
```

6.50 RowUiStateObserver.java

```
00001 package com.peternaggschga.gwent.ui.main;
00002
00003 import android.widget.ImageView;
00004 import android.widget.TextView;
00005
00006 import androidx.annotation.DrawableRes;
00007 import androidx.annotation.NonNull;
00008
00009 import com.peternaggschga.gwent.R;
00010 import com.peternaggschga.gwent.data.RowType;
00011
00012 import org.jetbrains.annotations.Contract;
00013
00014 import io.reactivex.rxjava3.functions.Consumer;
00015
00022 public class RowUiStateObserver implements Consumer<RowUiState> {
00027     @NonNull
00028     private final TextView damageView;
00029
00036     @NonNull
00037     private final ImageView weatherView;
00038
00045     @DrawableRes
00046     private final int weatherResource;
00047
00054     @NonNull
00055     private final ImageView hornView;
00056
00060     @NonNull
00061     private final TextView unitView;
00062
00074     private RowUiStateObserver(@NonNull TextView damageView, @NonNull ImageView weatherView,
00075                                @DrawableRes int weatherResource, @NonNull ImageView hornView,
```

```
00076                                @NonNull TextView unitView) {
00077         this.damageView = damageView;
00078         this.weatherView = weatherView;
00079         this.weatherResource = weatherResource;
00080         this.hornView = hornView;
00081         this.unitView = unitView;
00082     }
00083 
00097     @NonNull
00098     @Contract("_, _, _, _, _ -> new")
00099     public static RowUiStateObserver getObserver(@NonNull RowType row, @NonNull TextView damageView,
00100                                                  @NonNull ImageView weatherView, @NonNull ImageView
      hornView,
00101                                                  @NonNull TextView unitView) {
00102         int weatherResource;
00103         switch (row) {
00104             case MELEE:
00105                 weatherResource = R.drawable.frost_weather;
00106                 break;
00107             case RANGE:
00108                 weatherResource = R.drawable.fog_weather;
00109                 break;
00110             case SIEGE:
00111                 weatherResource = R.drawable.rain_weather;
00112                 break;
00113             default:
00114                 weatherResource = R.drawable.good_weather;
00115         }
00116         return new RowUiStateObserver(damageView, weatherView, weatherResource, hornView, unitView);
00117     }
00118 
00126     @Override
00127     public void accept(@NonNull RowUiState rowUiState) {
00128         damageView.setText(String.valueOf(rowUiState.getDamage()));
00129         ImageViewSwitchAnimator.animatedSwitch(weatherView, rowUiState.isWeather() ? weatherResource :
      R.drawable.good_weather)
00130                 .subscribe();
00131         ImageViewSwitchAnimator.animatedSwitch(hornView, rowUiState.isHorn() ? R.drawable.horn :
      R.drawable.horn_grey)
00132                 .subscribe();
00133         unitView.setText(String.valueOf(rowUiState.getUnits()));
00134     }
00135 }
```

6.51 RuleActivity.java

```
00001 package com.peternaggschga.gwent.ui.settings;
00002 
00003 import android.os.Bundle;
00004 import android.text.Html;
00005 import android.view.MenuItem;
00006 import android.widget.TextView;
00007 
00008 import androidx.annotation.NonNull;
00009 import androidx.appcompat.app.ActionBar;
00010 import androidx.appcompat.app.AppCompatActivity;
00011 import androidx.core.content.IntentCompat;
00012 
00013 import com.peternaggschga.gwent.R;
00014 
00015 import java.util.Objects;
00016 
00023 public class RuleActivity extends AppCompatActivity {
00028     public static final String INTENT_EXTRA_TAG = "rule_section";
00029 
00037     @Override
00038     public void onCreate(Bundle savedInstanceState) {
00039         super.onCreate(savedInstanceState);
00040 
00041         setContentView(R.layout.activity_rules);
00042 
00043         RuleSection section = Objects.requireNonNull(
00044                 IntentCompat.getSerializableExtra(getIntent(), INTENT_EXTRA_TAG, RuleSection.class));
00045 
00046         setSupportActionBar(findViewById(R.id.rulesToolbar));
00047 
00048         TextView textView = findViewById(R.id.rulesTextView);
00049 
00050         ActionBar actionBar = Objects.requireNonNull(getSupportActionBar());
00051         actionBar.setDisplayHomeAsUpEnabled(true);
00052 
00053         switch (section) {
00054             case GENERAL:
00055                 textView.setText(Html.fromHtml(getString(R.string.rules_general_text),
```

```
00056                         Html.FROM_HTML_MODE_LEGACY));
00057                 actionBar.setTitle(R.string.preference_rules_general_title);
00058                 break;
00059             case COURSE:
00060                 textView.setText(Html.fromHtml(getString(R.string.rules_course_text),
00061                         Html.FROM_HTML_MODE_LEGACY));
00062                 actionBar.setTitle(R.string.preference_rules_course_title);
00063                 break;
00064             case FACTIONS:
00065                 textView.setText(Html.fromHtml(getString(R.string.rules_factions_text),
00066                         Html.FROM_HTML_MODE_LEGACY));
00067                 actionBar.setTitle(R.string.preference_rules_factions_title);
00068                 break;
00069             case COMMANDER:
00070                 textView.setText(Html.fromHtml(getString(R.string.rules_commander_text),
00071                         Html.FROM_HTML_MODE_LEGACY));
00072                 actionBar.setTitle(R.string.preference_rules_commander_title);
00073                 break;
00074             case CARDS:
00075                 textView.setText(Html.fromHtml(getString(R.string.rules_cards_text),
00076                         Html.FROM_HTML_MODE_LEGACY));
00077                 actionBar.setTitle(R.string.preference_rules_cards_title);
00078                 break;
00079             case CARD_ABILITIES:
00080                 textView.setText(Html.fromHtml(getString(R.string.rules_card_abilities_text),
00081                         Html.FROM_HTML_MODE_LEGACY));
00082                 actionBar.setTitle(R.string.preference_rules_card_abilities_title);
00083                 break;
00084             case SPECIAL_CARDS:
00085                 textView.setText(Html.fromHtml(getString(R.string.rules_special_cards_text),
00086                         Html.FROM_HTML_MODE_LEGACY));
00087                 actionBar.setTitle(R.string.preference_rules_special_cards_title);
00088         }
00089     }
00090
00098     @Override
00099     public boolean onOptionsItemSelected(@NonNull MenuItem item) {
00100         if (item.getItemId() == android.R.id.home) {
00101             getOnBackPressedDispatcher().onBackPressed();
00102             return true;
00103         }
00104         return super.onOptionsItemSelected(item);
00105     }
00106 }
```

6.52 RuleSection.java

```
00001 package com.peternaggschga.gwent.ui.settings;
00002
00006 enum RuleSection {
00010     GENERAL,
00011
00015     COURSE,
00016
00020     FACTIONS,
00021
00025     COMMANDER,
00026
00030     CARDS,
00031
00035     CARD_ABILITIES,
00036
00040     SPECIAL_CARDS
00041 }
```

6.53 SettingsActivity.java

```
00001 package com.peternaggschga.gwent.ui.settings;
00002
00003 import android.content.SharedPreferences;
00004 import android.os.Bundle;
00005 import android.view.MenuItem;
00006
00007 import androidx.annotation.NonNull;
00008 import androidx.annotation.Nullable;
00009 import androidx.appcompat.app.ActionBar;
00010 import androidx.appcompat.app.AppCompatActivity;
00011 import androidx.fragment.app.Fragment;
00012 import androidx.preference.Preference;
00013 import androidx.preference.PreferenceFragmentCompat;
00014
```

```
00015 import com.peternaggschga.gwent.R;
00016 import com.peternaggschga.gwent.ui.main.FactionSwitchListener;
00017
00018 import java.util.Objects;
00019
00024 public class SettingsActivity extends AppCompatActivity implements
      PreferenceFragmentCompat.OnPreferenceStartFragmentCallback {
00029     private static final String CURRENT_FRAGMENT_KEY = "currentFragment";
00030
00037     @Override
00038     protected void onCreate(@Nullable Bundle savedInstanceState) {
00039         super.onCreate(savedInstanceState);
00040         FactionSwitchListener.setTheme(this);
00041
00042         setContentView(R.layout.activity_settings);
00043
00044         setSupportActionBar(findViewById(R.id.settingsToolbar));
00045         ActionBar actionBar = getSupportActionBar();
00046         if (actionBar != null) {
00047             actionBar.setDisplayHomeAsUpEnabled(true);
00048         }
00049
00050         Fragment currentFragment = null;
00051         if (savedInstanceState != null) {
00052             currentFragment = getSupportFragmentManager().getFragment(savedInstanceState,
      CURRENT_FRAGMENT_KEY);
00053         }
00054         currentFragment = currentFragment == null ? new SettingsHeaderFragment() : currentFragment;
00055         getSupportFragmentManager().beginTransaction()
00056                 .replace(R.id.settingsFrameLayout, currentFragment)
00057                 .commit();
00058     }
00059
00065     @Override
00066     protected void onSaveInstanceState(@NonNull Bundle outState) {
00067         super.onSaveInstanceState(outState);
00068
00069         getSupportFragmentManager().getFragments()
00070                 .stream()
00071                 .filter(Fragment::isVisible)
00072                 .findAny()
00073                 .ifPresent(fragment ->
00074                         getSupportFragmentManager().putFragment(outState, CURRENT_FRAGMENT_KEY,
      fragment));
00075     }
00076
00084     @Override
00085     public boolean onOptionsItemSelected(@NonNull MenuItem item) {
00086         if (item.getItemId() == android.R.id.home) {
00087             getOnBackPressedDispatcher().onBackPressed();
00088             return true;
00089         }
00090         return super.onOptionsItemSelected(item);
00091     }
00092
00101     @Override
00102     public boolean onPreferenceStartFragment(@NonNull PreferenceFragmentCompat caller, @NonNull
      Preference pref) {
00103         // Instantiate the new Fragment
00104         final Bundle args = pref.getExtras();
00105
00106         Fragment fragment = getSupportFragmentManager()
00107                 .getFragmentFactory()
00108                 .instantiate(getClassLoader(), Objects.requireNonNull(pref.getFragment()));
00109         fragment.setArguments(args);
00110
00111         // Replace the existing Fragment with the new Fragment
00112         getSupportFragmentManager().beginTransaction()
00113                 .replace(R.id.settingsFrameLayout, fragment)
00114                 .addToBackStack(null)
00115                 .commit();
00116         setTitle(pref.getTitle());
00117         return true;
00118     }
00119 }
```

6.54 SettingsHeaderFragment.java

```
00001 package com.peternaggschga.gwent.ui.settings;
00002
00003 import android.content.Intent;
00004 import android.os.Bundle;
00005
00006 import androidx.preference.Preference;
```

```
00007 import androidx.preference.PreferenceFragmentCompat;
00008
00009 import com.peternaggschga.gwent.R;
00010 import com.peternaggschga.gwent.ui.introduction.IntroductionActivity;
00011
00012 import java.util.Objects;
00013
00018 public class SettingsHeaderFragment extends PreferenceFragmentCompat {
00030     @Override
00031     public void onCreatePreferences(Bundle savedInstanceState, String rootKey) {
00032         setPreferencesFromResource(R.xml.header_preferences, rootKey);
00033
00034         Preference introductionPreference =
      Objects.requireNonNull(findPreference(getString(R.string.preference_key_introduction)));
00035         introductionPreference.setOnPreferenceClickListener(preference -> {
00036             startActivity(new Intent(getContext(), IntroductionActivity.class));
00037             return true;
00038         });
00039     }
00040 }
```

6.55 SettingsRuleFragment.java

```
00001 package com.peternaggschga.gwent.ui.settings;
00002
00003 import static com.peternaggschga.gwent.ui.settings.RuleSection.CARDS;
00004 import static com.peternaggschga.gwent.ui.settings.RuleSection.CARD_ABILITIES;
00005 import static com.peternaggschga.gwent.ui.settings.RuleSection.COMMANDER;
00006 import static com.peternaggschga.gwent.ui.settings.RuleSection.COURSE;
00007 import static com.peternaggschga.gwent.ui.settings.RuleSection.FACTIONS;
00008 import static com.peternaggschga.gwent.ui.settings.RuleSection.GENERAL;
00009 import static com.peternaggschga.gwent.ui.settings.RuleSection.SPECIAL_CARDS;
00010
00011 import android.content.Intent;
00012 import android.os.Bundle;
00013
00014 import androidx.annotation.Keep;
00015 import androidx.annotation.NonNull;
00016 import androidx.preference.Preference;
00017 import androidx.preference.PreferenceFragmentCompat;
00018
00019 import com.peternaggschga.gwent.R;
00020
00021 import org.jetbrains.annotations.Contract;
00022
00023 import java.io.Serializable;
00024 import java.util.Objects;
00025
00030 @Keep
00031 public class SettingsRuleFragment extends PreferenceFragmentCompat {
00041     @NonNull
00042     @Contract(pure = true)
00043     private Preference.OnPreferenceClickListener getSectionClickListener(@NonNull RuleSection section)
      {
00044         return preference -> {
00045             startActivity(
00046                     new Intent(getContext(), RuleActivity.class)
00047                             .putExtra(RuleActivity.INTENT_EXTRA_TAG, section)
00048             );
00049             return true;
00050         };
00051     }
00052
00065     @Override
00066     public void onCreatePreferences(Bundle savedInstanceState, String rootKey) {
00067         setPreferencesFromResource(R.xml.rule_preferences, rootKey);
00068
00069         Preference rulesGeneral =
      Objects.requireNonNull(findPreference(getString(R.string.preference_rules_general_key)));
00070         rulesGeneral.setOnPreferenceClickListener(getSectionClickListener(GENERAL));
00071
00072         Preference rulesCourse =
      Objects.requireNonNull(findPreference(getString(R.string.preference_rules_course_key)));
00073         rulesCourse.setOnPreferenceClickListener(getSectionClickListener(COURSE));
00074
00075         Preference ruleFactions =
      Objects.requireNonNull(findPreference(getString(R.string.preference_rules_factions_key)));
00076         ruleFactions.setOnPreferenceClickListener(getSectionClickListener(FACTIONS));
00077
00078         Preference rulesCommander =
      Objects.requireNonNull(findPreference(getString(R.string.preference_rules_commander_key)));
00079         rulesCommander.setOnPreferenceClickListener(getSectionClickListener(COMMANDER));
00080
00081         Preference rulesCards =
```

```
    Objects.requireNonNull(findPreference(getString(R.string.preference_rules_cards_key)));
00082         rulesCards.setOnPreferenceClickListener(getSectionClickListener(CARDS));
00083
00084         Preference ruleCardAbilities =
    Objects.requireNonNull(findPreference(getString(R.string.preference_rules_card_abilities_key)));
00085         ruleCardAbilities.setOnPreferenceClickListener(getSectionClickListener(CARD_ABILITIES));
00086
00087         Preference rulesSpecialCards =
    Objects.requireNonNull(findPreference(getString(R.string.preference_rules_special_cards_key)));
00088         rulesSpecialCards.setOnPreferenceClickListener(getSectionClickListener(SPECIAL_CARDS));
00089     }
00090 }
```

6.56 SettingsSoundFragment.java

```
00001 package com.peternaggschga.gwent.ui.settings;
00002
00003 import android.os.Bundle;
00004
00005 import androidx.annotation.Keep;
00006 import androidx.preference.Preference;
00007 import androidx.preference.PreferenceFragmentCompat;
00008
00009 import com.peternaggschga.gwent.R;
00010
00015 @Keep
00016 public class SettingsSoundFragment extends PreferenceFragmentCompat {
00026     @Override
00027     public void onCreatePreferences(Bundle savedInstanceState, String rootKey) {
00028         setPreferencesFromResource(R.xml.sound_preferences, rootKey);
00029     }
00030 }
```

6.57 Sound.java

```
00001 package com.peternaggschga.gwent.ui.sounds;
00002
00003 import android.content.Context;
00004 import android.content.SharedPreferences;
00005 import android.media.SoundPool;
00006
00007 import androidx.annotation.NonNull;
00008 import androidx.annotation.RawRes;
00009 import androidx.annotation.StringRes;
00010 import androidx.preference.PreferenceManager;
00011
00012 import com.peternaggschga.gwent.R;
00013
00014 import org.jetbrains.annotations.Contract;
00015
00020 class Sound {
00026     @NonNull
00027     private final String preferenceKey;
00028
00032     private final int soundId;
00033
00037     private boolean activated;
00038
00048     private Sound(@NonNull String preferenceKey, int soundId, boolean activated) {
00049         this.preferenceKey = preferenceKey;
00050         this.soundId = soundId;
00051         this.activated = activated;
00052     }
00053
00067     @NonNull
00068     @Contract("_, _, _, _ -> new")
00069     static Sound createSound(@NonNull Context context, @StringRes int preferenceRes,
00070                              @NonNull SoundPool soundPool, @RawRes int resId) {
00071         String preferenceKey = context.getString(preferenceRes);
00072         int soundId = soundPool.load(context, resId, 1);
00073         SharedPreferences pref = PreferenceManager.getDefaultSharedPreferences(context);
00074         boolean activated = context.getResources().getBoolean(R.bool.sound_preference_default);
00075         activated = pref.getBoolean(context.getString(R.string.preference_key_sounds_all), activated)
00076                 && pref.getBoolean(preferenceKey, activated);
00077         return new Sound(preferenceKey, soundId, activated);
00078     }
00079
00085     @NonNull
00086     String getPreferenceKey() {
00087         return preferenceKey;
00088     }
```

```
00089
00095     int getSoundId() {
00096         return soundId;
00097     }
00098
00104     boolean isActivated() {
00105         return activated;
00106     }
00107
00114     void setActivated(boolean activated) {
00115         this.activated = activated;
00116     }
00117
00124     void setActivated(@NonNull SharedPreferences sharedPreferences) {
00125         setActivated(sharedPreferences.getBoolean(preferenceKey, activated));
00126     }
00127 }
```

6.58 SoundManager.java

```
00001 package com.peternaggschga.gwent.ui.sounds;
00002
00003 import android.content.Context;
00004 import android.content.SharedPreferences;
00005 import android.media.AudioAttributes;
00006 import android.media.SoundPool;
00007 import android.os.Build;
00008
00009 import androidx.annotation.IntRange;
00010 import androidx.annotation.NonNull;
00011 import androidx.preference.PreferenceManager;
00012
00013 import com.peternaggschga.gwent.R;
00014 import com.peternaggschga.gwent.data.RowType;
00015
00016 import java.util.Arrays;
00017
00023 public class SoundManager {
00030     public static final int SOUND_WEATHER_GOOD = 0;
00031
00038     public static final int SOUND_WEATHER_FROST = 1;
00039
00046     public static final int SOUND_WEATHER_FOG = 2;
00047
00054     public static final int SOUND_WEATHER_RAIN = 3;
00055
00062     public static final int SOUND_HORN = 4;
00063
00070     public static final int SOUND_CARDS_EPIC = 5;
00071
00078     public static final int SOUND_CARDS_MELEE = 6;
00079
00086     public static final int SOUND_CARDS_RANGE = 7;
00087
00094     public static final int SOUND_CARDS_SIEGE = 8;
00095
00102     public static final int SOUND_RESET = 9;
00103
00110     public static final int SOUND_BURN = 10;
00111
00118     public static final int SOUND_COIN = 11;
00119
00127     @NonNull
00128     private final Sound[] sounds = new Sound[SOUND_COIN + 1];
00129
00133     @NonNull
00134     private final SoundPool soundPool;
00135
00143     @NonNull
00144     @SuppressWarnings("FieldCanBeLocal")
00145     private final SharedPreferences.OnSharedPreferenceChangeListener changeListener;
00146
00156     public SoundManager(@NonNull Context context) {
00157         AudioAttributes attributes = new AudioAttributes.Builder()
00158                 .setUsage(AudioAttributes.USAGE_GAME)
00159                 .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
00160                 .build();
00161
00162         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
00163             soundPool = new
      SoundPool.Builder().setMaxStreams(5).setAudioAttributes(attributes).setContext(context).build();
00164         } else {
00165             soundPool = new
      SoundPool.Builder().setMaxStreams(5).setAudioAttributes(attributes).build();
```

```
00166         }
00167
00168         sounds[SOUND_WEATHER_GOOD] = Sound.createSound(context,
      R.string.preference_key_sounds_weather, soundPool, R.raw.weather_good);
00169         sounds[SOUND_WEATHER_FROST] = Sound.createSound(context,
      R.string.preference_key_sounds_weather, soundPool, R.raw.weather_frost);
00170         sounds[SOUND_WEATHER_FOG] = Sound.createSound(context, R.string.preference_key_sounds_weather,
      soundPool, R.raw.weather_fog);
00171         sounds[SOUND_WEATHER_RAIN] = Sound.createSound(context,
      R.string.preference_key_sounds_weather, soundPool, R.raw.weather_rain);
00172         sounds[SOUND_HORN] = Sound.createSound(context, R.string.preference_key_sounds_horn,
      soundPool, R.raw.horn);
00173         sounds[SOUND_CARDS_EPIC] = Sound.createSound(context, R.string.preference_key_sounds_cards,
      soundPool, R.raw.card_epic);
00174         sounds[SOUND_CARDS_MELEE] = Sound.createSound(context, R.string.preference_key_sounds_cards,
      soundPool, R.raw.card_melee);
00175         sounds[SOUND_CARDS_RANGE] = Sound.createSound(context, R.string.preference_key_sounds_cards,
      soundPool, R.raw.card_range);
00176         sounds[SOUND_CARDS_SIEGE] = Sound.createSound(context, R.string.preference_key_sounds_cards,
      soundPool, R.raw.card_siege);
00177         sounds[SOUND_RESET] = Sound.createSound(context, R.string.preference_key_sounds_reset,
      soundPool, R.raw.reset);
00178         sounds[SOUND_BURN] = Sound.createSound(context, R.string.preference_key_sounds_burn,
      soundPool, R.raw.burn);
00179         sounds[SOUND_COIN] = Sound.createSound(context, R.string.preference_key_sounds_coin,
      soundPool, R.raw.coin);
00180
00181         SharedPreferences pref = PreferenceManager.getDefaultSharedPreferences(context);
00182         changeListener = (sharedPreferences, key) -> {
00183             if (context.getResources().getString(R.string.preference_key_sounds_all).equals(key)) {
00184                 if (sharedPreferences.getBoolean(key,
00185                         context.getResources().getBoolean(R.bool.sound_preference_default))) {
00186                     Arrays.stream(sounds).forEach(sound -> sound.setActivated(sharedPreferences));
00187                 } else {
00188                     Arrays.stream(sounds).forEach(sound -> sound.setActivated(false));
00189                 }
00190             } else {
00191                 Arrays.stream(sounds)
00192                         .filter(sound -> sound.getPreferenceKey().equals(key))
00193                         .forEach(sound -> sound.setActivated(sharedPreferences.getBoolean(key,
00194                                 context.getResources().getBoolean(R.bool.sound_preference_default))));
00195             }
00196         };
00197         pref.registerOnSharedPreferenceChangeListener(changeListener);
00198     }
00199
00213     public void playSound(@IntRange(from = SOUND_WEATHER_GOOD, to = SOUND_COIN) int soundId) {
00214         if (sounds[soundId].isActivated()) {
00215             soundPool.play(sounds[soundId].getSoundId(), 1, 1, 0, 0, 1);
00216         }
00217     }
00218
00225     public void playClearWeatherSound() {
00226         playSound(SOUND_WEATHER_GOOD);
00227     }
00228
00236     public void playWeatherSound(@NonNull RowType row) {
00237         switch (row) {
00238             case MELEE:
00239                 playSound(SOUND_WEATHER_FROST);
00240                 break;
00241             case RANGE:
00242                 playSound(SOUND_WEATHER_FOG);
00243                 break;
00244             case SIEGE:
00245                 playSound(SOUND_WEATHER_RAIN);
00246         }
00247     }
00248
00255     public void playHornSound() {
00256         playSound(SOUND_HORN);
00257     }
00258
00268     public void playCardAddSound(@NonNull RowType row, boolean epic) {
00269         if (epic) {
00270             playSound(SOUND_CARDS_EPIC);
00271             return;
00272         }
00273         switch (row) {
00274             case MELEE:
00275                 playSound(SOUND_CARDS_MELEE);
00276                 break;
00277             case RANGE:
00278                 playSound(SOUND_CARDS_RANGE);
00279                 break;
00280             case SIEGE:
00281                 playSound(SOUND_CARDS_SIEGE);
```

```
00282         }
00283     }
00284
00291     public void playCardRemovedSound() {
00292         playSound(SOUND_RESET);
00293     }
00294
00301     public void playResetSound() {
00302         playSound(SOUND_RESET);
00303     }
00304
00311     public void playBurnSound() {
00312         playSound(SOUND_BURN);
00313     }
00314
00320     public void playCoinSound() {
00321         playSound(SOUND_COIN);
00322     }
00323 }
```

Index

Ability, 11
    BINDING, 11
    HORN, 11
    MORAL_BOOST, 12
    NONE, 12
    REVENGE, 12
ability
    UnitEntity, 192
abilityImageId
    CardUiState, 36
abilityPicker
    CardNumberPickerAdapter, 30
abilityView
    CardListAdapter.CardViewHolder, 43
accept
    MenuUiStateObserver, 103
    RowUiStateObserver, 139
activated
    Sound, 162
adapteeBuilder
    ResetAlertDialogBuilderAdapter, 117
    RevengeAlertDialogBuilderAdapter, 125
AddCardDialog, 12
    AddCardDialog, 14
    disposables, 15
    onCreate, 14
    pickerAdapter, 15
    row, 15
    soundManager, 15
addSelectedUnits
    CardNumberPickerAdapter, 30
animatedSwitch
    ImageViewSwitchAnimator, 86
AppDatabase, 16
    rows, 17
    units, 17
ARG_SECTION_NUMBER
    IntroductionFragment, 94
AVENGER_ABILITY
    RevengeAlertDialogBuilderAdapter, 125
AVENGER_DAMAGE
    RevengeAlertDialogBuilderAdapter, 126
AVENGER_EPIC
    RevengeAlertDialogBuilderAdapter, 126
AVENGER_ROW
    RevengeAlertDialogBuilderAdapter, 126
AVENGER_SQUAD
    RevengeAlertDialogBuilderAdapter, 126

BACKGROUND
    OverlayDialog, 111
ballViews
    FactionSwitchListener, 70
BINDING
    Ability, 11
bindingView
    CardListAdapter.CardViewHolder, 43
BondDamageCalculatorDecorator, 17
    BondDamageCalculatorDecorator, 19
    calculateDamage, 19
    idToSquadSize, 20
    isBuffed, 20
BUFFED
    DamageCalculator.Color, 50
burn
    BurnDialogUseCase, 21, 22
    MenuUiState, 101
burnButton
    MenuUiStateObserver, 104
BurnDialogUseCase, 20
    burn, 21, 22
    getBurnUnits, 22
BUTTON_CLICK
    ResetDialogUseCase.Trigger, 180

calculateDamage
    BondDamageCalculatorDecorator, 19
    DamageCalculator, 52
    HornDamageCalculatorDecorator, 84
    MoralDamageCalculatorDecorator, 107
    UnitEntity, 187
    WeatherDamageCalculator, 212
calculator
    DamageCalculatorBuilder, 57
callback
    ChangeFactionDialog, 47
    IntroductionActivity, 91
CARD_ABILITIES
    RuleSection, 143
CardListAdapter, 25
    CardListAdapter, 26
    getItemId, 26
    onBindViewHolder, 27
    onCopy, 27
    onCreateViewHolder, 27
    onRemove, 28
cardListAdapter
    ShowUnitsDialog, 158
CardListAdapter.CardViewHolder, 40
    abilityView, 43

bindingView, 43
CardViewHolder, 42
damageView, 44
getAbilityView, 42
getBindingView, 42
getDamageView, 43
itemId, 44
setItemId, 43
CardNumberPickerAdapter, 28
abilityPicker, 30
addSelectedUnits, 30
CardNumberPickerAdapter, 29
damagePicker, 31
EPIC_UNIT_ABILITIES, 31
epicPicker, 31
getDelayedOnValueChangeListener, 30
numberPicker, 31
squadManager, 31
squadPicker, 31
CARDS
RuleSection, 143
CardUiState, 32
abilityImageId, 36
CardUiState, 33
damageBackgroundImageId, 36
damageString, 36
damageTextColor, 36
DIFF_CALLBACK, 37
equals, 34
getAbilityImageId, 34
getDamageBackgroundImageId, 34
getDamageString, 34
getDamageTextColor, 35
getSquadString, 35
getUnitId, 35
showAbility, 35
showSquad, 36
squadString, 37
unitId, 37
UNUSED, 37
CardUiStateFactory, 38
CardUiStateFactory, 39
createCardUiState, 39
damageTextColors, 40
horn, 40
weather, 40
CardViewHolder
CardListAdapter.CardViewHolder, 42
cardViews
FactionSwitchListener, 70
ChangeFactionDialog, 44
callback, 47
ChangeFactionDialog, 46
getOnThemeClickListener, 46
onCreate, 47
ChangeFactionDialog.Callback, 24
onThemeSelect, 24
changeListener
SoundManager, 167
clearRows
RowDao, 127
clearWeather
RowDao, 127
UnitRepository, 195
CoinFlipDialog, 47
CoinFlipDialog, 49
onCreate, 49
RANDOM, 50
collectionToString
UnitEntity, 187
COMMANDER
RuleSection, 144
component
DamageCalculatorDecorator, 59
copy
UnitRepository, 195
countUnits
UnitDao, 181
UnitRepository, 196
COURSE
RuleSection, 144
create
ResetAlertDialogBuilderAdapter, 116
RevengeAlertDialogBuilderAdapter, 124
createCardUiState
CardUiStateFactory, 39
createFragment
SectionsPagerAdapter, 147
createSound
Sound, 160
CURRENT_FRAGMENT_KEY
SettingsActivity, 150
currentView
IndicatorManager, 89

damage
MenuUiState, 101
RowUiState, 136
UnitEntity, 192
damageBackgroundImageId
CardUiState, 36
DamageCalculator, 51
calculateDamage, 52
isBuffed, 52
DamageCalculator.Color, 50
BUFFED, 50
DEBUFFED, 50
DEFAULT, 50
DamageCalculatorBuildDirector, 53
getCalculator, 53
setHorns, 54
setMoralBoosts, 54
setSquads, 55
DamageCalculatorBuilder, 55
calculator, 57
getResult, 56
setBond, 56

setHorn, 56
setMoral, 57
setWeather, 57
DamageCalculatorDecorator, 58
component, 59
DamageCalculatorDecorator, 59
DamageCalculatorUseCase, 59
getDamageCalculator, 60
damagePicker
CardNumberPickerAdapter, 31
damageString
CardUiState, 36
damageTextColor
CardUiState, 36
damageTextColors
CardUiStateFactory, 40
DamageValuePicker, 61
DamageValuePicker, 63
epicValues, 65
getDisplayString, 64
getValue, 64
setEpicValues, 64
setSelectableValues, 64
setValue, 65
damageView
CardListAdapter.CardViewHolder, 44
MenuUiStateObserver, 104
RowUiStateObserver, 140
Dandelion's Calculator, 1
database
GwentApplication, 81
UnitRepository, 203
DEBUFFED
DamageCalculator.Color, 50
DEFAULT
DamageCalculator.Color, 50
ResetDialogUseCase.DialogType, 66
delete
UnitRepository, 196
deleteUnit
UnitDao, 182
deleteUnits
UnitDao, 182
dialogCallback
ResetAlertDialogBuilderAdapter, 117
DIFF_CALLBACK
CardUiState, 37
dismissViewId
OverlayDialog, 111
displayIntegers
ValuePicker< T extends Comparable >, 209
disposables
AddCardDialog, 15
MainActivity, 98
ShowUnitsDialog, 158
doubleDamage
HornDamageCalculatorDecorator, 85

epic
UnitEntity, 192
EPIC_DAMAGE_VALUES
UnitEntity, 192
EPIC_UNIT_ABILITIES
CardNumberPickerAdapter, 31
epicPicker
CardNumberPickerAdapter, 31
epicValues
DamageValuePicker, 65
equals
CardUiState, 34
MenuUiState, 100
RowUiState, 135

FACTION_SWITCH
ResetDialogUseCase.Trigger, 180
factionButton
FactionSwitchListener, 70
FACTIONS
RuleSection, 144
FactionSwitchListener, 67
ballViews, 70
cardViews, 70
factionButton, 70
FactionSwitchListener, 68
getListener, 69
onSharedPreferenceChanged, 69
setTheme, 69
THEME_MONSTER, 70
THEME_NILFGAARD, 71
THEME_NORTHERN_KINGDOMS, 71
THEME_PREFERENCE_KEY, 71
THEME_SCOIATAEL, 71
unitViews, 71
factionSwitchListener
MainActivity, 98
finishButton
IntroductionActivity, 91

GameBoardViewModel, 71
GameBoardViewModel, 73
getMenuUiState, 74
getModel, 74
getRepository, 74
getRowUiState, 75
getSoundManager, 75
INITIALIZER, 78
menuUiState, 78
onBurnButtonPressed, 75
onFactionSwitchReset, 76
onHornViewPressed, 76
onResetButtonPressed, 76
onWeatherButtonPressed, 77
onWeatherViewPressed, 77
reset, 77
rowUiStates, 78
soundManager, 79
gameBoardViewModel
MainActivity, 99

GENERAL
RuleSection, 144
getAbility
UnitEntity, 188
getAbilityImageId
CardUiState, 34
getAbilityView
CardListAdapter.CardViewHolder, 42
getBindingView
CardListAdapter.CardViewHolder, 42
getBurnUnits
BurnDialogUseCase, 22
getCalculator
DamageCalculatorBuildDirector, 53
getContext
ValuePicker< T extends Comparable >, 206
getDamage
MenuUiState, 100
RowUiState, 136
UnitEntity, 188
getDamageBackgroundImageId
CardUiState, 34
getDamageCalculator
DamageCalculatorUseCase, 60
getDamageString
CardUiState, 34
getDamageTextColor
CardUiState, 35
getDamageView
CardListAdapter.CardViewHolder, 43
getDelayedOnValueChangeListener
CardNumberPickerAdapter, 30
getDialog
ShowUnitsDialog, 157
getDialogType
ResetDialogUseCase, 118
getDisplayIntegers
ValuePicker< T extends Comparable >, 206
getDisplayString
DamageValuePicker, 64
StringValuePicker< T extends Comparable >, 179
ValuePicker< T extends Comparable >, 206
getFirstSquadWithMembers
SquadManager, 171
getId
RowEntity, 131
UnitEntity, 188
getItemCount
SectionsPagerAdapter, 147
getItemId
CardListAdapter, 26
getListener
FactionSwitchListener, 69
getMemberBaseDamage
SquadState, 174
getMenuUiState
GameBoardViewModel, 74
getModel
GameBoardViewModel, 74
getObserver
RowUiStateObserver, 139
getOnThemeClickListener
ChangeFactionDialog, 46
getOverlayView
ImageViewSwitchAnimator, 87
getPicker
ValuePicker< T extends Comparable >, 207
getPreferenceKey
Sound, 161
getRandomUnit
ResetRepositoryUseCase, 120
getRepository
GameBoardViewModel, 74
GwentApplication, 81
UnitRepository, 197
getResult
DamageCalculatorBuilder, 56
getRevengeDialog
RemoveUnitsUseCase, 112
ResetRepositoryUseCase, 121
getRow
ShowUnitsDialog, 157
UnitEntity, 189
getRowUiState
GameBoardViewModel, 75
getSectionClickListener
SettingsRuleFragment, 152
getSelectableValues
ValuePicker< T extends Comparable >, 207
getSoundId
Sound, 161
getSoundManager
GameBoardViewModel, 75
getSquad
UnitEntity, 189
getSquadMembers
SquadState, 174
getSquadNumber
SquadState, 174
getSquadString
CardUiState, 35
getState
SquadState, 174
getUnit
UnitDao, 182
UnitRepository, 197
getUnitId
CardUiState, 35
getUnits
RowUiState, 136
UnitDao, 183
UnitRepository, 197, 198
getUnitsFlowable
UnitDao, 183
UnitRepository, 198
getValue

DamageValuePicker, 64
ValuePicker< T extends Comparable >, 207
gwent/data/Ability.java, 215
gwent/data/AppDatabase.java, 215
gwent/data/RowDao.java, 215
gwent/data/RowEntity.java, 216
gwent/data/RowType.java, 216
gwent/data/UnitDao.java, 217
gwent/data/UnitEntity.java, 217
gwent/data/UnitRepository.java, 220
gwent/domain/cases/BurnDialogUseCase.java, 223
gwent/domain/cases/DamageCalculatorUseCase.java, 224
gwent/domain/cases/RemoveUnitsUseCase.java, 224
gwent/domain/cases/ResetAlertDialogBuilderAdapter.java, 225
gwent/domain/cases/ResetDialogUseCase.java, 226
gwent/domain/cases/ResetRepositoryUseCase.java, 227
gwent/domain/cases/RevengeAlertDialogBuilderAdapter.java, 228
gwent/domain/damage/BondDamageCalculatorDecorator.java, 229
gwent/domain/damage/DamageCalculator.java, 230
gwent/domain/damage/DamageCalculatorBuildDirector.java, 230
gwent/domain/damage/DamageCalculatorBuilder.java, 231
gwent/domain/damage/DamageCalculatorDecorator.java, 231
gwent/domain/damage/HornDamageCalculatorDecorator.java, 232
gwent/domain/damage/MoralDamageCalculatorDecorator.java, 232
gwent/domain/damage/WeatherDamageCalculator.java, 233
gwent/GwentApplication.java, 233
gwent/ui/dialogs/addcard/AddCardDialog.java, 233
gwent/ui/dialogs/addcard/CardNumberPickerAdapter.java, 234
gwent/ui/dialogs/addcard/DamageValuePicker.java, 236
gwent/ui/dialogs/addcard/SquadManager.java, 237
gwent/ui/dialogs/addcard/SquadState.java, 238
gwent/ui/dialogs/addcard/StringValuePicker.java, 239
gwent/ui/dialogs/addcard/ValuePicker.java, 239
gwent/ui/dialogs/cards/CardListAdapter.java, 241
gwent/ui/dialogs/cards/CardUiState.java, 242
gwent/ui/dialogs/cards/CardUiStateFactory.java, 244
gwent/ui/dialogs/cards/ShowUnitsDialog.java, 245
gwent/ui/dialogs/ChangeFactionDialog.java, 247
gwent/ui/dialogs/CoinFlipDialog.java, 248
gwent/ui/dialogs/OverlayDialog.java, 248
gwent/ui/introduction/IndicatorManager.java, 249
gwent/ui/introduction/IntroductionActivity.java, 249
gwent/ui/introduction/IntroductionFragment.java, 250
gwent/ui/introduction/SectionsPagerAdapter.java, 251
gwent/ui/main/FactionSwitchListener.java, 251
gwent/ui/main/GameBoardViewModel.java, 253
gwent/ui/main/ImageViewSwitchAnimator.java, 256
gwent/ui/main/MainActivity.java, 256
gwent/ui/main/MenuUiState.java, 259
gwent/ui/main/MenuUiStateObserver.java, 260
gwent/ui/main/RowUiState.java, 260
gwent/ui/main/RowUiStateObserver.java, 261
gwent/ui/settings/RuleActivity.java, 262
gwent/ui/settings/RuleSection.java, 263
gwent/ui/settings/SettingsActivity.java, 263
gwent/ui/settings/SettingsHeaderFragment.java, 264
gwent/ui/settings/SettingsRuleFragment.java, 265
gwent/ui/settings/SettingsSoundFragment.java, 266
gwent/ui/sounds/Sound.java, 266
gwent/ui/sounds/SoundManager.java, 267
GwentApplication, 79
database, 81
getRepository, 81
onCreate, 81
repository, 82
hasMembers
SquadState, 175
hasNonEpicUnitsFlowable
UnitDao, 184
UnitRepository, 198
HORN
Ability, 11
horn
CardUiStateFactory, 40
RowEntity, 133
RowUiState, 136
HornDamageCalculatorDecorator, 82
calculateDamage, 84
doubleDamage, 85
HornDamageCalculatorDecorator, 84
isBuffed, 85
unitIds, 86
hornView
RowUiStateObserver, 140
id
RowEntity, 133
UnitEntity, 192
idToSquadSize
BondDamageCalculatorDecorator, 20
imageIds
IntroductionFragment, 94
ImageViewSwitchAnimator, 86
animatedSwitch, 86
getOverlayView, 87
IndicatorManager, 87
currentView, 89
IndicatorManager, 88
indicators, 89
updateIndicators, 88
indicatorManager
IntroductionActivity, 92
indicators
IndicatorManager, 89

inflateCoinFlipPopup
MainActivity, 97
inflateFactionPopup
MainActivity, 97
INITIALIZER
GameBoardViewModel, 78
initializeRows
UnitRepository, 199
initializeViewModel
MainActivity, 97
insertAvengers
RevengeAlertDialogBuilderAdapter, 124
insertRow
RowDao, 128
insertUnit
UnitDao, 184
UnitRepository, 199
INTENT_EXTRA_TAG
RuleActivity, 143
IntroductionActivity, 89
callback, 91
finishButton, 91
indicatorManager, 92
nextButton, 92
onCreate, 91
onResume, 91
skipButton, 92
updateBottomBar, 91
viewPager, 92
IntroductionFragment, 92
ARG_SECTION_NUMBER, 94
imageIds, 94
newInstance, 93
onCreateView, 94
PAGES_COUNT, 94
stringIds, 95
isActivated
Sound, 161
isBuffed
BondDamageCalculatorDecorator, 20
DamageCalculator, 52
HornDamageCalculatorDecorator, 85
MoralDamageCalculatorDecorator, 107
UnitEntity, 189
WeatherDamageCalculator, 212
isBurn
MenuUiState, 101
isEpic
UnitEntity, 189
isHorn
RowDao, 128
RowEntity, 132
RowUiState, 136
UnitRepository, 200
isHornFlowable
RowDao, 128
UnitRepository, 200
isReset
MenuUiState, 101
isWeather
MenuUiState, 101
RowDao, 129
RowEntity, 132
RowUiState, 136
UnitRepository, 201
isWeatherFlowable
RowDao, 129
UnitRepository, 201
itemId
CardListAdapter.CardViewHolder, 44

layout
OverlayDialog, 111

MainActivity, 95
disposables, 98
factionSwitchListener, 98
gameBoardViewModel, 99
inflateCoinFlipPopup, 97
inflateFactionPopup, 97
initializeViewModel, 97
onCreate, 97
onDestroy, 98
onResume, 98
onWindowFocusChanged, 98
MAX_NR_SQUADS
SquadManager, 172
MELEE
RowType, 134
memberBaseDamage
SquadState, 175
MenuUiState, 99
burn, 101
damage, 101
equals, 100
getDamage, 100
isBurn, 101
isReset, 101
isWeather, 101
MenuUiState, 100
reset, 101
weather, 101
menuUiState
GameBoardViewModel, 78
MenuUiStateObserver, 102
accept, 103
burnButton, 104
damageView, 104
MenuUiStateObserver, 103
resetButton, 104
weatherButton, 104
MONSTER
ResetDialogUseCase.DialogType, 66
MORAL_BOOST
Ability, 12
MoralDamageCalculatorDecorator, 104
calculateDamage, 107

isBuffed, 107
MoralDamageCalculatorDecorator, 106
unitIds, 107

newInstance
IntroductionFragment, 93
nextButton
IntroductionActivity, 92
NO_DISMISS_VIEW
OverlayDialog, 111
NON_EPIC_DAMAGE_VALUES_UPPER_BOUND
UnitEntity, 192
NONE
Ability, 12
ResetDialogUseCase.DialogType, 66
numberPicker
CardNumberPickerAdapter, 31

onBindViewHolder
CardListAdapter, 27
onBurnButtonPressed
GameBoardViewModel, 75
onCopy
CardListAdapter, 27
onCreate
AddCardDialog, 14
ChangeFactionDialog, 47
CoinFlipDialog, 49
GwentApplication, 81
IntroductionActivity, 91
MainActivity, 97
OverlayDialog, 111
RuleActivity, 142
SettingsActivity, 149
ShowUnitsDialog, 158
onCreatePreferences
SettingsHeaderFragment, 151
SettingsRuleFragment, 153
SettingsSoundFragment, 154
onCreateView
IntroductionFragment, 94
onCreateViewHolder
CardListAdapter, 27
onDestroy
MainActivity, 98
onFactionSwitchReset
GameBoardViewModel, 76
onHornViewPressed
GameBoardViewModel, 76
onItemRangeInserted
ShowUnitsDialog.ScrollToEndObserver, 145
onOptionsItemSelected
RuleActivity, 142
SettingsActivity, 149
onPreferenceStartFragment
SettingsActivity, 149
onRemove
CardListAdapter, 28
onResetButtonPressed
GameBoardViewModel, 76
onResume
IntroductionActivity, 91
MainActivity, 98
onSaveInstanceState
SettingsActivity, 149
onSharedPreferenceChanged
FactionSwitchListener, 69
onSquadChanged
SquadManager, 171
onThemeSelect
ChangeFactionDialog.Callback, 24
onValueChange
ValuePicker< T extends Comparable >.OnValueChangeListener< T extends Comparable >, 108
onWeatherButtonPressed
GameBoardViewModel, 77
onWeatherViewPressed
GameBoardViewModel, 77
onWindowFocusChanged
MainActivity, 98
OverlayDialog, 109
BACKGROUND, 111
dismissViewId, 111
layout, 111
NO_DISMISS_VIEW, 111
onCreate, 111
OverlayDialog, 110

PAGES_COUNT
IntroductionFragment, 94
picker
ValuePicker< T extends Comparable >, 209
pickerAdapter
AddCardDialog, 15
playBurnSound
SoundManager, 165
playCardAddSound
SoundManager, 165
playCardRemovedSound
SoundManager, 165
playClearWeatherSound
SoundManager, 165
playCoinSound
SoundManager, 165
playHornSound
SoundManager, 166
playResetSound
SoundManager, 166
playSound
SoundManager, 166
playWeatherSound
SoundManager, 167
preferenceKey
Sound, 162

RANDOM
CoinFlipDialog, 50
RANGE

RowType, 134
remove
RemoveUnitsUseCase, 113, 114
RemoveUnitsUseCase, 112
getRevengeDialog, 112
remove, 113, 114
repository
GwentApplication, 82
reset
GameBoardViewModel, 77
MenuUiState, 101
ResetAlertDialogBuilderAdapter.Callback, 23
ResetDialogUseCase, 118, 119
ResetRepositoryUseCase, 121, 122
UnitRepository, 202
ResetAlertDialogBuilderAdapter, 114
adapteeBuilder, 117
create, 116
dialogCallback, 117
ResetAlertDialogBuilderAdapter, 115
setMonsterDialog, 116
setTrigger, 116
ResetAlertDialogBuilderAdapter.Callback, 23
reset, 23
resetButton
MenuUiStateObserver, 104
ResetDialogUseCase, 117
getDialogType, 118
reset, 118, 119
ResetDialogUseCase.DialogType, 66
DEFAULT, 66
MONSTER, 66
NONE, 66
ResetDialogUseCase.Trigger, 180
BUTTON_CLICK, 180
FACTION_SWITCH, 180
ResetRepositoryUseCase, 120
getRandomUnit, 120
getRevengeDialog, 121
reset, 121, 122
REVENGE
Ability, 12
RevengeAlertDialogBuilderAdapter, 122
adapteeBuilder, 125
AVENGER_ABILITY, 125
AVENGER_DAMAGE, 126
AVENGER_EPIC, 126
AVENGER_ROW, 126
AVENGER_SQUAD, 126
create, 124
insertAvengers, 124
RevengeAlertDialogBuilderAdapter, 124
setNegativeCallback, 125
setPositiveCallback, 125
row
AddCardDialog, 15
ShowUnitsDialog, 158
UnitEntity, 193
RowDao, 127
clearRows, 127
clearWeather, 127
insertRow, 128
isHorn, 128
isHornFlowable, 128
isWeather, 129
isWeatherFlowable, 129
updateHorn, 129
updateWeather, 130
RowEntity, 130
getId, 131
horn, 133
id, 133
isHorn, 132
isWeather, 132
RowEntity, 131
setHorn, 132
setWeather, 132
weather, 133
rows
AppDatabase, 17
RowType, 133
MELEE, 134
RANGE, 134
SIEGE, 134
RowUiState, 134
damage, 136
equals, 135
getDamage, 136
getUnits, 136
horn, 136
isHorn, 136
isWeather, 136
RowUiState, 135
units, 137
weather, 137
RowUiStateObserver, 137
accept, 139
damageView, 140
getObserver, 139
hornView, 140
RowUiStateObserver, 139
unitView, 140
weatherResource, 140
weatherView, 141
rowUiStates
GameBoardViewModel, 78
RuleActivity, 141
INTENT_EXTRA_TAG, 143
onCreate, 142
onOptionsItemSelected, 142
RuleSection, 143
CARD_ABILITIES, 143
CARDS, 143
COMMANDER, 144
COURSE, 144
FACTIONS, 144

GENERAL, 144
SPECIAL_CARDS, 144

SectionsPagerAdapter, 145
createFragment, 147
getItemCount, 147
SectionsPagerAdapter, 146
selectableValues
ValuePicker< T extends Comparable >, 210
setAbility
UnitEntity, 190
setActivated
Sound, 161, 162
setBond
DamageCalculatorBuilder, 56
setDamage
UnitEntity, 190
setEpic
UnitEntity, 190
setEpicValues
DamageValuePicker, 64
setHorn
DamageCalculatorBuilder, 56
RowEntity, 132
setHorns
DamageCalculatorBuildDirector, 54
setId
UnitEntity, 191
setItemId
CardListAdapter.CardViewHolder, 43
setMonsterDialog
ResetAlertDialogBuilderAdapter, 116
setMoral
DamageCalculatorBuilder, 57
setMoralBoosts
DamageCalculatorBuildDirector, 54
setNegativeCallback
RevengeAlertDialogBuilderAdapter, 125
setOnValueChangedListener
ValuePicker< T extends Comparable >, 207
setPositiveCallback
RevengeAlertDialogBuilderAdapter, 125
setRow
UnitEntity, 191
setSelectableValues
DamageValuePicker, 64
ValuePicker< T extends Comparable >, 208
setSquad
UnitEntity, 191
setSquads
DamageCalculatorBuildDirector, 55
setTheme
FactionSwitchListener, 69
SettingsActivity, 147
CURRENT_FRAGMENT_KEY, 150
onCreate, 149
onOptionsItemSelected, 149
onPreferenceStartFragment, 149
onSaveInstanceState, 149
SettingsHeaderFragment, 150
onCreatePreferences, 151
SettingsRuleFragment, 151
getSectionClickListener, 152
onCreatePreferences, 153
SettingsSoundFragment, 153
onCreatePreferences, 154
setTrigger
ResetAlertDialogBuilderAdapter, 116
setValue
DamageValuePicker, 65
ValuePicker< T extends Comparable >, 209
setWeather
DamageCalculatorBuilder, 57
RowEntity, 132
showAbility
CardUiState, 35
showSquad
CardUiState, 36
ShowUnitsDialog, 155
cardListAdapter, 158
disposables, 158
getDialog, 157
getRow, 157
onCreate, 158
row, 158
ShowUnitsDialog, 157
soundManager, 158
unitRecyclerView, 159
ShowUnitsDialog.ScrollToEndObserver, 144
onItemRangeInserted, 145
SIEGE
RowType, 134
skipButton
IntroductionActivity, 92
Sound, 159
activated, 162
createSound, 160
getPreferenceKey, 161
getSoundId, 161
isActivated, 161
preferenceKey, 162
setActivated, 161, 162
Sound, 160
soundId, 162
SOUND_BURN
SoundManager, 167
SOUND_CARDS_EPIC
SoundManager, 167
SOUND_CARDS_MELEE
SoundManager, 167
SOUND_CARDS_RANGE
SoundManager, 168
SOUND_CARDS_SIEGE
SoundManager, 168
SOUND_COIN
SoundManager, 168
SOUND_HORN

SoundManager, 168
SOUND_RESET
SoundManager, 168
SOUND_WEATHER_FOG
SoundManager, 169
SOUND_WEATHER_FROST
SoundManager, 169
SOUND_WEATHER_GOOD
SoundManager, 169
SOUND_WEATHER_RAIN
SoundManager, 169
soundId
Sound, 162
SoundManager, 162
changeListener, 167
playBurnSound, 165
playCardAddSound, 165
playCardRemovedSound, 165
playClearWeatherSound, 165
playCoinSound, 165
playHornSound, 166
playResetSound, 166
playSound, 166
playWeatherSound, 167
SOUND_BURN, 167
SOUND_CARDS_EPIC, 167
SOUND_CARDS_MELEE, 167
SOUND_CARDS_RANGE, 168
SOUND_CARDS_SIEGE, 168
SOUND_COIN, 168
SOUND_HORN, 168
SOUND_RESET, 168
SOUND_WEATHER_FOG, 169
SOUND_WEATHER_FROST, 169
SOUND_WEATHER_GOOD, 169
SOUND_WEATHER_RAIN, 169
SoundManager, 164
soundPool, 169
sounds, 170
soundManager
AddCardDialog, 15
GameBoardViewModel, 79
ShowUnitsDialog, 158
soundPool
SoundManager, 169
sounds
SoundManager, 170
SPECIAL_CARDS
RuleSection, 144
squad
UnitEntity, 193
SquadManager, 170
getFirstSquadWithMembers, 171
MAX_NR_SQUADS, 172
onSquadChanged, 171
SquadManager, 171
states, 172
squadManager
CardNumberPickerAdapter, 31
squadMembers
SquadState, 175
squadNumber
SquadState, 176
squadPicker
CardNumberPickerAdapter, 31
SquadState, 172
getMemberBaseDamage, 174
getSquadMembers, 174
getSquadNumber, 174
getState, 174
hasMembers, 175
memberBaseDamage, 175
squadMembers, 175
squadNumber, 176
SquadState, 173
squadString
CardUiState, 37
states
SquadManager, 172
stringIds
IntroductionFragment, 95
StringValuePicker
StringValuePicker< T extends Comparable >, 178
StringValuePicker< T extends Comparable >, 176
getDisplayString, 179
StringValuePicker, 178
switchHorn
UnitRepository, 202
switchWeather
UnitRepository, 202

THEME_MONSTER
FactionSwitchListener, 70
THEME_NILFGAARD
FactionSwitchListener, 71
THEME_NORTHERN_KINGDOMS
FactionSwitchListener, 71
THEME_PREFERENCE_KEY
FactionSwitchListener, 71
THEME_SCOIATAEL
FactionSwitchListener, 71
toString
UnitEntity, 191

UnitDao, 180
countUnits, 181
deleteUnit, 182
deleteUnits, 182
getUnit, 182
getUnits, 183
getUnitsFlowable, 183
hasNonEpicUnitsFlowable, 184
insertUnit, 184
UnitEntity, 185
ability, 192
calculateDamage, 187
collectionToString, 187

damage, 192
epic, 192
EPIC_DAMAGE_VALUES, 192
getAbility, 188
getDamage, 188
getId, 188
getRow, 189
getSquad, 189
id, 192
isBuffed, 189
isEpic, 189
NON_EPIC_DAMAGE_VALUES_UPPER_BOUND, 192
row, 193
setAbility, 190
setDamage, 190
setEpic, 190
setId, 191
setRow, 191
setSquad, 191
squad, 193
toString, 191
UnitEntity, 186
unitId
CardUiState, 37
unitIds
HornDamageCalculatorDecorator, 86
MoralDamageCalculatorDecorator, 107
unitRecyclerView
ShowUnitsDialog, 159
UnitRepository, 193
clearWeather, 195
copy, 195
countUnits, 196
database, 203
delete, 196
getRepository, 197
getUnit, 197
getUnits, 197, 198
getUnitsFlowable, 198
hasNonEpicUnitsFlowable, 198
initializeRows, 199
insertUnit, 199
isHorn, 200
isHornFlowable, 200
isWeather, 201
isWeatherFlowable, 201
reset, 202
switchHorn, 202
switchWeather, 202
UnitRepository, 195
units
AppDatabase, 17
RowUiState, 137
unitView
RowUiStateObserver, 140
unitViews
FactionSwitchListener, 71
UNUSED
CardUiState, 37
updateBottomBar
IntroductionActivity, 91
updateHorn
RowDao, 129
updateIndicators
IndicatorManager, 88
updateWeather
RowDao, 130

ValuePicker
ValuePicker< T extends Comparable >, 205
ValuePicker< T extends Comparable >, 203
displayIntegers, 209
getContext, 206
getDisplayIntegers, 206
getDisplayString, 206
getPicker, 207
getSelectableValues, 207
getValue, 207
picker, 209
selectableValues, 210
setOnValueChangedListener, 207
setSelectableValues, 208
setValue, 209
ValuePicker, 205
ValuePicker< T extends Comparable >.OnValueChangeListener< T extends Comparable >, 108
onValueChange, 108
viewPager
IntroductionActivity, 92

weather
CardUiStateFactory, 40
MenuUiState, 101
RowEntity, 133
RowUiState, 137
WeatherDamageCalculator, 213
weatherButton
MenuUiStateObserver, 104
WeatherDamageCalculator, 210
calculateDamage, 212
isBuffed, 212
weather, 213
WeatherDamageCalculator, 211
weatherResource
RowUiStateObserver, 140
weatherView
RowUiStateObserver, 141